

Základy umělé inteligence

Studijní text

Obsah

1	Úvod.....	5
1.1	Definice pojmů inteligence a umělé inteligence.....	5
1.2	Stručná historie umělé inteligence.....	6
1.3	Klasická umělá inteligence	9
2	Řešení úloh.....	10
2.1	Metody řešení úloh prohledáváním stavového prostoru.....	10
2.1.1	Úlohy řešitelné prohledáváním stavového prostoru.....	12
2.1.1.1	Úloha dvou džbánů	12
2.1.1.2	Úloha Loydovy patnáctky	13
2.1.1.3	Úloha nalezení nejkratší cesty.....	13
2.1.1.4	Úloha obchodního cestujícího.....	13
2.1.2	Neinformované metody	14
2.1.2.1	Metoda slepého prohledávání do šířky (BFS)	14
2.1.2.2	Metoda slepého prohledávání do hloubky (DFS)	18
2.1.2.3	Metoda omezeného prohledávání do hloubky (DLS).....	21
2.1.2.4	Metoda postupného zanořování do hloubky (IDS)	21
2.1.2.5	Metoda obousměrného prohledávání (BS)	22
2.1.2.6	Metoda stejných cen (UCS)	24
2.1.2.7	Metoda zpětného navracení (Backtracking)	25
2.1.3	Informované metody	28
2.1.3.1	Metoda hltavého prohledávání (GS)	29
2.1.3.2	Metoda A*	30
2.1.4	Metody lokálního prohledávání	33
2.1.4.1	Metoda horolezecká (HC).....	33
2.1.4.2	Metoda simulovaného žíhání (SA)	34
2.2	Metody řešení úloh s omezujícími podmínkami	35
2.2.1	Úlohy s omezujícími podmínkami	35
2.2.1.1	Úloha osmi dam	36
2.2.1.2	Úloha barvení map	37
2.2.2	Metody řešení CSP.....	37
2.2.2.1	Metoda zpětného navracení pro CSP (Backtracking for CSP)	37
2.2.2.2	Metoda dopředné kontroly (Forward checking)	38
2.2.2.3	Metoda minimálního konfliktu (Min-conflict)	41
2.3	Metody řešení úloh rozkladem na podproblémy	43
2.3.1	Úlohy vhodné pro řešení rozkladem na podproblémy	43
2.3.1.1	Úloha hanojských věží	44

2.3.1.2	Úloha dvanácti mincí	45
2.3.2	Neinformovaná metoda (AO algoritmus)	47
2.3.3	Informovaná metoda (AO* algoritmus).....	49
2.4	Metody hraní her	51
2.4.1	Hry se dvěma hráči.....	51
2.4.1.1	Jednoduché hry	51
2.4.1.2	Složité hry	52
2.4.1.3	Hry s neurčitostí	53
2.4.2	Metody hraní složitých her dvou hráčů.....	53
2.4.2.1	Metoda Minimax.....	53
2.4.2.2	Alfa-beta řezy.....	56
2.4.2.3	Metody hraní her s neurčitostí	58
2.4.3	Metody hraní her n hráčů	61
2.5	Metody řešení úloh inspirované přírodou.....	63
2.5.1	Genetické algoritmy	63
2.5.1.1	Biologická inspirace.....	63
2.5.1.2	Princip činnosti genetického algoritmu.....	63
2.5.2	Optimalizace mravenčí kolonií	67
2.5.2.1	Biologická inspirace.....	67
2.5.2.2	Algoritmus Ant System.....	68
2.5.3	Optimalizace hejnem částic.....	72
3	Logika a umělá inteligence	74
3.1	Výroková logika	74
3.2	Predikátová logika	77
3.3	Rezoluční metoda	81
3.3.1	Unifikace	81
3.3.2	Zobecněná rezoluční metoda.....	83
3.4	Hornovy klauzule	86
4	PROLOG.....	87
4.1	Syntax jazyka.....	87
4.2	Plnění cílů	88
4.2.1	Jednoduchá konverzace	89
4.3	Práce se seznamy	90
4.4	Definice nových klauzulí	91
4.5	Vybrané zabudované predikáty	92
4.5.1	Predikáty pro práci s databází	92
4.5.2	I/O predikáty	93

4.5.3	Predikáty – operátory	93
4.5.4	Některé další užitečné predikáty	94
4.6	Abstraktní datové struktury (zásobník, fronta, množina)	95
4.7	Základní prohledávací algoritmy	96
4.7.1	BFS (se seznamem CLOSED) - úloha dvou džbánů.....	96
4.7.2	DFS - úloha dvou džbánů.....	98
4.7.3	Algoritmus A* (se seznamem CLOSED) – Loydův hlavolam.....	100
5	Strojové učení	104
5.1	Učení s učitelem	104
5.1.1	Rozhodovací stromy.....	105
5.1.1.1	Algoritmus Decision Tree	105
5.1.1.2	Algoritmus ID3	106
5.1.2	Prohledávání prostoru verzí.....	111
5.1.2.1	Algoritmus Specific_to_general_search	112
5.1.2.2	Algoritmus General_to_specific_search:	113
5.1.2.3	Algoritmus Candidate_elimination	113
5.1.3	Rozpoznávání a klasifikace	115
5.1.3.1	Příznakové rozpoznávání	116
5.1.3.2	Strukturální rozpoznávání	122
5.1.4	Neuronové sítě.....	125
5.1.4.1	Biologická inspirace.....	125
5.1.4.2	Neuronové sítě s LBF neurony	125
5.1.4.3	Neuronové sítě s RBF neurony	129
5.2	Učení bez učitele	131
5.3	Posilované učení	133
5.3.1	Policy-only learning	133
5.3.2	Metoda TD learning (On-Policy)	134
5.3.3	Metoda Q learning (Off-Policy).....	136
5.3.4	SARSA (On-Policy).....	137
6	Expertní systémy.....	138
6.1	Pravidlové expertní systémy.....	139
6.1.1	Pravidlové systémy pracující s neurčitostí	143
7	Počítačové vidění	149
7.1	Transformace obrazů	149
7.1.1	Monadické transformace	149
7.1.2	Dyadické transformace.....	150
7.1.3	Prostorové transformace.....	151

7.1.3.1	Dolní propusti	151
7.1.3.2	Horní propusti	153
7.2	Detekce přímk a rohů.....	155
7.2.1	Detekce přímek	155
7.2.2	Detekce rohů	157
7.3	Metrické a topologické vlastnosti snímku	158
7.4	Morfologické operace.....	159
7.5	Analýza scény.....	161
7.5.1	Dekompozice obrazů.....	161
7.5.2	Rozpoznání objektů.....	163
8	Zpracování přirozeného jazyka.....	165
8.1	Akustická analýza.....	165
8.1.1	Rozpoznávání porovnáváním se vzory	166
8.1.2	Rozpoznávání s využitím statistických metod	167
8.1.2.1	Skrytý Markovův model	168
8.1.2.2	Akustický model	170
8.1.2.3	Jazykový model	171
8.2	Zpracování textu	171
8.2.1	Znakové jazykové modely	171
8.2.2	Slovní jazykové modely	172
8.2.3	Lingvistická analýza.....	173
9	Agentní systémy.....	179
9.1	Plánování činnosti agenta	181
9.2	Multiagentní systémy	185
10	Závěr.....	189

1 Úvod

Cílem této úvodní kapitoly je seznámit čtenáře s různými definicemi pojmů inteligence a umělé inteligence, uvést, jak bude pojem umělé inteligence v dalším textu chápán, stručně seznámit čtenáře s dosavadním vývojem a nastínit současný stav problematiky.

1.1 Definice pojmů inteligence a umělé inteligence

Existuje celá řada různých definic pojmu inteligence (z lat. *intelligentia*, tj. rozlišovat, racionálně poznávat a chápat), dosud však obecně nebyla přijata žádná z nich. Například:

- Intelligence je složitou kognitivní vlastností osobnosti, která umožňuje jedinci poznávat svět a využívat získané poznatky pro přizpůsobení se změnám prostředí. Pojem kognitivní se přitom používá pro souhrnné označení vnímání a racionálního myšlení, které zahrnuje chápavost, představivost, hodnocení, paměť a usuzování.
- Intelligence je dispozice pro myšlení, učení a adaptaci a projevuje se intelektovým výkonem.
- Intelligence je schopnost účinně a rychle řešit na základě vlastní rozvahy obtížné nebo nové situace a problémy, nacházet podstatné prvky a jejich souvislosti a vztahy v těchto situacích, náležitě užívat výsledky vlastního myšlení, učit se ze zkušenosti.
- Intelligence je všeobecná schopnost individua vědomě orientovat vlastní myšlení na nové požadavky, je to všeobecná duchovní schopnost přizpůsobit se novým životním úkolům a podmínkám.
- Intelligence je vnitřně členitá a zároveň globální schopnost individua účelně jednat, rozumně myslet a efektivně se vyrovnávat se svým okolím.
- Intelligence je schopnost zpracovávat informace. Informacemi je třeba chápat všechny dojmy, které člověk vnímá.
- Intelligence je způsobilost determinující kognitivní operace, jejichž podstatou je patrně objevování a chápání různých vztahů. V tomto smyslu je inteligence složitá dispozice k myšlení, tj. k řešení problémů.
- ...

Odborníci v čele s psychology vymezují 8 různých druhů inteligence:

- jazykově-verbální inteligence,
- logicko-matematická inteligence,
- zvukově-hudební inteligence,
- tělesně-pohybová inteligence,
- vizuálně-prostorová inteligence,
- vnitřní nebo také intrapersonální inteligence,
- společenská nebo také interpersonální inteligence,
- přírodní inteligence.

Intelligence je navíc zřejmě jedinou vlastností, která není dostatečně definována, a přesto se přesně vyhodnocuje. Její jednotkou je tzv. inteligenční kvocient (IQ), který se zjišťuje pomocí inteligenčních testů. Jedna z definic inteligence pak dokonce zní: "Intelligence je to, co měří inteligenční test".

Pozn.: Intelligence byla dlouhou dobu přisuzována pouze lidem, v poslední době je studována také u zvířat, a dokonce i u některých rostlin.

Umělá intelligence (Artificial Intelligence) by pak měla jistým způsobem napodobovat inteligenci přirozenou. Proto i pro pojem umělá intelligence existuje celá řada různých definic, například:

- Umělá intelligence je vlastnost uměle vytvořeného systému, který má schopnost rozpoznávat předměty a jevy, analyzovat vztahy mezi nimi, a tak si vytvářet modely světa, dělat účelná rozhodnutí a předvídat jejich důsledky, řešit problémy včetně objevování nových zákonitostí a zdokonalování své činnosti.
- Umělá intelligence je modelování intelektuální činnosti člověka počítačem při řešení složitých úloh, kde postup vyžaduje schopnost výběru z mnoha nebo z nezřetelně popsanych variant; též samočinné rozpoznávání tvarů nebo předmětů, usuzování z jednoho výroku na jiný, vytváření analogií mezi jednotlivými úsudky, generování a ověřování hypotéz, tvorba a uplatnění znalostí na základě přijatých vstupních dat a informací, schopnost eliminovat nepříznivé reakce na podněty z okolí a usměrňovat činnost systému v probíhajících procesech s ohledem na měnící se a často nezřetelné vnější podmínky.

Pojem umělá intelligence se současně používá i pro označení vědní disciplíny, která se zabývá studiem a realizací umělé intelligence jako výše uvedené vlastnosti, a pouze takto bude tento pojem chápán v dalším textu.

Umělá intelligence jako vědní disciplína pak zastřešuje několik relativně samostatných oblastí, jejichž popisu jsou věnovány následující kapitoly. Velmi stručná, avšak výstižná charakteristika takto chápaného pojmu umělé intelligence je tato (Marvin Minsky, 1967):

- Umělá intelligence je věda o tom, jak konstruovat stroje, jejichž činnost, kdyby ji vykonávali lidé (a kdybychom nevěděli, že ji vykonávají stroje!), bychom považovali za projev jejich intelligence.

1.2 Stručná historie umělé intelligence

Za první práce spadající do problematiky umělé intelligence (dále UI) jsou považovány práce, na jejichž základě Warren McCulloch a Walter Pitts prezentovali v roce 1943 umělý neuron a naznačili možnosti umělých neuronových sítí, včetně jejich učení.

V roce 1949 formuloval Donald Hebb základní pravidlo pro učení neuronových sítí, které se používá pro učení některých neuronových sítí dodnes.

V roce 1950 publikoval Alan Turing článek *Computing Machinery and Intelligence*, ve kterém formuloval principy strojového učení (*Machine learning*) a především zde navrhl test kvality umělé intelligence (*Turing test*). Tento test je založen na imitační hře, ve které vystupují tři osoby: žena, muž a odděleně umístěný testující. Testující má za úkol zjistit pomocí nepřímých otázek, zda mu odpovídá žena, nebo muž. Allan Turing zaměnil dvojici žena muž za dvojici osoba a stroj; úkolem testujícího bylo nyní pomocí nepřímých otázek zjistit, zda mu na ně odpovídá osoba, nebo stroj. Tento test byl dlouho považován za základní měřítko schopností uměle vytvořeného inteligentního systému, nicméně jeho závažný nedostatek byl ukázán na tzv. argumentu čínského pokoje (John R. Searle, 1980): předpokládá uzavřenou místnost s knihovnou naplněnou čínskými texty, ve kterých se nachází každá smysluplná věta tohoto jazyka. V místnosti je také člověk/stroj, který je schopný se v knihovně velmi dobře orientovat. Tomuto člověku/stroji se písemně předávají otázky a ten je schopen nalézt na každou položenou

otázku smysluplnou odpověď. Tazatel se pak domnívá, že člověk/stroj uvnitř místnosti čínštinu dokonale ovládá, přestože ten jí vůbec nemusí rozumět.

V roce 1951 postavili Marvin Minsky a Dean Edmond v rámci svých disertačních prací na Princeton University první neuronový počítač, který nazvali SNARC (Stochastic Neural Analog Reinforcement Calculator).

Pojem Artificial intelligence se poprvé objevil v názvu dvouměsíční pracovní konference The Dartmouth Summer Research Project on Artificial Intelligence, která se uskutečnila v roce 1956 v Dartmouth College v Hanoveru (New Hampshire, USA) a které se zúčastnilo celkem deset vědeckých pracovníků: John McCarthy, Marvin Minsky, Claude Shannon, Nathaniel Rochester (Dartmouth College), Trenchard More (Princeton), Arthur Samuel (IBM), Ray Solomonoff, Olivier Selfridge (MIT), Allen Newell, Herbert Simon (CMU).

Tato konference byla věnována především úvahám o možnostech počítačové simulace procesů probíhajících v mozku a je považována za počátek umělé inteligence jako samostatné vědní disciplíny.

Výše uvedení účastníci konference se na mnoho dalších let stali vůdčími pracovníky v tomto novém oboru, a to spolu se svými kolegy a studenty na MIT (Massachusetts Institute of Technology), CMU (Carnegie Mellon University), SRI (Stanford Research Institute) a IBM (International Business Machines).

Na konferenci byl mj. předveden program nazvaný *Logic Theorist* (Newell, Simon), který byl určen pro dokazování matematických teorémů. Tento program byl prvním programem, který místo s čísly pracoval se symboly, a je proto považován za první funkční "inteligentní" program.

Závěry konference byly velmi optimistické. Převládl zde názor, že inteligentní činnost je založena především na dedukci, kterou brzy zvládnou výkonnější počítače pomocí ad hoc technik. Účastníci konference dokonce předpovídali, že počítače nahradí veškerou lidskou intelektuální činnost nejpozději do dvaceti pěti let.

Skutečnost však byla zcela jiná. UI prošla od zmíněné konference poměrně složitým vývojem, který se obvykle rozděluje do několika etap.

První etapa (zbytek padesátých a šedesátá léta) byla charakterizována nadšenou prací a velkým očekáváním. Z významných výsledků této etapy stojí za zmínku zejména:

- LISP (List Processor; J. McCarthy, 1958) - nejpoužívanější programovací jazyk pro UI v USA.
- GPS (General Problem Solver, A. Newell, 1960) - program určený k řešení obecných úloh.
- Adaline (Adaptive Linear Neuron, B. Widrow, M. Hoff) - algoritmus pro učení neuronů.
- Perceptron (F. Rosenblatt, 1962) - neuronová síť pro rozpoznávání obrazu, důkaz konvergence algoritmu pro učení této sítě.
- První programy určené pro hraní her: dáma (A. Samuel, 1960), šachy (A. Newell, J. C. Shaw, H. A. Simon, 1962).
- Program pro symbolickou integraci (J. Slag, 1961).
- Rezoluční metoda (J. Weizenbaum, 1966) pro strojové odvozování nových klauzulí.
- ELIZA (J. Weizenbaum, 1966) - program pro práci s přirozeným jazykem, který simuloval nepřímou psychoterapii.
- A* Algoritmus (P. E. Hart, N. J. Nilsson, B. Raphael, 1968) - základní algoritmus pro řešení úloh.

- QA3 (Question-Answering System, C. Green, 1969) - program postavený na rezoluční metodě. Byl určen k řešení jednodušších úloh (pohyby robotů, hříčky).

Koncem šedesátých let se začalo zřetelně ukazovat, že problematika UI je mnohem obtížnější, než se původně předpokládalo. Přispěla k tomu velmi omezená síla tehdejších počítačů, problémy s monotónností tradičních logik, a především skutečnost, že dosažené výsledky zdaleka nesplnily všechna očekávání. Například zcela selhaly pokusy o strojový překlad založený na slovnících a základních gramatických pravidlech. Poukazováno bývá zejména na překlad věty *The spirit is willing but the flesh is weak* do ruštiny a zpět, který údajně vedl ke větě *The vodka is good but the meat is rotten*. Stále zřetelněji se ukazovalo, že základem inteligence jsou znalosti, včetně znalostí kontextových a že při hledání řešení obtížnějších úloh jsou nezbytné heuristiky ke zvládnutí prohledávání kombinatorické exploze možných stavů těchto úloh. Také se ukázalo, že na rozdíl od původních předpokladů je řešení high-level problémů (například hraní her) jednodušší, než řešení low-level problémů (především senzomotorických). Období mezi roky 1968-1973 je proto označováno jako návrat k realitě (skepse, první „zimní“ období UI).

V dalším průběhu sedmdesátých let docházelo postupně k obnovení zájmu o UI, především díky expertním systémům. Mezi dosaženými výsledky v tomto období stojí za zmínku:

- SRI Vision Module (G. J. Agin, G. J. Gleason, 1970) - prototyp "vidícího" systému pro průmyslové použití (rozpoznávání součástek podle obrysů).
- STRIPS (R. Fikes, N. J. Nilsson, 1971) - program/jazyk určený pro plánování činnosti robota Shakey.
- DENDRAL (E. A. Feigenbaum, 1971) - první expertní systém. Byl určen pro identifikaci organických sloučenin.
- SHRDLU (T. Winograd, 1972) - program pro komunikaci v přirozeném jazyce o jednoduchém světě kostek (barvy kostek, vzájemné pozice ap.). Pracoval na základě syntaktické a sémantické analýzy.
- PROLOG (PROgramming in LOGic; A. Colmerauer, 1972, P. Roussel, 1975, D. H. D. Warren, L. M. Pereira, F. Pereira, 1977) - dosud nejpoužívanější jazyk pro UI v Evropě a v Japonsku.
- MYCIN (E. H. Shortliffe, 1976) - expertní systém pro diagnostiku infekčních onemocnění a návrh terapie.
- PROSPECTOR (R. Duda, P. Hart, 1978) - expertní systém pro hledání ložisek rud.
- HEARSAY II (L.D. Erman a kol., 1980) - systém pro rozpoznávání řeči se slovníkem asi 1000 slov.

Třetí etapa (léta osmdesátá a devadesátá) se vyznačovala zvýšeným zájmem o problematiku umělé inteligence a bývá charakterizována těmito výsledky:

- Expertní systém R1 pro konfiguraci počítačů firmy DEC, který údajně přinášel této firmě úsporu 40 milionů dolarů ročně, 1982.
- Počítače s architekturami vhodnými pro umělou inteligenci (LISPovské a PROLOGovské počítače).
- Rozsáhlé projekty s využitím výsledků UI, především japonský projekt páté generace počítačových systémů 1982-1992.
- Metoda backpropagation, (D. Rumelhart, G. Hinton, R. Williams, 1986) - metoda zpětného šíření chyby, která vedla k renesanci výzkumu neuronových sítí.

- Počítač Hitech (Carnegie-Mellon University, H. J. Berliner, 1988) - porazil šachového velmistra (A. Denkera).
- Bayesovské sítě (J. Pearl, 1990).
- Počátek výzkumu inteligentních agentů (1991).
- Posilované učení, program TD-gammon (G. Tesauro, 1992).
- RoboCup, 1. ročník mezinárodních soutěží/turnajů autonomních robotů v kopané, 1997.
- Počítač DeepBlue (IBM, F. Hsu, 1997) - porazil v sérii šesti zápasů poměrem 4:2 tehdejšího světového šachového šampiona G. Kasparova.

Za zmínku stojí, že v průběhu let 1987-1993 nastalo krátké druhé „zimní“ období UI. Bylo zapříčiněno zejména:

- Prudkým rozvojem stolních počítačů Apple a IBM, který způsobil v roce 1987 náhlé zhroucení trhu s UI hardware, především s LISPovskými počítači – uvádí se, že celý tento průmysl v hodnotě půl miliardy dolarů byl zničen prakticky přes noc.
- Neúspěchem japonského projektu počítačů páté generace.
- Nenaplněním očekávání přínosů expertních systémů.

V průběhu třetí etapy se UI stala uznávanou vědou. Byla tvořena odborná terminologie a byly publikovány příručky a monografie k jednotlivým oblastem UI (expertní systémy, počítačové vidění, zpracování přirozeného jazyka ap.).

Poslední etapa vývoje UI trvá přibližně od roku 2000 dodnes:

- Počátek vývoje komerčních chytrých zařízení: telefonů, praček, fotoaparátů, navigací, hodinek apod. (2000 - ...).
- Počátek vývoje komerčních servisních robotů (2006 - ...).
- První samostatná jízda autonomního auta Google po dálnici (2009).
- IBM Watson (systém dotaz-odpověď) zvítězil v americké vědomostní televizní soutěži Jeopardy (2011).
- Boom hlubokého učení při klasifikaci obrazů (2015 - ...).
- Počítač AlphaGo (Google) zvítězil poměrem 4:1 v sérii pěti zápasů nad osmnáctinásobným světovým mistrem Lee-Se-dolem ve hře Go (2016).

1.3 Klasická umělá inteligence

Klasická UI je zaměřena na základní teoretické přístupy k řešení úloh, reprezentaci znalostí, strojové učení, klasifikaci a rozpoznávání, a v aplikační oblasti pak na expertní systémy, počítačové vidění a zpracování přirozeného jazyka.

V poslední době se dostává do popředí zájmu distribuovaná UI (multiagentní systémy) a práce s neurčitostí (tzv. soft computing) reprezentovaná neuronovými sítěmi, genetickými algoritmy, pravděpodobnostním usuzováním, fuzzy množinami a fuzzy logikou, hrubými množinami a teorií chaosu.

2 Řešení úloh

Klasická UI úloha je definovaná počátečním stavem, množinou cílových stavů a množinou operátorů, které umožňují stavy této úlohy měnit. Řešením úlohy je nalezení posloupnosti operátorů, jejichž postupnou aplikací se dostaneme z počátečního stavu do nějakého stavu z množiny cílových stavů. V některých úlohách je tento postup nezajímavý a hledá se pouze cílový stav, který splňuje předem dané podmínky. Takové úlohy se nazývají CSP (*Constraint Satisfaction Problems*).

Z dnešního pohledu je klasická UI úloha úlohou v plně pozorovatelném prostředí (v každém časovém okamžiku je možné zjistit úplný stav tohoto prostředí), deterministickém prostředí (následující stav prostředí je určen pouze aktuálním stavem a použitým operátorem), statickém prostředí (stavy lze měnit výhradně definovanými operátory) a diskrétním prostředí (stavy se mění v diskrétních časových okamžicích).

Prvním úkolem při řešení každé úlohy je jednoznačná formulace jejích cílů a jednoznačná definice operátorů, která zahrnuje i případné podmínky omezující jejich použití. Metody řešení úloh pak nabízejí postupy, kterými lze cílové stavy (resp. posloupnosti operací vedoucí k těmto stavům) nalézat a představují tak prostředky nepostradatelné ve všech oblastech UI.

V tomto studijním textu se budeme zabývat pouze jednoduchými úlohami, jejichž formulace je snadná a většinou i známá. U takových úloh můžeme zaměřit svoji pozornost na principy jednotlivých metod, aniž bychom museli ztrácet čas potřebný k pochopení podstaty složitějších problémů.

Pro další výklad rozdělíme metody řešení úloh do pěti skupin (s vědomím, že poslední skupina je novější a do klasické UI nepatří):

- Metody řešení úloh prohledáváním stavového prostoru (*State Space Search Methods*),
- Metody řešení úloh s omezujícími podmínkami (*Constraint Satisfaction Methods*),
- Metody řešení úloh rozkladem na podproblémy (*Problem Reduction Methods*),
- Metody hraní her (*Game Playing Methods*),
- Metody řešení úloh algoritmy inspirovanými přírodou (*Nature-inspired algorithms*).

Metody řešení úloh se pak hodnotí podle čtyř kritérií, kterými jsou:

1. Úplnost (nalezne metoda řešení, pokud toto existuje?).
2. Optimálnost (nalezne metoda nejlepší řešení?).
3. Časová náročnost.
4. Paměťová/prostorová náročnost.

2.1 Metody řešení úloh prohledáváním stavového prostoru

Stavový prostor (*State space*) je orientovaný graf, jehož uzly představují jednotlivé stavy úlohy a jehož hrany reprezentují přechody mezi těmito stavy způsobené definovanými operátory. Formálně je stavový prostor definovaný dvojicí (S, O) , kde symbol S (*States*) označuje množinu všech možných stavů konkrétní úlohy a symbol O (*Operators*) označuje množinu všech operátorů, kterými lze stavy této úlohy měnit. Poznamenejme, že na konkrétní stavy nemusí být některé operátory aplikovatelné.

Úloha je definována dvojicí (s_0, G) kde symbol $s_0 \in S$ značí počáteční stav úlohy a symbol $G \subset S$ množinu cílových stavů této úlohy (Goals). Množina G obsahuje často jediný prvek.

Řešením úlohy je posloupnost operátorů $s_1 = o_1(s_0), s_2 = o_2(s_1), \dots, s_n = o_n(s_{n-1}), s_n \in G$, tzv. cesta, jejíž cena je dána součtem cen jednotlivých přechodů, které musí být kladné. V řadě úloh existuje takových cest více a obvykle se pak hledá cesta s nejmenší cenou (optimální cesta).

Metody řešení úloh prohledáváním stavového prostoru se dělí na dvě základní skupiny:

- Metody neinformované/slepé (*Uninformed/Blind Search*)
- Metody informované (*Informed Search*)

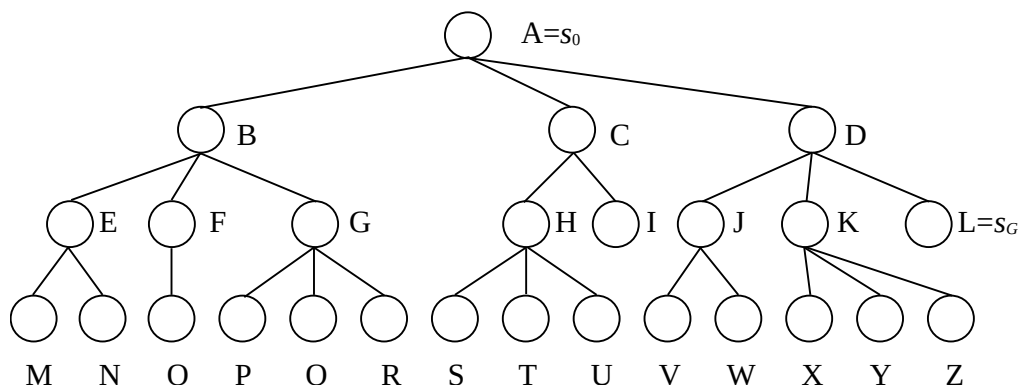
Neinformované metody:

- Metoda slepého prohledávání do šířky (*BFS, Breadth First Search*)
- Metoda slepého prohledávání do hloubky (*DFS, Depth First Search*)
- Metoda omezeného prohledávání do hloubky (*DLS, Depth Limited Search*)
- Metoda postupného zanořování do hloubky (*IDS, Iterative deepening Search*)
- Metoda obousměrného prohledávání (*BS, Bidirectional Search*)
- Metoda stejných cen (*UCS, Uniform Cost Search*)
- Metoda zpětného navracení (*Backtracking*)

Informované metody:

- Metody uspořádaného prohledávání (*BestFS, Best First Search*):
 - Metoda hltavého prohledávání (*GS, Greedy Search*)
 - Metoda A s hvězdičkou (*A*, A star Search*)
- Metody lokálního prohledávání (*Local Search*)
 - Metoda horolezecká (*HC, Hill climbing*)
 - Metoda simulovaného žíhání (*SA, Simulated annealing*)

Uvažujme jednoduchý demonstrační stavový prostor uvedený na Obr. 2.1.



Obr. 2.1 Demonstrační stavový prostor

V dalším výkladu budeme používat následující terminologii:

- uzel A je uzel kořenový,

- uzly I, L, M, ..., Z jsou uzly listové,
- uzel C je bezprostředním předchůdcem uzlu H atp.,
- uzly A, D, J jsou předchůdci uzlu V atp.,
- uzel K je bezprostředním následníkem uzlu D atp.,
- uzly H, I, S, T, U jsou následníci uzlu C atp.,
- uzel A má hloubku 0, uzly B, C, D mají hloubku 1 atp.,
- expanzí uzlu se rozumí určení všech jeho bezprostředních následníků,
- generací uzlu se nazývá proces jeho vytvoření,
- ohodnocením uzlu součet cen přechodů z kořenového do tohoto uzlu,
- uzel A označuje počáteční stav (s_0),
- uzel L označuje cílový stav (s_G).

2.1.1 Úlohy řešitelné prohledáváním stavového prostoru

Úlohy, které lze řešit prohledáváním stavového prostoru lze rozdělit na umělé úlohy (využívané především pro demonstraci činnosti jednotlivých metod řešení) a na úlohy reálné.

Umělé úlohy:

- Úloha dvou džbánů
- Úloha Loydovy patnáctky
- Rubikova kostka
- ...

Reálné úlohy:

- Úloha nalezení optimální cesty (z hlediska délky, času, ceny),
- Úloha obchodního cestujícího, TSP (*Travelling Salesman Problem*)
- ...

Dále si stručně popíšeme principy úloh, které budou použity v demonstračních příkladech při výkladu principů jednotlivých metod dalším textu.

2.1.1.1 Úloha dvou džbánů

Stav úlohy s = (obsah většího džbánu, obsah menšího džbánu). Pro klasickou úlohu se džbány o velikosti 4 a 3 litry je množina stavů $S = \{(0, 0), (0, 1), (0, 2), (0, 3), (1, 0), \dots, (4, 3)\}$.

Operátorů je celkem šest: Naplnění většího džbánu ($\rightarrow V$), naplnění menšího džbánu ($\rightarrow M$), vyprázdnění většího džbánu ($V \rightarrow$), vyprázdnění menšího džbánu ($M \rightarrow$), přelití (části) obsahu menšího do většího džbánu ($M \rightarrow V$), menší džbán je pak buď prázdný, nebo/a větší džbán je plný, přelití (části) obsahu většího do menšího džbánu ($V \rightarrow M$), větší džbán je pak buď prázdný, nebo/a menší džbán je plný, tj. množina operátorů $O = \{\rightarrow V, \rightarrow M, V \rightarrow, M \rightarrow, M \rightarrow V, V \rightarrow M\}$. Je zřejmé, že operace prováděné jednotlivými operátory nejsou obecně vratné. Například použití operátoru $\rightarrow M$ na stav $(0, 2)$ vede ke stavu $(0, 0)$, ale žádný operátor nemůže sám o sobě naplnit zpětně malý džbán dvěma litry vody/kapaliny.

Konkrétní úloha pak může být zadána například takto:

$$s_0 = (0, 0)$$

$$G = \{(2, 0), (0, 2)\}$$

Stavový prostor této úlohy je sice velmi jednoduchý (celkem pouze 20 stavů), přesto v něm existují stavy, kterých není možné dosáhnout, například stav (1, 1). Proto konkrétní úloha nemusí být řešitelná.

2.1.1.2 Úloha Loydovy patnáctky

Loydova patnáctka (*Sam Loyd's Fifteen*) je hlavolam, ve kterém je 15 očíslovaných kamenů o velikosti 1×1 umístěno v krabici 4×4 , a zbývá tedy v ní jedno volné místo. Kromě základní verze 4×4 existují verze tohoto hlavolamu o jiné velikosti, my budeme dále používat pouze jednoduchou verzi 3×3 (Loydovu osmičku).

Stav s = matice s čísly kamenů na jednotlivých políčkách, například: $s = \begin{bmatrix} 1 & 3 & 5 \\ 8 & 4 & 2 \\ 7 & 6 & _ \end{bmatrix}$
 resp. ve vektorovém zápisu (po řádcích) $s = (1, 3, 5, 8, 4, 2, 7, 6, _)$

Operátory:

- Při pohybech kameny se používá 32 operátorů: $O = \{o_{1\uparrow}, o_{1\rightarrow}, o_{1\downarrow}, o_{1\leftarrow}, o_{2\uparrow}, o_{2\rightarrow}, \dots, o_{8\leftarrow}\}$, které provádí tahy jednotlivými kameny nahoru, doprava, dolů a doleva.
- Při pohybu prázdným políčkem se používají pouze čtyři operátory: $O = \{\uparrow, \rightarrow, \downarrow, \leftarrow\}$.

Operátory z obou uvedených množin jsou ekvivalentní, například pohyb kamene č. 2 z výše ukázaného stavu dolů odpovídá pohybu prázdného políčka nahoru, a proto pohyb prázdným políčkem je výrazně jednodušší. Je také zřejmé, že operace jsou vždy vratné, tj. z každého stavu se lze dostat zpět do stavu předcházejícího.

Konkrétní úloha pak může být zadána například takto:

$$s_0 = \begin{bmatrix} 1 & 4 & 2 \\ 7 & _ & 3 \\ 6 & 8 & 5 \end{bmatrix} \quad s_G = \begin{bmatrix} 1 & 2 & 3 \\ 8 & _ & 4 \\ 7 & 6 & 5 \end{bmatrix} \quad \text{resp.} \quad s_0 = (1, 4, 2, 7, _, 3, 6, 8, 5) \\ s_G = (1, 2, 3, 8, _, 4, 7, 6, 5)$$

Poznamenejme, že stavové prostory Loydovy patnáctky, resp. osmičky mají $16! \cong 2 \cdot 10^{13}$, resp. $9! = 362880$ různých stavů a v obou případech jsou množiny těchto stavů rozdělené na dvě stejně velké disjunktní podmnožiny. Proto konkrétní úlohy nemusí být řešitelné, podobně jako v předchozí úloze dvou džbánů.

2.1.1.3 Úloha nalezení nejkratší cesty

Úloha nalezení nejkratší cesty (*Shortest path problem*) spočívá v hledání nejkratší cesty mezi dvěma různými místy (nejkratší z hlediska délky, času, ceny, ...). Pro n plně propojených míst (včetně výchozího a cílového) je počet různých možných cest dán výrazem $(n-2)!$ a jde tak o NP-úplný problém (například pro pouhých 12 míst existuje 3628800 různých možných cest).

2.1.1.4 Úloha obchodního cestujícího

Úloha obchodního cestujícího (*TSP, Traveling salesman problem*) je podobná předchozí úloze. Obchodní cestující musí navštívit n míst, každé pouze jedenkrát, a vrátit se do výchozího místa tak, aby celková délka cesty byla minimální. V tomto případě je počet různých možných cest dán výrazem $\frac{(n-1)!}{2}$ a jde opět o NP-úplný problém. Pro podobný příklad jako výše, tj. pro cestu přes 12 míst s návratem do výchozího místa, existuje téměř $2 \cdot 10^7$ různých možných cest.

2.1.2 Neinformované metody

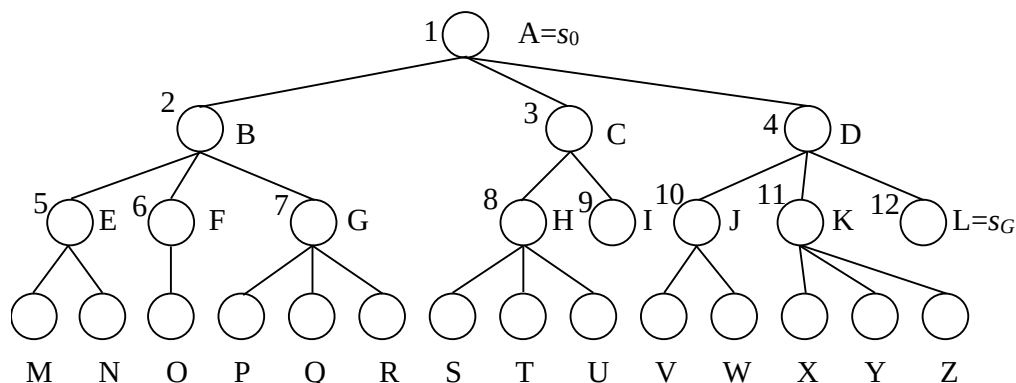
Neinformované metody nemají k dispozici žádnou informaci o cílovém stavu a nemají ani žádné prostředky, jak aktuální stavy hodnotit. Podobné metody musí někdy používat i člověk; například při hledání cesty na mapě z nějakého (výchozího) místa do jiného (cílového) místa a nemá-li vůbec žádnou představu, ve kterém směru od výchozího místa se cílové místo nachází.

2.1.2.1 Metoda slepého prohledávání do šířky (BFS)

Základní algoritmus metody BFS je následující:

1. Sestrojte frontu OPEN (bude obsahovat všechny uzly určené k expanzi) a umístěte do ní počáteční uzly.
2. Je-li fronta OPEN prázdná, pak úloha nemá řešení, a proto ukončete prohledávání jako neúspěšné. Jinak pokračujte.
3. Vyberte z čela fronty OPEN první uzly.
4. Je-li vybraný uzly uzlem cílovým, ukončete prohledávání jako úspěšné a vraťte cestu od kořenového uzlu k uzlu cílovému (vrací se posloupnost stavů, nebo operátorů). Jinak pokračujte.
5. Vybraný uzly expandujte, všechny jeho bezprostřední následníky umístěte do fronty OPEN a vraťte se na bod 2.

Na Obr. 2.2 je ukázáno pořadí výběru a expanze uzlů z fronty OPEN pro demonstrační prostor z Obr. 2.1 (čísla u jednotlivých uzlů ukazují pořadí jejich expanze).

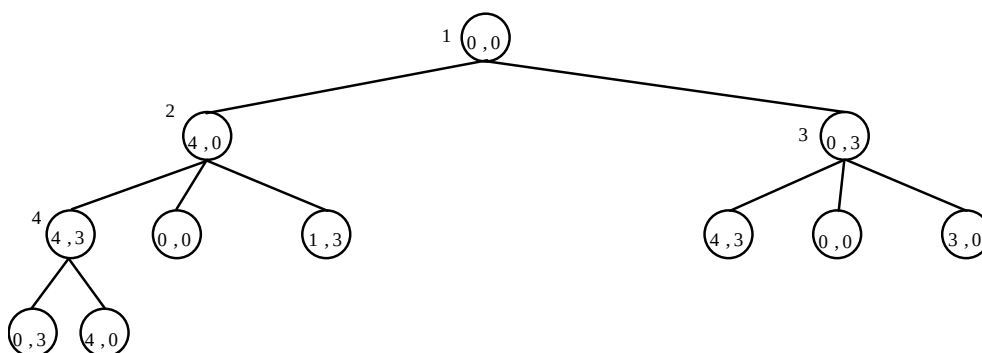


Krok	Fronta OPEN
0	[[A,-]]
1	[[B,A,-],[C,A,-],[D,A,-]]
2	[[C,A,-],[D,A,-],[E,B,A,-],[F,B,A,-],[G,B,A,-]]
3	[[D,A,-],[E,B,A,-],[F,B,A,-],[G,B,A,-],[H,C,A,-],[I,C,A,-]]
...	...
...	...
11	[[L,D,A,-],[M,E,B,A,-], ..., [T,H,C,A,-], ..., [Z,K,D,A,-]]
12	[[M,E,B,A,-], ..., [Z,K,D,A,-]]
Řešení:	[L,D,A,-] tj. $A \Rightarrow D \Rightarrow L$

Obr. 2.2 Prohledávání demonstračního stavového prostoru metodou BFS

I když je algoritmus BFS velmi jednoduchý, je velmi náročný na čas i paměť, a proto je pro řešení složitějších úloh prakticky nepoužitelný. Přesto je však považován za jeden ze základních algoritmů.

Algoritmu BFS vede ke zbytečně opakovanému generování již expandovaných uzlů. Tento problém je ukázán na Obr. 2.3 při řešení úlohy dvou džbánů popsané v kap. 2.1.1.1. Obsahy džbánů jsou 4 a 3 litry, počáteční stav úlohy je definován prázdnými džbány a cílovým stavem je naplnění některého z obou džbánů dvěma litry vody $(0,0) \rightarrow \{(0,2), (2,0)\}$. Operátory se budeme snažit aplikovat v pořadí $\rightarrow V$, $\rightarrow M$, $V \rightarrow$, $M \rightarrow$, $M \rightarrow V$, $V \rightarrow M$. Na obrázku je znázorněna situace pro první čtyři kroky řešení (čísla v uzlech označují stavy, čísla mimo uzlů udávají pořadí jejich expanzí). Tyto čtyři kroky stačí k tomu, aby nám ukázaly, že problém opakovaného generování již expandovaných uzlů je velmi významný, navíc opakovaně generované uzly mají stále více předchůdců.



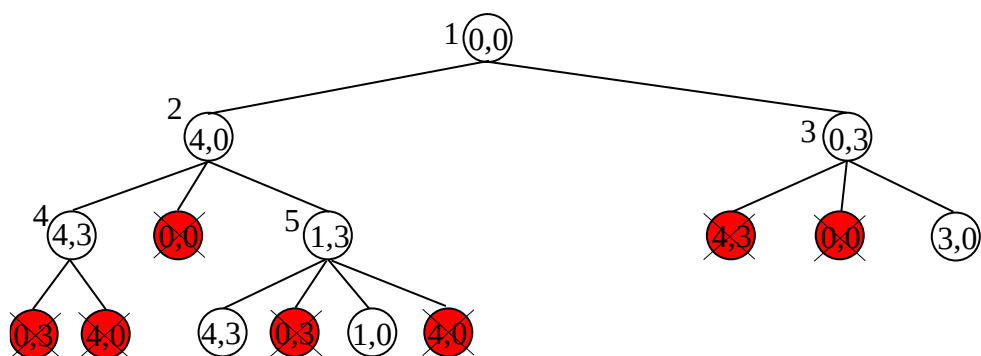
Krok	Fronta OPEN
0	<u>[(0,0),-]</u>
1	<u>[(4,0),(0,0),-]</u> , [(0,3),(0,0),-]
2	<u>[(0,3),(0,0),-]</u> , [(4,3),(4,0),(0,0),-], [(0,0),(4,0),(0,0),-], [(1,3),(4,0),(0,0),-]
3	<u>[(4,3),(4,0),(0,0),-]</u> , [(0,0),(4,0),(0,0),-], [(1,3),(4,0),(0,0),-], [(4,3),(0,3),(0,0),-], [(0,0),(0,3),(0,0),-], [(3,0),(0,3),(0,0),-]
4	<u>[(0,0),(4,0),(0,0),-]</u> , ..., [(3,0),(0,3),(0,0),-], [(0,3),(4,3),(4,0),(0,0),-], [(4,0),(4,3),(4,0),(0,0),-]
5	...

Obr. 2.3 Prohledávání stavového prostoru úlohy dvou džbánů metodou BFS

Možná úprava algoritmu BFS se týká bodu 5:

5. Vybraný uzel expandujte, do fronty OPEN umístěte všechny jeho bezprostřední následníky, kteří v ní ještě nejsou a kteří nejsou ani předky expandovaného uzlu, a vraťte se na bod 2.

Tato úprava však není příliš významná, algoritmus sice pracuje efektivněji, přesto však i dále dochází k ukládání a poté k expanzi uzlů, které již dříve byly expandovány (uzel nemusí být předkem expandovaného uzlu, ale může být předkem jiného uzlu, viz uzel (4,3) na Obr. 2.4. Na tomto obrázku jsou generované uzly nezařazené do fronty OPEN podle bodu 5 upraveného algoritmu vyplněné červeně. Porovnávání generovaných uzlů s uzly ve frontě OPEN a s uzly (předchůdci) uloženými v seznamech expandovaných uzlů jsou navíc výpočetně poměrně náročné operace a značně prodlužují doby výpočtů.



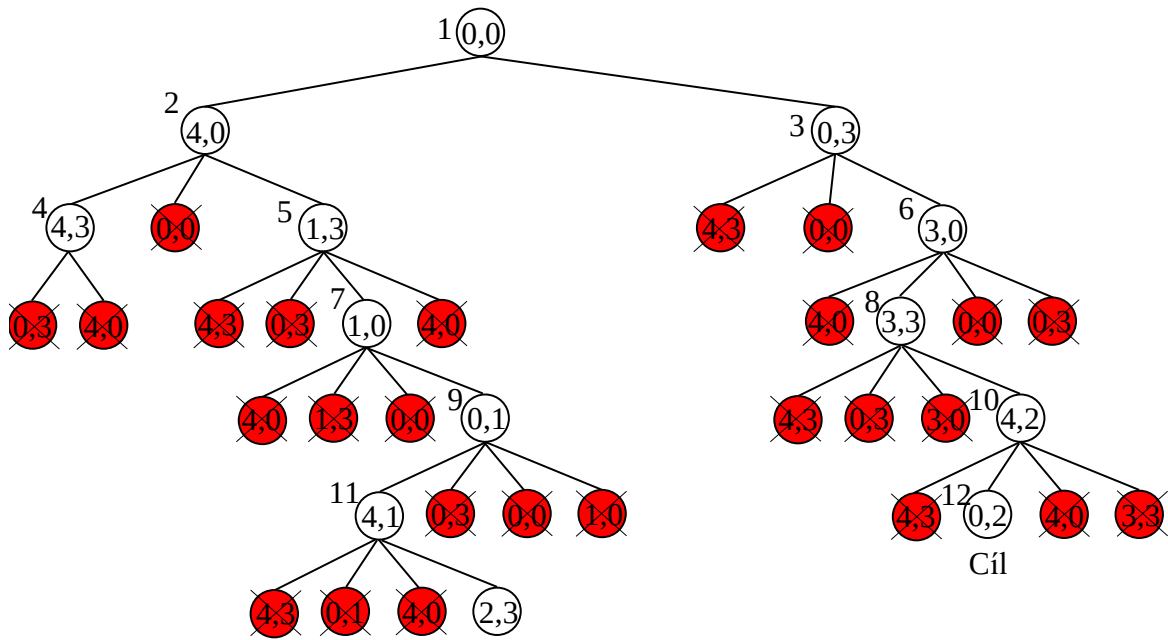
Krok	Fronta OPEN
0	[[(0,0), -]]
1	[[(4,0), (0,0), -], [(0,3), (0,0), -]]
2	[[(0,3), (0,0), -], [(4,3), (4,0), (0,0), -], [(1,3), (4,0), (0,0), -]]
3	[[(4,3), (4,0), (0,0), -], [(1,3), (4,0), (0,0), -], [(3,0), (0,3), (0,0), -]]
4	[[(1,3), (4,0), (0,0), -], [(3,0), (0,3), (0,0), -]]
5	[[(3,0), (0,3), (0,0), -], [(4,3), (1,3), (4,0), (0,0), -], [(1,0), (1,3), (4,0), (0,0), -]]
6	...

Obr. 2.4 Prohledávání stavového prostoru úlohy dvou džbánů metodou BFS s eliminací stejných stavů v OPEN a s eliminací předků generovaných uzlů

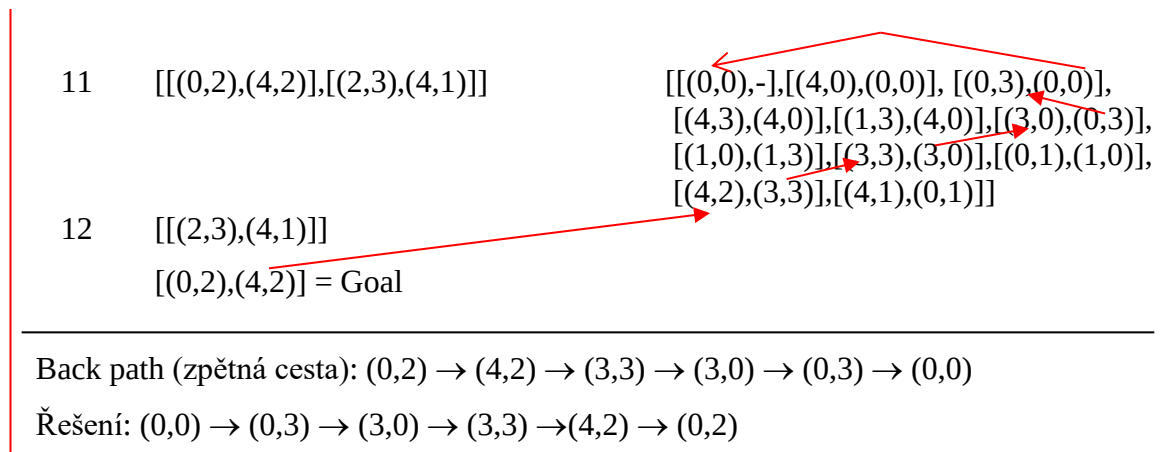
K úplnému zabránění opakování expanzí dříve expandovaných uzlů je nutné do algoritmu BFS přidat seznam pro ukládání expandovaných uzlů. Takový seznam se nazývá seznamem CLOSED (uzavřeno), a kromě původního účelu umožňuje, aby u generovaných uzlů byly místo všech předchůdců ukládány pouze jejich bezprostřední předchůdci. K rekonstrukci řešení (cesty stromem) pak totiž stačí postupný průchod uzly v seznamu CLOSED a jejich bezprostředními následníky tak, jak je naznačeno na Obr. 2.5 (čísla v uzlech označují stavy, čísla mimo udávají pořadí jejich expanzí).

Algoritmus metody BFS se seznamem CLOSED:

1. Sestrojte frontu OPEN (bude obsahovat všechny uzly určené k expanzi) a seznam CLOSED (bude obsahovat všechny již expandované uzly). Do fronty OPEN umístěte počáteční uzel.
2. Je-li fronta OPEN prázdná, pak úloha nemá řešení, a proto ukončete prohledávání jako neúspěšné. Jinak pokračujte.
3. Vyberte z čela fronty OPEN první uzel.
4. Je-li vybraný uzel uzlem cílovým, ukončete prohledávání jako úspěšné a vraťte cestu od kořenového uzlu k uzlu cílovému. Jinak pokračujte.
5. Vybraný uzel expandujte, jeho bezprostřední následníky, kteří nejsou ani ve frontě OPEN, ani v seznamu CLOSED, umístěte do fronty OPEN, expandovaný uzel umístěte do seznamu CLOSED, a vraťte se na bod 2.



Krok	OPEN	CLOSED
0	[[(0,0), -]]	[]
1	[[(4,0), (0,0)], [(0,3), (0,0)]]	[[(0,0), -]]
2	[[(0,3), (0,0)], [(4,3), (4,0)], [(1,3), (4,0)]]	[[(0,0), -], [(4,0), (0,0)]]
3	[[(4,3), (4,0)], [(1,3), (4,0)], [(3,0), (0,3)]]	[[(0,0), -], [(4,0), (0,0)], [(0,3), (0,0)]]
4	[[(1,3), (4,0)], [(3,0), (0,3)]]	[[(0,0), -], [(4,0), (0,0)], [(0,3), (0,0)], [(4,3), (4,0)]]
5	[[(3,0), (0,3)], [(1,0), (1,3)]]	[[(0,0), -], [(4,0), (0,0)], [(0,3), (0,0)], [(4,3), (4,0)], [(1,3), (4,0)]]
6	[[(1,0), (1,3)], [(3,3), (3,0)]]	[[(0,0), -], [(4,0), (0,0)], [(0,3), (0,0)], [(4,3), (4,0)], [(1,3), (4,0)], [(3,0), (0,3)]]
7	[[(3,3), (3,0)], [(0,1), (1,0)]]	[[(0,0), -], [(4,0), (0,0)], [(0,3), (0,0)], [(4,3), (4,0)], [(1,3), (4,0)], [(3,0), (0,3)], [(1,0), (1,3)]]
8	[[(0,1), (1,0)], [(4,2), (3,3)]]	[[(0,0), -], [(4,0), (0,0)], [(0,3), (0,0)], [(4,3), (4,0)], [(1,3), (4,0)], [(3,0), (0,3)], [(1,0), (1,3)], [(3,3), (3,0)]]
9	[[(4,2), (3,3)], [(4,1), (0,1)]]	[[(0,0), -], [(4,0), (0,0)], [(0,3), (0,0)], [(4,3), (4,0)], [(1,3), (4,0)], [(3,0), (0,3)], [(1,0), (1,3)], [(3,3), (3,0)], [(0,1), (1,0)]]
10	[[(4,1), (0,1)], [(0,2), (4,2)]]	[[(0,0), -], [(4,0), (0,0)], [(0,3), (0,0)], [(4,3), (4,0)], [(1,3), (4,0)], [(3,0), (0,3)], [(1,0), (1,3)], [(3,3), (3,0)], [(0,1), (1,0)], [(4,2), (3,3)]]



Obr. 2.5 Prohledávání stavového prostoru úlohy dvou džbánů metodou BFS s použitím seznamu CLOSED a rekonstrukce řešení (cesty stromem)

Pozn.: Metodu BFS lze modifikovat testováním cílového stavu již při vygenerování uzlu. Pak se body 4 a 5 algoritmu u všech variant metody sloučí v bod jediný:

4. Vybraný uzel expandujte. Pokud je některý z jeho bezprostředních následníků uzlem cílovým, ukončete prohledávání jako úspěšné a vraťte cestu od kořenového uzlu k uzlu cílovému. Jinak ... a vraťte se na bod 2.

Takto modifikovaná metoda má poněkud nižší časovou i paměťovou náročnost, není však „kompatibilní“ s dále uváděnými metodami.

Hodnocení metody BFS:

- Metoda je úplná
- Metoda je optimální.
- Časová složitost je exponenciální: $O_{\text{BFS}}(b^{d+1})$, resp. $O_{\text{BFSmodif}}(b^d)$.
- Prostorová složitost je exponenciální: $O_{\text{BFS}}(b^{d+1})$, resp. $O_{\text{BFSmodif}}(b^d)$.

Symbol b značí průměrný počet bezprostředních následníků expandovaných uzlů (tzv. faktor větvení) a symbol d hloubku stromu, ve které se nachází optimální řešení.

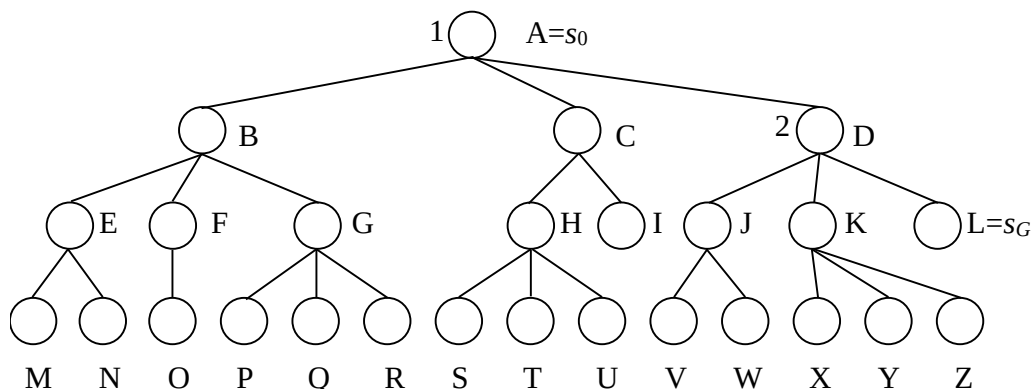
2.1.2.2 Metoda slepého prohledávání do hloubky (DFS)

Algoritmus metody DFS je prakticky stejný, jako algoritmus metody BFS, pouze místo fronty OPEN používá zásobník OPEN.

Základní algoritmus metody DFS:

1. Sestrojte zásobník OPEN (bude obsahovat všechny uzly určené k expanzi) a umístěte do něj počáteční uzel.
2. Je-li zásobník OPEN prázdný, pak úloha nemá řešení, a proto ukončete prohledávání jako neúspěšné. Jinak pokračujte.
3. Vyberte z vrcholu zásobníku OPEN první uzel.
4. Je-li vybraný uzel uzlem cílovým, ukončete prohledávání jako úspěšné a vraťte cestu od kořenového uzlu k uzlu cílovému (vrací se posloupnost stavů, nebo operátorů). Jinak pokračujte.
5. Vybraný uzel expandujte, všechny jeho bezprostřední následníky umístěte do zásobníku OPEN a vraťte se na bod 2.

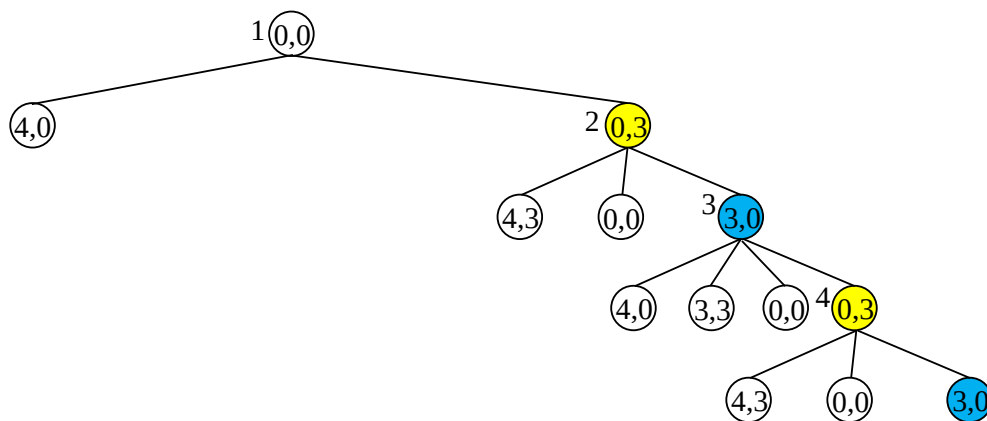
Na Obr. 2.6 je ukázáno pořadí výběru a expanze uzlů ze zásobníku OPEN pro demonstrační prostor z Obr. 2.1.



Krok	OPEN
0	[[A,-]]
1	[[D,A,-],[C,A,-],[B,A,-]]
2	[[L,D,A,-],[K,D,A,-],[J,D,A,-],[C,A,-],[B,A,-]]
3	[K,D,A,-],[J,D,A,-],[C,A,-],[B,A,-]]
Řešení: [L,D,A,-], tj. $A \Rightarrow D \Rightarrow L$	

Obr. 2.6 Prohledávání demonstračního stavového prostoru metodou DFS

Metoda DFS se na první pohled zdá být výhodnější než metoda BFS, může však zcela selhat i při řešení tak jednoduché úlohy, jakou je úloha dvou džbánů (Obr. 2.7). Algoritmus opakovaně generuje uzly (3,0) a (0,3) a řešení nikdy nenalezne.

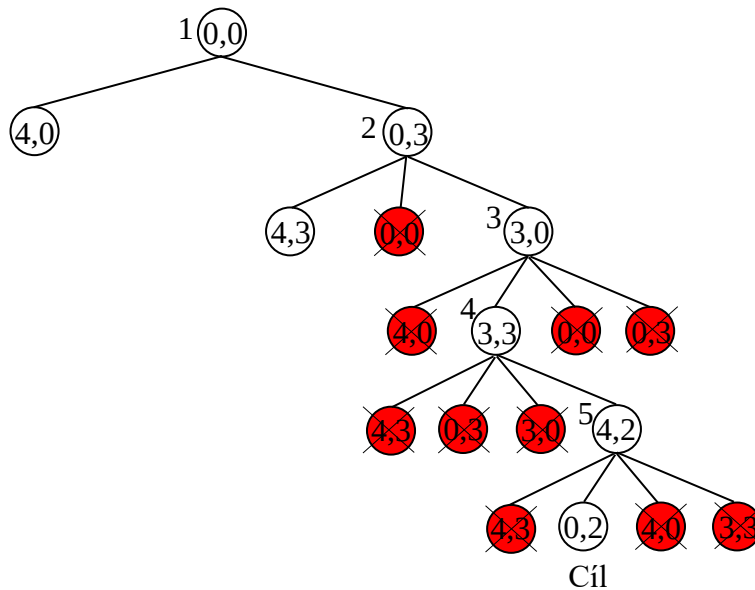


Obr. 2.7 Prohledávání stavového prostoru úlohy dvou džbánů metodou DFS

Pro praktické použití je u metody DFS nutná podobná úprava, jako u metody BFS, a opět se týká bodu 5 algoritmu:

5. Vybraný uzel expandujte, do zásobníku OPEN umístěte všechny jeho bezprostřední následníky, kteří v něm ještě nejsou a kteří nejsou ani předky generovaného uzlu, a vraťte se na bod 2.

Na Obr. 2.8 je ukázáno řešení úlohy dvou džbánů modifikovanou metodou DFS (tj. metodou s výše uvedenou úpravou bodu 5 algoritmu). Čísla v uzlech opět označují stavy a čísla mimo udávají pořadí jejich expanzí.



Krok	OPEN
0	[[(0,0), -]]
1	[[(0,3), (0,0), -], [(4,0), (0,0), -]]
2	[[(3,0), (0,3), (0,0), -], [(4,3), (0,3), (0,0), -], [(4,0), (0,0), -]]
3	[[(3,3), (3,0), (0,3), (0,0), -], [(4,3), (0,3), (0,0), -], [(4,0), (0,0), -]]
4	[[(4,2), (3,3), (3,0), (0,3), (0,0), -], [(4,3), (0,3), (0,0), -], [(4,0), (0,0), -]]
5	[[(0,2), (4,2), (3,3), (3,0), (0,3), (0,0), -], [(4,3), (0,3), (0,0), -], [(4,0), (0,0), -]]
6	[[(4,3), (0,3), (0,0), -], [(4,0), (0,0), -]]
Řešení:	[(0,2), (4,2), (3,3), (3,0), (0,3), (0,0), -]

Obr. 2.8 Prohledávání stavového prostoru úlohy dvou džbánů modifikovanou metodou DFS (s eliminací předchůdců a uzlů, které jsou v zásobníku OPEN).

Pozn.: Ověřte si, že záměna prvních dvou operátorů $\{\rightarrow M, \rightarrow V, V \rightarrow, M \rightarrow, M \rightarrow V, V \rightarrow M\}$ vede k řešení, které není optimální (je v hloubce 7)!

Hodnocení metody DFS

- Základní metoda není úplná, modifikovaná metoda s eliminací předchůdců a uzlů v zásobníku je úplná.
- Metoda není optimální.
- Časová složitost: $O_{DFS}(\infty)$, resp. $O_{DFS_{modif}}(b^m)$.
- Prostorová složitost: $O_{DFS}(\infty)$, resp. $O_{DFS_{modif}}(m)$.

Symbol b opět značí faktor větvení a symbol m maximální počet možných různých stavů úlohy (velikost stavového prostoru).

Poznamenejme, že použití seznamu CLOSED pro metodu DFS je teoreticky také možné, ale ztratila by se tím jediná výhoda metody DFS (lineární prostorová složitost, která by pak byla exponenciální), a proto se taková modifikace metody DFS nikdy nepoužívá.

2.1.2.3 Metoda omezeného prohledávání do hloubky (DLS)

Pokud řešíme úlohu, u které dokážeme odhadnout hloubku řešení, můžeme problém s opakujícím se generováním stejných stavů vyřešit omezením hloubky prohledávání (například výše uváděná úloha dvou džbánů má celkem 20 různých stavů, a proto řešení musí ležet v nejhorším případě v hloubce 19).

Algoritmus DLS:

1. Sestrojte zásobník OPEN a umístěte do něj počáteční uzel s označením hloubky (0).
2. Je-li zásobník OPEN prázdný, pak úloha nemá řešení, a proto ukončete prohledávání jako neúspěšné. Jinak pokračujte.
3. Vyberte z vrcholu zásobníku OPEN první uzel.
4. Je-li vybraný uzel uzlem cílovým, ukončete prohledávání jako úspěšné a vraťte cestu od kořenového uzlu k uzlu cílovému (vrací se posloupnost stavů nebo operátorů). Jinak pokračujte.
5. Pokud je hloubka vybraného uzlu menší než zadaná maximální hloubka, tak tento uzel expandujte, všem jeho bezprostředním následníkům přiřaďte hloubku o jedničku větší než je hloubka expandovaného uzlu, a umístěte je do zásobníku OPEN. Jinak pokračujte.
6. Vraťte se na bod 2.

Hodnocení metody DLS

- Metoda není úplná.
- Metoda není optimální.
- Časová složitost: $O_{DLS}(b^l)$.
- Prostorová složitost: $O_{DLS}(b \cdot l)$.

Symbol b značí faktor větvení a symbolem l je označena hloubka prohledávaného stromu.

2.1.2.4 Metoda postupného zanořování do hloubky (IDS)

Nelze-li stanovit hloubku řešení a nemáme-li k dispozici dostatek paměti pro použití metody BFS, můžeme se pokusit vyřešit tento problém použitím metody postupného zanořování do hloubky. Princip této metody spočívá v opakovaném použití metody DLS s postupným zvyšováním hloubky prohledávání.

Algoritmus IDS:

1. Nastavte aktuální maximální hloubku prohledávání na hodnotu 1.
2. Zavolejte proceduru DLS s omezením na aktuální hloubku.
3. Skončí-li procedura DLS úspěchem, ukončete prohledávání jako úspěšné a vraťte cestu nalezenou procedurou DLS.
4. Skončí-li procedura DLS neúspěchem, pak
 - a) pokud v proceduře DLS nebyl alespoň jeden uzel expandován z důvodu dosažení maximální hloubky (bod 5. procedury DLS), inkrementujte aktuální maximální hloubku prohledávání a vraťte se na bod 2.

b) jinak ukončete prohledávání jako neúspěšné (úloha nemá řešení).

Hodnocení metody IDS

- Metoda je úplná.
- Metoda je optimální.
- Časová složitost: $O_{IDS}(b^d)$.
- Prostorová složitost: $O_{IDS}(b \cdot d)$.

Symbol b značí faktor větvení a symbol d hloubku optimálního řešení.

Metoda postupného zanořování do hloubky se zdá být na první pohled velmi neefektivní, protože při každém dalším zanoření opakuje prohledávání, které již provedeno při předcházející hloubce. Časová složitost je podobná složitosti metody BFS, ale prostorová složitost metody IDS je lineární, a tedy nesrovnatelně menší.

Porovnání metody IDS s metodou BFS:

Pro počty generovaných uzlů zřejmě platí:

$N_{BFS} = b^1 + b^2 + b^3 + \dots + (b^{d+1} - b)$ optimální uzel může být i posledním uzlem v hloubce d

$N_{BFS_{modif}} = b^1 + b^2 + b^3 + \dots + b^d$ optimální uzel může být libovolným uzlem v hloubce d

$N_{IDS} = d \cdot b + (d-1) \cdot b^2 + (d-2) \cdot b^3 + \dots + 1 \cdot b^d$ kořenový uzel se expanduje d krát, uzly v hloubce 1 se expandují $(d-1)$ krát, až konečně uzly v hloubce d se expandují pouze jednou

Uvažujeme-li například úlohu s faktorem větvení $b = 10$, s potřebnou pamětí 100 B/stav a počítač, který generuje 10000 stavů (uzlů)/s, pak například pro hodnotu $d = 10$ dostáváme:

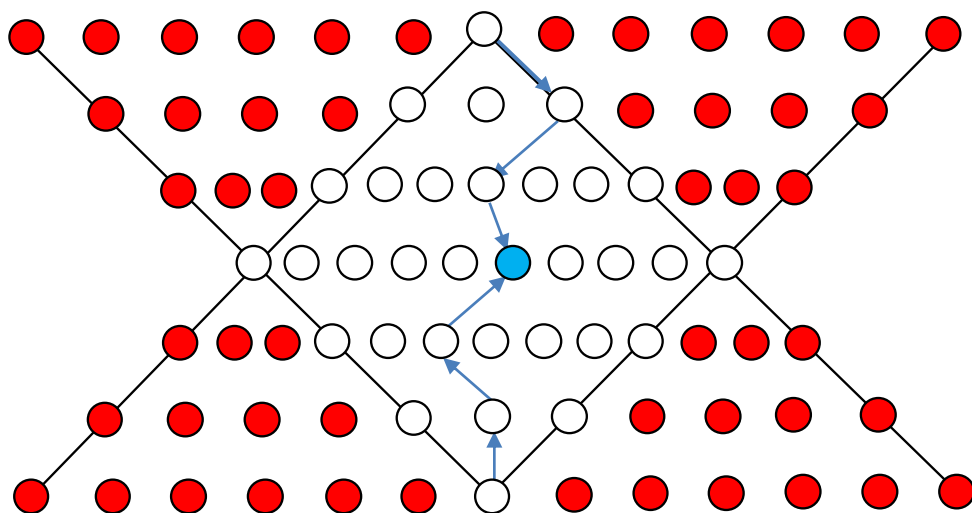
	BFS/BFS _{modif}	IDS
Počet generovaných stavů	$10^{11}/10^{10}$	10^{10}
Potřebný čas	128/13 dnů	14.3 dnů
Potřebná paměť	11.1/1.1 TB	10 kB

2.1.2.5 Metoda obousměrného prohledávání (BS)

U úloh s vratnými operátory je možné prohledávat stavový prostor oběma směry, tj. buď od počátečního stavu k cílovému stavu, nebo od cílového stavu k počátečnímu stavu. Metoda obousměrného prohledávání je založena na metodě BFS, přičemž stavový prostor prohledává střídavě oběma směry. Používá proto dvě fronty OPEN a dva seznamy CLOSED. Po každém kroku řešení se porovnávají obsahy front OPEN a hledá se stejný stav, tzv. most. Na Obr. 2.9 je most označen modrou barvou. Cesta od cílového stavu k mostu se pomocí vratných operátorů převede na cestu z mostu k uzlu cílovému a připojí se k cestě od počátečního uzlu k mostu. Tím se získá celá cesta od počátečního k cílovému stavu, tj. řešení úlohy.

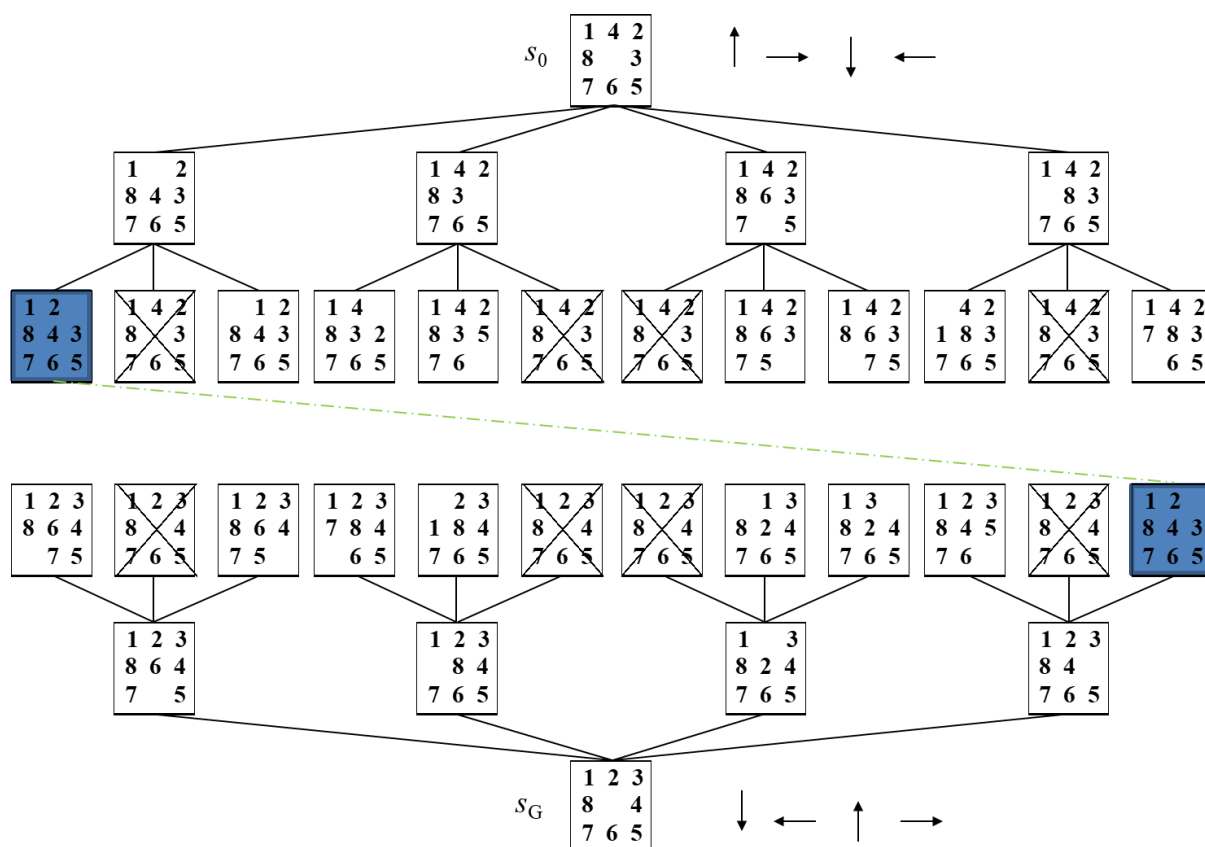
V každém směru se tak prohledává poloviční hloubka, a proto klesne paměťová i časová složitost generování uzlů z $O(b^d)$ na $O(2b^{d/2}) \sim O(b^{d/2})$. Například pro $b = 10$ a $d = 6$ se počet generovaných uzlů sníží z 11111100 na 2220.

Metoda BS je úplná a optimální.



Obr. 2.9 Princip prohledávání stavového prostoru metodou obousměrného prohledávání

Praktické použití metody BS je ukázán na řešení úlohy Loydovy osmičky (Obr. 2.10)



Obr. 2.10 Prohledávání stavového prostoru Loydovy osmičky metodou BS

Řešením úlohy je následující posloupnost operátorů (posunů prázdného políčka): $\uparrow, \rightarrow, \downarrow, \leftarrow$.

2.1.2.6 Metoda stejných cen (UCS)

Metoda UCS je prakticky stejná jako metoda BFS, ale uvažuje skutečné ceny cest, které jsou dány součtem cen příslušných přechodů. Místo fronty OPEN používá seznam OPEN a pro expanzi vybírá ze seznamu OPEN uzel s nejmenším ohodnocením, tj. uzel s nejnižší cenou cesty. Metoda UCS má stejné možnosti úprav jako metoda BFS, dále si ukážeme pouze metodu UCS doplněnou seznamem CLOSED.

Algoritmus metody UCS se seznamem CLOSED:

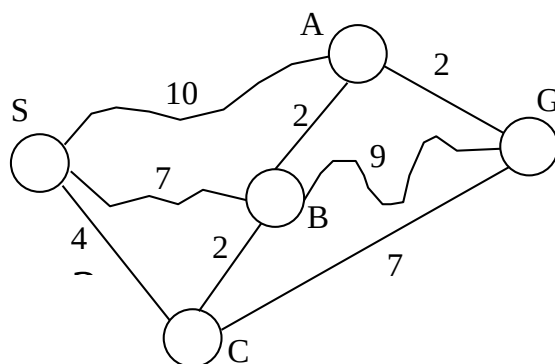
1. Sestrojte dva seznamy, OPEN (bude obsahovat uzly určené k expanzi) a CLOSED (bude obsahovat expandované uzly). Do seznamu OPEN umístěte počáteční uzel včetně jeho (nulového) ohodnocení.
2. Je-li seznam OPEN prázdný, pak úloha nemá řešení, a proto ukončete prohledávání jako neúspěšné. Jinak pokračujte.
3. Vyberte ze seznamu OPEN uzel s nejnižším ohodnocením.
4. Je-li vybraný uzel uzlem cílovým, ukončete prohledávání jako úspěšné a vraťte cestu od počátečního uzlu k uzlu cílovému. Jinak pokračujte.
5. Vybraný uzel expandujte a jeho bezprostřední následníky, kteří nejsou v seznamu CLOSED, umístěte do seznamu OPEN včetně jejich ohodnocení. Expandovaný uzel pak umístěte do seznamu CLOSED. Z uzlů, které se v seznamu OPEN vyskytují vícekrát, ponechte pouze uzel s nejlepším ohodnocením, ostatní ze seznamu OPEN vyškrtněte a vraťte se na bod 2.

Pro hodnocení algoritmu UCS platí stejné závěry, jako pro algoritmus BFS. Algoritmus UCS je úplný a optimální, jeho časová i paměťová náročnost jsou exponenciální a jsou dány výrazy $O(b^k)$, kde b je faktor větvení a k je koeficient získaný podílem ceny optimálního řešení C^* a nejmenšího přírůstku ceny mezi dvěma uzly ΔC_{min}

$$k = C^* / \Delta C_{min}$$

Poznamenejme, že metoda UCS není vhodná pro úlohy, kdy cesta z výchozího do cílového uzlu prochází pouze několika málo jinými uzly s vysokými cenami přechodů, protože pak se prohledává zbytečně mnoho cest s nízkými cenami (například při hledání cesty z Brna do Plzně přes jediný uzel (Prahu) dojde k prohledávání cest Brno-Česká, Brno-Jinačovice, Brno-Ostopovice, ..., Česká-Lelekovice, Česká-Kuřim atd.).

Použití metody UCS je ukázáno na úloze hledání nejkratší cesty (Obr. 2.11). Cílem úlohy je nalézt nejkratší cestu z místa startu S do cílového místa G. Čísla na jednotlivých cestách označují vzdálenosti mezi příslušnými dvěma místy.



Krok	OPEN	CLOSED
0	[(S,-),0]	[]
1	[(A,S),10],[B,S),7],[C,S),4]	[(S,-),0]
2	[(A,S),10],[B,S),7],[B,C),6], [(G,C),11]]	[(S,-),0],[C,S),4]
3	[(A,S),10],[G,C),11],[A,B),8], [(G,B),15]]	[(S,-),0],[C,S),4],[B,C),6]
4	[(G,C),11],[G,A),10]	[(S,-),0],[C,S),4],[B,C),6], [(A,B),8]]
5	[] [(G,A),10] = Goal	

Zpětná cesta: $G \rightarrow A \rightarrow B \rightarrow C \rightarrow S$
 Řešení (optimální cesta): $S \rightarrow C \rightarrow B \rightarrow A \rightarrow G$, vzdálenost = 10

Obr. 2.11 Prohledávání stavového prostoru úlohy nejkratší cesty metodou UCS.

2.1.2.7 Metoda zpětného navracení (Backtracking)

Metoda zpětného navracení je speciální metodou prohledávání do hloubky, kdy místo expanze vybraného uzlu, tj. generování všech jeho bezprostředních následníků, se generuje pouze následník jeden a teprve v případě neúspěchu se generuje následník další atd. Pokud uzel není na cestě k řešení (tj. všichni jeho bezprostřední následníci „ohlásili“ neúspěch), vrací uzel neúspěch svému bezprostřednímu předchůdci a popsaná situace se s postupnou generací následníků opakuje v tomto uzlu.

Algoritmus Backtracking:

1. Sestrojte zásobník OPEN (bude obsahovat uzly určené k expanzi) a umístěte do něj počáteční uzel.
2. Je-li zásobník OPEN prázdný, pak úloha nemá řešení, a ukončete proto prohledávání jako neúspěšné. Jinak pokračujte.
3. Jde-li na uzel na vršku zásobníku aplikovat první/další operátor, tak tento operátor aplikujte a pokračujte bodem 4, v opačném případě odstraňte testovaný uzel z vrcholu zásobníku a vraťte se na bod 2.
4. Je-li právě vygenerovaný uzel uzlem cílovým, ukončete prohledávání jako úspěšné a vraťte cestu od počátečního uzlu k uzlu cílovému (vrací se posloupnost stavů nebo operátorů). Jinak uložte nový uzel na vršek zásobníku.
5. Vraťte se na bod 2.

Metoda má extrémně nízkou paměťovou náročnost, protože v paměti, tj. v zásobníku OPEN, jsou uloženy pouze uzly aktuální cesty. Každý uzel si ukládá informaci o svém bezprostředním předchůdci, nebo o použitém operátoru. Pro časovou náročnost, úplnost a optimálnost platí stejné závěry jako pro metodu DFS: časová náročnost je exponenciální, metoda není ani optimální, ani úplná.

Modifikace metody spočívá v úpravě bodu 4. Nový uzel se do zásobníku OPEN neukládá, pokud se již v tomto zásobníku nachází, tj. pokud nový uzel je již předchůdcem uzlu na vršku zásobníku.

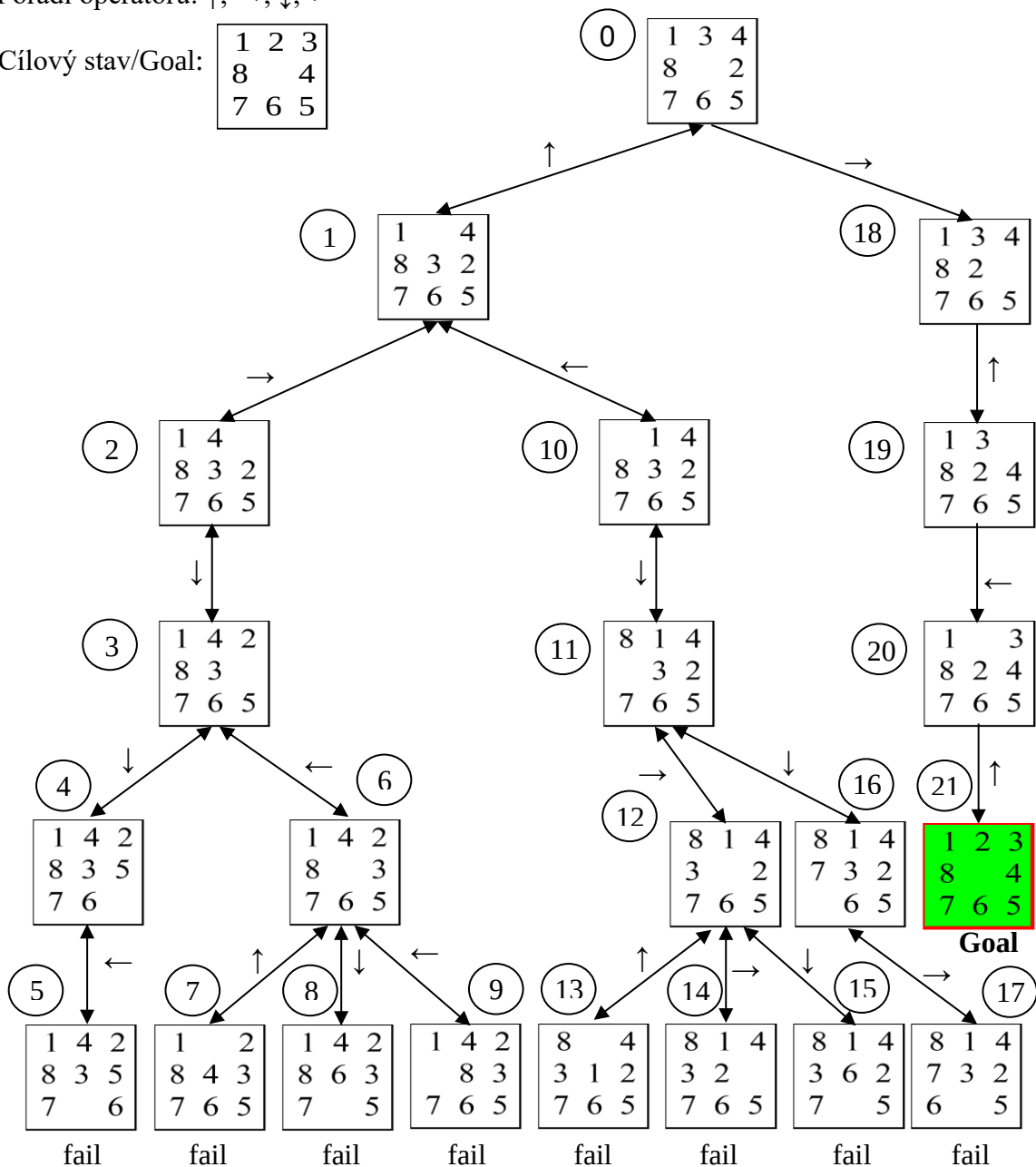
Metodu zpětného navracení lze dále upravit tak, že se podobně jako u metody DLS zadá maximální hloubka prohledávání (pokud uzel leží v maximální hloubce, pak v bodu 3 nelze aplikovat žádný operátor) a následně tím, že podobně jako u metody IDS se tato hloubka postupně zvyšuje. Pro hodnocení takto upravené metody pak platí všechny závěry uvedené u metod DLS a IDS.

Následující příklad (Obr. 2.12) uvažuje modifikaci metody jak s úpravou bodu 4 algoritmu, tak i s omezenou hloubkou prohledávání. Jde o řešení úlohy Loydovy osmičky s operátory pro posun prázdného políčka v pořadí $\uparrow$, $\rightarrow$, $\downarrow$, $\leftarrow$ a s omezením hloubky prohledávání na hodnotu 5. Čísla v kroužcích označují stavy. V zásobníku jsou pak stavy označené příslušnými čísly a každý stav je doplněn posledním použitým operátorem. Goal znamená cíl, fail značí neúspěch.

Pořadí operátorů: $\uparrow$, $\rightarrow$, $\downarrow$, $\leftarrow$

Cílový stav/Goal:

1	2	3
8		4
7	6	5



Krok	OPEN	Poznámka
0	[(0,-)]	Počáteční stav, (další) operátor možný
1	[(1,-),(0,↑)]	Nový uzel, (další) operátor možný
2	[(2,-),(1,→),(0,↑)]	Nový uzel, (další) operátor možný
3	[(3,-),(2,↓),(1,→),(0,↑)]	Nový uzel, (další) operátor možný
4	[(4,-),(3,↓),(2,↓),(1,→),(0,↑)]	Nový uzel, (další) operátor možný
5	[(5,-),(4,←),(3,↓),(2,↓),(1,→),(0,↑)]	Uzel 5 není cílový a je v max. hloubce
6	[(4,-),(3,↓),(2,↓),(1,→),(0,↑)]	Návrat k uzlu 4, další operátor není
7	[(3,↓),(2,↓),(1,→),(0,↑)]	Návrat k uzlu 3, další operátor možný
8	[(6,-),(3,←),(2,↓),(1,→),(0,↑)]	Nový uzel, (další) operátor možný
9	[(7,-),(6,↑),(3,←),(2,↓),(1,→),(0,↑)]	Uzel 7 není cílový a je v max. hloubce
10	[(6,↑),(3,←),(2,↓),(1,→),(0,↑)]	Návrat k uzlu 6, další operátor možný
11	[(8,-),(6,↓),(3,←),(2,↓),(1,→),(0,↑)]	Uzel 8 není cílový a je v max. hloubce
12	[(6,↓),(3,←),(2,↓),(1,→),(0,↑)]	Návrat k uzlu 6, další operátor možný
13	[(9,-),(6,←),(3,←),(2,↓),(1,→),(0,↑)]	Uzel 9 není cílový a je v max. hloubce
14	[(6,←),(3,←),(2,↓),(1,→),(0,↑)]	Návrat k uzlu 6, další operátor není
15	[(3,←),(2,↓),(1,→),(0,↑)]	Návrat k uzlu 3, další operátor není
16	[(2,↓),(1,→),(0,↑)]	Návrat k uzlu 2, další operátor není
17	[(1,→),(0,↑)]	Návrat k uzlu 1, (další) operátor možný
18	[(10,-),(1,←),(0,↑)]	Nový uzel, (další) operátor možný
19	[(11,-),(10,↓),(1,←),(0,↑)]	Nový uzel, (další) operátor možný
20	[(12,-),(11,→),(10,↓),(1,←),(0,↑)]	Nový uzel, (další) operátor možný
21	[(13,-),(12,↑),(11,→),(10,↓),(1,←),(0,↑)]	Uzel 13 není cílový a je v max. hloubce
22	[(12,↑),(11,→),(10,↓),(1,←),(0,↑)]	Návrat k uzlu 12, další operátor možný
23	[(14,-),(12,→),(11,→),(10,↓),(1,←),(0,↑)]	Uzel 14 není cílový a je v max. hloubce
24	[(11,→),(7,→),(4,↓),(1,←),(0,↑)]	Návrat k uzlu 12, (další) operátor možný
25	[(15,-),(12,↓),(11,→),(10,↓),(1,←),(0,↑)]	Uzel 15 není cílový a je v max. hloubce
26	[(12,↓),(11,→),(10,↓),(1,←),(0,↑)]	Návrat k uzlu 12, další operátor není
27	[(11,→),(10,↓),(1,←),(0,↑)]	Návrat k uzlu 11, další operátor možný
28	[(16,-),(11,↓),(10,↓),(1,←),(0,↑)]	Nový uzel, (další) operátor možný
29	[(17,-),(16,→),(11,↓),(10,↓),(1,←),(0,↑)]	Uzel 17 není cílový a je v max. hloubce
30	[(16,→),(11,↓),(10,↓),(1,←),(0,↑)]	Návrat k uzlu 16, další operátor není
31	[(11,↓),(10,↓),(1,←),(0,↑)]	Návrat k uzlu 11, další operátor není
32	[(10,↓),(1,←),(0,↑)]	Návrat k uzlu 10, další operátor není
33	[(1,←),(0,↑)]	Návrat k uzlu 1, další operátor není
34	[(0,↑)]	Návrat k uzlu 0, (další) operátor možný
35	[(18,-),(0,→)]	Nový uzel, (další) operátor možný
36	[(19,-),(18,↑),(0,→)]	Nový uzel, (další) operátor možný
37	[(20,-),(19,←),(18,↑),(0,→)]	Nový uzel, (další) operátor možný
38	[(21,-),(20,↑),(19,←),(18,↑),(0,→)]	Cíl/Goal, řešení je přímo v zásobníku!

Obr. 2.12 Prohledávání stavového prostoru úlohy Loydovy osmičky modifikovanou metodou zpětného navracení s omezením hloubky prohledávání

2.1.3 Informované metody

Informované metody využívají dostupné informace o cílovém stavu. Jejich společný algoritmus (algoritmus BestFS) je následující:

1. Sestrojte seznam OPEN (bude obsahovat všechny uzly určené k expanzi) a vložte do něj počáteční uzel včetně jeho ohodnocení.
2. Je-li seznam OPEN prázdný, pak úloha nemá řešení, a ukončete proto prohledávání jako neúspěšné. Jinak pokračujte.
3. Vyberte ze seznamu OPEN uzel s nejlepším (nejnižším) ohodnocením.
4. Je-li vybraný uzel uzlem cílovým, ukončete prohledávání jako úspěšné a vraťte cestu od počátečního uzlu k uzlu cílovému. Jinak pokračujte.
5. Vybraný uzel expandujte. Všechny jeho bezprostřední následníky, kteří nejsou jeho předky, umístěte do seznamu OPEN včetně jejich ohodnocení. Z uzlů, které se v seznamu OPEN vyskytují vícekrát, ponechte pouze uzel s nejlepším ohodnocením, ostatní ze seznamu OPEN vyškrtněte a vraťte se na bod 2.

resp. se seznamem CLOSED:

1. Sestrojte dva seznamy, OPEN (bude obsahovat uzly určené k expanzi) a CLOSED (bude obsahovat expandované uzly). Do seznamu OPEN umístěte počáteční uzel včetně jeho ohodnocení.
2. Je-li seznam OPEN prázdný, pak úloha nemá řešení, a proto ukončete prohledávání jako neúspěšné. Jinak pokračujte.
3. Vyberte ze seznamu OPEN uzel s nejnižším ohodnocením.
4. Je-li vybraný uzel uzlem cílovým, ukončete prohledávání jako úspěšné, a vraťte cestu od počátečního uzlu k uzlu cílovému. Jinak pokračujte.
5. Vybraný uzel expandujte a jeho bezprostřední následníky, kteří nejsou v seznamu CLOSED, umístěte do seznamu OPEN včetně jejich ohodnocení. Expandovaný uzel pak umístěte do seznamu CLOSED. Z uzlů, které se v seznamu OPEN vyskytují vícekrát, ponechte pouze uzel s nejlepším ohodnocením, ostatní ze seznamu OPEN vyškrtněte a vraťte se na bod 2.

Metoda vybírá k expanzi vždy nejlépe ohodnocený uzel, a proto se nazývá Best First Search (BestFS). Ohodnocení $f(s_k)$ každého uzlu s_k (Obr. 2.13) zahrnuje cenu dosavadní cesty do tohoto uzlu z počátečního uzlu $g(s_k)$ i předpokládanou cenu cesty z tohoto uzlu do cílového uzlu $h(s_k)$

$$f(s_k) = g(s_k) + h(s_k)$$

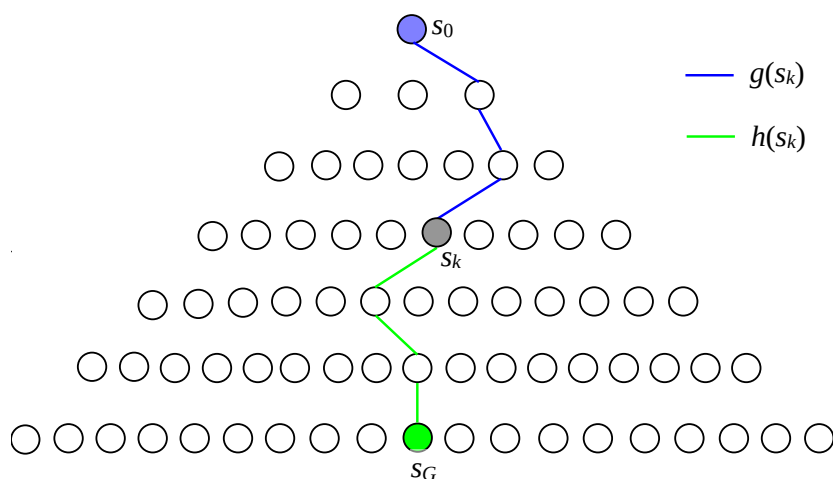
Funkce $h(s_k)$ se nazývá heuristickou funkcí, nebo krátce heuristikou. Čím přesnější je tato funkce, tím méně stavového prostoru je nutné prohledávat. V extrémním případě, pokud je heuristika přesným odhadem, tj. pokud známe řešení, tak samozřejmě nemusíme prohledávat žádný stavový prostor.

Je zřejmé, že cena cesty v počátečním uzlu s_0 je nulová ($g(s_0) = 0$) a že nulová musí být i hodnota heuristické funkce v cílovém uzlu s_G ($h(s_G) = 0$).

Zdůrazněme, že ceny přechodů ze stavů s_k do stavů s_{k+1} ($k = 0, 1, 2, \dots$) musí být kladné, tj. že musí platit vztah $g(s_{k+1}) > g(s_k)$.

Metodu UCS lze považovat za extrémní metodu BestFS, pro kterou jsou však hodnoty heuristické funkce všech uzlů nulové ($h(s_k) = 0$), a proto metoda UCS nepatří mezi informované metody.

Druhou extrémní metodou je metoda Greedy search, která naopak při hodnocení uzlů ignoruje ceny dosavadních cest ($g(s_k) = 0$).

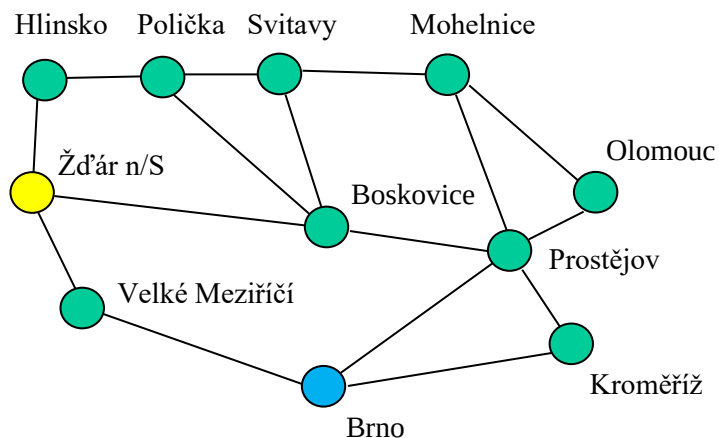


Obr. 2.13 Ohodnocení uzlu

2.1.3.1 Metoda hltavého prohledávání (GS)

Metoda GS (Greedy Search) ohodnocuje uzly pouze heuristickou funkcí, tj. odhadovanou cenou z daného uzlu do uzlu cílového. Metoda je úplná, ale není optimální. Časová i prostorová složitost metody je stejná ($O_{greedy}(b^d)$), dobrá heuristika může skutečnou složitost výrazně zmenšit!

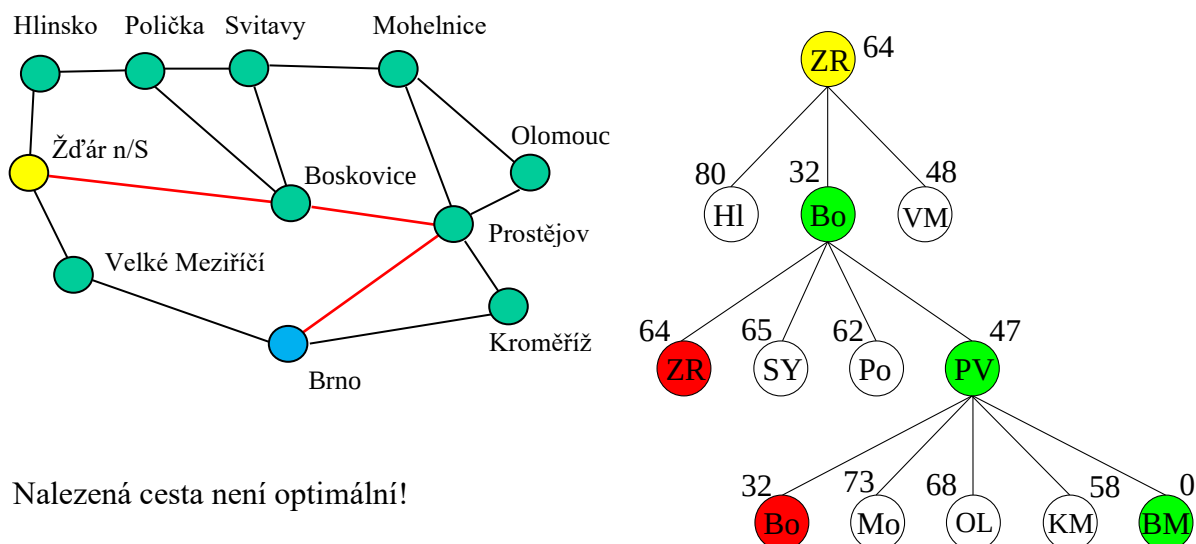
Princip metody Greedy search ukážeme na úloze hledání cesty (Obr. 2.14) a použijeme k tomu algoritmus BestFS. Je nutné upozornit, že ani algoritmus BestFS nezaručuje nalezení optimální cesty (Obr. 2.15)!



Přímá vzdálenost [km] z:	do BM
Žďár nad Sázavou (ZR)	64
Hlinsko (Hl)	80
Velké Meziříčí (VM)	48
Polička (Po)	62
Boskovice (Bo)	32
Svitavy (SY)	65
Mohelnice (Mo)	73
Prostějov (PV)	47
Kroměříž (KM)	58
Olomouc (OL)	68

Obr. 2.14 Příklad úlohy hledání cesty

Nejpoužívanější heuristikou pro podobné úlohy je přímá vzdálenost mezi místy, a to i v případech, kdy mezi těmito místy přímé spojení není (například z Hlinska do Brna).



Krok	OPEN
0	[(ZR,-),64]
1	[[(HL,ZR,-),80], [(BO,ZR,-),32] ,[(VM,ZR,-),48]]
2	[[(HL,ZR,-),80],[(VM,ZR,-),48],[(SY,BO,ZR,-),65], [(PO,BO,ZR,-),62], [(PV,Bo,ZR,-),47]]
3	[[(HL,ZR,-),80],[(VM,ZR,-),48],[(SY,BO,ZR,-),65], [(PO,BO,ZR,-),62],[(MO,PV,BO,ZR,-),73], [(BM,PV,BO,ZR,-),0] , [(KM,PV,BO,ZR,-),58],[(OL,PV,BO,ZR,-),68]]
Řešení:	[(BM,PV,Bo,ZR,-),0]

Obr. 2.15 Prohledávání stavového prostoru úlohy nalezení cesty metodou Greedy search

2.1.3.2 Metoda A*

Metoda A* je nejznámější a nejpoužívanější metodou pro řešení úloh prohledáváním stavového prostoru. Jde opět o metodu BestFS, ve které heuristická funkce $h(s_k)$ musí být spodním odhadem skutečné ceny cesty od ohodnocovaného uzlu s_k k cíli (odhad musí být optimistický, tj. menší než skutečná cena) – taková heuristika se pak nazývá přípustnou heuristikou, resp. přípustnou heuristickou funkcí.

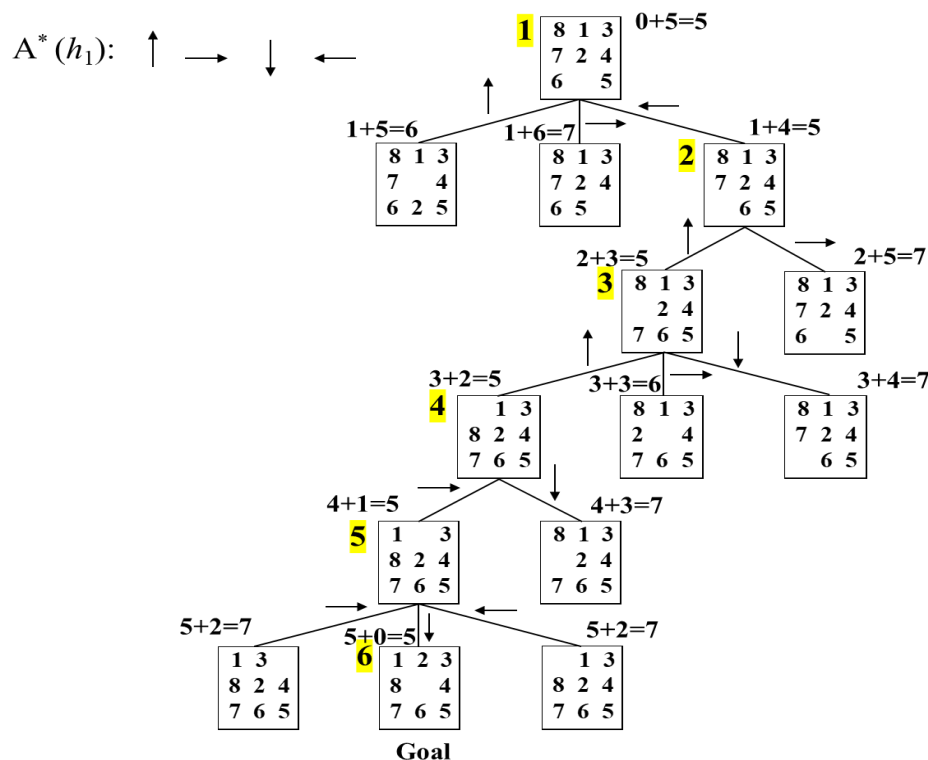
Metoda A* je úplná a optimální. Důkaz optimálnosti je poměrně jednoduchý:

1. Označme cenu optimální cesty do optimálního cílového uzlu G_{opt} jako $f_{opt}(s_{G_{opt}})$ a cenu jiné (neoptimální) cesty do stejného nebo jiného cílového uzlu jako $f(s_G)$. Zřejmě musí platit $f_{opt}(s_{G_{opt}}) < f(s_G)$.
2. Necht' s_k je uzel na optimální cestě k optimálnímu cíli. Pro přípustnou heuristiku musí platit $f_{opt}(s_{G_{opt}}) \geq f(s_k)$.
3. Pokud by uzel s_k nebyl vybrán z OPEN, zatímco cílový uzel s_G s ohodnocením $f(s_G)$ ano, muselo by platit $f(s_k) \geq f(s_G)$.
4. Z bodů 2 a 3 vyplývá, že $f_{opt}(s_{G_{opt}}) \geq f(s_k) \geq f(s_G)$, tedy že $f_{opt}(s_{G_{opt}}) \geq f(s_G)$, což je ve sporu se závěrem bodu 1.
5. Uzel s_k proto musí být vybrán k expanzi, stejně jako každý jiný uzel na optimální cestě k optimálnímu cíli, a metoda proto musí nalézt optimální cestu.

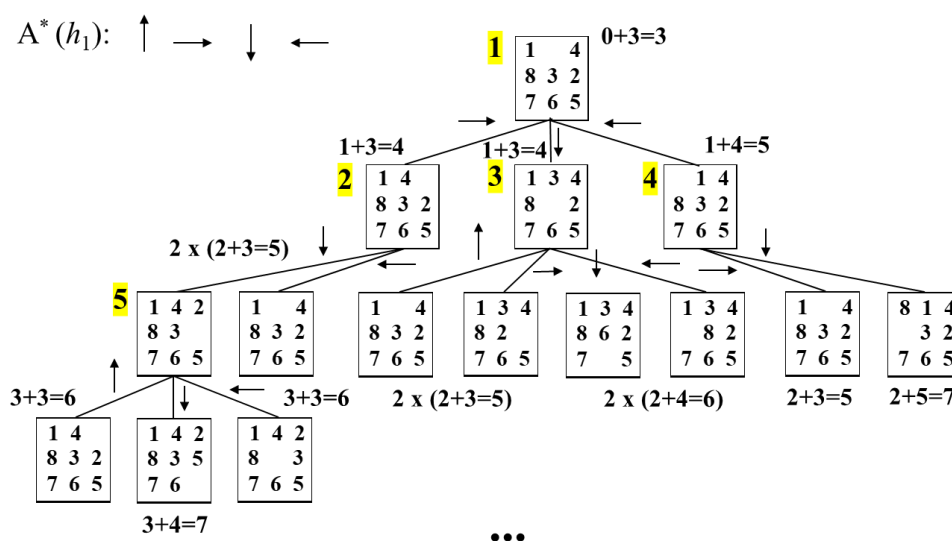
Metodu A* použijeme pro řešení úlohy Loydovy osmičky a pro následující dvě heuristiky, které jsou jistě spodním odhadem skutečné ceny:

- počet kamenů na nesprávných pozicích (heuristiku označíme jako h_1)
- součet vzdáleností kamenů (použijeme Manhattanovu vzdálenost) od jejich cílových pozic (heuristiku označíme jako h_2)

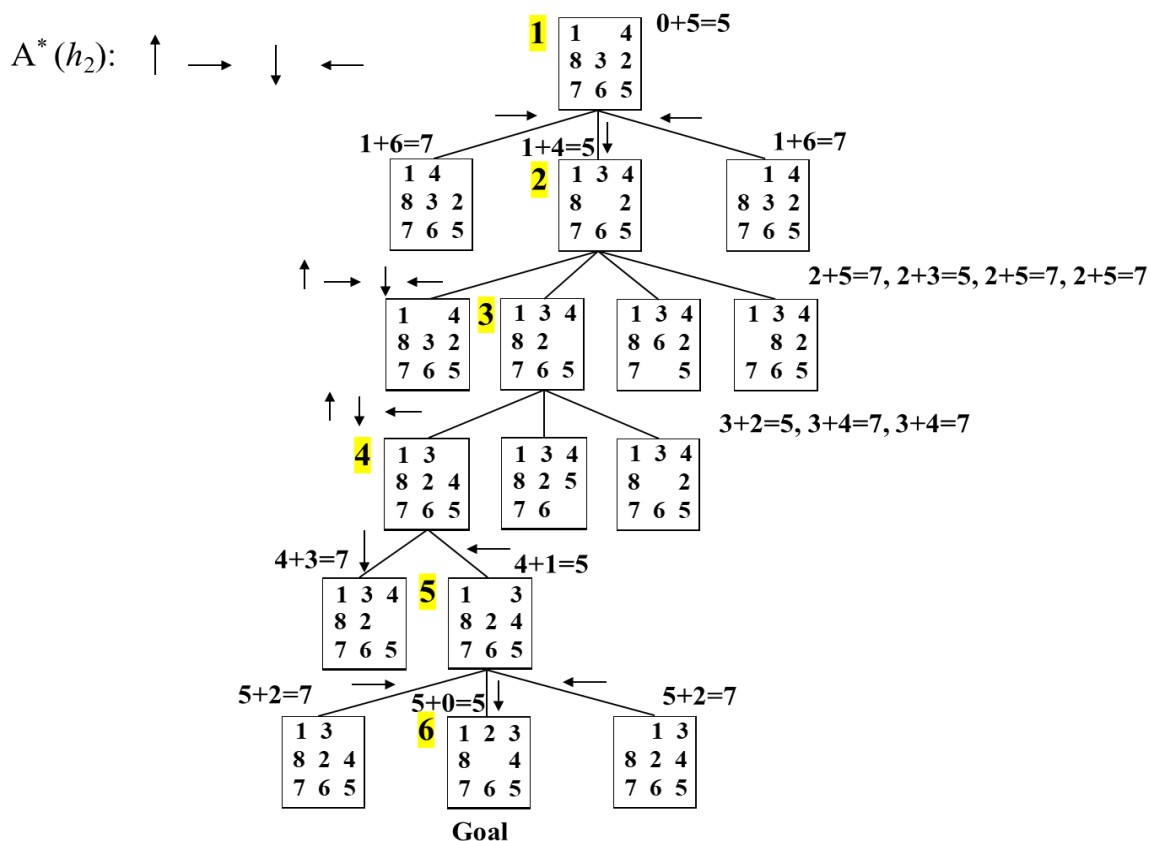
Na následujících obrázcích znamenají čísla ve žlutých podkladech pořadí vybírání uzlů ze zásobníku (bod 3 algoritmu), výrazy u uzlů pak hodnoty výrazu $g(s_k) + h(s_k) = f(s_k)$.



Obr. 2.16 Prohledávání stavového prostoru Loydovy osmičky metodou A* s heuristikou h_1



Obr. 2.17 Prohledávání stavového prostoru Loydovy osmičky metodou A* s heuristikou h_1 (jiný počáteční stav než v Obr. 2.16)



Obr. 2.18 Prohledávání stavového prostoru Loydovy osmičky metodou A^* s heuristikou h_2 (stejný počáteční stav jako v Obr. 2.17)

Na Obr. 2.16 je ukázáno řešení úlohy Loydovy osmičky, pro které je heuristika h_1 plně dostatečná. Expanduje pouze pět uzlů a nalezne optimálních pět tahů. Na Obr. 2.17 je použita stejná heuristika pro řešení stejného hlavolamu, pouze s jiným počátečním stavem. Zde již tato heuristika tak úspěšná není. I když by algoritmus také jistě našel optimální řešení, potřeboval by k tomu expandovat mnohem více uzlů, než je žádoucí. Heuristika h_2 je lepším spodním odhadem skutečné ceny, a proto v tomto případě nalezne optimální řešení expanzí menšího počtu uzlů (Obr. 2.18).

Závěr: Platí-li podmínka $h \geq h_2 > h_1$, kde h je skutečná cena, pak h_2 je lepší heuristika než h_1 !

Algoritmus A^* expanduje pouze uzly s_k pro které platí $f(s_k) \leq f_{opt}$. Jeho časovou i prostorovou složitost proto výrazně ovlivňuje použitá heuristika – pokud se pro všechny uzly blíží k nule, a to je jistě spodní odhad skutečné ceny, pak složitost algoritmu se blíží exponenciální složitosti (pro nulovou heuristiku jde o neinformovanou metodu UCS). Pokud je použitá heuristika dobrým spodním odhadem skutečné ceny, pak jsou expandovány pouze uzly kolem optimální cesty a je-li přesným odhadem, pak jsou expandovány pouze uzly na optimální cestě.

Volba heuristických funkcí je závislá na konkrétní úloze. Někdy je vhodné začít definovat heuristické funkce pro úlohy s menším omezením (*relaxed problems*), například pro Loydovu osmičku, kdy kamenem lze táhnout z pole A na pole B, jestliže pole A sousedí s polem B a pole B je prázdné. Úlohy s menším omezením pak mohou být

- Kamenem lze táhnout z A na B (h_1)
- Kamenem lze táhnout z A na B, jestliže A sousedí s B (heuristika h_2)
- Kamenem lze táhnout z A na B, pokud B je prázdné (?)

2.1.4 Metody lokálního prohledávání

Existuje řada úloh, jejichž stavy jsou ohodnoceny nějakou funkcí a cílem řešení je nalézt optimální stav, tj. stav s minimální, resp. maximální hodnotou hodnotící funkce. Takové úlohy se nazývají optimalizačními úlohami a k jejich řešení se používají metody, které místo optimální cesty hledají optimální stav a nazývají se optimalizačními metodami. Mezi tyto metody patří mj. i metody lokálního prohledávání. Metody lokálního prohledávání nejsou úplné, a tedy ani optimální, mají často velkou časovou složitost, ale zcela zanedbatelnou paměťovou složitost. Často pak dospějí k přijatelnému řešení v rozsáhlých stavových prostorech, kdy použití výše popsaných systematických metod není prakticky možné.

V této kapitole si ukážeme dvě klasické metody, a to metodu horolezeckou (*Hill-climbing*) a metodu simulovaného žitání (*Simulated Annealing*). Více se pak optimalizačním metodám budeme věnovat v kapitole 2.5, kde si popíšeme principy vybraných optimalizačních algoritmů inspirovaných přírodou.

2.1.4.1 Metoda horolezecká (HC)

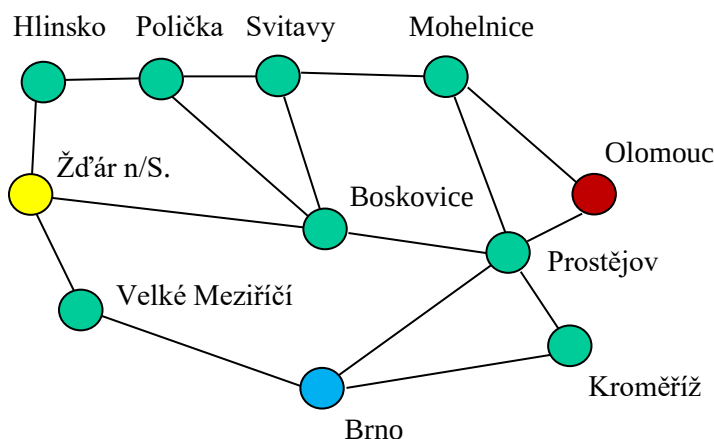
Metoda HC (*Hill-climbing*) používá k ohodnocení uzlu s_k , podobně jako metoda Greedy search, pouze heuristiku $h(s_k)$. Obvykle je však touto heuristikou funkce, která ohodnocuje „kvalitu řešení“ spíše než vzdálenost od cílového řešení. Čím lepší řešení představuje ohodnocovaný uzel, tím vyšší je mu přiřazena hodnota a algoritmus pak vybírá k expanzi uzly s nejvyšším ohodnocením (proto i název horolezecký algoritmus).

Vlastní algoritmus je zcela jednoduchý:

1. Vytvořte uzel *Current* (počáteční stav včetně jeho ohodnocení).
2. Expandujte uzel *Current*, ohodnoťte jeho bezprostřední následníky, vyberte z nich nejlépe ohodnoceného a označte ho jako uzel *Next*.
3. Je-li ohodnocení uzlu *Current* lepší než ohodnocení uzlu *Next*, ukončete řešení a vraťte jako výsledek uzlu *Current*. Jinak pokračujte.
4. Nahradejte uzel *Current* uzlem *Next* (uzel *Current* odstraňte a uzel *Next* přejmenujte na *Current*) a vraťte se na bod 2.

Na následujícím obrázku (Obr. 2.19) jsou ukázána řešení získaná metodou Hill-climbing pro dvě úlohy nalezení cesty: a) ze Žďáru n/S. do Olomouce b) ze Žďáru na/S. do Brna.

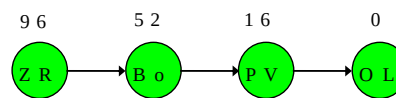
K ohodnocení uzlů můžeme použít například funkci $h(s_k) = 1/(\text{abs}(h(s_k) - h(s_g)) + c)$, kde c je libovolná kladná konstanta zabraňující dělení nulou při hodnocení cílového uzlu. Ohodnocení touto funkcí pak zaručuje, že hodnoty uzlů $h(s_k)$ budou tím vyšší, čím budou tyto uzly blíže k cílovému uzlu s_g .



Přímá vzdálenost [km] výchozí místo:	do BM	do OL
Boskovice	32	52
Brno	0	68
Hlinsko	48	100
Kroměříž	58	28
Mohelnice	73	52
Olomouc	68	0
Polička	62	76
Prostějov	47	16
Svitavy	65	60
Velké Meziříčí	48	96
Žďár nad Sázavou	64	96

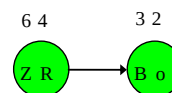
a) cesta ze Žďáru n/S do Olomouce

Krok	Current	Next
0	[ZR,96]	[Bo,52]
1	[Bo,52]	[PV,16]
2	[PV,16]	[OL,0]



b) cesta ze Žďáru n/S do Brna

Krok	Current	Next
0	[ZR,96]	[Bo,32]
1	[Bo,32]	[PV,47]



Obr. 2.19 Řešení úlohy hledání cesty metodou Hill-climbing

První úlohu vyřeší metoda Hill-climbing velmi rychle a bez jakýchkoliv problémů. Druhou úlohu však nevyřeší, protože zůstane uvázná v lokálním extrému. Tento extrém představují Boskovice, z nichž je sice přímou čarou do Brna nejbližší, ale cesta do Brna z nich podle naší mapy nevede (hodnota uzlu *Next* je větší, než hodnota uzlu *Current*).

Metoda je neúplná, a proto není ani optimální. Časová i prostorová složitost jsou lineární a velmi malé.

2.1.4.2 Metoda simulovaného žihání (SA)

Metoda simulovaného žihání je stochastickou metodou, jejímž cílem je překonání lokálních extrémů vedoucích k častým neúspěchům metody Hill-climbing. Vychází z principů žihání kovů a používá i stejnou terminologii. Algoritmus je následující:

1. Vytvořte tabulku pro „teplotu“ klesající v závislosti na kroku výpočtu $T(k+1) \leq T(k)$.
2. Vytvořte pracovní uzel *Current* a uložte do něj počáteční stav spolu s jeho ohodnocením (obvykle je ohodnocení stavu opět dáno pouze hodnotou heuristické funkce h , tj. $\text{value}(\text{Current}) = h$). Nastavte krok výpočtu k na nulu.
3. Expandujte uzel *Current*, z jeho bezprostředních následníků vyberte náhodně jednoho z nich a označte ho jako uzel *Next*.
4. Z tabulky zjistěte aktuální teplotu $T(k)$. Je-li tato teplota nulová a je-li ohodnocení uzlu *Current* lepší (nižší) než ohodnocení uzlu *Next*, pak ukončete řešení a vraťte jako výsledek uzel *Current*. Jinak pokračujte.
5. Vypočítejte rozdíl ohodnocení uzlů $\Delta E = \text{value}(\text{Current}) - \text{value}(\text{Next})$.
6. Jestliže $\Delta E > 0$, pak nahraďte uzel *Current* uzlem *Next*, jinak proveďte tuto náhradu s pravděpodobností $P(\text{náhrady}) = e^{\Delta E/T}$.
7. Inkrementujte krok výpočtu k a vraťte se na bod 3.

Je zřejmé, že algoritmus přechází do nového stavu nepodmíněně, pokud je ohodnocení tohoto stavu nižší než ohodnocení aktuálního stavu (nový stav je blíže cílovému stavu). V opačném případě je přechod dán pravděpodobností, která je pro vysoké počáteční teploty také vysoká a která se postupně se snižující se teplotou také postupně snižuje, až konečně pro nulovou teplotu se, stejně jako v algoritmu Hill-climbing, akceptuje pouze následník *Next* s ohodnocením lepším, než je ohodnocení uzlu *Current*.

V bodu 6 se někdy místo vztahu $P(\text{náhrady}) = e^{\frac{\Delta E}{T}}$ používá vztah $P(\text{náhrady}) = \frac{1}{1 + e^{-\frac{\Delta E}{T}}}$.

Při vysokých počátečních teplotách a při stejných hodnotách obou stavů dojde k prvním případě k akceptování nového stavu s pravděpodobností blízkou jedničce (nový stav má přednost), zatímco ve druhém případě s pravděpodobností $P(\text{náhrady}) \approx 0.5$ (stavy mají stejnou šanci).

Metoda simulovaného žitání má opět extrémně malou prostorovou složitost, ale jeho časová složitost je velmi vysoká a je závislá na rychlosti klesání „teploty“. Pro reálné časové možnosti pak není zaručena ani úplnost, a tedy ani optimálnost této metody.

2.2 Metody řešení úloh s omezujícími podmínkami

V některých úlohách musí cílový stav pouze splňovat předepsané podmínky a posloupnost operátorů vedoucí k nalezení tohoto stavu není důležitá. Takové úlohy se nazývají CSP (*Constraint Satisfaction Problems*).

Formálně je úloha s omezujícími podmínkami definována takto:

- Nechť existuje množina proměnných $\{x_1, x_2, \dots, x_n\}$, z nichž každá proměnná x_i může nabývat hodnot z neprázdné domény D_i .
- Nechť existuje množina omezujících podmínek $\{c_1, c_2, \dots, c_m\}$, z nichž každá předepisuje vztah mezi nějakou podmnožinou proměnných z výše zmíněné množiny proměnných.
- Nechť stav úlohy je definován hodnotami přiřazenými jednotlivým proměnným a nechť legální stav znamená, že proměnné s přiřazenými hodnotami vyhovují předepsaným podmínkám.
- V počátečním stavu není přiřazena hodnota žádné proměnné.
- Obecný operátor přiřazuje hodnotu libovolné volné proměnné tak, aby následný stav byl legálním stavem.
- Řešením úlohy je legální stav, ve kterém všechny proměnné mají přiřazené hodnoty.

2.2.1 Úlohy s omezujícími podmínkami

Úlohy s omezujícími podmínkami lze opět rozdělit na umělé úlohy a na úlohy reálné.

Umělé úlohy:

- Úloha osmi dam
- Krypto-aritmetické úlohy
- Sudoku
- ...

Reálné úlohy:

- Úloha barvení map
- Úlohy rozvrhů
- ...

Dále si popíšeme podrobněji dvě úlohy, které použijeme v dalším textu při výkladu principů metod CSP.

2.2.1.1 Úloha osmi dam

Osm dam je úloha, ve které se má postavit 8 dam na šachovnici o rozměrech 8 x 8 tak, aby žádná dáma neohrožovala jinou. V obecnější verzi jde o úlohu N dam a šachovnici o rozměrech $N \times N$. Označíme-li počet políček symbolem n_p ($n_p = N \times N$), pak počet všech teoreticky možných stavů n_s (tj. i těch stavů, ve kterých se dámy ohrožují) je dán výrazem

$$n_s = \binom{n_p}{N} = \frac{n_p!}{N!(n_p-N)!}. \text{ Pro klasickou úlohu } 8 \times 8 \text{ je počet stavů přibližně } 4.5 \cdot 10^9.$$

Budeme-li ale respektovat fakt, že v každém sloupci může být jediná dáma, pak počet možných stavů (bez respektování vzájemného ohrožení) je dán výrazem $n_s = N^N$, což pro $N = 8$ znamená přibližně $1.7 \cdot 10^7$ stavů. Počet přípustných stavů, kdy dáma ve sloupci nesmí být na řádku ohroženém dāmami v předcházejících sloupcích (neuvažujeme ohrožení v diagonálách) je dán výrazem $N!$ a pro $N = 8$ vede pouze ke 40320 různým přípustným stavům.

Stav s může být matice s dāmami na jednotlivých políčkách, například pro $N = 4$ a po umístění prvních dvou dam (s uvažováním, že ve sloupci může být jediná dáma) nebo jednodušeji zápisem čísel řádků v jednotlivých sloupcích

$$s = (1, 3, _, _)$$

Q			
	Q		

Operátory mohou být tyto:

- Postavte (další) dāmu na libovolné prázdné políčko. Jde o celkem $\frac{n_p!}{(n_p-N)!}$ možností. Počet dosažitelných stavů je výrazně vyšší než počet možných stavů, protože dámy si mohou vzájemně vyměňovat místa: například postavení první dāmy na políčko (i, j) a druhé dāmy na políčko $(i, j+2)$ vede na stejný stav, pokud postavíme první dāmu na políčko $(i, j+2)$ a druhou dāmu na políčko (i, j) . Pro $N = 8$ je pak počet takových stavů přibližně $1.8 \cdot 10^{14}$. Tento přístup je nepraktický, a v praxi se proto vůbec nepoužívá.
- Postav dāmu na nějaké políčko v následujícím sloupci.

Úloha N dam s operátory podle předchozího bodu b) je definována takto:

- Proměnné z množiny $\{Q_1, Q_2, \dots, Q_N\}$ představují dāmy v jednotlivých sloupcích a jejich hodnotami jsou řádky, na kterých jsou příslušné dāmy postavené.
- Domény D_i jsou pro všechny proměnné stejné a obsahují čísla řádků $\{1, 2, \dots, N\}$.
- Omezující podmínky:
 - Dāmy nesmí být ve stejných sloupcích (zajišťuje již definice proměnných).
 - Dāmy nesmí být ve stejných řádcích: $Q_i \neq Q_j, i \neq j, i \in D_i, j \in D_j$.
 - Dāmy nesmí být ve stejných úhlopříčkách: $|Q_i - Q_j| \neq |i - j|, i \neq j, i \in D_i, j \in D_j$.

Testování porušení podmínek až po dosažení koncového stavu vede ke zbytečným výpočtům, a proto se porušení podmínek obvykle testuje po postavení každé dāmy.

Poznamenejme, že úloha není řešitelná pro $N < 4$ a že zajímavou se stává pro $N \geq 8$.

2.2.1.2 Úloha barvení map

Cílem úlohy je nalézt obarvení jednotlivých ploch mapy barvami z dané množiny barev takové, aby sousední plochy měly jiné barvy.

Úloha je definována takto:

- Proměnné z množiny $\{x_1, x_2, \dots, x_N\}$ představují jednotlivé plochy mapy a jejich hodnotami jsou barvy, kterými jsou obarvené.
- Domény D_i jsou pro všechny proměnné stejné a obsahují jednotlivé použitelné barvy.
- Omezující podmínka: Sousední plochy musí být obarvené různými barvami.

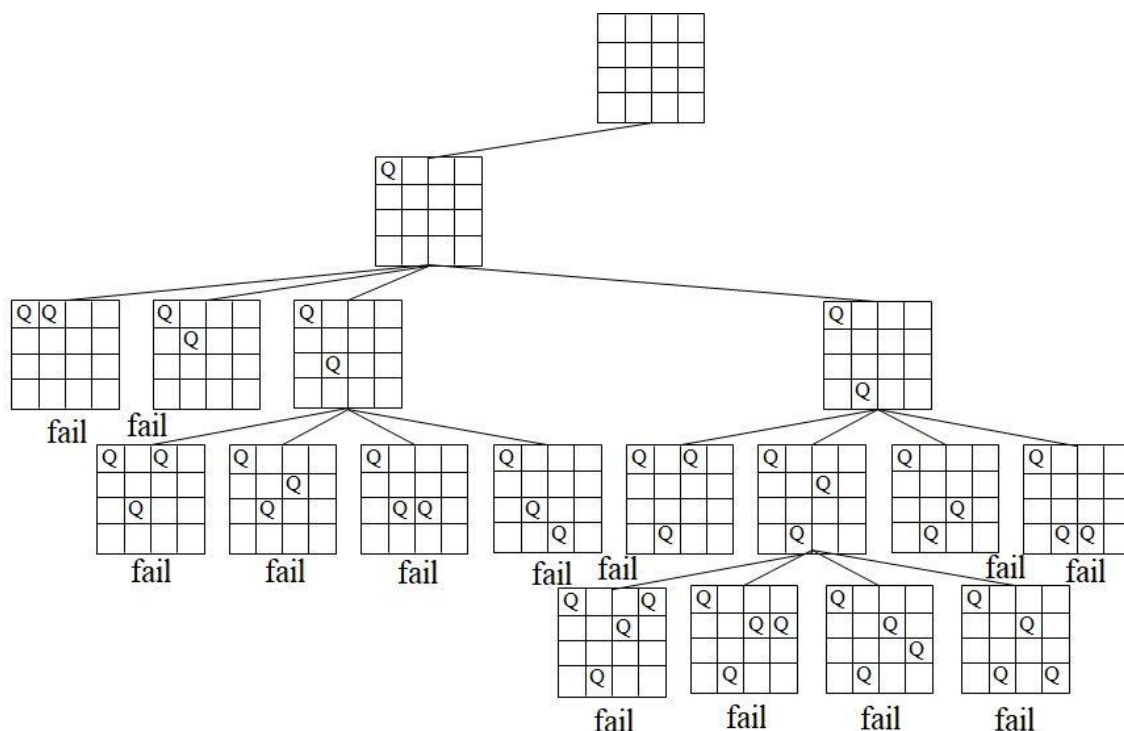
2.2.2 Metody řešení CSP

Úlohy s omezujícími podmínkami lze řešit všemi metodami prohledávání stavového prostoru, které byly popsány v předchozí kapitole. Existují však speciální a výrazně efektivnější metody, z nichž si dále ukážeme principy tří nejznámějších:

- Metody zpětného navracení pro CSP (Backtracking for CSP)
- Metody dopředné kontroly (Forward checking)
- Metody minimálního konfliktu (Min-conflict)

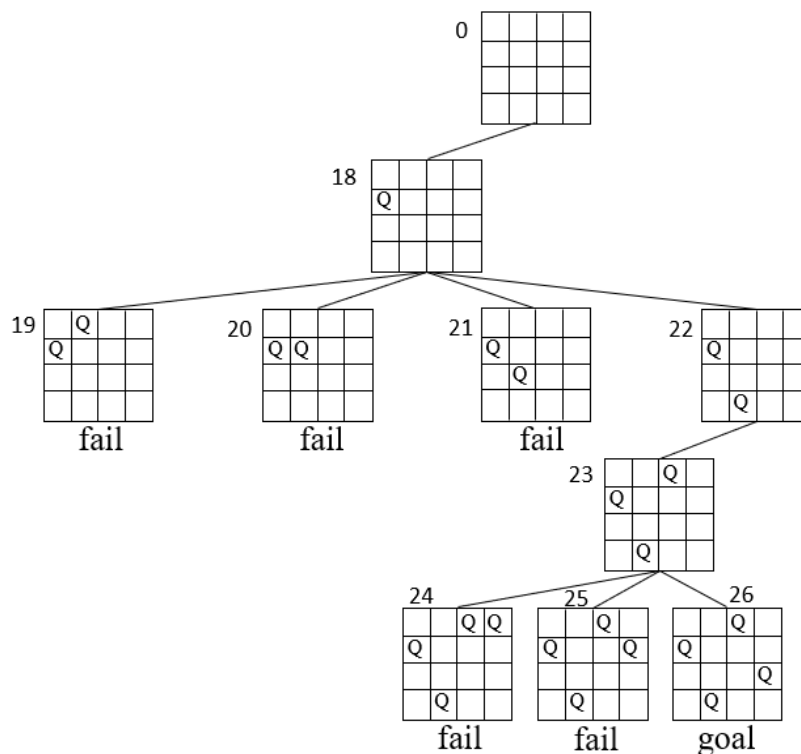
2.2.2.1 Metoda zpětného navracení pro CSP (Backtracking for CSP)

Princip metody, resp. její algoritmus je velmi jednoduchý: v každém kroku přiřazuje hodnotu (nejprve první a pak postupně další) proměnné a pokud nový stav je legálním stavem, přiřazuje hodnotu další proměnné atd. Pokud nový stav porušuje předepsané podmínky, pak se algoritmus vrací o krok zpět. Na Obr. 2.20 je ukázána činnost algoritmu na úloze čtyř dam.



Pokračování na další straně

Pokračování z předcházející strany



Obr. 2.20 Řešení úlohy čtyř dam metodou Backtracking pro CSP

Metoda je úplná a jako každé řešení CSP je i optimální. Označíme-li symbolem n počet proměnných a symbolem m maximální počet přiřaditelných hodnot, pak platí:

Pro prostorovou složitost: $O_{bck-CSP}(n)$
 Pro časovou složitost: $O_{bck-CSP}(m^n)$

2.2.2.2 Metoda dopředné kontroly (Forward checking)

Princip metody Forward checking spočívá v tom, že po každém přiřazení hodnoty nějaké proměnné vyřazuje z množin hodnot dosud volných proměnných ty hodnoty, které jsou s právě přiřazenou hodnotou v konfliktu. Algoritmus je dán procedurou, která rekurzivně volá sama sebe:

- Přiřaďte každé proměnné x_i ($i = 1, \dots, n$) množinu přípustných hodnot $S_i = D_i$.
- Zavolejte proceduru Forward_Checking(1).

Procedura Forward_Checking:

1. Odstraňte první hodnotu z množiny S_i a přiřaďte tuto hodnotu proměnné x_i , necht' X je nový stav (je dán hodnotami všech proměnných x_i , $i = 1, \dots, n$).
2. Je-li X cílovým, tj. legálním stavem, skonči proceduru úspěchem (vrať stav X), jinak pokračujte.
3. Odstraňte z množin S_j ($j = i + 1, \dots, n$) všechny hodnoty, které jsou v konfliktu s dosud přiřazenými hodnotami.
4. Jestliže je nějaká množina S_j ($j = i + 1, \dots, n$) prázdná, obnovte původní stav množin S_j (tj. stav před bodem 3) a přejděte na bod 7. Jinak pokračujte.
5. Volejte proceduru Forward_Checking($i + 1$).

6. Skončí-li předchozí procedura Forward_Checking úspěchem, skončete také úspěchem (vraťte vrácený stav). Jinak pokračujte.
7. Není-li množina S_i prázdná, vraťte se na bod 1. Jinak skončete neúspěchem.

Použití algoritmu Forward checking je ukázán na úloze čtyř dam (Obr. 2.21). Z obrázku je zřejmé vyřazování hodnot, které jsou v konfliktu. Například po postavení 1. dámy na 1. políčko, jsou z přípustných hodnot pro 2. dámu vyřazeny hodnoty 1 a 2, z přípustných hodnot pro 3. dámu hodnoty 1 a 3, a z přípustných hodnot pro 4. dámu hodnoty 1 a 4. Dále je vidět mechanismus navracení, například po postavení druhé dámy na třetí políčko, kdy seznam přípustných hodnot pro 3. dámu je prázdný – druhá dáma se z třetího políčka odstraní, hodnota 3 se vypustí ze seznamu přípustných hodnot pro 2. dámu a seznamy přípustných hodnot pro 3. a 4. dámu se obnoví na stav po postavení 1. dámy.

	$S_1 = \{1,2,3,4\}, S_2 = \{1,2,3,4\}, S_3 = \{1,2,3,4\}, S_4 = \{1,2,3,4\}$
	$x_1 = 1$ $S_1 = \{2,3,4\}, S_2 = \{3,4\}, S_3 = \{2,4\}, S_4 = \{2,3\}$
	$x_1 = 1, x_2 = 3$ $S_1 = \{2,3,4\}, S_2 = \{4\}, S_3 = \{\}, S_4 = \{2\}$ fail! $S_3 = \{2,4\}, S_4 = \{2,3\}$
	$x_1 = 1, x_2 = 4$ $S_1 = \{2,3,4\}, S_2 = \{\}, S_3 = \{2\}, S_4 = \{3\}$
	$x_1 = 1, x_2 = 4, x_3 = 2$ $S_1 = \{2,3,4\}, S_2 = \{\}, S_3 = \{\}, S_4 = \{\}$ fail! $S_2 = \{1,2,3,4\}, S_3 = \{1,2,3,4\}, S_4 = \{1,2,3,4\}$
	$x_1 = 2,$ $S_1 = \{3,4\}, S_2 = \{4\}, S_3 = \{1,3\}, S_4 = \{1,3,4\}$
	$x_1 = 2, x_2 = 4,$ $S_1 = \{3,4\}, S_2 = \{\}, S_3 = \{1\}, S_4 = \{1,3\}$
	$x_1 = 2, x_2 = 4, x_3 = 1,$ $S_1 = \{3,4\}, S_2 = \{\}, S_3 = \{\}, S_4 = \{3\}$
	$x_1 = 2, x_2 = 4, x_3 = 1, x_4 = 3,$ $S_1 = \{3,4\}, S_2 = \{\}, S_3 = \{\}, S_4 = \{\}$ $X = (x_1, x_2, x_3, x_4) = (2, 4, 1, 3)$ Goal

Obr. 2.21 Řešení úlohy čtyř dam metodou Forward checking

Užitečná doporučení pro metodu Forward checking:

Kterou proměnnou z dosud volných proměnných vybrat pro přiřazení hodnoty?

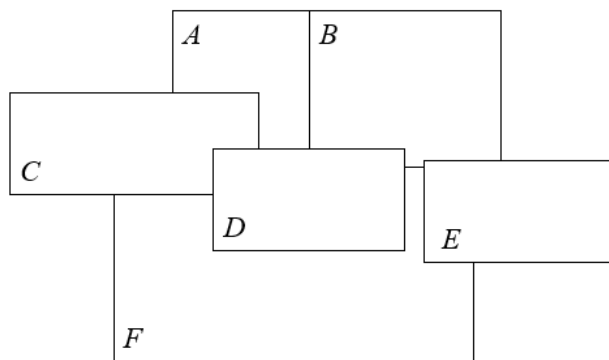
Vyberte proměnnou s nejmenším počtem přípustných hodnot (*Most constrained variable heuristic*).

Vyberte proměnnou, která má největší vliv na omezení zbývajících volných proměnných (*Most constraining variable heuristic*).

Kterou z přípustných hodnot vybrané proměnné přiřadit?

Vybrané proměnné přiřaďte hodnotu, která vylučuje nejméně hodnot proměnných, které mají společná omezení s vybranou proměnnou (*Least constraining value heuristic*).

Použití doporučených heuristik je ukázáno na Obr. 2.22 (barvení mapy třemi barvami: Red, Green, Blue).



$$S_A = \{R, G, B\}$$

$$S_B = \{R, G, B\}$$

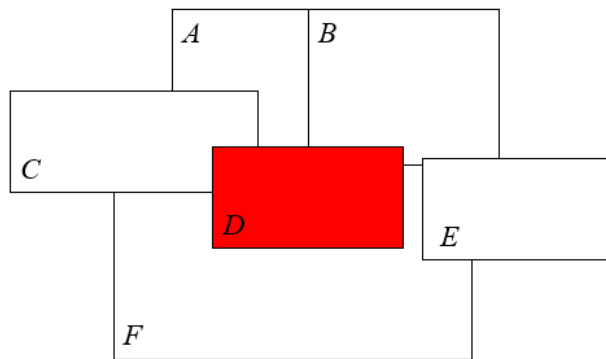
$$S_C = \{R, G, B\}$$

$$S_D = \{R, G, B\}$$

$$S_E = \{R, G, B\}$$

$$S_F = \{R, G, B\}$$

Největší vliv na omezení zbývajících proměnných má proměnná D. Výběr barvy této proměnné nemá na řešení úlohy žádný vliv.



Vybere se proto první barva z množiny S_D a tato barva se odstraní ze všech množin sousedních proměnných

$$S_A = \{G, B\}$$

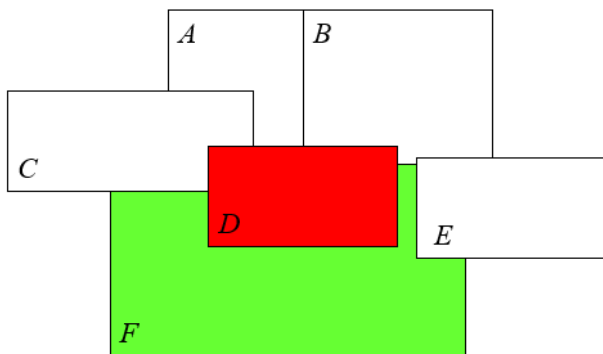
$$S_B = \{G, B\}$$

$$S_C = \{G, B\}$$

$$S_D = \{G, B\}$$

$$S_E = \{R, G, B\}$$

$$S_F = \{G, B\}$$



Největší vliv na omezení zbývajících proměnných má nyní proměnná F. Výběr její barvy (G nebo B) nemá na řešení vliv.

$$S_A = \{G, B\}$$

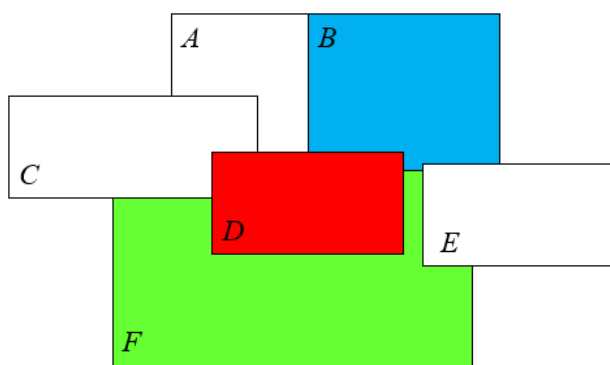
$$S_B = \{B\}$$

$$S_C = \{B\}$$

$$S_D = \{G, B\}$$

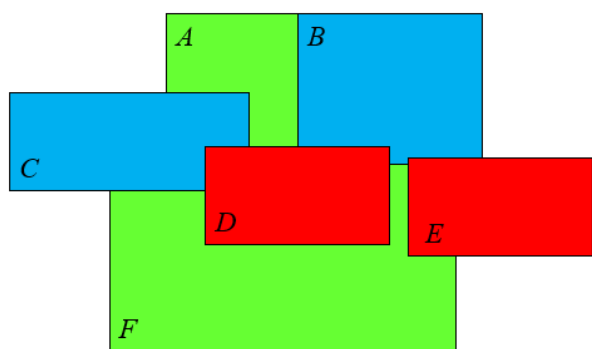
$$S_E = \{R, B\}$$

$$S_F = \{B\}$$



Největší vliv na omezení zbývajících proměnných mají nyní proměnné A a B. Z těchto dvou proměnných má méně přípustných hodnot proměnná B.

$S_A = \{G\}$
 $S_B = \{\}$
 $S_C = \{B\}$
 $S_D = \{G, B\}$
 $S_E = \{R\}$
 $S_F = \{B\}$



Zbývající neobarvené proměnné mají přípustnou vždy jedinou hodnotu a pro tyto hodnoty je celý problém řešitelný.

$S_A = \{\}$
 $S_B = \{\}$
 $S_C = \{\}$
 $S_D = \{G, B\}$
 $S_E = \{\}$
 $S_F = \{G\}$

Obr. 2.22 Barvení mapy metodou Forward checking

Pro hodnocení metody Forward checking platí stejné závěry, jako pro hodnocení metody Backtracking pro CSP. Metoda je úplná a optimální. Pro prostorovou a časovou složitost platí opět vztahy:

Pro prostorovou složitost: $O_{Fw-Ch}(n)$
 Pro časovou složitost: $O_{Fw-Ch}(m^n)$

kde n je počet proměnných a m maximální počet přiřaditelných hodnot.

2.2.2.3 Metoda minimálního konfliktu (Min-conflict)

Metoda Min-conflict vychází z libovolného úplného (všem proměnným jsou přiřazeny nějaké hodnoty), ale obecně nelegálního stavu (přiřazené hodnoty nesplňují podmínky) a snaží se zmenšovat počet konfliktů (tj. počet porušených podmínek). Tento velmi jednoduchý přístup je realizován algoritmem Min-conflict a je překvapivě efektivní pro mnoho CSP.

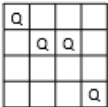
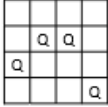
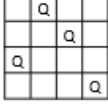
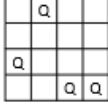
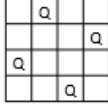
Algoritmus Min-conflict:

1. Přiřaďte každé proměnné x_i ($i = 1, \dots, n$) libovolnou hodnotu z množiny S_i jejích přípustných hodnot ($S_i = D_i$). Nastavte pomocné proměnné i a j na hodnoty 1 ($i = j = 1$).
2. Pro každou hodnotu proměnné x_i spočítejte počet možných (včetně teoreticky možných) konfliktů.
3. Pokud existuje pro jinou možnou hodnotu proměnné x_i počet konfliktů menší než počet konfliktů pro její aktuální hodnotu, změňte hodnotu této proměnné na hodnotu s minimálním počtem konfliktů. Pokud taková hodnota neexistuje, ale pokud existuje jiná hodnota proměnné x_i se stejným počtem konfliktů, tak změňte hodnotu proměnné

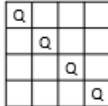
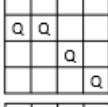
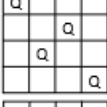
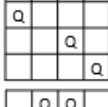
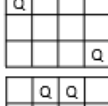
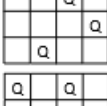
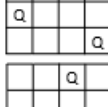
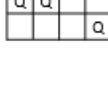
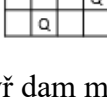
x_i na tuto novou hodnotu. V obou uvedených případech nastavte hodnotu proměnné j na 1, jinak pouze inkrementujte hodnotu této proměnné.

4. Pokud hodnota proměnné $j = n$, pak bylo nalezeno optimální řešení (tj. řešení s minimálním počtem konfliktů). Jinak inkrementujte hodnotu proměnné i a pokračujte.
5. Je-li hodnota proměnné $i > n$, pak změňte její hodnotu na hodnotu 1 ($i = 1$).
6. Vraťte se na bod 2.

Na Obr. 2.23 je ukázáno použití algoritmu Min-conflict opět na řešení problému čtyř dam.

1 2 3 4	Počet ohrožení dámy i na jednotlivých řádcích sloupce i :
	$Q_1 = \{2,2,1,2\}$ Dámu Q_1 teoreticky ohrožují na prvním řádku dámy Q_2 a Q_4 , na druhém řádku dámy Q_2 a Q_3 , na třetím řádku dáma Q_2 a na čtvrtém řádku dámy Q_3 a Q_4 .
	$Q_2 = \{1,3,2,2\}$
	$Q_3 = \{2,1,2,1\}$
	$Q_4 = \{1,0,3,1\}$
	$Q_1 = \{1,3,0,1\}, Q_2 = \{0,2,2,3\}, Q_3 = \{3,2,2,0\}, Q_4 = \{1,0,3,1\}$ $\Rightarrow$ Goal

Příklad 4 dam, jiný výchozí stav:

	$Q_1 = \{3,1,2,1\}$		$Q_4 = \{2,1,3,0\}$ $Q_1 = \{1,2,1,3\}$
	$Q_2 = \{1,3,2,2\}$		$Q_2 = \{2,3,1,1\}$
	$Q_3 = \{1,2,1,2\}$		$Q_3 = \{1,0,3,2\}$ $Q_4 = \{2,2,1,2\}$
	$Q_4 = \{2,2,1,0\}$ $Q_1 = \{3,1,1,1\}$		$Q_1 = \{0,1,2,2\}$ $Q_2 = \{3,2,2,0\}$ $Q_3 = \{1,1,3,2\}$
	$Q_2 = \{1,3,1,2\}$		$Q_4 = \{2,2,0,2\}$ $Q_1 = \{1,0,3,1\}$
	$Q_3 = \{1,1,3,2\}$		Goal

Obr. 2.23 Řešení úlohy čtyř dam metodou Min-conflict

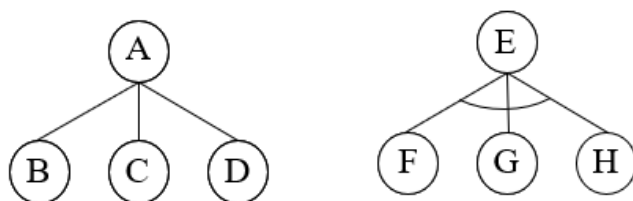
Metoda Min-conflict má extrémně malou prostorovou složitost, protože v paměti se udržuje pouze jediný stav s několika málo dodatečnými informacemi.

Zatím však není znám důkaz konvergence této metody, a tedy ani důkaz její úplnosti a s tím související odhad časové složitosti.

2.3 Metody řešení úloh rozkladem na podproblémy

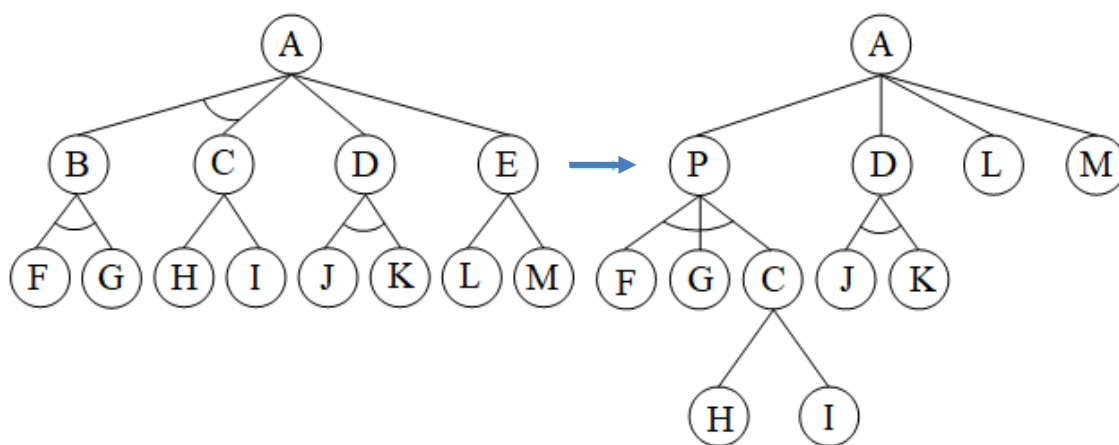
Metody řešení úloh rozkladem na podproblémy (dílčí úlohy) jsou přirozenými metodami, které při řešení obtížných problémů používá i člověk.

Existují dva typy problémů (Obr. 2.24). Problém A na tomto obrázku je řešitelný, je-li řešitelný alespoň jeden z podproblémů B, C, D, zatímco problém E je řešitelný, jsou-li řešitelné všechny podproblémy F, G a H. Problém A budeme nazývat problémem OR a problém E problémem AND (poznamenejme, že někteří autoři označují tyto uzly právě naopak). Zdůrazňeme, že uzly nyní označují problémy, a ne stavy jak tomu bylo v předcházející kapitole!



Obr. 2.24 Problémy OR a AND

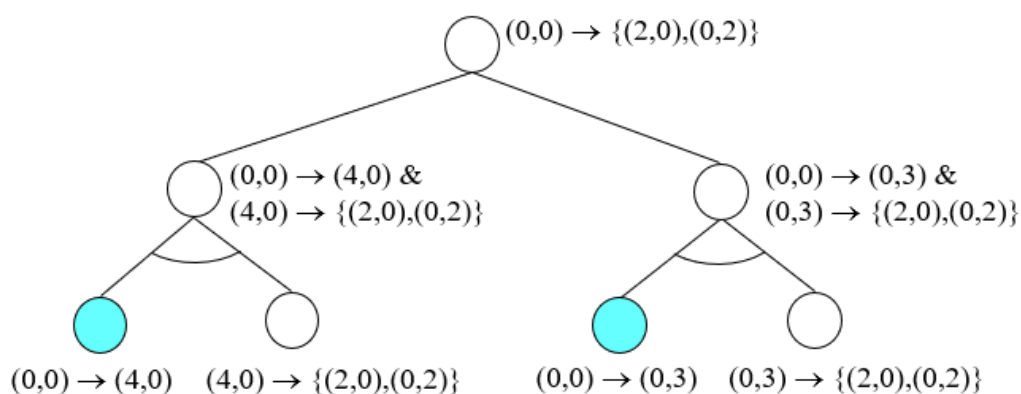
Případný smíšený problém (uzel A na Obr. 2.25) můžeme převést na „čisté“ AND a OR problémy zavedením pomocných uzlů (uzel P na Obr. 2.25), resp. vynecháním nedůležitých uzlů (uzel E na Obr. 2.25).



Obr. 2.25 Převod smíšených uzlů na uzly AND a OR

2.3.1 Úlohy vhodné pro řešení rozkladem na podproblémy

Obecně lze každou úlohu řešitelnou prohledáváním stavového prostoru řešit také rozkladem na dílčí úlohy (podproblémy). Každý uzel OR v AND/OR grafu/stromu pak představuje konjunkci dvou podproblémů, z nichž jeden má vždy triviální řešení spočívající v bezprostředním použití některého operátoru. Princip převodu je ukázán na úloze dvou džbánů (Obr. 2.26).



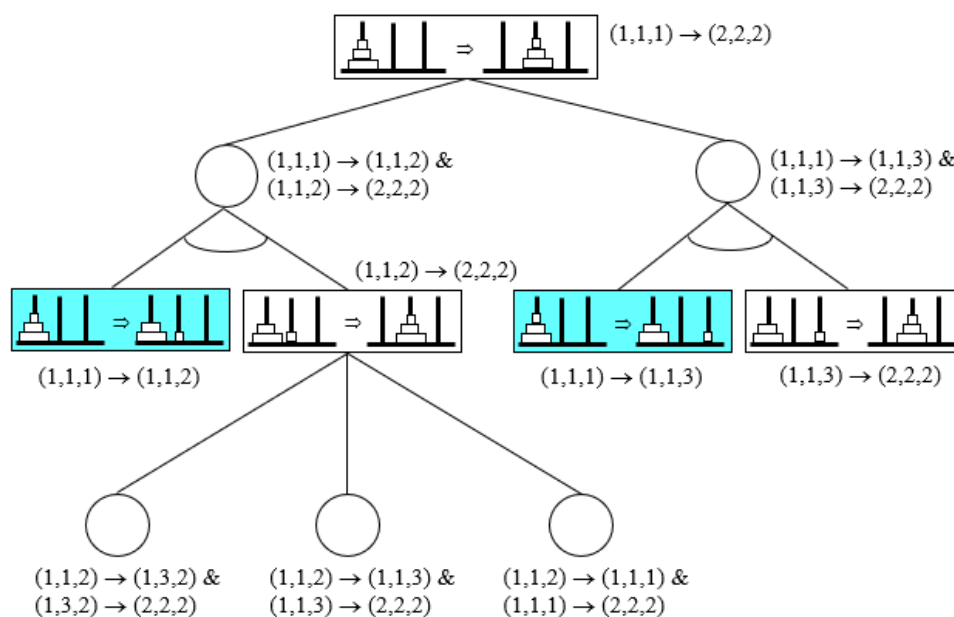
Obr. 2.26 Princip převodu úlohy dvou džbánů na podproblémy
barevně jsou označena triviální řešení

2.3.1.1 Úloha hanojských věží

Hlavolam Hanojských věží se skládá ze tří věží/kolíků. Na začátku je na první věž nasazeno několik kotoučů různých poloměrů, seřazených od největšího (dole) po nejmenší (nahore). Úlohou je přemístit všechny kotouče na druhou věž, třetí věž je pomocná. Pravidla pro přesun kotoučů jsou následující:

1. V jednom tahu lze přemístit pouze jeden kotouč.
2. Jeden tah spočívá v přesunu vrchního kotouče z některé věže na vrchol jiné věže.
3. Větší kotouč se nesmí položit na menší.

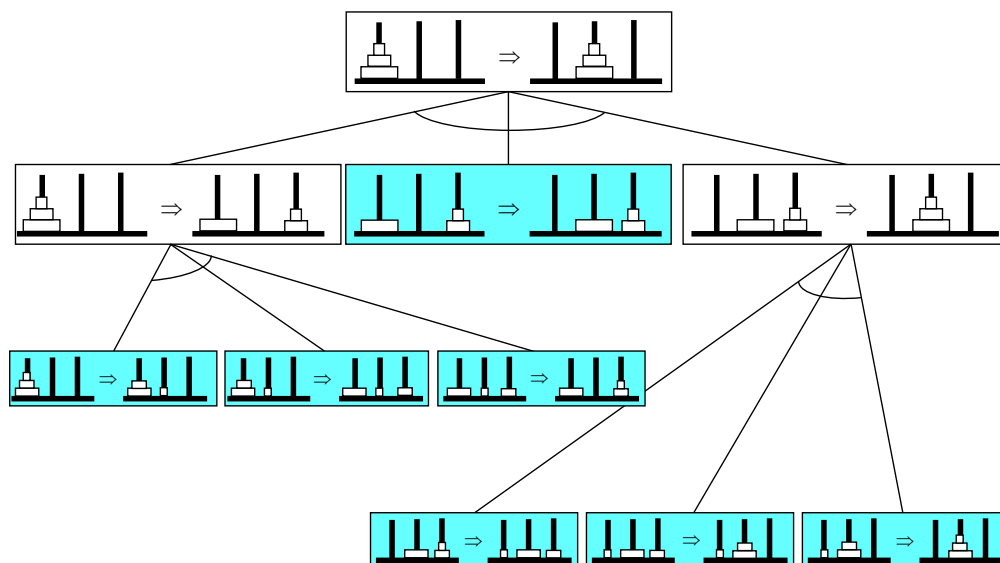
Princip převodu úlohy Hanojských věží na podproblémy je ukázán na Obr. 2.27 (stav je dán seznamem pozic kotoučů od největšího k nejmenšímu: například stav (1,1,3) znamená, že největší kotouč je na věži 1, střední kotouč je také na věži 1 a nejmenší kotouč je na věži 3). Barevně jsou opět označena triviální řešení.



Obr. 2.27 Princip převodu úlohy Hanojských věží na podproblémy

Formálně je pak úloha Hanojských věží definována takto: $s_0 = (1,1,1)$, $G = \{s_g\}$, $s_g = (2,2,2)$.

Poznamenejme, že některé úlohy je výhodné řešit pomocí rekurze, tj. pomocí AND grafů, například právě úloha Hanojských věží (Obr. 2.28).



Obr. 2.28 Řešení úlohy Hanojských věží AND grafem

2.3.1.2 Úloha dvanácti mincí

Máme 12 mincí z nichž 11 má stejnou váhu, ale jedna má váhu nepatrně rozdílnou (je buď lehčí, nebo těžší). Úlohou je nalézt tuto minci pomocí vážení na jazýčkových (balančních) váhách a minimalizovat přitom potřebná vážení.

Stav s je dán čtveřicí seznamů $s = (M, M_S, M_{SL}, M_{ST})$, kde značí

M množinu mincí s dosud nerozhodnutou váhou,

M_S množinu mincí se stejnými vahami,

M_{SL} množinu mincí, které jsou buď všechny stejné, nebo jedna z nich je lehčí

M_{ST} množinu mincí, které jsou buď všechny stejné, nebo jedna z nich je těžší

Počáteční stav $s_0 = (\{m_1, m_2, \dots, m_{12}\}, \{\}, \{\}, \{\})$

Množina cílových stavů $G = \{s_{g1}, s_{g2}\}$

$$s_{g1} = (\{\}, \{m_1, m_2, \dots, m_{12}\} - \{m_i\}, \{m_i\}, \{\})$$

$$s_{g2} = (\{\}, \{m_1, m_2, \dots, m_{12}\} - \{m_i\}, \{\}, \{m_i\})$$

Operátory provedou přesuny mincí mezi příslušnými množinami jednotlivých stavů na základě výsledků vážení (v miskách na obou miskách jazýčkové váhy musí být samozřejmě stejné počty mincí, tj. jedna-jedna, dva-dva, ..., šest-šest, přičemž mince se mohou vybírat ze všech čtyř seznamů M, M_S, M_{SL}, M_{ST}).

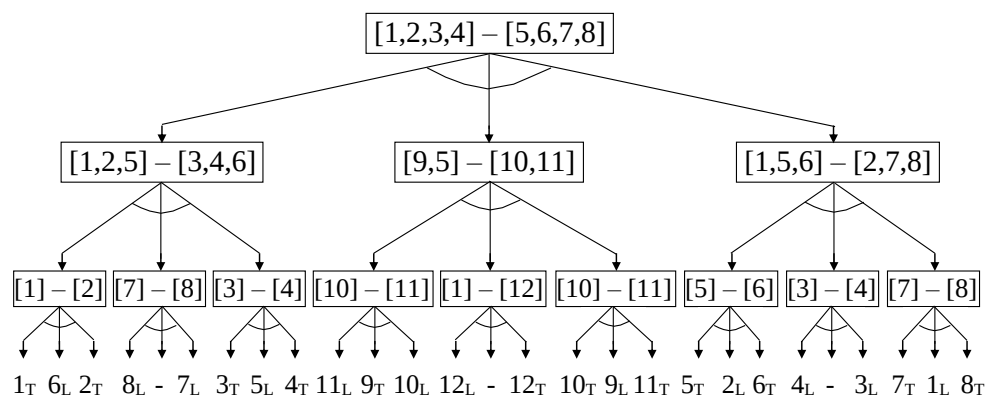
- Pokud klesne levá miska, tak je buď na levé misce těžší mince, nebo na pravé misce lehčí mince. Proto se přesunou všechny mince z levé misky, které dosud byly v množině M , do množiny M_{ST} , všechny mince z pravé misky, které dosud byly v množině M , do množiny M_{SL} a všechny zbývající mince z množiny M do množiny M_S .
- Pokud klesne pravá miska, tak je buď na pravé misce těžší mince, nebo na levé misce lehčí mince. Proto se přesunou všechny mince z pravé misky, které dosud byly v množině M , do množiny M_{ST} , všechny mince z levé misky, které dosud byly v množině M , do množiny M_{SL} a všechny zbývající mince z množiny M do množiny M_S .

- Pokud zůstane váha ve vodorovné poloze, pak se přesunou všechny mince z obou misek do množiny M_S .

Ke každému vážení vybereme různé kombinace mincí. Jde tedy o problémy/uzly typu OR.

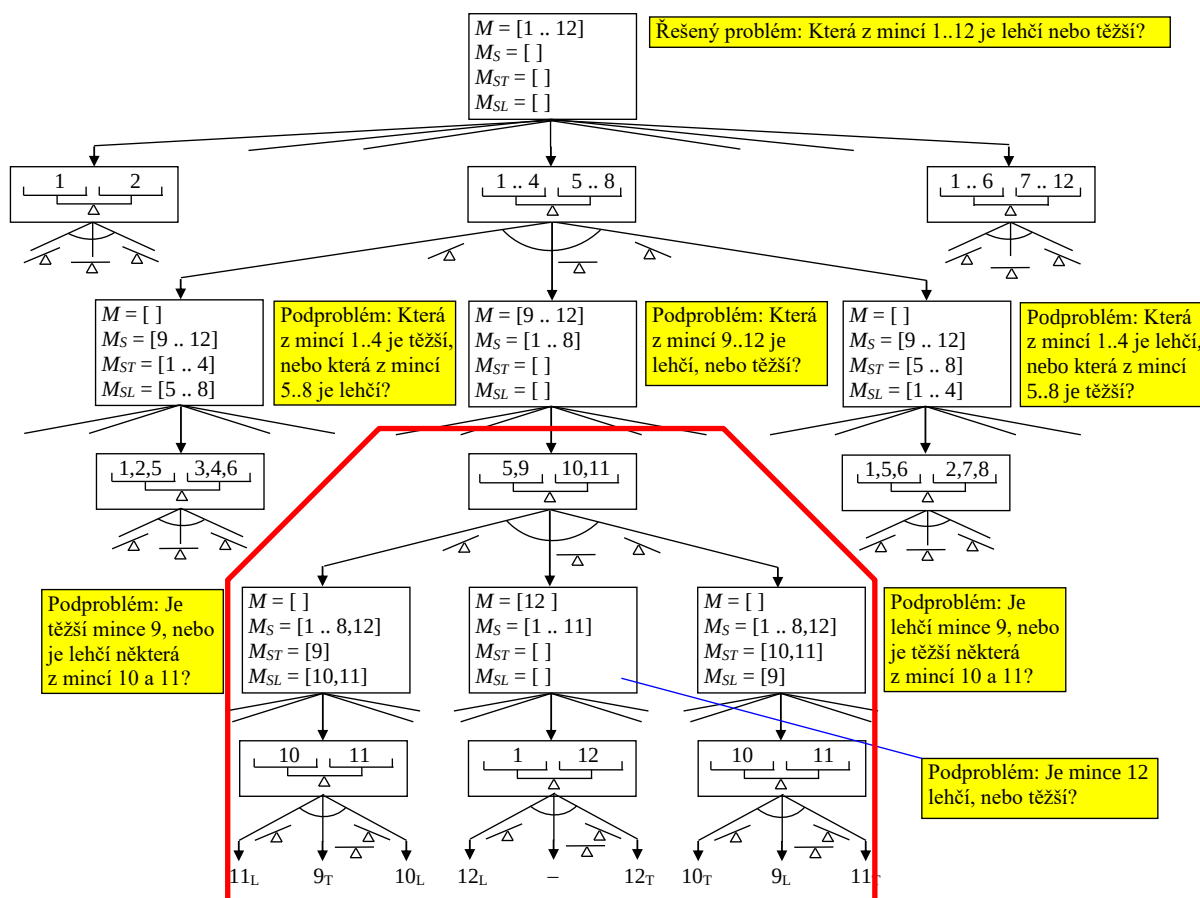
Po každém vážení musíme být schopni zpracovat výsledek tohoto vážení, tj. upravit obsah výše uvedených seznamů. Jde tedy o problémy/uzly typu AND.

Na Obr. 2.29 je ukázáno optimální řešení úlohy dvanácti mincí rozkladem na podproblémy

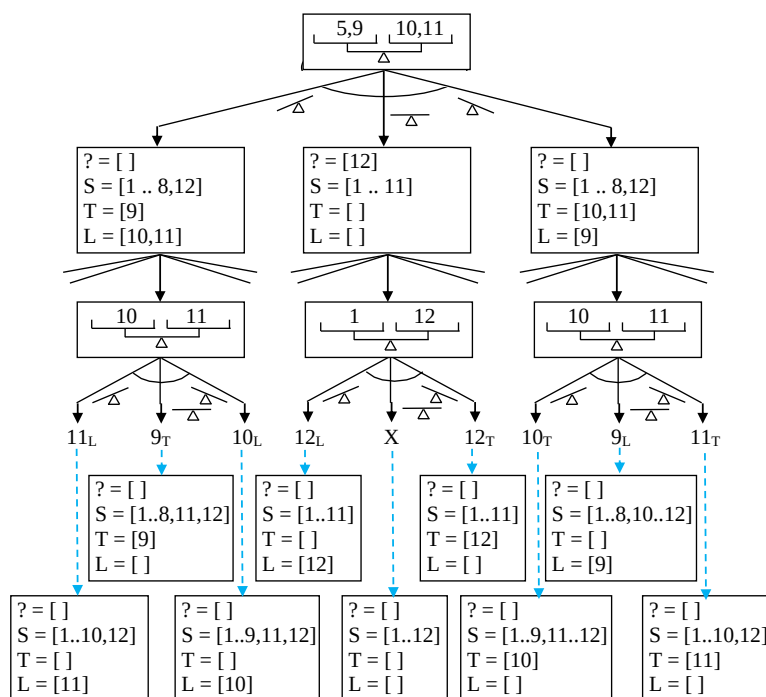


Obr. 2.29 Optimální řešení úlohy dvanácti mincí rozkladem na podproblémy

Na Obr. 2.30 a Obr. 2.31 je ukázán princip řešení úlohy (Obr. 2.31 popisuje podrobněji část stromu označenou na Obr. 2.30 červeným polygonem):

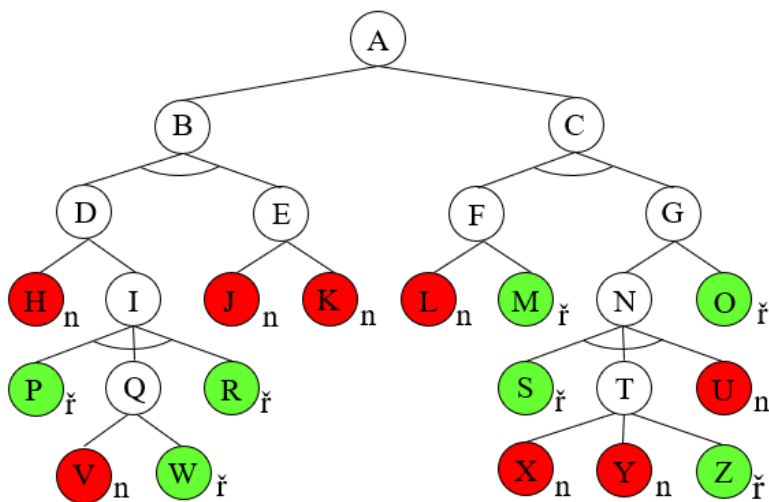


Obr. 2.30 Princip převodu úlohy dvanácti mincí na podproblémy



Obr. 2.31 Detail označený na Obr. 2.30 červeným polygonem

Pro vysvětlení principů algoritmů používaných v metodách řešení úloh rozkladem na podproblémy budeme dále používat jednoduchý demonstrační AND/OR graf/strom ukázaný na Obr. 2.32.



Obr. 2.32 Demonstrační AND/OR graf

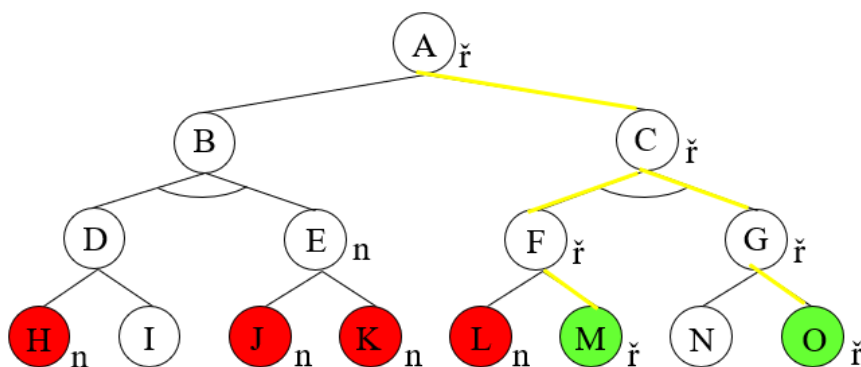
2.3.2 Neinformovaná metoda (AO algoritmus)

Základní neinformovanou metodou pro prohledávání AND-OR grafů je metoda AO (And-Or), která je realizována AO algoritmem. Při generování podproblémů může pracovat podobně jako algoritmy pro řešení úloh prohledáváním stavového prostoru (BFS, DFS, DLS, IDS, Backtracking).

AO algoritmus (BFS, DFS):

1. Sestrojte prázdný seznam OPEN (frontu pro BFS, zásobník pro DFS) a prázdný graf/strom G a do obou uložte počáteční uzel (problém), kterým nesmí být elementárně řešitelný nebo neřešitelný problém.
2. Vyjměte uzel z OPEN a označte jej jako uzel X .
3. Expandujte uzel X (rozložte X na podproblémy) a všechny jeho následníky připojte ke stromu G .
 - a) Pro všechny řešitelné následníky uzlu X přeneste informaci o jejich řešitelnosti jejich předchůdcům. Je-li řešitelný počáteční problém, ukončete řešení jako úspěšné (vraťte relevantní část AND/OR grafu).
 - b) Pro všechny neřešitelné následníky uzlu X přeneste informaci o jejich neřešitelnosti jejich předchůdcům. Není-li řešitelný počáteční problém, ukončete řešení jako neúspěšné.
 - c) Všechny ostatní následníky uzlu X uložte do OPEN.
4. Odstraňte z OPEN všechny uzly, které mají vyřešené předchůdce.
5. Je-li seznam OPEN prázdný, ukončete řešení jako neúspěšné, jinak se vraťte na bod 2.

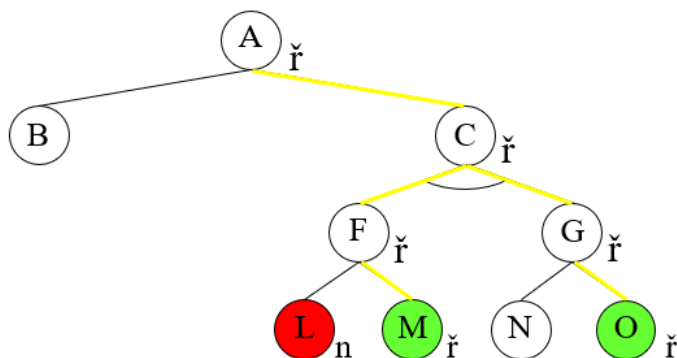
Na Obr. 2.33 je ukázán postup při řešení demonstrační úlohy (AND/OR grafu) algoritmem AO BFS.



Krok	OPEN	Graph
0	[A]	[(A,?)]
1	[B,C]	[(A,?),(B,?),(C,?)]
2	[C,D,E]	[(A,?),(B,?),(C,?),(D,?),(E,?)]
3	[D,E,F,G]	[(A,?),(B,?),(C,?),(D,?),(E,?),(F,?),(G,?)]
4	[E,F,G,I]	[(A,?),(B,?),(C,?),(D,?),(E,?),(F,?),(G,?),(H,n),(I,?)]
5	[F,G,I]	[(A,?),(B,n),(C,?),(D,?),(E,n),(F,?),(G,?),(H,n),(I,?),(J,n),(K,n)]
6	[G,I]	[(A,?),(B,n),(C,?),(D,?),(E,n),(F,n),(G,?),(H,n),(I,?),(J,n),(K,n),(L,n),(M,ř)]
7	[I]	[(A,ř),(B,n),(C,ř),(D,?),(E,n),(F,ř),(G,ř),(H,n),(I,?),(J,n),(K,n),(L,n),(M,ř),(N,?),(O,ř)]

Obr. 2.33 Ukázka řešení demonstrační úlohy pomocí AO algoritmu BFS

Na Obr. 2.34 je ukázán postup při řešení demonstrační úlohy algoritmem AO DFS.



Krok	OPEN	Graph
0	[A]	[(A,?)]
1	[B,C]	[(A,?),(B,?),(C,?)]
2	[B,F,G]	[(A,?),(B,?),(C,?),(F,?),(G,?)]
3	[B,F]	[(A,?),(B,?),(C,?),(F,?),(G,ř),(N,?),(O,ř)]
4	[]	[(A,ř),(B,?),(C,ř),(F,ř),(G,ř),(N,?),(O,ř),(L,n),(M,ř)]

Obr. 2.34 Postup při řešení demonstrační úlohy algoritmem AO DFS

Na první pohled by se mohlo zdát, že metoda AO DFS je výhodnější než metoda AO BFS. Stačí však záměna generování následníků a prohledávaný graf bude větší než u metody AO BFS.

2.3.3 Informovaná metoda (AO* algoritmus)

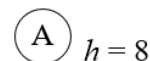
Pokud lze jednotlivé podproblémy ohodnocovat pomocí nějaké heuristické funkce, můžeme v každém uzlu OR „přepínat“ mezi řešenými podproblémy a řešit tak nejnadějnější podstrom. Ohodnocení bývá číselné, řešitelný uzel se označuje jako SOLVED a neřešitelný uzel jako FUTILITY (marnost). Informovaný AO algoritmus se nazývá AO* algoritmem:

1. Sestrojte prázdný strom G a umístěte do něj počáteční uzel označený INIT spolu s jeho ohodnocením. Stanovte hodnotu FUTILITY, která určuje maximální povolenou cenu řešení.
2. Je-li uzel INIT označen jako SOLVED, ukončete řešení jako úspěšné (řešení je dáno nejnadějnějším podstromem). Je-li hodnota uzlu INIT větší nebo rovna hodnotě FUTILITY, ukončete prohledávání jako neúspěšné. Jinak pokračujte.
3. Procházejte nejnadějnějším podstromem až narazíte na neexpandovaný uzel. Tento uzel označte jako NODE.
4. Expandujte uzel NODE. Jestliže NODE nemá žádného následníka, přiřaďte mu hodnotu FUTILITY (je ekvivalentní sdělení, že uzel není řešitelný) a přejděte na bod 7. Jinak pokračujte.
5. Představují-li někteří bezprostřední následníci elementární úlohy, označte je jako SOLVED (tj. přiřaďte jim nulové ohodnocení), u zbývajících ohodnocení odhadněte (vypočítejte).

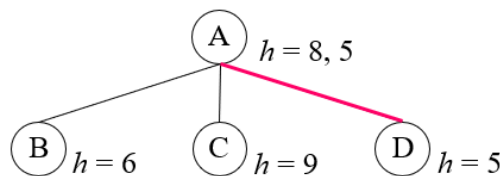
6. Připojte bezprostřední následníky uzlu NODE ke stromu G a přeneste informace o jejich ohodnocení směrem nahoru k uzlu INIT takto:
 - ohodnocení uzlů AND je rovno součtu ohodnocení všech jeho bezprostředních následníků.
 - ohodnocení uzlů OR je rovno nejnižšímu ohodnocení jeho bezprostředních následníků.
7. Ve všech uzlech OR označte nejnadějnější podstromy (hrany k nejlépe ohodnoceným bezprostředním následníkům).
8. Vraťte se na bod 2.

Princip činnosti algoritmu AO* je ukázán na Obr. 2.35.

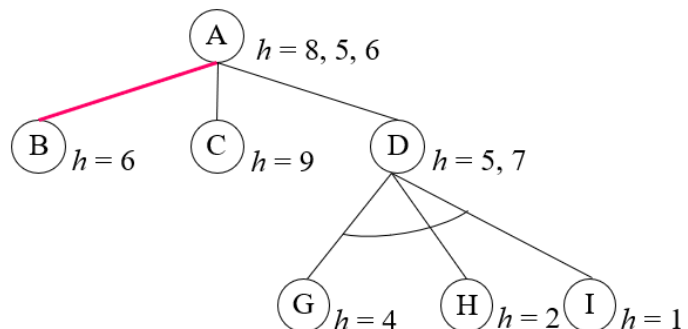
Krok 0 (počáteční uzel A s heuristickým ohodnocením $h = 8$):



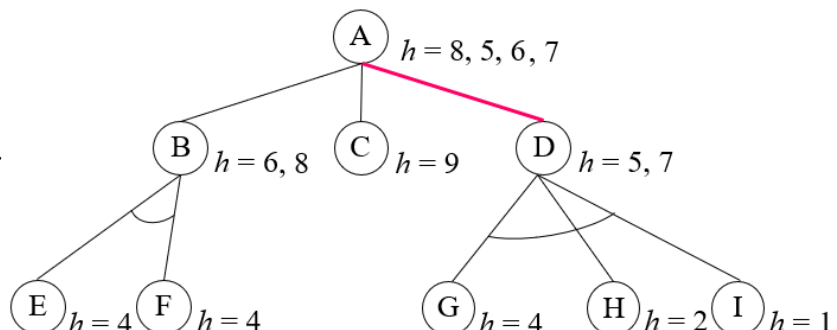
Krok 1 (expanze uzlu A s ohodnocením následníků. Novou hodnotou uzlu A je minimální hodnota následníků, nejnadějnějším podstromem je pravá větev):



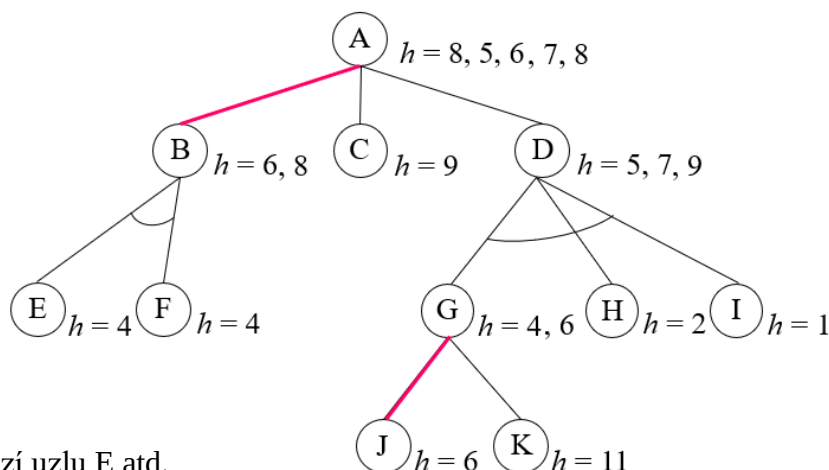
Krok 2 (expanze uzlu D s ohodnocením následníků. Novou hodnotou uzlu D je součet hodnot jeho následníků. Tato hodnota je vyšší než hodnota uzlu B, a proto novým nejnadějnějším podstromem se stává levá větev. Změní se i ohodnocení uzlu A):



Krok 3 (expanze uzlu B s ohodnocením následníků. Novou hodnotou uzlu B je součet hodnot jeho následníků. Tato hodnota je vyšší, než aktuální ohodnocení uzlu D, a proto novým nejnadějnějším podstromem se opět stává pravá větev. Změní se samozřejmě i ohodnocení uzlu A):



Krok 3 (expanze uzlu G s ohodnocením následníků. Novou hodnotou uzlu G je minimální hodnota následníků a nejnadějnějším podstromem uzlu G je jeho levá větev. Změní se i hodnoty uzlů D a A, a novým nejnadějnějším podstromem uzlu A se zase stává jeho levá větev):



Dále by se pokračovalo expanzí uzlu E atd.

Obr. 2.35. Princip činnosti algoritmu AO*

2.4 Metody hraní her

Cílem všech metod hraní her je **určit tah hráče na tahu**, který povede k jeho vítězství nebo pro který je pravděpodobnost jeho vítězství nejvyšší.

S jedinou výjimkou budeme dále uvažovat pouze hry, které hrají dva pravidelně se střídající hráči A a B. Oba mají úplnou informaci o aktuálním stavu hry, hrají čestně a oba si přejí zvítězit.

Pro takové hry obecně platí:

- Hráč na tahu (označuje se vždy jako hráč A) zvítězí, pokud vede k jeho vítězství alespoň jeden tah – jde tedy o problém OR.
- Hráč na tahu (tj. hráč A) zvítězí, vedou-li po jeho tahu k jeho vítězství všechny možné tahy protivráce (hráče B) – problém AND.

Hledání tahu vedoucího k výhře hráč A tak vede na klasické prohledávání AND/OR grafu.

Po výběru a následném uskutečnění tahu hráče A se „vše zapomene“ a na dalším tahu je hráč B. Hráč A vybírá svůj další tah až po tahu hráče B, a to z již nového konkrétního stavu úlohy!!!

Poznamenejme, že pokud svůj tah hledá hráč B, tak se on považuje za hráče A, protivníka považuje za hráče B a řešitelnost stavů hry hodnotí přirozeně opačně.

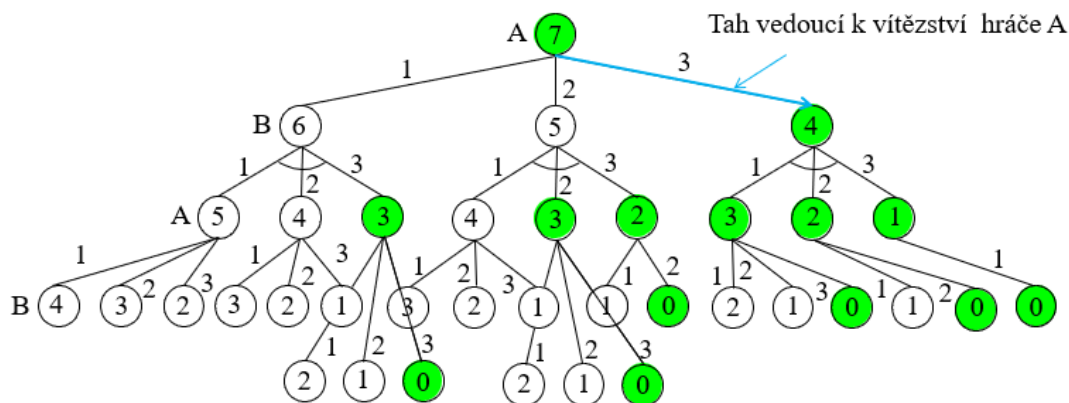
2.4.1 Hry se dvěma hráči

Hry můžeme formálně rozdělit na jednoduché hry (NIM), složité hry (Piškvorky, Dáma, Šachy, Go) a hry s neurčitostí (Člověče nezlob se, Vrháčky).

2.4.1.1 Jednoduché hry

Za jednoduché hry lze považovat hry, u kterých lze v reálném čase prohledat celý jejich AND/OR graf. K řešení takových her lze použít algoritmy AO, resp. AO* s tím, že není nutné vracet celou část prohledávaného AND/OR grafu, ale pouze tah hráče A, který vede k jeho výhře nebo který je pro tohoto hráče v daném stavu hry nejvýhodnější.

K demonstraci hledání tahu u jednoduché hry použijeme následující hru se zápalkami (zjednodušený NIM): Hráči střídavě odebírají z hromádky zápalek jednu, dvě, nebo tři zápalky a prohrává ten hráč, který již nemá co odebrat. Výchozím stavem hry je hromádka s libovolným počtem zápalek, dále budeme pro jednoduchost uvažovat pouze 7 zápalek. Řešení této úlohy metodou AO BFS je ukázáno na Obr. 2.36, zelenou barvou jsou označeny řešitelné uzly pro hráče A, žádný neřešitelný uzel v prohledávané části grafu není. Čísla v uzlech značí aktuální stav hry (počet zápalek na hromádce), čísla u hran značí počet odebíraných zápalek.



Obr. 2.36 Prohledávání AND/OR grafu úlohy 7 zápalek algoritmem AO BFS

Je zřejmé, že výchozí stav je pro hráče A příznivý – tah spočívající v odebrání tří zápalek vede k jeho výhře.

2.4.1.2 Složitě hry

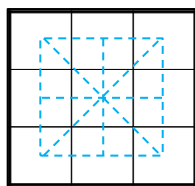
Pro složité hry je prohledávání jejich AND/OR grafů do větších hloubek nemožné. Proto se u těchto her prohledává pouze do předem zadané hloubky, tj. zkoumá se pouze předem zadaný počet tahů. Pokud se pak v této hloubce nachází uzly, u kterých nelze rozhodnout o jejich řešitelnosti/neřešitelnosti, je nutné tyto uzly nějakým způsobem ohodnotit.

V dále popsaných metodách se k ohodnocení problémů/uzlů používají celočíselné hodnotící funkce, jejichž kladné hodnoty znamenají příznivé stavy pro hráče A (čím je hodnota uzlu vyšší, tím je stav pro hráče A příznivější), záporné pak příznivý stav pro hráče B (čím je hodnota uzlu menší, tím je pro hráče A příznivější). Vítězství, resp. prohry jsou hodnoceny maximálními, resp. minimálními hodnotami uvažovaného číselného intervalu.

Je proto zřejmé, že hráč A si bude vybírat tahy vedoucí ke stavům s maximálními ohodnoceními a hráč B si bude vybírat tahy vedoucí ke stavům s minimálními ohodnoceními. Základní metoda hraní her pracuje na tomto principu a nazývá se Minimax nebo zkráceně MinMax.

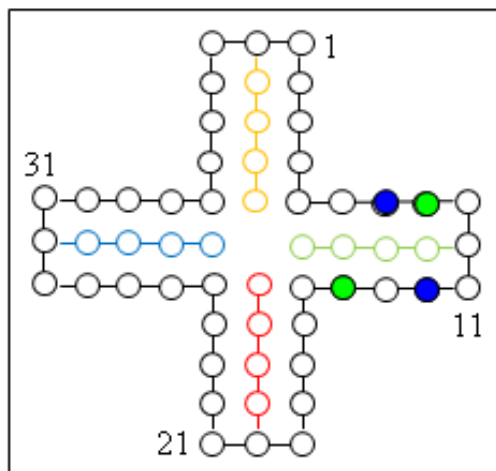
K demonstraci hledání tahu u složité hry použijeme zjednodušené Piškvorky (budeme uvažovat pouze pole 3 x 3). Hráč A obsazuje pole například křížky a hráč B kolečky, na tahu je hráč A. Hráči se postupně střídají a pokládají své symboly na políčka. Cílem je vytvořit řadu tří svých symbolů za sebou: vodorovně, svisle nebo šikmo. Tato řada musí tvořit přímku a nesmí být nikde přerušena soupeřovým symbolem. Kdo takovou řadu vytvoří jako první, vyhrává.

Je zřejmé, že v této velmi jednoduché verzi hry existuje pouze 8 možných řad:



2.4.1.3 Hry s neurčitostí

Hry s neurčitostí jsou většinou hry, ve kterých se hází kostkou. K demonstraci hledání tahu u takové hry použijeme velmi zjednodušenou verzi hry „Člověče nezlob se“. Budeme uvažovat



pouze dva hráče, každý se dvěma kameny na desce. Startovním místem zeleného hráče je pozice 11, startovním místem modrého hráče pak pozice 31. Zelený hráč tak má jeden svůj kámen před svým domečkem a druhý kámen na počátku cesty, modrý hráč má oba své kameny přibližně uprostřed cesty. Hráči střídavě hází kostkou a přesouvají jeden (libovolný) svůj kámen o tolik pozic, kolik ukazuje kostka. Pokud hráč postoupí svým kamenem na pozici obsazenou kamenem protihráče, tak jeho kámen z desky vyhodí (vyhozený kámen se pak vrací na své startovní políčko obvykle až po hodu maximálního čísla na kostce). Do svého domečku postupuje hráč, pokud mu na kostce padne číslo,

které by vedlo k posunu kamene na/přes jeho startovní políčko. Cílem hry je dostat kameny do jejich domečků a vyhrává hráč, který bude mít oba své kameny ve svém domečku jako první.

2.4.2 Metody hraní složitých her dvou hráčů

Jak již bylo zmíněno v předchozí kapitole, je nutné všechny listové uzly grafu ohodnotit tak, aby kladné hodnoty znamenaly příznivé stavy pro hráče A (čím je hodnota vyšší, tím je stav pro hráče A příznivější) a záporné hodnoty pak příznivý stav pro hráče B (čím je hodnota menší, tím je stav pro hráče A nepříznivější). Bylo zde také uvedeno, že hráč A si proto vybírá tahy vedoucí ke stavům s maximálními ohodnoceními, hráč B tahy vedoucí ke stavům s minimálními ohodnoceními a že základní metoda hraní her pracující na uvedeném principu se nazývá metodou Minimax.

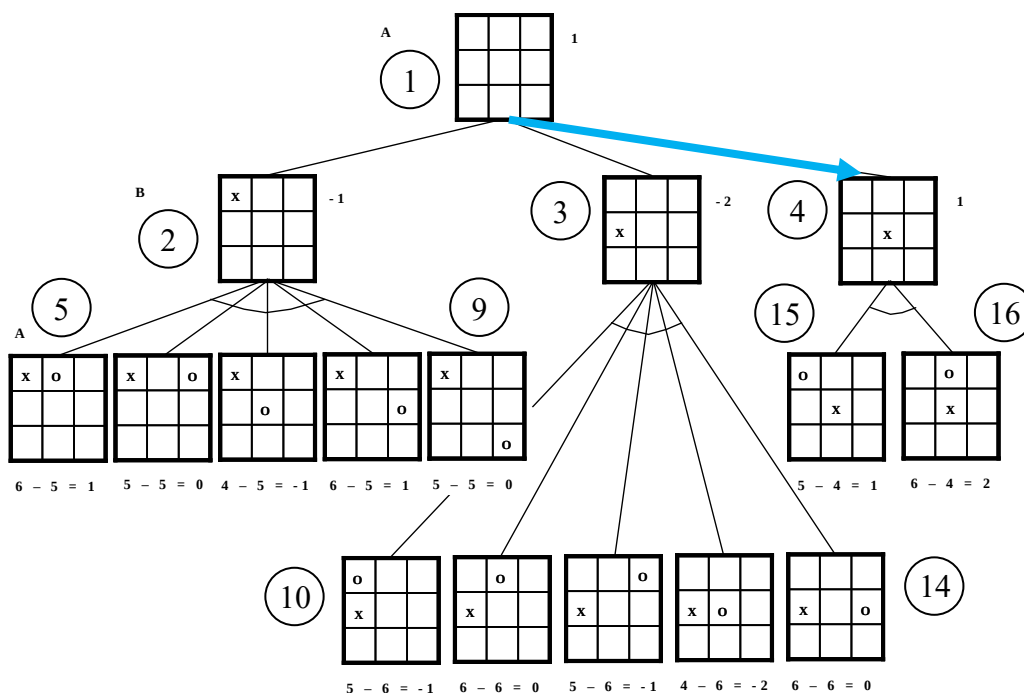
2.4.2.1 Metoda Minimax

Základem metody Minimax je stejnojmenná procedura, která je rekurzivní a zahajuje se, tj. poprvé se volá, když je na tahu hráč A. Vstupními parametry procedury jsou aktuální stav hry X , maximální hloubka prohledávání a informace o hráči, který je právě na tahu (A nebo B). Procedura vrací ohodnocení aktuálního stavu a tah, který k tomuto ohodnocení vede.

Procedura Minimax

1. Je-li uzel X listem (konečným stavem hry, nebo uzlem v maximální hloubce) vrací ohodnocení tohoto uzlu.
2. Je-li na tahu hráč A, tak postupně pro všechny jeho možné tahy (bezprostřední následníky uzlu X , tj. stavy před tahem hráče B) volá proceduru Minimax pro hráče B, vrací maximální z navrácených hodnot a tah, který vede k nejlépe ohodnocenému bezprostřednímu následníkovi (tento tah má význam pouze u kořenového uzlu, kdy představuje nejvýhodnější reálný tah hráče A).
3. Je-li na tahu hráč B, tak postupně pro všechny jeho možné tahy (bezprostřední následníky uzlu X , tj. stavy před tahem hráče A) volá proceduru Minimax pro hráče A a vrací minimální z navrácených hodnot.

Postup při použití procedury Minimax ukážeme na hledání optimálního počátečního tahu pro hru Piškvorky zjednodušenou podle kap. 2.4.1.2 (hráč A používá křížky a hráč B kolečka). Necht' ohodnocení stavu je dáno rozdílem počtu řad dosažitelných hráčem A a počtu řad dosažitelných hráčem B; ohodnocení počátečního stavu je tedy $8 - 8 = 0$. Úplný tah obou hráčů (hloubka = 2) bez zrcadlových variant je ukázán na Obr. 2.37. Zrcadlové stavy jsou takové stavy, které vzniknou buď otočením nebo překlopením nějakého stavu (například první stav na úrovni hráče B, tj. křížek v levém horním rohu, má zrcadlové varianty křížek v pravém horním rohu, křížek v levém dolním rohu a křížek v pravém dolním rohu), které nemají na řešení úlohy žádný vliv.



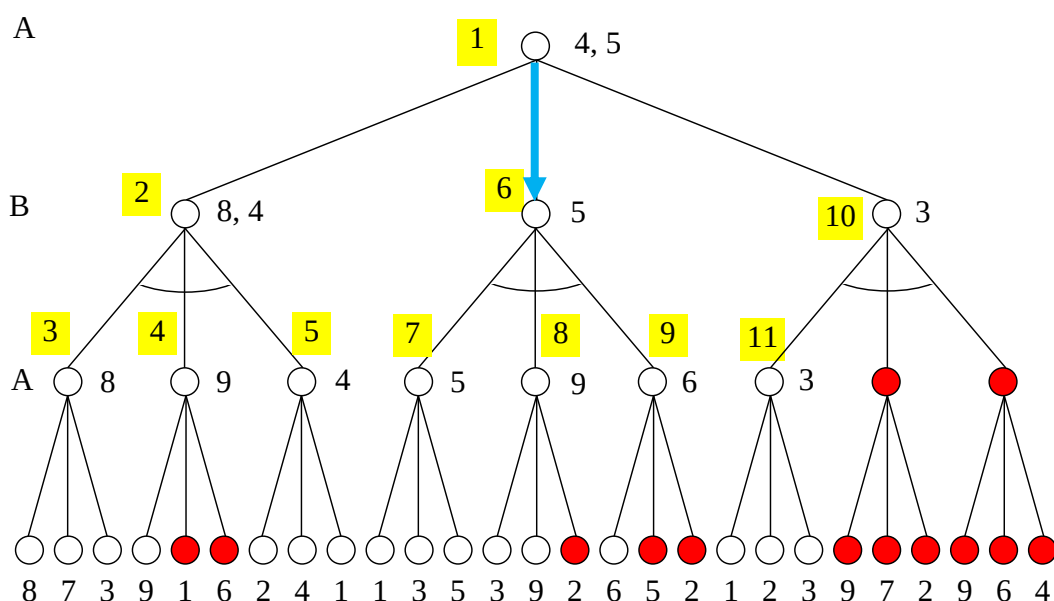
Obr. 2.37 Hledání optimálního tahu z počátečního stavu hry Piškvorky metodou Minimax

Čísla v kroužcích označují stav (pro níže popsanou činnost procedury), čísla u stavů bez kroužků jsou hodnoty ohodnocení uzlů. Parametry procedury jsou stav, hráč, vrácená hodnota a vrácený tah (má smysl pouze pro výchozí stav hráče A). Postup činnosti procedury je následující:

- 1 Minimax(1, A, -, -)
 1. Minimax(2, B, -, -)
 1. Minimax(5, A, 1, 5)
 2. Minimax(6, A, 0, 6)
 3. Minimax(7, A, -1, 7)
 4. Minimax(8, A, 1, 8)
 5. Minimax(9, A, 0, 9)
 - Minimax(2, B, -1, -) návrat - vrací minimum z obdržných hodnot
 2. Minimax(3, B, -, -)
 1. Minimax(10, A, -1, 10)
 2. Minimax(11, A, 0, 11)
 3. Minimax(12, A, -1, 12)
 4. Minimax(13, A, -2, 13)
 5. Minimax(14, A, 0, 14)

Minimax(3,B, -2, -)	návrat - vrací minimum z obdržených hodnot
3. Minimax(4,B, -, -)	
1. Minimax(15, A, 1, 15)	
2. Minimax(16, A, 2, 16)	
Minimax(4,B, 1, -)	návrat - vrací minimum z obdržených hodnot
Minimax(1, A, 1, 4)	návrat - vrací maximum obdržených hodnot a optimální tah, který je na obrázku zvýrazněný modrou šipkou

Procedura Minimax je sice jednoduchá, ale při jejím použití dochází ke zbytečnému vyšetřování velké části AND/OR grafu z důvodu, který je blíže vysvětlen na Obr. 2.38. Na následujících obrázcích znamenají čísla ve žlutých podkladech pořadí volání procedury Minimax. Čísla bez žlutého podkladu znamenají (měnící se) ohodnocení příslušného uzlu.



Obr. 2.38 Příklad na zbytečná vyšetřování některých uzlů AND/OR grafu metodou Minimax

Hráč A (uzel 1) volá proceduru Minimax na svůj první tah (uzel 2). Hráč B volá tuto proceduru na svůj první tah (uzel 3). Hráč A (uzel 3) volá proceduru Minimax postupně na všechny své možné tahy, a protože všichni jeho bezprostřední následníci jsou uzlovými listy, tak procedura Minimax postupně vrací ohodnocení těchto listů. Hráč A si vybere maximální hodnotu z jejich ohodnocení (tj. hodnotu 8). Hráč B (uzel 2) tuto hodnotu akceptuje a volá proceduru Minimax na svůj druhý tah (uzel 4). První bezprostřední následník tohoto uzlu (listový uzel) vrací hodnotu 9. Je zřejmé, že prohledávání dalších následníků uzlu 4 je zbytečné, protože již nyní je jasné, že tento uzel vrátí hodnotu ≥ 9 (hráč A vybírá maximum). Hráč B (uzel 2), který si vybírá tah s minimálním ohodnocením, si tento tah určitě nevybere, protože ohodnocení jeho prvního tahu je pro něj lepší. Hráč B (uzel 2) pak volá proceduru Minimax na svůj poslední tah (uzel 5). Hráč A (uzel 5) pak opět volá postupně proceduru Minimax na všechny své bezprostřední následníky a z vrácených hodnot vybere (vrátí) hodnotu maximální, tj. hodnotu 4. Protože tato hodnota je menší než hodnota 8, hráč B (uzel 2) si vybere a vrátí tuto hodnotu. Hráč A (uzel 1) hodnotu 4 akceptuje a volá proceduru Minimax na svůj druhý možný tah (uzel 6). Další postup pro tento tah je velmi podobný postupu pro první tah. Hráč B (uzel 6) volá postupně proceduru Minimax na své bezprostřední následníky (uzly 7, 8 a 9) a vrátí minimum z vrácených hodnot,

tj. číslo 5. Listové uzly, které jsou na obrázku označené červenou barvou, se opět vyšetřují zbytečně, z důvodů popsaných výše. Protože hodnota uzlu 6 je větší než hodnota uzlu 1 ($5 > 4$) a hráč A si vybírá maximum, akceptuje hráč A (uzel 1) tuto hodnotu (druhý tah je pro něj výhodnější, než tah první) a volá proceduru Minimax na svůj třetí tah (uzel 10). Hráč B (uzel 10) volá proceduru Minimax na svůj první tah (uzel 11) a od tohoto uzlu dostane navracenu hodnotu 3. Proto je již v tomto okamžiku zřejmé, že hráč B (uzel 10), který si vybírá minimum, vrátí kořenovému uzlu hodnotu ≤ 3 a hráč A (uzel 1) si tento tah nevybere. Další vyšetřování tahů hráče B (uzlu 10) je tak zbytečné.

Zbytečně vyšetřované uzly metodou Minimax jsou na obrázku zvýrazněny červenou barvou; je jich přibližně 30 % (13 z celkového počtu 40 uzlů).

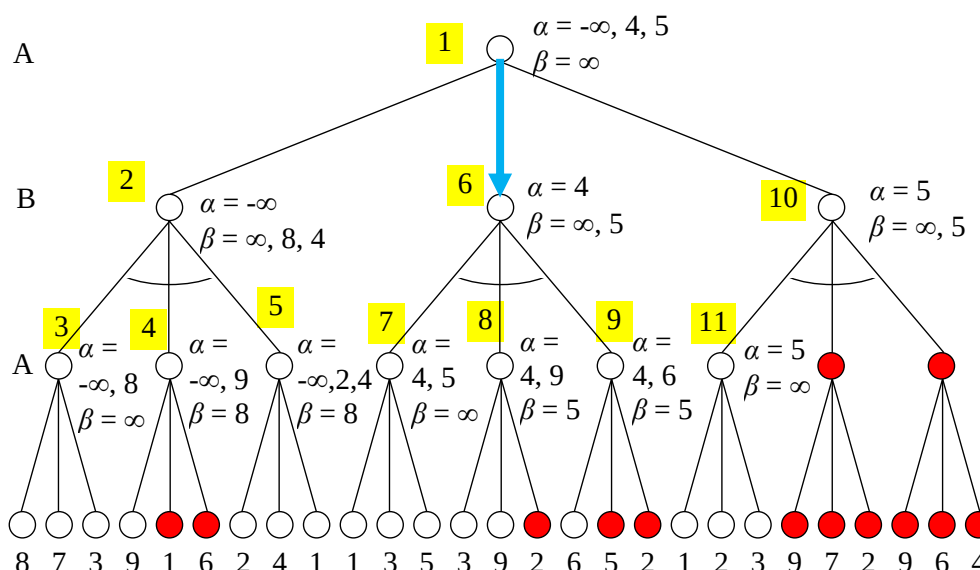
2.4.2.2 Alfa-beta řezy

Uvedenému zbytečnému vyšetřování lze zabránit velmi jednoduchým způsobem, a to zavedením tzv. alfa-beta řezů. Alfa řezy zabrání zbytečnému vyšetřování tahů hráče A, beta řezy pak zbytečnému vyšetřování tahů hráče B. Procedura Minimax se zavedením alfa-beta řezů se pak nazývá Alfa-beta procedura. Protože tato procedura vychází z procedury Minimax, je této proceduře také velmi podobná. Vstupními parametry procedury jsou opět stav hry X , maximální hloubka prohledávání a informaci o hráči, který je právě na tahu (A nebo B), používá však navíc dva parametry označené standardně symboly α a β , které se při jejím prvním volání nastavují na hodnoty $\alpha = -\infty$, $\beta = \infty$ (v praxi na minimum a maximum). Procedura je opět rekurzivní, poprvé se volá, když je na tahu hráč A, vrací ohodnocení aktuálního stavu a tah, který k tomuto ohodnocení vede.

Procedura Alfa-beta

1. Je-li uzel X listem (konečným stavem hry nebo uzlem v maximální hloubce) vrací ohodnocení tohoto uzlu.
2. Je-li na tahu hráč A:
 - a) Dokud platí nerovnost $\alpha < \beta$, tak postupně pro první/další tah (tj. pro prvního/dalšího bezprostředního následníka uzlu X) volá proceduru Alfa-beta s aktuálními hodnotami parametrů α a β . Po každém vyšetřovém tahu nastaví hodnotu parametru α na maximum z aktuální a navracené hodnoty.
 - b) Je-li $\alpha \geq \beta$, nebo nemá-li uzel X žádného dalšího bezprostředního následníka, vrací aktuální hodnotu parametru α a tah, který vede k nejlépe ohodnocenému bezprostřednímu následníkovi (tento tah má opět význam pouze u kořenového uzlu, kdy představuje nejvýhodnější tah hráče A; pokud má ale stejné ohodnocení více možných tahů, je relevantní pouze tah, který byl takto ohodnocen jako první z těchto tahů !!!).
3. Je-li na tahu hráč B:
 - a) Dokud platí nerovnost $\alpha < \beta$, tak postupně pro první/další tah (tj. pro prvního/dalšího bezprostředního následníka uzlu X) volá proceduru Alfa-beta s aktuálními hodnotami parametrů α a β . Po každém vyšetřovém tahu nastaví hodnotu parametru β na minimum z aktuální a navracené hodnoty.
 - b) Je-li $\alpha \geq \beta$, nebo nemá-li uzel X žádného dalšího bezprostředního následníka, vrací aktuální hodnotu parametru β

Na Obr. 2.39 je ukázán princip alfa a beta řezů při vyšetřování grafu z Obr. 2.38.



Obr. 2.39 Princip práce procedury Alfa-beta (alfa-beta řezů)

Při zavolání procedury Alfa-beta na počáteční uzel se nastaví hodnoty $\alpha = -\infty$, $\beta = \infty$. Hráč A (uzel 1) volá proceduru Alfa-beta na svůj první tah (uzel 2) s hodnotami $\alpha = -\infty$, $\beta = \infty$, hráč B (uzel 2) volá tuto proceduru na svůj první tah (uzel 3), opět s hodnotami $\alpha = -\infty$, $\beta = \infty$. Hráč A (uzel 3) volá proceduru Alfa-beta postupně na všechny své možné tahy. Protože všichni jeho bezprostřední následníci jsou uzlovými listy, tak procedura pouze postupně vrací ohodnocení těchto listů a hráč A upravuje hodnotu své proměnné α na maximum z původní a navracených hodnot. Poznamenejme, že hodnota proměnné β je stále větší než hodnota proměnné α . Hráč A (uzel 3) nakonec vrátí hodnotu 8. Hráč B (uzel 2) tuto hodnotu akceptuje a nastaví na tuto novou hodnotu proměnné $\beta = \min(8, \infty)$. Zdůrazněme, že hodnota proměnné α se v tomto uzlu nemění ($\alpha = -\infty$). Hráč B (uzel 2) pak volá proceduru Alfa-beta na svůj druhý tah (uzel 4) s aktuálními hodnotami proměnných $\alpha = -\infty$, $\beta = 8$. Hráč A (uzel 4) volá proceduru Alfa-beta na svůj první tah. Vracená hodnota je 9 a tato hodnota je novou hodnotou α uzlu 4. V tomto okamžiku je však hodnota proměnné α ($\alpha = 9$) větší než hodnota proměnné β ($\beta = 8$) a vyšetřování uzlu 4 je ukončeno (alfa řez). Procedura vrací hodnotu 9 a procedura příslušející uzlu 2 tuto hodnotu ignoruje (hráč B si vybírá minimum). Hráč B pak volá proceduru Alfa-beta na svůj třetí a poslední tah A (uzel 5) s aktuálními hodnotami proměnných $\alpha = -\infty$, $\beta = 8$. Hráč A (uzel 5) volá postupně proceduru Alfa-beta na všechny své bezprostřední následníky a postupně upravuje aktuální hodnotu proměnné $\alpha = -\infty$, 2, 4 (Hodnota $\beta = 8$ je stále větší než hodnota proměnné α). Konečnou hodnotu proměnné α , tj. hodnotu 4, vrací hráč A (uzel 5) hráči B (uzlu 2). Hráč B (uzel 2) přepíše touto hodnotou hodnotu proměnné β ($\beta = 4$) a tuto hodnotu vrátí hráči A (uzlu 1). Nová hodnota proměnné α uzlu 1 je tak 4. Hráč A (uzel 1) volá proceduru Alfa-beta na svůj druhý tah (uzel 6) s aktuálními hodnotami $\alpha = 4$, $\beta = \infty$ a postup uvedený výše se v podstatě opakuje. Aktuální hodnota proměnné α hráče A (uzlu 1) se pak zvýší na 5 ($\alpha = 5$).

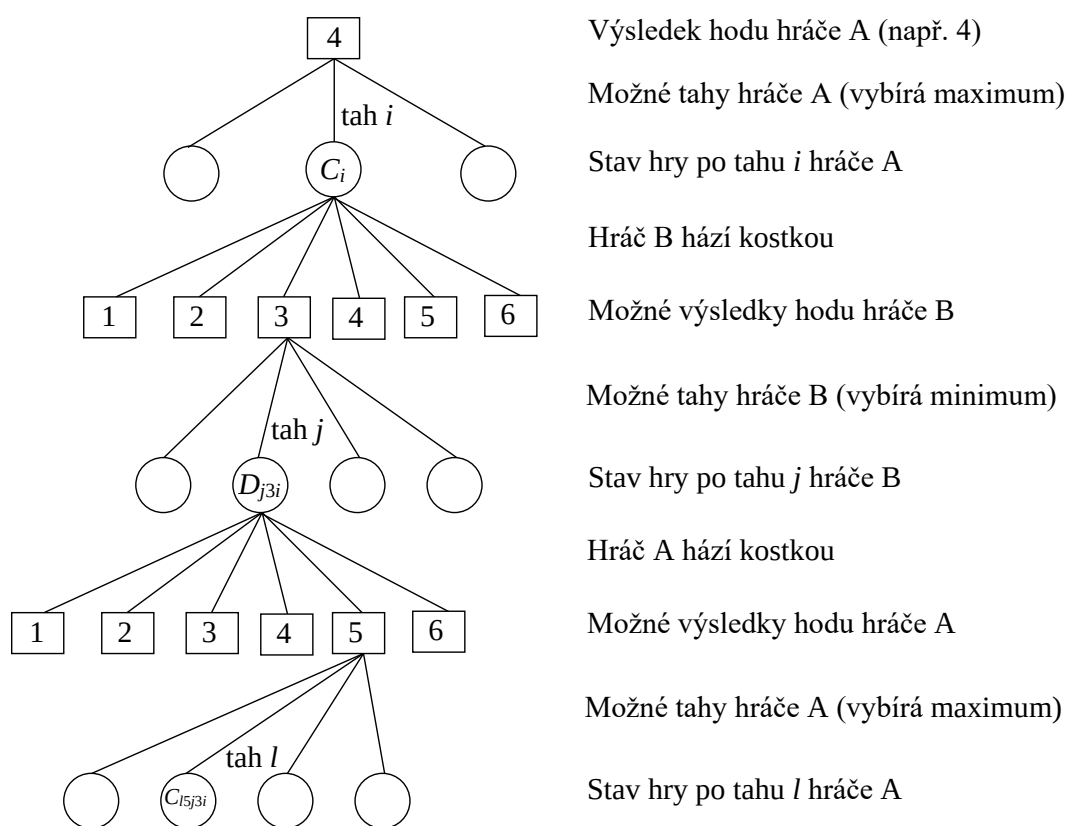
Za povšimnutí stojí vyšetřování uzlů 10 a 11. Hráči B (uzlu 10) jsou hráčem A (uzlem 1) předány hodnoty $\alpha = 5$, $\beta = \infty$. Hráč B (uzel 10) předá tyto hodnoty pro svůj první tah hráči A (uzlu 11). Hráč A (uzel 11) volá se stejnými hodnotami proceduru Alfa-beta postupně na všechny své bezprostřední následníky. Protože tyto následníci vrací hodnoty menší než 5, hodnota α hráče A (uzel 11) se nezmění, a nakonec je vrácena hráči B (uzlu 10). Ten upraví hodnotu své proměnné $\beta = 5$ a protože v tomto okamžiku se hodnota β rovná hodnotě $\alpha = 5$,

vyšetřování uzlu 10 okamžitě končí (beta řez). Hráči A (uzlu 1) je vrácena hodnota 5 a protože tato hodnota je stejná jako aktuální hodnota α uzlu 1, tak se již nic nemění (ukazatel na nejlepší tah hráče A (uzlu 1) ukazuje stále na druhý tah (uzel 6), který jako první tuto hodnotu uzlu 1 vrátil!!!).

2.4.2.3 Metody hraní her s neurčitostí

Existují hry, které opět hrají dva hráči, kteří se po jednotlivých tazích hry pravidelně střídají, mají úplnou informaci o stavu hry, hrají čestně a oba si přejí zvítězit. Při hře však používají kostku, resp. kostky a do hry tak vstupuje neurčitost/náhoda. Hráči neznají výsledky hodů kostkou a nemohou si vybírat minimální nebo maximální hodnoty, jako v proceduře Minimax.

Hráč A proto bude postupovat při hodnocení každého možného stavu po svém hodu kostkou složitějším způsobem (Obr. 2.40).



Obr. 2.40 Výběr nejlepšího tahu pro hry s kostkou

Hráč A vyjde z úvahy, že po následujícím hodu hráčem B by si tento hráč vybral tah do stavu s minimálním ohodnocením D_{jki} a že tedy hodnotou uzlu C_i by byla minimální hodnota z možných hodnot jednotlivých stavů po hodu hráče B. Proto hráč A může pracovat pouze s ohodnocením očekávaným, které se označuje jako *expectimin* (očekávané minimum) a pro stav C_i má tvar

$$expectimin(C_i) = \sum_k P(h_k) \cdot \min_j (D_{jki})$$

kde $P(h_k)$ je pravděpodobnost výsledku h_k (hodnoty na horní straně po hodu kostky). Pro standardní kostku jsou hodnoty $h_k \in \{1, 2, 3, 4, 5, 6\}$ a pravděpodobnosti $P(h_k) = 1/6$.

Ohodnocení *expectimin* je tedy dáno součtem ohodnocení po všech možných výsledcích hodu kostky, kdy každé jednotlivé ohodnocení je dáno součinem pravděpodobnosti daného výsledku hodu kostky a následného minimálního ohodnocení stavu, kterého je možné po daném hodu dosáhnout. Hráč A si pak vybírá tah do stavu C_i s maximální hodnotou *expectimin*.

Podobným způsobem se postupuje při vyšetřování očekávaného ohodnocení uzlu D_{jki} , který pro jednoduchost označme pouze jako D_j . Protože hráč A vybírá maximum z možných ohodnocení, označuje se toto ohodnocení jako *expectimax* (očekávané maximum):

$$expectimax(D_j) = \sum_k P(h_k) \cdot \max_l (C_{lkj})$$

Procedura Expectiminimax

Vstupními parametry procedury jsou (stejně jako u procedury Minimax) stav hry X , maximální hloubka prohledávání a informaci o hráči, který je právě na tahu (A nebo B). Procedura vrací ohodnocení aktuálního stavu a tah, který k tomuto ohodnocení vede.

1. Je-li uzel X listem (konečným stavem hry, nebo uzlem v maximální hloubce) procedura vrací ohodnocení tohoto uzlu.
2. Je-li na tahu hráč A, tak postupně pro všechny jeho možné tahy (bezprostřední následníky uzlu X) volá proceduru Expectiminimax pro hráče B, vrací maximální hodnotu z navrácených hodnot *expectimin* a tah, který k nejlépe ohodnocenému bezprostřednímu následníkovi vede (tento tah má opět význam pouze u kořenového uzlu, kdy představuje nejvýhodnější reálný tah hráče A).
3. Je-li na tahu hráč B, tak postupně pro všechny jeho možné tahy (bezprostřední následníky uzlu X) volá proceduru Expectiminimax pro hráče A a vrací minimální hodnotu z navrácených hodnot *expectimax*.

Praktické použití procedury Expectiminimax si ukážeme na příkladu zjednodušené hry Člověče nezlob se, tak jak bylo naznačeno v kap. 2.4.1.3.

Každý stav hry popíšeme seznamem $[Z, M, h]$, kde Z značí seznam pozic zelených figurek, M seznam pozic modrých figurek a h ohodnocení stavu. Stavy hry můžeme ohodnotit například takto (hráč na tahu, tj. hráč A, hraje se zelenými figurkami):

$$h = 15 \cdot (dZ - dM) + 5 \cdot (pZ - pM) + 2 \cdot (ddZ - ddM) + (oZM - oMZ)$$

kde jednotlivé symboly značí počty:

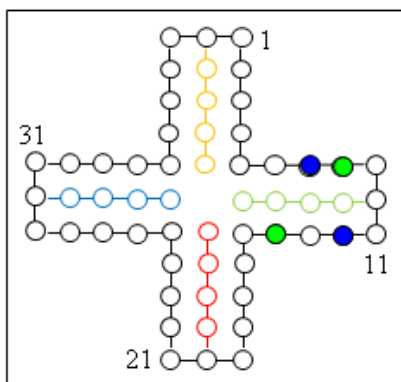
dZ/dM	zelených/modrých figurek v domečkích
pZ/pM	zelených/modrých figurek na hracím poli mimo domeček
ddZ/ddM	zelených/modrých figurek, které se mohou dostat po jednom hodu kostky do domečku
oZM/oMZ	bezprostředních ohrožení modrých/zelených figurek zelenými/modrými figurkami

Výchozí stav uvedený na obrázku Obr. 2.41 má ohodnocení

$$h = 15 \cdot (0 - 0) + 5 \cdot (2 - 2) + 2 \cdot (1 - 0) + (0 - 2) = 0$$

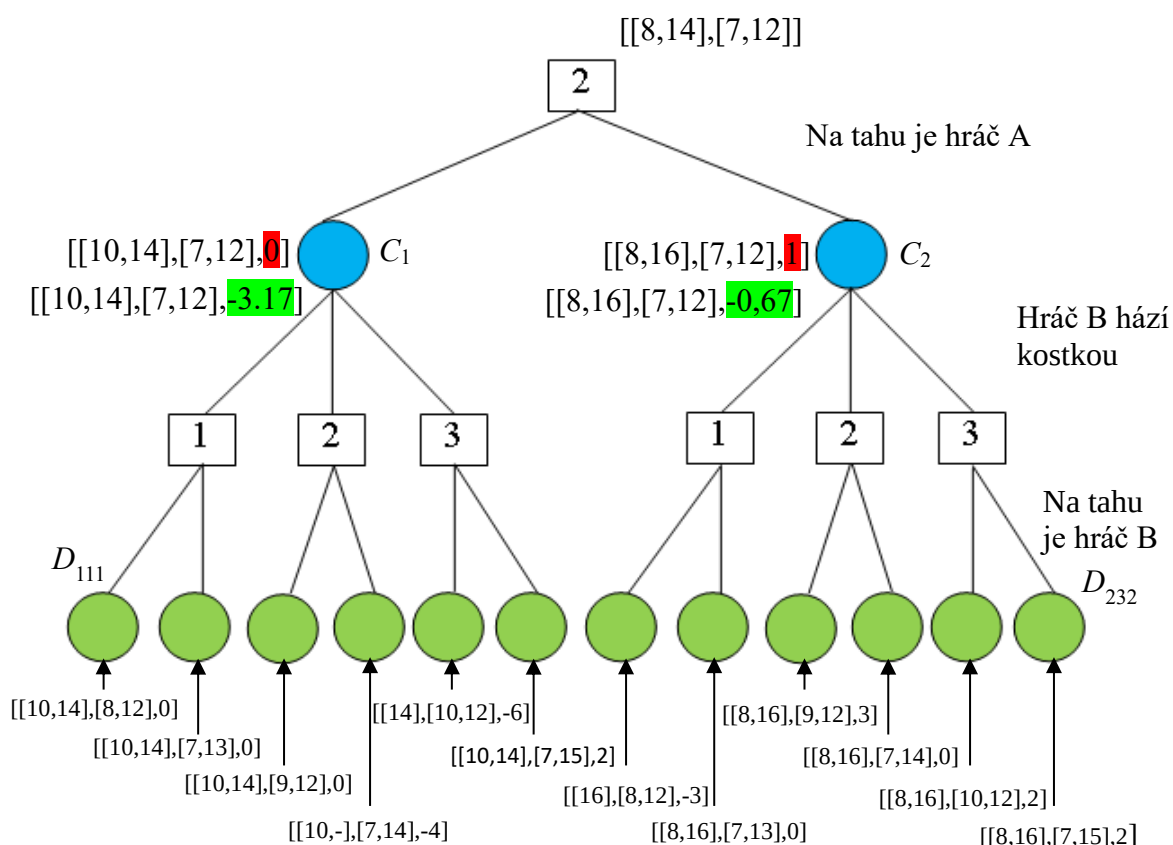
a je popsán seznamem $[[8,14],[7,12],0]$.

Uvažujme šestistrannou hrací kostku s čísly 1,1,2,2,2,3, pro kterou jsou pravděpodobnosti hodů jednotlivých číslic $P(h_1) = 2/6$, $P(h_2) = 3/6$, $P(h_3) = 1/6$.



Nechť hráč na tahu (hráč A) právě hodil kostkou číslo 2. Musí se nyní rozhodnout mezi dvěma možnými tahy: první tah vede na stav $C_1 = [[10,14],[7,12],0]$ a druhý tah na stav $C_2 = [[8,16],[7,12],1]$. Hráč na tahu si vybírá tah do lépe ohodnoceného stavu, a proto, pokud by neuvažoval další průběh hry, by si vybral tah do stavu $C_2 = [[8,16],[7,12],1]$. Tato ohodnocení uzlů C_1 a C_2 jsou na Obr. 2.41 zvýrazněna červenou barvou.

Pokud bude hráč A vyšetřovat možné stavy i po hodu hry hráče B, musí nejprve ohodnotit všechny možné stavy po hodu kostkou a výběrem tahu hráče B.



Obr. 2.41 Použití procedury Expectiminimax na řešení zjednodušené hry Člověče nezlob se

Výpočet nového ohodnocení stavů C_1 a C_2 :

$$expectimin(C_1) = \sum_k P(h_k) \cdot \min_j (D_{jki}) = \frac{2}{6} \cdot (0) + \frac{3}{6} \cdot (-4) + \frac{1}{6} \cdot (-7) = -3,17$$

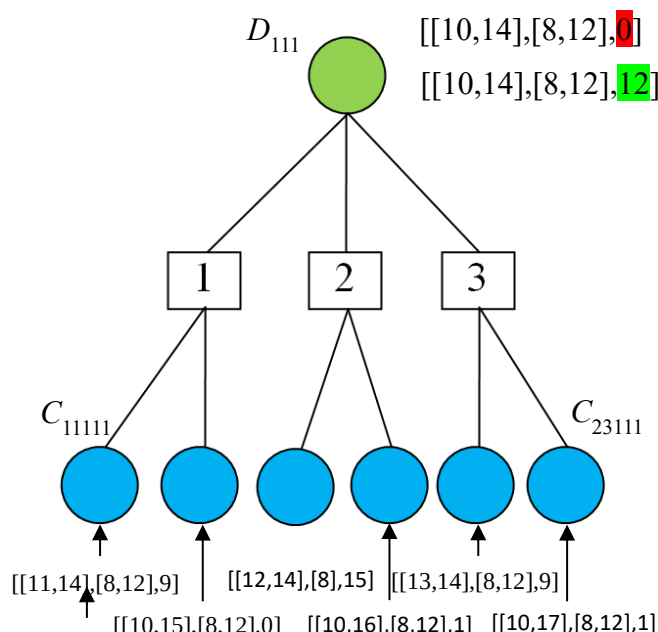
$$expectimin(C_2) = \sum_k P(h_k) \cdot \min_j (D_{jki}) = \frac{2}{6} \cdot (-3) + \frac{3}{6} \cdot (0) + \frac{1}{6} \cdot (2) = -0,67$$

Tato nová ohodnocení uzlů C_1 a C_2 jsou na Obr. 2.41 zvýrazněna zelenou barvou.

Při uvažování dalšího tahu se postupně ohodnocují dolní zelené stavy z Obr. 2.41, pak se přepočítají hodnoty předchozích modrých stavů atd. Na Obr. 2.42 je ukázáno přepočítání

ohodnocení stavu D_{111} (původní ohodnocení je zvýrazněno opět červenou barvou a nové ohodnocení zelenou barvou).

$$expectimax(D_{111}) = \sum_k P(h_k) \cdot \max_f (C_{fk111}) = \frac{2}{6} \cdot (9) + \frac{3}{6} \cdot (15) + \frac{1}{6} \cdot (9) = 12$$



Obr. 2.42 Pokračování ukázky použití procedury Expectiminimax z Obr. 2.41

2.4.3 Metody hraní her n hráčů

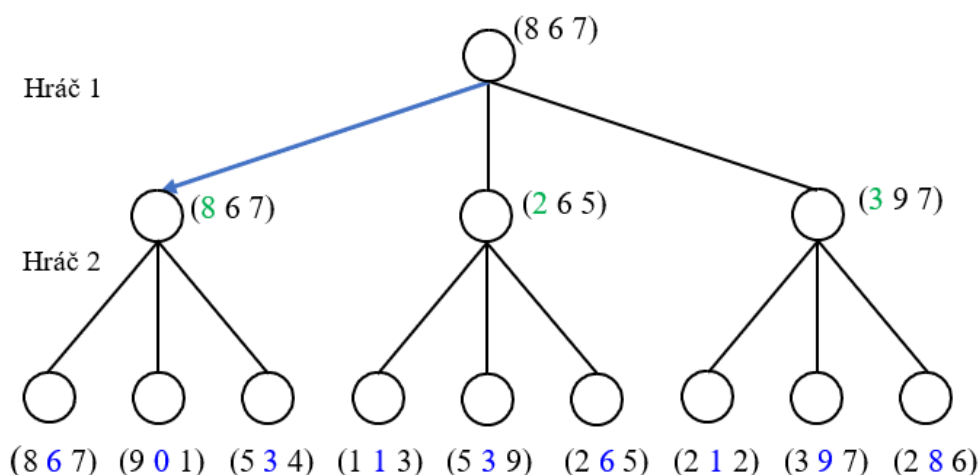
Pokud hraje hru více než dva hráči, nelze předchozí postup, tj. metodu Minimax použít. Při řešení těchto her, tj. při hledání optimálního tahu pro prvního hráče, se předpokládá, že hráči označení indexy $i = 1 \dots n$ se pravidelně střídají (druhý po prvním, třetí po druhém až poslední po předposledním a pak první po posledním atd.). Každý hráč chce vyhrát a všichni mají úplný přehled o aktuálním stavu hry.

Jednotlivé stavy hry budeme nyní ohodnocovat jedinou hodnotící funkcí aplikovanou na každého hráče zvlášť, přičemž hodnota této funkce musí být tím vyšší, čím je daná pozice pro hodnoceného hráče výhodnější.

U každého stavu je pak jeho hodnocení dáno seznamem hodnot hodnotící funkce pro jednotlivé hráče. Například pro hru se třemi hráči ohodnocení stavu (8 3 5) znamená, že daný stav má ohodnocení 8 pro prvního hráče, 3 pro druhého hráče a 5 pro třetího hráče.

Každý hráč si pak vybírá stav s ohodnocením, které je pro něho nejvýhodnější. Příklad pro tři hráče, kdy hráč 1 zvažuje situaci, která může nastat po tahu hráče 2, je ukázán na Obr. 2.43.

Hráč 2 si z první trojice následníků vybere stav (8 6 7), z druhé trojice stav (2 6 5) a z třetí trojice (3 9 7), protože jsou tyto stavy pro hráče 2 nejvýhodnější (hodnoty výhodné pro hráče 2 jsou v seznamech ohodnocení označeny modrými číslicemi). Z vybraných stavů hráčem 2 si hráč 1 vybere stav (8 6 7), protože je tento stav pro hráče 1 nejvýhodnější (hodnoty výhodné pro hráče 1 jsou v seznamech ohodnocení označeny zelenými číslicemi). Nejvýhodnější tah hráče 1 je označen modrou šipkou.



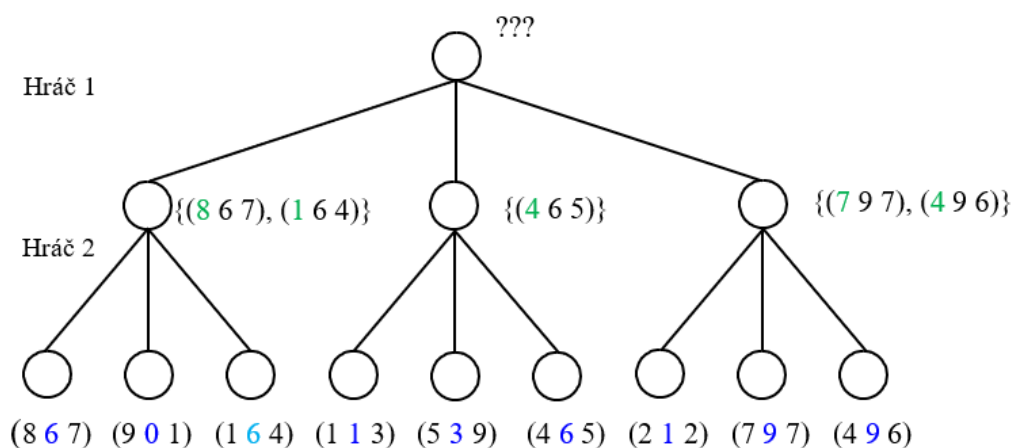
Obr. 2.43 Výběr optimálního tahu procedurou Max^n

Zobecněním procedury Minimax pro n hráčů je procedura nazývaná procedura Max^n . Proceduru volá hráč na tahu, vstupním parametrem je aktuální stav (označený jako stav/uzel X) a výstupním parametrem je ohodnocení tohoto stavu.

Procedura Max^n

1. Je-li uzel X listem (konečným stavem hry nebo uzlem v maximální hloubce), tak procedura vrací ohodnocení tohoto uzlu ($v_1, v_2, \dots, v_n$).
2. Jinak tato procedura pro aktuálního hráče i volá rekurzivně sama sebe na všechny své bezprostřední následníky (tj. stavy před tahem následujícího hráče j ($j = i + 1$ a pokud $j > n$, pak $j = 1$) a vrací ohodnocení, které je pro daného hráče, tj hráče i , nejvýhodnější.

Problémem procedury Max^n je nejednoznačnost hodnocení uzlu (stavu hry) v případech, kdy stejné maximální ohodnocení má několik jeho bezprostředních následníků. Tento problém (částečně) řeší procedura Soft-Max^n , která vrací množiny se stejnými ohodnoceními pro příslušného hráče, a tím umožňuje hráči na tahu informovanější rozhodování (Obr. 2.44):



Obr. 2.44 Výběr optimálního tahu procedurou Soft-Max^n

Pokud hráči 1 stačí k vítězství 4 body, pak si tento hráč zřejmě vybere tah rovně dolů k uzlu (4 6 5). V opačném případě může buď riskovat, tj. zvolit tah vlevo (hráč 2 si však může vybrat tah (1 6 4), který je pro hráče 1 velmi nevýhodný), nebo sázet raději na jistotu, tj. zvolit tah vpravo.

2.5 Metody řešení úloh inspirované přírodou

Úlohami, které lze řešit metodami/algoritmy inspirovanými přírodou, jsou prakticky výhradně optimalizační úlohy. Prvním algoritmem inspirovaným přírodou byl genetický algoritmus, publikovaný v roce 1975. Dalších několik algoritmů bylo navrženo v průběhu devadesátých let minulého století, většina pak ještě později, a proto tyto algoritmy nebyly do klasické UI zařazené. Ke dnešnímu dni je známo několik desítek metod/algoritmů inspirovaných biologií (často chováním hejna/roje/smečky živočichů), fyzikou, chemií apod.

Dále si podrobněji popíšeme tři nejstarší a nejznámější metody/algoritmy, a to

- Genetické algoritmy (GA, *Genetic Algorithms*)
- Optimalizace mravenčí kolonií (ACO, *Ant Colony Optimization*)
- Optimalizace hejnem částic (PSO, *Particle Swarm Optimization*)

2.5.1 Genetické algoritmy

2.5.1.1 Biologická inspirace

Každý živočišný druh má ve svých buňkách charakteristický počet útvarů složených z DNA (kyseliny deoxyribonukleové) a proteinů (organických makromolekulárních látek), které se nazývají chromozomy. Každý chromozom obsahuje tisíce až miliony genů, přičemž každý gen může nabývat různých forem, které se nazývají alely.

Soubor všech různých alel se nazývá genotyp a určuje soubor znaků jedince, tzv. fenotyp, který se pak projeví navenek (krevní skupina, barva očí atd.).

Buňky člověka mají 46 chromozomů, které tvoří 23 párů. V každém z těchto párů je jeden chromozom mateřského a druhý otcovského původu. Výjimku tvoří pohlavní buňky (ve vajíčkách a spermiích), které obsahují pouze jeden náhodně vybraný chromozom příslušného páru (princip dědičnosti).

Po oplodnění vytvoří odpovídající si chromozomy pár, a proto každý potomek má ve svých buňkách opět 23 párů chromozomů. Přitom není důležité, který gen je v těchto párech převzat od matky a který od otce.

Příležitostně, vlivem neobvyklých fyzikálních, chemických nebo biologických efektů, např. kosmického záření, může být některý gen náhodně změněn a tato změna se nazývá mutací.

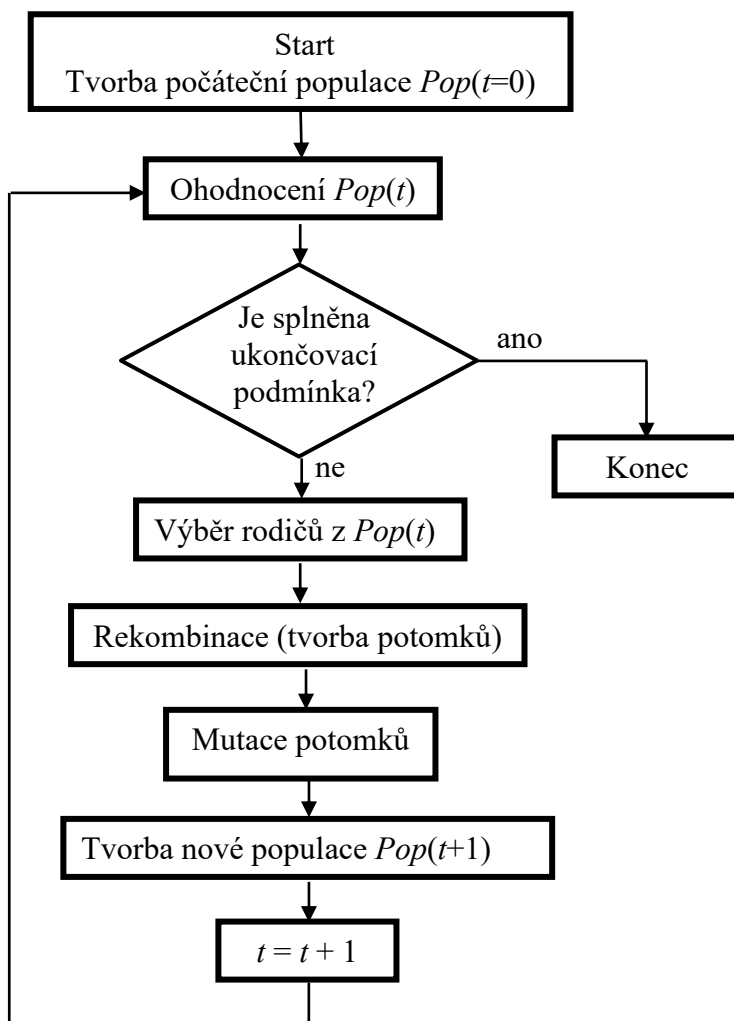
Jedinci, kteří se lépe adaptují na okolní prostředí, mají větší šanci na přežití a tím na plození dalších potomků (Darwinova teorie vývoje, evoluce).

2.5.1.2 Princip činnosti genetického algoritmu

Chromozomy v GA jsou obvykle reprezentovány řetězci znaků, ale obecně jsou možné i jiné jejich reprezentace, například grafy, matice ap. Kombinace jednotlivých znaků v chromozomu nese informaci, která se nazývá řešením dané optimalizační úlohy, přestože toto „řešení“ může mít ke skutečnému (tj. k optimálnímu) řešení velmi daleko.

Populaci v GA tvoří předem daný počet chromozomů. Tento počet je závislý na řešené úloze, obvykle jde o desítky až tisíce jedinců. Jedinci počáteční/výchozí populace se generují náhodně, může se však samozřejmě využít libovolná dostupná heuristika, která umožní generování jedinců s počátečními hodnotami (tj. řešeními) blízkými optimálnímu řešení.

Cílem GA je nalézt chromozom, který je (sub)optimálním řešením dané optimalizační úlohy. Programový model genetického algoritmu je ukázán na Obr. 2.45.



Obr. 2.45 Programový model genetického algoritmu

Hodnocení jedince/chromozomu spočívá ve vyhodnocení, jak se řešení představované tímto jedincem liší od správného/optimálního řešení dané úlohy (*fitness of solution*).

Funkce, jejíž hodnota udává kvalitu řešení chromozomu, se nazývá fitness funkce. Hodnota této funkce se musí zvyšovat s kvalitou řešení, tj. musí být tím vyšší, čím je řešení představované ohodnocovaným chromozomem bližší správnému řešení.

Fitness funkce mají výrazný vliv jak na kvalitu řešení, tak i na délku výpočtu, a proto jejich návrhu je nutné věnovat velkou pozornost.

Kvalita populace je dána ohodnocením populace, což je průměr hodnot fitness funkcí všech jedinců populace.

Pokud některý jedinec představuje správné/optimální řešení (což však u některých úloh nelze zjistit!), výpočet samozřejmě okamžitě končí. Jinak se výpočet GA ukončí, pokud se kvalita populace delší dobu nezvyšuje, nebo když bylo dosaženo předem stanoveného maximálního počtu iterací (vytváření nových populací). Řešení problému v těchto případech představuje nejlépe hodnocený chromozom.

Výběr rodičů se provádí na základě ohodnocení všech jedinců populace fitness funkcí, a to buď podle jejich proporcí, nebo podle jejich pořadí.

Nejjednodušším výběrem rodičů je výběr elity, kdy se k rodičovství vybere potřebný počet nejlépe ohodnocených jedinců. I když je tento přístup jednoduchý a zdá se být i nejvýhodnější, často vede k uváznutí řešení v lokálním extrému.

Dalším jednoduchým výběrem je turnajový výběr, kdy z náhodně vybraných jedinců zvítězí, tj. vybírá se jako rodič, nejlépe ohodnocený jedinec. Počet „turnajů“ se samozřejmě musí rovnat potřebnému počtu rodičů. V turnajovém výběru dostávají šanci stát se rodiči i hůře ohodnocení jedinci, protože některé turnaje mohou být díky náhodnému výběru „slaběji“ obsazené.

Rekombinace (křížení) produkuje nové jedince z informací obsažených v genech rodičů a spočívá ve vzájemné výměně částí chromozomů. Místa, od kterých změny začínají, se nazývají místy křížení a bývají vybírána náhodně. Konkrétních přístupů ke křížení je celá řada a závisí mj. i na použité reprezentaci jedinců.

Nejjednodušším a nepoužívanějším křížením je tzv. jednobodové křížení: Stanoví se náhodně místo křížení – pro m genů v chromozomu může toto místo nabývat hodnot 1 až $(m-1)$:

	← místo křížení
1. rodič:	a b c d e f g h i j k
2. rodič:	a b c d e f g h i j k
1. potomek:	a b c d e f g h i j k
2. potomek:	a b c d e f g h i j k

Potomci pak mají vzájemně zaměněné části chromozomů rodičů. Poznamenejme, že uvedený způsob křížení není použitelný obecně, nelze jej použít například pro permutační problémy, jako je problém obchodního cestujícího.

Mutace spočívá ve změně hodnoty náhodně vybraného genu náhodně vybraného potomka. Pravděpodobnost mutace je obvykle velmi nízká, například $P_m = 1/n$, kde n je počet jedinců v populaci. U binárních hodnot genů spočívá mutace ve změně hodnoty bitu, u genů s reálnými hodnotami se k hodnotě genu přičte/odečte malá náhodná reálná hodnota, u permutačních problémů se vzájemně vymění hodnoty náhodně vybraných genů.

Pro tvorbu nové populace se používají tři modely:

- Generační model: všichni jedinci původní populace jsou v nové populaci nahrazeni potomky.
- Inkrementační model: v nové populaci je nahrazen potomkem jediný jedinec původní populace.
- Modely s překrytím generací: v nové populaci je nahrazena potomky část jedinců původní populace; je zřejmé, že předchozí dva typy modelů jsou extrémními případy tohoto modelu.

Postup činnosti GA si ukážeme na triviálním příkladu hledání hodnoty proměnné x , pro kterou funkce $y = 5 - x$ prochází nulovou hodnotou.

Budeme předpokládat, že chromozomy mají čtyři binární geny (jejich hodnotou je kladné celé číslo x), že populace obsahuje čtyři chromozomy, že geny chromozomů počáteční populace mají hodnoty 1100, 0011, 0111, 1001, a že fitness funkce je definována takto: $f = 8 - abs(y)$.

Použijeme turnajový výběr rodičů (potřebujeme čtyři rodiče, tj. 2 x 2 turnaje a nechť tyto turnaje mají vždy pouze dva účastníky) a generační model tvorby nové populace.

Pro počáteční populaci platí:

i	chromozom $_i$	x_i	y_i	f_i
1	1100	12	-7	1
2	0011	3	2	6
3	0111	7	-2	6
4	1001	9	-4	4
Σ				17
ϕ				4.25

Hodnota $\phi = 17/4 = 4.25$ udává průměrnou hodnotu fitness funkce jedinců v populaci (kvalitu populace).

Nechť:

- pro první turnaj byli náhodně vybráni jedinci 2 a 4, vítězem je jedinec 2,
- pro druhý turnaj byli náhodně vybráni jedinci 1 a 3, vítězem je jedinec 3,
- pro třetí turnaj byli náhodně vybráni jedinci 2 a 4, vítězem je jedinec 2,
- pro čtvrtý turnaj byli náhodně vybráni jedinci 1 a 4, vítězem je jedinec 4.

Nechť první dvojici rodičů tvoří jedinci 2 a 3 (vítězové prvních dvou turnajů) a druhou dvojici tvoří jedinci 2 a 4 (vítězové druhých dvou turnajů).

Nechť pro první pár bylo náhodně zvoleno místo křížení 3 a pro druhý pár místo křížení 1:

První pár:

R2 0 0 1 | 1
R3 0 1 1 | 1
P1 0 0 1 1
P2 0 1 1 1

Druhý pár:

R2 0 | 0 1 1
R4 1 | 0 0 1
P3 0 0 0 1
P4 1 0 1 1

Nová populace je pak následující:

i	chromozom $_i$	x_i	y_i	f_i
1	0011	3	2	6
2	0111	7	-2	6
3	0001	1	4	4
4	1011	11	-6	2
Σ				18
ϕ				4.5

Kvalita populace (hodnota fitness funkce průměrného jedince) ϕ se zvýšila z hodnoty 4.25 na hodnotu 4.5.

Nechť v dalších čtyřech turnajích byly k reprodukci vybrány dvojice (3 a 2) a (1 a 2) a necht' místa křížení jsou 2 a 3:

První pár:	Druhý pár:
R3 0 0 0 1	R1 0 0 1
R2 0 1 1 1	R2 0 1 1
P1 0 0 1 1	P3 0 0 1 1
P2 0 1 0 1	P4 0 1 1 1

Další nová populace je následující:

i	chromozom $_i$	x_i	y_i	f_i	
1	0011	3	2	6	
2	0101	5	0	8	... hledané řešení
3	0011	3	2	6	
4	0111	7	2	6	
Σ				26	
ϕ				6.5	

Na příkladu je vidět postupné zlepšování kvality populace vedoucí k nalezení hledaného řešení.

2.5.2 Optimalizace mravenčí kolonií

2.5.2.1 Biologická inspirace

Optimalizace mravenčí kolonií je inspirována postupem, který používají mravenci při hledání potravy (nejkratší cesty mezi mraveništěm a zdrojem potravy):

- Mravenci nejprve náhodně prohledávají blízké okolí mraveniště.
- Během svého pohybu vypouští na zem chemickou látku, tzv. feromon.
- Ostatní mravenci tuto látku cítí a pravděpodobnost výběru jejich dalších cest je dána aktuálními koncentracemi feromonů na začátcích možných dalších cest.
- Když některý mravenec narazí na zdroj potravy, vyhodnotí její kvalitu i kvantitu a vrací se s částí této potravy do mraveniště.
- Během zpáteční cesty je množství vypouštěného feromonu tímto mravencem úměrné kvalitě a množství nalezené potravy.
- Feromonové stopy tak směřují ostatní mravence ke zdrojům potravy, s časem však tyto stopy vyprchávají.

Experimenty bylo zjištěno, že popsaná nepřímá komunikace mezi mravenci pomocí feromonových stop, tzv. stigmergie (komunikace hmyzu prostřednictvím chemických značek v prostředí), umožňuje mravencům nalézt nejkratší cestu mezi mraveništěm a zdrojem potravy. Toto zjištění bylo inspirací pro návrh optimalizačních metod s mravenci umělými, které byly souhrnně nazvány Ant Colony Optimization (ACO).

Hlavní rozdíly mezi chováními skutečných a umělých mravenců jsou tyto:

- Skuteční mravenci se pohybují v reálném (spojitém) prostředí asynchronně, umělí mravenci se pohybují v diskretním prostředí synchronně.
- Skuteční mravenci se při návratu do mraveniště řídí feromonovými stopami, umělí mravenci se v každém cyklu vrací do mraveniště po stejných cestách, kterými se v tomto cyklu pohybovali od mraveniště.
- Skuteční mravenci vypouští feromon neustále, umělí mravenci značkují cestu „feromonem“ pouze při návratu do mraveniště.
- Chování skutečných mravenců je založeno na implicitním vyhodnocení cest. To spočívá v tom, že pohyb po kratších cestách trvá mravencům kratší dobu, proto mohou tyto cesty opakovat častěji a tím se množství feromonu na těchto cestách zvyšuje. Umělí mravenci vyhodnocují cesty explicitně, a to při návratech do mraveniště podle kvality a délky cesty.
- Skuteční mravenci jsou prakticky slepí, umělí mravenci „zrak“ mají.
- Umělí mravenci mají paměť.

2.5.2.2 Algoritmus Ant System

Algoritmus Ant System (AS) byl prvním ACO algoritmem a byl navržen pro řešení úlohy obchodního cestujícího. Princip tohoto algoritmu je následující:

- Každému mravenci se přidělí seznam, do kterého si v průběhu každého cyklu řešení ukládá místa, která již navštívil.
- V každém kroku řešení se každý mravenec rozhoduje, do kterého dosud nenavštíveného místa se přesune. Pravděpodobnost výběru nového místa se přitom řídí množstvím feromonu na příslušné cestě a jeho vzdáleností od aktuálního místa.
- Po skončení cyklu každý mravenec zpětně označuje feromonem cestu, po které se pohyboval. Množství feromonu přitom závisí na kvalitě této cesty (čím je kratší, tím je lepší).
- Na lépe ohodnocených, tj. na kratších cestách, resp. na úsecích těchto cest, se množství feromonu zvyšuje, a tím se také zvyšuje pravděpodobnost jejich výběru v dalších cyklech řešení.

Algoritmus AS (řešení úlohy TSP)

1. Vynulujte čítač cyklů $c = 0$ a nastavte počáteční stav intenzit feromonů $\tau_{ij} = \alpha$ na všech cestách mezi místy i a j ($i = 1 \dots n$, $j = 1 \dots n$, $i \neq j$, n je počet míst a α je malé kladné číslo). Dále vytvořte m n -místných seznamů $Tabu_k$, $k = 1 \dots m$, kde m je uvažovaný počet mravenců, dále jeden n -místný seznam T_{min} pro ukládání dosud nalezené nejkratší cesty a proměnou L_{min} , do které se bude ukládat délka této nejkratší cesty.
2. Vynulujte přírůstky intenzity feromonů $\Delta \tau_{ij} = 0$ na všech cestách $i-j$.
3. Umístěte náhodně m mravenců do libovolných míst.
4. Nastavte index s na první položky seznamů $Tabu_k$ (tj. $s = 1$) a do každého seznamu $Tabu_k(s)$ uložte index místa, ve kterém se právě nachází k -tý mravenec.
5. Inkrementujte hodnotu indexu s .
6. Pro každého mravence k (nachází se právě v místě s indexem $j = Tabu_k(s-1)$) vyberte cestu do následujícího místa i a vložte index tohoto místa do seznamu $Tabu_k(s)$.

Cesta k -tého mravence z místa j (odkud) do místa i (kam) se vybírá na základě pravděpodobnosti této cesty, která je dána vztahem

$$P_{ij}^k = \begin{cases} \frac{\tau_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_{l=1, l \notin Tabu_k}^n \tau_{il}^\alpha \cdot \eta_{il}^\beta} & \text{pro } j \notin Tabu_k \\ 0 & \text{jinak} \end{cases}$$

Symbol η_{ij} označuje vzájemnou „viditelnost“ míst i a j , která je nepřímo úměrná jejich vzdálenosti d_{ij} ($\eta_{ij} = 1/d_{ij}$), α a β jsou parametry, které nastavují vlivy aktuální intenzity feromonové stopy a vzájemné viditelnosti míst.

7. Pokud nejsou seznamy $Tabu_k$ plné, tj. platí-li vztah $s < n$, pak dosud nebyla navštívena všechna místa, a proto se vraťte na bod 5. Jinak pokračujte.
8. Ze seznamů $Tabu_k$ vypočítejte délky cest L_k uskutečněných jednotlivými mravenci v aktuálním cyklu a k těmto délkám připočítejte délky cest z posledních do prvních míst v těchto seznamech (obchodní cestující se musí vrátit do výchozího místa)!
9. Aktualizujte nejkratší nalezenou cestu T_{min} a její délku L_{min} (po prvním cyklu pouze uložte nejkratší nalezenou cestu, po každém dalším cyklu nejprve porovnejte délky aktuální a dosud nejkratší cesty a do T_{min} a L_{min} uložte kratší z nich).
10. Spočítejte přírůstky intenzit feromonových stop na všech cestách $i-j$ uložených jednotlivými mravenci k takto:

$$\Delta\tau_{kij} = \begin{cases} \frac{Q}{L_k} & \text{pokud je cesta } ij \text{ v seznamu } Tabu_k \\ 0 & \text{jinak} \end{cases}$$

kde Q je konstanta reprezentující celkové množství feromonu vyloučeného jedním mravencem během jednoho cyklu.

11. Upravte intenzitu feromonových stop na všech cestách $i-j$:

$$\begin{aligned} \Delta\tau_{ij} &= \sum_{k=1}^n \Delta\tau_{kij} \\ \tau_{ij} &= (1 - \rho) \cdot \tau_{ij} + \Delta\tau_{ij} \end{aligned}$$

Symbolem $\rho \in \langle 0,1 \rangle$ je označen koeficient, který reprezentuje vypařování feromonů.

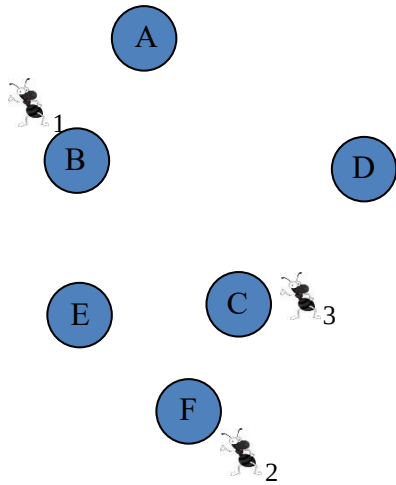
12. Inkrementujte hodnotu čítače cyklů c a pokud platí podmínka $c < c_{max}$, kde c_{max} je předem stanovený maximální počet cyklů, tak se vraťte na bod 2, jinak výpočet ukončete: řešením je pak nejkratší nalezená cesta T_{min} (s uvažováním návratu do prvního místa této cesty), která má délku uloženou v proměnné L_{min} .

Algoritmus se na první pohled zdá složitý, ale ve skutečnosti je poměrně jednoduchý. Jeho použití si ukážeme na příkladu nalezení cesty obchodního cestujícího mezi šesti místy s použitím tří mravenců, tj. $n = 6$ a $m = 3$ (rozšíření na více míst více mravenců je pak již velmi jednoduché). Rozmístění míst a vzdálenosti d_{ij} mezi nimi jsou ukázány na Obr. 2.46.

Nejprve nastavíme $c = 0$, $\tau_{ij} = 1$, $Q = 100$, $\alpha = \beta = 1$, $\rho = 0.1$, $\eta_{ij} = 1/d_{ij}$, $\Delta\tau_{ij} = 0$ ($i, j = 1 \dots n$, $i \neq j$), $Tabu_1 = []$, $Tabu_2 = []$, $Tabu_3 = []$.

Pak umístíme mravence náhodně do tří míst, například do B, F, a C a tato místa uložíme do seznamů $Tabu_1 = [B]$, $Tabu_2 = [F]$, $Tabu_3 = [C]$.

Poznamenejme, že výchozí místa mohou být i stejná, následující cesty však mohou být, a pro více míst většinou i budou, rozdílné.



d_{ij}	A	B	C	D	E	F
A	0	20	40	35	40	55
B	20	0	30	40	20	40
C	40	30	0	25	20	15
D	35	40	25	0	45	45
E	40	20	20	45	0	20
F	55	40	15	45	20	0

Obr. 2.46 Úloha TSP pro šest míst a tři mravence

První mravenec se nyní rozhoduje, do kterého dosud nenavštíveného místa půjde. Nejprve vypočítá pravděpodobnosti cest do těchto míst a pak například algoritmem rulety se rozhodne.

Výpočet pravděpodobností cest

$$P_{ij}^k = \begin{cases} \frac{\tau_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_{l=1, l \notin Tabu_k}^n \tau_{il}^\alpha \cdot \eta_{il}^\beta} = \frac{1/d_{ij}}{\sum_{l=1, l \notin Tabu_k}^n 1/d_{il}} & \text{pro } j \notin Tabu_k \\ 0 & \text{jinak.} \end{cases}$$

a pro možnou cestu prvního mravence z místa B do místa A tedy

$$P_{AB}^1 = \frac{1^1 \cdot 1/20^1}{1^1 \cdot 1/20^1 + 1^1 \cdot 1/30^1 + 1^1 \cdot 1/40^1 + 1^1 \cdot 1/20^1 + 1^1 \cdot 1/40^1} = 0.273$$

Pravděpodobnosti dalších možných cest prvního mravence se vypočítají stejným způsobem:

$$P_{EB}^1 = P_{AB}^1 = 0.273$$

$$P_{DB}^1 = P_{FB}^1 = \frac{1^1 \cdot \frac{1}{40^1}}{0.183} = 0.136$$

$$P_{CB}^1 = \frac{1^1 \cdot 1/30^1}{0.183} = 0.182$$

Výběr algoritmem rulety spočívá v tom, že se nejprve interval pro generování náhodného čísla $\langle 0,1 \rangle$ rozdělí podle pravděpodobností

$$\langle 0, P_{AB}^1 \rangle, \langle P_{AB}^1, P_{AB}^1 + P_{CB}^1 \rangle, \langle P_{AB}^1 + P_{CB}^1, P_{AB}^1 + P_{CB}^1 + P_{DB}^1 \rangle, \langle P_{AB}^1 + P_{CB}^1 + P_{DB}^1, P_{AB}^1 + P_{CB}^1 + P_{DB}^1 + P_{EB}^1 \rangle, \\ \langle P_{AB}^1 + P_{CB}^1 + P_{DB}^1 + P_{EB}^1, 1 \rangle = \langle 0, 0.273 \rangle, \langle 0.273, 0.455 \rangle, \langle 0.455, 0.591 \rangle, \langle 0.591, 0.864 \rangle, \langle 0.864, 1 \rangle.$$

a pak se generuje náhodné číslo $\langle 0,1 \rangle$, např. 0.672, které odpovídá čtvrtému intervalu $\Rightarrow$ první mravenec si vybere cestu do místa E: $Tabu_1 = [B, E]$.

Podobně postupují zbývající dva mravenci a vyberou si např. cesty do míst C a B: $Tabu_2 = [F, C]$, $Tabu_3 = [C, B]$. Postup pak se opakuje, dokud mravenci neprošli všemi místy.

Cesty mravenců těsně před návratem do výchozích míst jsou dány seznamy $Tabu$, například:

$$Tabu_1 = [B, E, D, A, C, F],$$

$$Tabu_2 = [F, C, D, B, A, E],$$

$$Tabu_3 = [C, B, D, A, E, F].$$

Délky těchto cest včetně návratů do původních míst jsou pak tyto:

$$L_1 = 20 + 45 + 35 + 40 + 15 + 40 = 195$$

$$L_2 = 15 + 25 + 40 + 20 + 40 + 20 = 160$$

$$L_3 = 30 + 40 + 35 + 40 + 20 + 15 = 180$$

Nejkratší cestou je cesta druhého mravence, a proto se uloží do seznamu T_{min} a její délka do proměnné L_{min} ($T_{min} = Tabu_2$, $L_{min} = L_2$).

Nyní se vypočítají přírůstky feromonových stop:

$$\Delta\tau_{k_{ij}} = \begin{cases} \frac{Q}{L_k} & \text{pokud je cesta } ij \text{ v seznamu } Tabu_k \\ 0 & \text{jinak} \end{cases}$$

$$Tabu_1 = [B, E, D, A, C, F], \quad L_1 = 195,$$

$$Tabu_3 = [C, B, D, A, E, F], \quad L_3 = 180,$$

$$\Delta\tau_{AB} = 0 + 100/160 + 0 = 0.625$$

$$\Delta\tau_{AD} = 100/195 + 0 + 100/180 = 1.069$$

$$\Delta\tau_{AF} = 0 + 0 + 0 = 0$$

$$\Delta\tau_{BC} = 0 + 0 + 100/180 = 0.556$$

$$\Delta\tau_{BE} = 100/195 + 0 + 0 = 0.513$$

$$\Delta\tau_{CD} = 0 + 100/160 + 0 = 0.625$$

$$\Delta\tau_{CF} = 100/195 + 100/160 + 100/180 = 1.694$$

$$\Delta\tau_{DE} = 100/195 + 0 + 0 = 0.513$$

$$\Delta\tau_{EF} = 0 + 100/160 + 100/180 = 1.181$$

$$Tabu_2 = [F, C, D, B, A, E], \quad L_2 = 160$$

$$Q = 100$$

$$\Delta\tau_{AC} = 100/195 + 0 + 0 = 0.513$$

$$\Delta\tau_{AE} = 0 + 100/160 + 100/180 = 1.181$$

$$\Delta\tau_{BD} = 0 + 100/160 + 100/180 = 1.181$$

$$\Delta\tau_{BF} = 100/195 + 0 + 0 = 0.513$$

$$\Delta\tau_{CE} = 0 + 0 + 0 = 0$$

$$\Delta\tau_{DF} = 0 + 0 + 0 = 0$$

Pak se aktualizují feromonové stopy

$$\tau_{ij} = (1 - \rho) \cdot \tau_{ij} + \Delta\tau_{ij}$$

$$\tau_{AB} = 0.9 \cdot 1 + 0.625 = 1.525$$

$$\tau_{AD} = 0.9 \cdot 1 + 1.069 = 1.969$$

$$\tau_{AF} = 0.9 \cdot 1 + 0 = 0.9$$

$$\tau_{BC} = 0.9 \cdot 1 + 0.556 = 1.456$$

$$\tau_{BE} = 0.9 \cdot 1 + 0.513 = 1.413$$

$$\tau_{CD} = 0.9 \cdot 1 + 0.625 = 1.525$$

$$\tau_{CF} = 0.9 \cdot 1 + 1.694 = 2.594$$

$$\tau_{DE} = 0.9 \cdot 1 + 0.513 = 1.413$$

$$\tau_{EF} = 0.9 \cdot 1 + 1.181 = 2.081$$

Počáteční $\tau_{ij} = 1$ a necht' $\rho = 0.1$

$$\tau_{AC} = 0.9 \cdot 1 + 0.513 = 1.413$$

$$\tau_{AE} = 0.9 \cdot 1 + 1.181 = 2.081$$

$$\tau_{BD} = 0.9 \cdot 1 + 1.181 = 2.081$$

$$\tau_{BF} = 0.9 \cdot 1 + 0.513 = 1.413$$

$$\tau_{CE} = 0.9 \cdot 1 + 0 = 0.9$$

$$\tau_{DF} = 0.9 \cdot 1 + 0 = 0.9$$

Nakonec se testuje, zda bude výpočet opakován, nebo zda se ukončí. Inkrementuje se hodnota čítače c a pokud platí nerovnost $c \leq 100$, pak se výpočet opakuje (od bodu 2 algoritmu na str. 68), jinak se vrací nejkratší nalezené řešení (T_{min} , L_{min}).

2.5.3 Optimalizace hejnem částic

Algoritmus optimalizace hejnem částic (PSO) byl původně navržen pro simulaci pohybů ptačího hejna, dnes se však používá především k řešení spojitých optimalizačních úloh.

Algoritmus PSO má několik společných rysů s GA:

- Pracuje s populací jedinců nazývanými „částice“, které představují „řešení“ podobně jako chromozomy v GA.
- Hodnocení kvality částic je prováděno pomocí hodnotících (*fitness*) funkcí.
- Počáteční rozložení částic v prohledávaném prostoru jsou náhodná.

Každá částice k je reprezentována polohou v n -rozměrném prostoru, vektorem rychlosti jejího pohybu a pamětí předchozích úspěchů při hledání optima hodnotící funkce.

V průběhu výpočtu se jak poloha, tak rychlost každé částice mění, a to v závislosti na dosud nejlepší poloze této částice a na dosud nejlepší poloze nalezené hejnem, tj. nejúspěšnější částicí hejna, podle následujících vztahů:

$$\vec{x}^k(t + \Delta t) = \vec{x}^k(t) + \vec{v}^k(t) \cdot \Delta t$$

$$\vec{v}^k(t + \Delta t) = \omega \cdot \vec{v}^k(t) + c_p \cdot r_p \cdot (\vec{x}_{best}^k - \vec{x}^k(t)) + c_g \cdot r_g \cdot (\vec{x}_{best} - \vec{x}^k(t))$$

a pro prakticky vždy uvažovaný jednotkový přírůstek času ($\Delta t = 1$)

$$\vec{x}^k(t + 1) = \vec{x}^k(t) + \vec{v}^k(t)$$

$$\vec{v}^k(t + 1) = \omega \cdot \vec{v}^k(t) + c_p \cdot r_p \cdot (\vec{x}_{best}^k - \vec{x}^k(t)) + c_g \cdot r_g \cdot (\vec{x}_{best} - \vec{x}^k(t))$$

Koeficient setrvačnosti ω se v původním algoritmu nevyskytoval (byl tedy roven jedné), dnes se doporučuje volit jeho hodnotu v rozmezí 0.4 až 0.9.

Koeficienty c_p (kognitivní) a c_g (sociální) jsou váhové koeficienty a udávají míru důležitosti individuální paměti a sociálního vlivu. Obvykle se volí oba tyto koeficienty stejné s hodnotami mírně vyššími než 2, většinou $c_p = c_g = 2.05$.

Koeficienty r_p a r_g vkládají do algoritmu neurčitost a nastavují se náhodně v intervalu $\langle 0, 1 \rangle$ s rovnoměrným rozložením pravděpodobnosti.

Pro doposud nejlepší pozice nalezené částicí k a všemi částicemi hejna pak zřejmě platí vztahy (pro aktuální čas τ):

$$f(\vec{x}_{best}^k) \geq f(\vec{x}^k(\tau)), \quad \tau = 0 \dots t$$

$$f(\vec{x}_{best}) \geq f(\vec{x}_{best}^k), \quad k = 1 \dots m$$

kde $f(\cdot)$ je hodnotící (fitness) funkce a m je počet částic hejna.

Algoritmus PSO

1. Zvolte počet částic m (obvykle v rozmezí 20 až 100).
2. Zvolte hodnoty koeficientů ω , c_p a c_g .
3. Nastavte náhodně počáteční hodnoty pozic a rychlostí částic:
 - $x_i^k = \text{random}(x_{ilow}, x_{iup})$ pro každou částici $k = 1 \dots m$ a pro každou souřadnici i n -rozměrného prostoru. Symboly x_{ilow} , resp. x_{iup} značí dolní a horní hranici prohledávaného prostoru v i -té souřadnici.

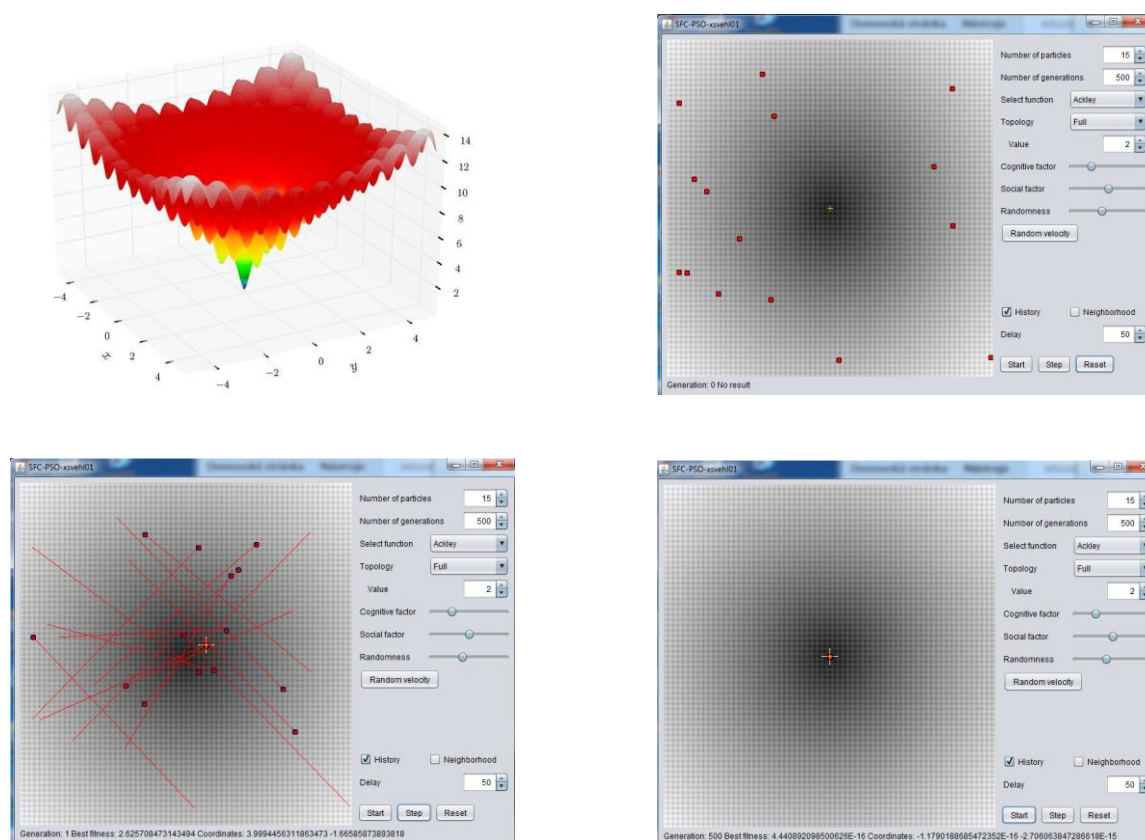
- $v_i^k = \text{random}(-v_{imax}, v_{imax})$ pro každou částici $k = 1 \dots m$ a pro každou souřadnici i n -rozměrného prostoru. Symbol v_{imax} značí maximální rychlost ve směru i -té souřadnice.
4. Nastavte pozici $\vec{x}^k$ částice k jako její dosud nejlepší pozici $\vec{p}^k$.
 5. Určete nejlepší pozici hejna $\vec{g}$, které je dáno pozicí nejlépe hodnocené částice $\vec{g} \geq \vec{p}^k$, $k = 1 \dots m$.
 6. Nastavte index k na první částici ($k = 1$).
 7. Určete náhodná čísla r_p a r_g z intervalu $<0,1>$, rovnoměrné rozdělení pravděpodobnosti.
 8. Vypočítejte novou rychlost částice a její novou pozici podle vztahů

$$\vec{x}^k(t+1) = \vec{x}^k(t) + \vec{v}^k(t)$$

$$\vec{v}^k(t+1) = \omega \cdot \vec{v}^k(t) + c_p \cdot r_p \cdot (\vec{x}_{best}^k - \vec{x}^k(t)) + c_g \cdot r_g \cdot (\vec{x}_{best} - \vec{x}^k(t))$$
 9. Ohodnoťte novou polohu částice $\vec{x}^k$ a pokud je toto hodnocení lepší než její dosavadní nejlepší ohodnocení, pak upravte její nejlepší pozici $\vec{p}^k = \vec{x}^k$.
 10. Pokud je hodnocení částice lepší než hodnocení dosud nejlepší pozice hejna, pak upravte nejlepší pozici hejna $\vec{g} = \vec{p}^k$.
 11. Inkrementujte index k , a pokud platí $k \leq m$, tak se vraťte na bod 7, jinak pokračujte.
 12. Pokud se řešení zlepšuje a pokud nebyl překročen zvolený maximální čas výpočtu, tak se vraťte na bod 6, jinak řešení ukončete (řešení je dáno nejlepší pozicí hejna $\vec{g}$).

Příklad: hledání globálního minima Ackleyho funkce (Obr. 2.47):

$$f(x) = -20e^{-0.2 \cdot \sqrt{0.5(x^2 + y^2)}} - e^{0.5(\cos(2\pi x) + \cos(2\pi y))} + e + 20$$



Obr. 2.47 Hledání minima Ackleyho funkce algoritmem PSO

3 Logika a umělá inteligence

Logika je považována za jeden ze základních „stavebních kamenů“ klasické umělé inteligence. Umožňuje jak snadnou reprezentaci znalostí, tak i práci s těmito znalostmi. Má však také některé nedostatky, z nichž nejzávažnějším je velmi obtížná práce s nejistými, neúplnými a dočasnými znalostmi.

V dalším textu nejprve stručně popíšeme principy výrokové logiky. Ta má sice poměrně slabou vyjadřovací sílu a v UI se prakticky nepoužívá, dovoluje nám však snadněji pochopit některé principy, které používá predikátová logika. Predikátové logice prvního řádu pak bude věnována převážná část této kapitoly.

3.1 Výroková logika

Formální systém každé logiky, a tedy i výrokové logiky, tvoří tři složky:

- 1) *jazyk* (množiny symbolů pro označení logických konstant, atomických formulí, spojek a pomocných objektů),
- 2) *axiomy* (formule představující základní formule jazyka),
- 3) *odvozovací pravidla* (pravidla umožňující odvodit ze základních formulí nové formule).

Axiomy výrokové logiky a odvozovací pravidlo (ve výrokové logice je jediné) nebudeme dále používat a uvádíme je pouze pro informaci:

- Axiomy výrokové logiky:

- $A \Rightarrow (B \Rightarrow A)$,
- $(A \Rightarrow (B \Rightarrow C)) \Rightarrow ((A \Rightarrow B) \Rightarrow (A \Rightarrow C))$,
- $(\neg A \Rightarrow \neg B) \Rightarrow (B \Rightarrow A)$.

- Odvozovací pravidlo výrokové logiky (modus ponens):

z formulí A , $A \Rightarrow B$ odvod' formuli B .

Základními prvky jazyka výrokové logiky jsou prvotní formule (atomy), logické spojky ($\neg$, $\vee$, $\wedge$, $\Rightarrow$, $\Leftrightarrow$ (negace, disjunkce, konjunkce, implikace a ekvivalence) a kulaté závorky.

S prvotními formulemi je vždy spojena jedna ze dvou logických hodnot T/F (True/False, pravda/nepravda).

Z prvotních formulí se vytvářejí výrokové formule (dále jen formule) takto:

- každá prvotní formule je formule,
- jsou-li A a B formule, pak i $\neg A$, $A \vee B$, $A \wedge B$, $A \Rightarrow B$ a $A \Leftrightarrow B$ jsou formule.

Logická hodnota formulí s logickými spojkami je dána následující tabulkou:

A	B	$\neg A$	$\neg B$	$A \vee B$	$A \wedge B$	$A \Rightarrow B$	$A \Leftrightarrow B$
F	F	T	T	F	F	T	T
F	T	T	F	T	F	T	F
T	F	F	T	T	F	F	F
T	T	F	F	T	T	T	T

V případě složitějších výrazů se respektují priority logických spojek podle výše uvedeného pořadí (nejvyšší prioritu má spojka $\neg$ a nejnižší spojka $\Leftrightarrow$), kulaté závorky umožňují prioritu standardním způsobem měnit.

Spojky $\vee$, $\wedge$, $\Leftrightarrow$ jsou symetrické, a proto pořadí argumentů nemá na hodnotu formule vliv, spojka $\Rightarrow$ je však nesymetrická a výsledná hodnota formule na pořadí argumentů závisí. (formule A se nazývá antecedent a formule B konsekvent formule $A \Rightarrow B$).

Pravdivostní ohodnocení prvotních formulí (přiřazení logických hodnot) v dané formuli se nazývá interpretací této formule.

Formule se nazývá tautologií (logicky pravdivou formulí), je-li pravdivá ve všech interpretacích, a kontradikcí (nesplnitelnou formulí), je-li ve všech interpretacích nepravdivá.

Formule je splnitelná, existuje-li alespoň jedna interpretace, ve které je pravdivá. Formule není logicky pravdivá (není tautologií), existuje-li alespoň jedna interpretace, ve které je nepravdivá.

Dvě formule jsou ekvivalentní, když nabývají stejných logických hodnot ve všech interpretacích.

Příklady: formule $(A \vee \neg A)$ je tautologií
 formule $(A \wedge \neg A)$ je kontradikcí
 formule $(A \vee \neg B)$ je splnitelnou formulí
 formule $\neg(A \vee B)$ a $(\neg A \wedge \neg B)$ jsou ekvivalentní

Literálem se rozumí prvotní formule nebo její negace.

Formule je v disjunktivní normální formě, je-li disjunkcí formulí, které jsou konjunkcemi literálů. V žádné z těchto formulí se však nesmí vyskytovat některá prvotní formule společně se svou negací.

Formule je v konjunktivní normální formě, je-li konjunkcí formulí, které jsou disjunkcemi literálů. V žádné z těchto formulí se opět nesmí vyskytovat některá prvotní formule společně se svou negací.

Disjunkce literálů $L_1 \vee L_2 \vee, \dots, \vee L_n$ se nazývá klauzulí. Jednotková klauzule obsahuje jeden literál, prázdná klauzule neobsahuje žádný literál, označuje se symbolem $\square$ a je nesplnitelná.

Klauzule s komplementárním párem literálů (např. $A \vee \neg A$) je tautologií.

Formule G je logickým důsledkem (logicky vyplývá z) formulí $\{F_1, F_2, \dots, F_n\}$, je-li pravdivá v každé interpretaci I, v níž je pravdivá formule $F_1 \wedge F_2 \wedge, \dots, \wedge F_n$, tj. je-li formule $F_1 \wedge F_2 \wedge, \dots, \wedge F_n \Rightarrow G$ tautologií.

Formule $F_1 \wedge F_2 \wedge, \dots, \wedge F_n \Rightarrow G$ se nazývá teorémem, formule $\{F_1, F_2, \dots, F_n\}$ premisami (postuláty) tohoto teorému a formule G tvrzením.

Konečná posloupnost formulí $A_1, A_2, \dots, A_n$ je důkazem formule A, jestliže A_n je A a pro libovolné $i \in \langle 1, n \rangle$ je A_i buď axiómem, nebo bezprostředním důsledkem aplikace pravidla modus ponens na nějaké dva prvky z množiny $\{A_1, \dots, A_{i-1}\}$.

Formule dokazatelné ve výrokové logice jsou právě tautologie a výroková logika proto tvoří bezesporný (korektní / konzistentní) formální systém.

Pozn.: Formální systém je sporný, je-li v něm dokazatelná libovolná formule.

Následující dokazatelné formule jsou považovány za zákony výrokové logiky:

1. zákon ekvivalence $A \Leftrightarrow B \Leftrightarrow (A \Rightarrow B) \wedge (B \Rightarrow A)$
2. zákon implikace $A \Rightarrow B \Leftrightarrow \neg A \vee B$
3. zákony komutativní $A \vee B \Leftrightarrow B \vee A$
 $A \wedge B \Leftrightarrow B \wedge A$
4. zákony asociativní $A \wedge (B \wedge C) \Leftrightarrow (A \wedge B) \wedge C$
 $A \vee (B \vee C) \Leftrightarrow (A \vee B) \vee C$
5. zákony distributivní $A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$
 $A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$
6. zákony zjednodušení disjunkce
 $A \vee T \Leftrightarrow T$
 $A \vee F \Leftrightarrow A$
 $A \vee A \Leftrightarrow A$
 $A \vee (A \wedge B) \Leftrightarrow A$
7. zákony zjednodušení konjunkce
 $A \wedge T \Leftrightarrow A$
 $A \wedge F \Leftrightarrow F$
 $A \wedge A \Leftrightarrow A$
 $A \wedge (A \vee B) \Leftrightarrow A$
8. zákon vyloučeného třetího
 $A \vee \neg A \Leftrightarrow T$
9. zákon sporu $\neg(A \vee \neg A) \Leftrightarrow F$
10. zákon identity $A \Leftrightarrow A$
11. zákon negace $\neg(\neg A) \Leftrightarrow A$
12. zákony De Morganovy $\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$
 $\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$
13. eliminace AND $A_1 \wedge A_2 \wedge \dots \wedge A_n \Rightarrow A_i \quad i \in \langle 1, n \rangle$
14. zavedení OR $A_i \Rightarrow A_1 \vee A_2 \vee \dots \vee A_n \quad i \in \langle 1, n \rangle$

Formule G je logickým důsledkem formulí $\{F_1, F_2, \dots, F_n\}$, je-li formule $(F_1 \wedge F_2 \wedge \dots \wedge F_n \Rightarrow G)$ tautologií. Pak negovaná formule, tj. formule $\neg(F_1 \wedge F_2 \wedge \dots \wedge F_n \Rightarrow G)$, musí být kontradikcí. Tuto negovanou formuli pak lze pomocí výše uvedených zákonů snadno převést na tvar

$$(F_1 \wedge F_2 \wedge \dots \wedge F_n \wedge \neg G)$$

což vede k důležitému závěru: Formule G je logickým důsledkem formulí $\{F_1, F_2, \dots, F_n\}$, když a jen když je formule $(F_1 \wedge F_2 \wedge \dots \wedge F_n \wedge \neg G)$ nespílitelná.

Důkaz nespílitelnosti množiny je základem strojového dokazování a bude podrobněji popsán v podkapitole věnované rezoluční metodě.

Příklad: Dokažte, že formule C je logickým důsledkem formulí $(A \Rightarrow B)$, $(B \Rightarrow C)$, A

a) Postup při důkazu, že formule $((A \Rightarrow B) \wedge (B \Rightarrow C) \wedge A) \Rightarrow C$ je tautologií:

$$\begin{aligned}
 & ((A \Rightarrow B) \wedge (B \Rightarrow C) \wedge A) \Rightarrow C && \Leftrightarrow \\
 & \neg((\neg A \vee B) \wedge (\neg B \vee C) \wedge A) \vee C && \Leftrightarrow \\
 & (\neg(\neg A \vee B) \vee \neg(\neg B \vee C) \vee \neg A) \vee C && \Leftrightarrow \\
 & (A \wedge \neg B) \vee (B \wedge \neg C) \vee \neg A \vee C && \Leftrightarrow \\
 & ((A \wedge \neg B) \vee \neg A) \vee ((B \wedge \neg C) \vee C) && \Leftrightarrow \\
 & ((A \vee \neg A) \wedge (\neg B \vee \neg A)) \vee ((B \vee C) \wedge (\neg C \vee C)) && \Leftrightarrow \\
 & (T \wedge (\neg B \vee \neg A)) \vee ((B \vee C) \wedge T) && \Leftrightarrow \\
 & \neg B \vee \neg A \vee B \vee C && \Leftrightarrow \\
 & T
 \end{aligned}$$

b) Postup při důkazu, že formule $((A \Rightarrow B) \wedge (B \Rightarrow C) \wedge A) \Rightarrow C$ je kontradikcí:

$$\begin{aligned}
 & \neg(((A \Rightarrow B) \wedge (B \Rightarrow C) \wedge A) \Rightarrow C) && \Leftrightarrow \\
 & \neg(\neg((A \Rightarrow B) \wedge (B \Rightarrow C) \wedge A) \vee C) && \Leftrightarrow \\
 & \neg(\neg((\neg A \vee B) \wedge (\neg B \vee C) \wedge A) \vee C) && \Leftrightarrow \\
 & (\neg A \vee B) \wedge (\neg B \vee C) \wedge A \wedge \neg C && \Leftrightarrow \\
 & ((\neg A \vee B) \wedge A) \wedge ((\neg B \vee C) \wedge \neg C) && \Leftrightarrow \\
 & ((\neg A \wedge A) \vee (B \wedge A)) \wedge ((\neg B \wedge \neg C) \vee (C \wedge \neg C)) && \Leftrightarrow \\
 & ((F \vee (B \wedge A)) \wedge ((\neg B \wedge \neg C) \vee F)) && \Leftrightarrow \\
 & B \wedge A \wedge \neg B \wedge \neg C && \Leftrightarrow \\
 & F
 \end{aligned}$$

3.2 Predikátová logika

Formální systém predikátové logiky prvního řádu (dále jen predikátová logika) opět tvoří tři složky:

- jazyk predikátové logiky (JPL),
- axiomy predikátové logiky:
 - axiomy výrokové logiky,
 - schéma specifikace $\forall x(A) \Rightarrow Ax[t]$ ($Ax[t]$ je formule vzniklá substitucí termu t za proměnnou x ve formuli A),
 - schéma kvantifikace implikace $\forall x(A \Rightarrow B) \Rightarrow (A \Rightarrow \forall x(B))$, proměnná x nesmí být ve formuli A volná.
- odvozovací pravidla:
 - modus ponens,
 - pravidlo generalizace: pro libovolnou proměnnou x odvod' z formule A formuli $\forall x(A)$.

Axiomy predikátové logiky a odvozovací pravidla nebudeme dále používat stejně jako u výrokové logiky a v tomto textu jsou uvedena opět pouze pro informaci.

Jazyk predikátové logiky je bohatší než jazyk výrokové logiky a jeho základními složkami jsou:

- individuové konstanty: $a, b, c, \dots$,
- individuové proměnné: $x, y, z, \dots$,
- funkční symboly: $f, g, h, \dots$,
- predikátové symboly: $P, Q, R, \dots$,
- logické spojky, stejné jako u výrokové logiky ($\neg, \vee, \wedge, \Rightarrow, \Leftrightarrow$),
- kvantifikátory (univerzální $\forall$ a existenční $\exists$),
- pomocné symboly (kulaté závorky, hranaté závorky a čárka).

S každým funkčním a predikátovým symbolem je spojeno přirozené číslo, které udává počet jeho argumentů (míst).

Za term jazyka predikátové logiky se považuje individuová konstanta, individuová proměnná a struktura $f(t_1, t_2, \dots, t_n)$, kde f je n -místný funkční symbol a t_i jsou termy.

Atomickou formulí jazyka predikátové logiky je struktura $P(t_1, t_2, \dots, t_n)$, kde P je n -místný predikátový symbol a t_i jsou termy. Za speciální případ atomických formulí se považují pravdivostní symboly T a F .

Atomické formule a jejich negace se nazývají literály.

Formule jazyka predikátové logiky se vytvářejí takto:

- Každá atomická formule je formule,
- jsou-li A a B formule, pak i $\neg A, A \vee B, A \wedge B, A \Rightarrow B$ a $A \Leftrightarrow B$ jsou formule a pro jejich vyhodnocení lze použít tabulku uvedenou u výrokové logiky,
- je-li x individuová proměnná a A formule, pak i $\forall x(A)$ a $\exists x(A)$ jsou formule.

Pravidlo pro tvorbu formulí kvantifikací proměnných omezuje obecný jazyk predikátové logiky na jazyk predikátové logiky 1. řádu (predikátové jazyky vyšších řádů povolují kvantifikovat i predikátové a funkční symboly).

Individuová proměnná je vázaná, jestliže se vyskytuje v oblasti působnosti některého kvantifikátoru, jinak je volná.

Kvantifikátor $\forall x$ znamená „pro každé x “, a pravdivostní hodnota formule $\forall x(A)$ je proto dána konjunkcemi formulí A pro všechny možné hodnoty proměnné x .

Kvantifikátor $\exists x$ znamená „existuje x “, a pravdivostní hodnota formule $\exists x(A)$ je proto dána disjunkcemi formulí A pro všechny možné hodnoty proměnné x .

Formule je otevřená, pokud neobsahuje žádnou vázanou proměnnou, resp. uzavřená, pokud neobsahuje žádnou volnou proměnnou.

Formule $\forall x_1 \forall x_2 \dots \forall x_n(A)$ se nazývá univerzálním uzávěrem formule A , jsou-li všechny volné proměnné ve formuli A z množiny $\{x_1, x_2, \dots, x_n\}$. Obdobně je definován i existenční uzávěr formule A .

Interpretací I formule A nad libovolnou neprázdnou množinou D (oborem interpretace, univerzem) se rozumí:

- přiřazení pevně zvoleného prvku z D každé individuové konstantě,
- přiřazení libovolného prvku z D každé individuové proměnné,
- přiřazení n -ární operace každému n -místnému funkčnímu symbolu ($D^n \rightarrow D$),
- přiřazení n -ární relace každému n -místnému predikátovému symbolu ($D^n \rightarrow \{T, F\}$).

Příklad: Dokažte, že formule $\forall x(P(a, x) \Rightarrow Q(f(x)))$ je pravdivá v této interpretaci I:

$D = \{1,2,3\}$	$a = 2$	$f(x) = 4 - x$
$P(1, 1) = F$	$P(1, 2) = T$	$P(1, 3) = T$
$P(2, 1) = T$	$P(2, 2) = F$	$P(2, 3) = F$
$P(3, 1) = T$	$P(3, 2) = T$	$P(3, 3) = F$
$Q(1) = T$	$Q(2) = F$	$Q(3) = T$

Postup odvození důkazu:

$$\begin{aligned} & \forall x(P(a, x) \Rightarrow Q(f(x))) \\ & (P(2, 1) \Rightarrow Q(f(1))) \wedge (P(2, 2) \Rightarrow Q(f(2))) \wedge (P(2, 3) \Rightarrow Q(f(3))) \\ & (T \Rightarrow T) \wedge (F \Rightarrow F) \wedge (F \Rightarrow T) \\ & T \wedge T \wedge T \\ & T \end{aligned}$$

Definice logicky pravdivých, logicky nepravdivých, splnitelných a nesplnitelných formulí jsou stejné jako ve výrokové logice. Rovněž stejné jsou definice klauzulí, ekvivalentních formulí, konjunktivní i disjunktivní normální formy a logických důsledků.

Formule A je v prenexní normální formě, je-li ve tvaru $A = Q_1x_1 \dots Q_nx_n(M)$, kde Q_i je buď univerzální, nebo existenční kvantifikátor, x_i jsou navzájem různé proměnné a M je otevřená formule vzniklá z atomických formulí použitím logických spojek. Formulí M se pak říká jádro (matice) a posloupnosti kvantifikátorů $Q_1x_1 \dots Q_nx_n$ prefix formule A.

Libovolnou formuli lze převést do prenexní normální formy pomocí zákonů výrokové logiky, doplněných zákony pro práci s kvantifikátory:

15. přesun kvantifikátorů doleva (x se nevyskytuje v B!)

$$\begin{aligned} Qx(A) \vee B & \Leftrightarrow Qx(A \vee B) \\ Qx(A) \wedge B & \Leftrightarrow Qx(A \wedge B) \end{aligned}$$

16. přesun negace dovnitř

$$\begin{aligned} \neg(\forall x(A)) & \Leftrightarrow \exists x(\neg A) \\ \neg(\exists x(A)) & \Leftrightarrow \forall x(\neg A) \end{aligned}$$

17. distributivní zákony pro kvantifikátory

$$\begin{aligned} \forall x(A) \wedge \forall x(B) & \Leftrightarrow \forall x(A \wedge B) \\ \exists x(A) \vee \exists x(B) & \Leftrightarrow \exists x(A \vee B) \end{aligned}$$

18. přejmenování vázaných proměnných

$$\begin{aligned} \forall x(A(x)) \vee \forall x(B(x)) & \Leftrightarrow \forall x \forall y(A(x) \vee B(y)) \\ \exists x(A(x)) \wedge \exists x(B(x)) & \Leftrightarrow \exists x \exists y(A(x) \wedge B(y)) \end{aligned}$$

Symbol $A(x)$ znamená, že se ve formuli A vyskytuje volná proměnná x apod.

Poslední zákon říká, že se ve formuli B nahradí proměnná x novou proměnnou y, která v této formuli dosud nebyla!

Při převodu formule do prenexní normální formy se nejprve eliminují spojky $\Leftrightarrow$ a $\Rightarrow$ (zákony 1, 2), poté se přesouvají všechny negace těsně k atomům (zákony 11, 12), provede se nezbytné přejmenování proměnných (zákon 18) a s použitím zákonů (15, 17, 18) se přesunou všechny

kvantifikátory do prefixu formule. S použitím ostatních zákonů lze navíc převést matici formule do konjunktivní nebo disjunktivní normální formy.

Příklad: Převod formule $\forall x \forall y (\exists z (A(x, z) \wedge B(y, z)) \Rightarrow \exists v (C(v, x, y)))$ do disjunktivní prenexní normální formy:

$$\forall x \forall y (\exists z (A(x, z) \wedge B(y, z)) \Rightarrow \exists v (C(v, x, y)))$$

$$\forall x \forall y (\neg \exists z (A(x, z) \wedge B(y, z)) \vee \exists v (C(v, x, y)))$$

$$\forall x \forall y \forall z (\neg A(x, z) \vee \neg B(y, z) \vee \exists v (C(v, x, y)))$$

$$\forall x \forall y \forall z \exists v (\neg A(x, z) \vee \neg B(y, z) \vee C(v, x, y))$$

Odstranění existenčních kvantifikátorů ve formuli A, která je v prenexní normální formě a která má navíc matici M v konjunktivní normální formě, se provádí postupem zvaným skolemizace:

1. Nechť $\exists x_1$ je prvním kvantifikátorem v prefixu formule A zleva. Pak se vybere libovolná individuová konstanta c, která se dosud v matici M nevyskytuje (tzv. Skolemova konstanta), touto konstantou se nahradí všechny výskyty proměnné x_1 v matici M a kvantifikátor $\exists x_1$ se z prefixu formule odstraní.
2. Nechť $\exists x_{m+1}$ je první existenční kvantifikátor v prefixu formule A zleva a nechť se před ním (zleva) nachází m univerzálních kvantifikátorů. Pak se vybere libovolná m-místná funkce $f(x_1, \dots, x_m)$, která dosud v matici M není (tzv. Skolemova funkce), touto funkcí se nahradí všechny výskyty proměnné x_{m+1} v matici M a kvantifikátor $\exists x_{m+1}$ se z prefixu formule odstraní.

Formule F v normální formě je konjunkcí klauzulí, a proto se nazývá formulí v klauzulárním tvaru nebo v klauzulární formě. Zapisuje se pak často ve tvaru množiny klauzulí S a zřejmě platí, že formule F je splnitelná právě tehdy, když je splnitelná množina klauzulí S a naopak.

Příklad: Převod formule $\exists x \forall y \exists z (\neg A(x, y) \wedge (B(x, z) \vee C(x, y, z)))$ do Skolemovy normální formy:

$$\exists x \forall y \exists z (\neg A(x, y) \wedge (B(x, z) \vee C(x, y, z)))$$

$$\forall y \exists z (\neg A(a, y) \wedge (B(a, z) \vee C(a, y, z)))$$

$$\forall y (\neg A(a, y) \wedge (B(a, f(y)) \vee C(a, y, f(y))))$$

Odpovídající množina klauzulí: $S = \{\neg A(a, y), (B(a, f(y)) \vee C(a, y, f(y)))\}$

Důkazem formule A, podobně jako ve výrokové logice, je konečná posloupnost formulí $A_1, A_2, \dots, A_n$, jestliže A_n je A a pro libovolné $i \in \langle 1, n \rangle$ je A_i buď axiomem, nebo bezprostředním důsledkem aplikace odvozovacích pravidel na nějaké dva prvky z množiny $\{A_1, \dots, A_{i-1}\}$. Přestože jde o úlohu podobnou hledání důkazu ve výrokové logice, nelze v případě predikátové logiky podobný důkaz jednoduše provést. Predikátová logika je obecně nerozhodnutelná, což jinými slovy znamená, že proces důkazu nepravdivé formule nemusí nikdy skončit (není možné dokázat logickou pravdivost nebo nespílitelnost formulí predikátové logiky ověřováním jejich interpretací, protože počet oborů těchto interpretací není konečný). Nedokáže-li se tedy pravdivost formule, nelze tento neúspěch nikdy považovat za důkaz její nepravdivosti, protože proces dokazování mohl být ukončen příliš brzy.

V běžné praxi se naštěstí nepotřebují dokazovat libovolné formule, ale formule jistým způsobem omezené, které jsou pak dokazatelné.

3.3 Rezoluční metoda

Rezoluční metoda je nejznámější metodou automatického dokazování. Umožňuje dokázat nespílitelnost dané množiny klauzulí procesem postupného rozšiřování této množiny o odvozené klauzule (rezolventy). Daná množina klauzulí je pak nespílitelná, podaří-li se odvodit prázdnou klauzuli $\square$ (Robinsonův rezoluční princip).

Pravidlo základní rezoluce je určeno k dokazování nespílitelnosti množiny klauzulí, ve kterých se nevyskytují žádné proměnné, a je proto vhodné především pro výrokovou logiku:

Nechť C_1 a C_2 jsou klauzule (tzv. rodičovské klauzule) a nechť jedna z nich, např. C_1 , obsahuje literál L a druhá klauzule (C_2) negaci tohoto literálu $\neg L$. Pak rezolventou klauzulí C_1 a C_2 je klauzule

$$C = (C_1 - L) \vee (C_2 - \neg L),$$

pro kterou platí, že je logickým důsledkem klauzulí C_1 a C_2 . Důkaz je jednoduchý:

Nechť $C_1 = (A_1 \vee A_2 \vee A_n \vee L)$ a $C_2 = (B_1 \vee B_2 \vee B_m \vee \neg L)$.

Protože obě klauzule (disjunkce literálů) jsou pravdivé, pak je-li nepravdivý literál L musí být pravdivá klauzule $(A_1 \vee A_2 \vee A_n)$. Naopak, je-li nepravdivý literál $\neg L$ musí být pravdivá klauzule $(B_1 \vee B_2 \vee B_m)$. Z této jednoduché úvahy bezprostředně vyplývá, že disjunkce klauzulí $(A_1 \vee A_2 \vee A_n) \vee (B_1 \vee B_2 \vee B_m)$ musí být rovněž pravdivou klauzulí.

Poznamenejme, že pravidlo modus ponens je speciálním případem pravidla základní rezoluce: $\{A, (A \Rightarrow B)\}$ lze pomocí zákona implikace upravit na $\{A, (\neg A \vee B)\}$ a rezolventou těchto dvou klauzulí je literál B .

Příklad: Dokažte nespílitelnost množiny klauzulí:

- 1) $P(a, b)$
- 2) $\neg P(a, b) \vee \neg Q \vee R(c)$
- 3) $\neg R(c)$
- 4) $\neg T(a, c) \vee Q$
- 5) $T(a, c)$

Postup důkazu:

- | | | |
|----|--------------------|--------------------|
| 6) | $\neg Q \vee R(c)$ | rezolventa z 1 a 2 |
| 7) | $\neg Q$ | rezolventa z 6 a 3 |
| 8) | $\neg T(a, c)$ | rezolventa z 7 a 4 |
| 9) | $\square$ | rezolventa z 8 a 5 |

Podařilo se odvodit prázdnou klauzuli a tím je důkaz proveden.

3.3.1 Unifikace

V případě množiny klauzulí s proměnnými je situace s rezolucí složitější. Nejprve se musí provést tzv. unifikace, která spočívá ve vhodné substituci termů za proměnné:

- Substitucí σ je konečná množina dvojic $\{x_1/t_1, \dots, x_n/t_n\}$, kde x_i jsou proměnné a t_i termy.
- Instancí klauzule C je klauzule C_σ získaná z klauzule C náhradou každého výskytu proměnné x_i termem t_i ze substituce σ .
- Unifikátorem λ množiny literálů $\{L_1, L_2, \dots, L_k\}$ je substituce, pro kterou platí $L_{1\lambda} = L_{2\lambda} = \dots = L_{k\lambda} = L_\lambda$.

Jestliže unifikátor λ existuje, nazývá se množina unifikovatelnou.

Je nutné poznamenat, že v řadě případů může být unifikace provedena více unifikátory. Proto se zavádí pojem nejobecnějšího unifikátoru (MGU – Most General Unifier), což je unifikátor, ze kterého jdou všechny jiné unifikátory dalšími substitucemi odvodit.

Následující algoritmus pro unifikaci dvou klauzulí (obecně dvou výrazů E_1 a E_2) předpokládá zápis klauzulí ve tvaru seznamů, jejichž prvky jsou predikátové symboly následované termy (a pro termy/funkce pak platí stejná konvence zápisu):

Klauzule:	Odpovídající seznam:
$P(a, b)$	$(P \ a \ b)$
$P(f(a), g(x, y))$	$(P \ (f \ a) \ (g \ x \ y))$
$P(x) \vee Q(y)$	$((P \ x) \vee (Q \ y))$

Tento algoritmus (procedura Unify) buď nalezne nejobecnější unifikátor výrazů, nebo zjistí, že dané výrazy nejsou unifikovatelné.

Procedura $\text{Unify}(E_1, E_2)$ unifikuje výrazy/seznamy E_1 a E_2 , vrací buď nejobecnější unifikátor, nebo Fail:

1. E_1 i E_2 jsou buď konstanty nebo prázdné seznamy (současně):
 - Jestliže $E_1 = E_2$, vrátí $\{\}$ (tj. prázdnou substituci), jinak vrátí Fail (výrazy nelze unifikovat).
2. E_1 je proměnná:
 - Jestliže E_1 je proměnnou v E_2 , vrátí Fail, jinak vrátí substituci $\{E_1/E_2\}$.
3. E_2 je proměnná:
 - Jestliže E_2 je proměnnou v E_1 , vrátí Fail, jinak vrátí substituci $\{E_2/E_1\}$.
4. jinak:
 - 4.1. Volá proceduru $\text{Unify}(HE1, HE2)$, $HE1$ je první prvek E_1 a $HE2$ první prvek E_2 .
 - 4.2. Vratí-li procedura volaná v bodě 4.1. Fail, vrátí také Fail, jinak pokračuje.
Nechť SUBST1 je vrácená substituce procedurou volanou v bodě 4.1., dále nechť $TE1$ je zbytek seznamu E_1 (bez prvku $HE1$) a $TE2$ zbytek seznamu E_2 (bez prvku $HE2$), s respektováním substituce SUBST1 .
 - 4.3. Volá proceduru $\text{Unify}(TE1, TE2)$.
 - 4.4. Vratí-li procedura volaná v bodě 4.4. Fail, vrátí také Fail, jinak pokračuje.
Nechť SUBST2 je vrácená substituce procedurou volanou v bodě 4.3.
 - 4.5. Vratí kompozici substitucí SUBST1 a SUBST2 .

Příklad: Najděte nejobecnější unifikátor klauzulí

$\text{Parents}(x, \text{father}(x), \text{mother}(\text{john})), \text{Parents}(\text{john}, \text{father}(\text{john}), y)$

Postup řešení:

Obě klauzule se převedou na seznamy (budeme je zapisovat v kulatých závorkách)

$(\text{Parents } x \ (\text{father } x) \ (\text{mother john})), (\text{Parents john } (\text{father john}) \ y)$

a zavolá se procedura Unify:

```

Unify((Parents x (father x) (mother john)), (Parents john (father john) y))
  Unify(Parents, Parents)
  return { }
  Unify((x (father x) (mother john)), (john (father john) y))
    Unify(x, john)
    return {x/john}
    Unify(((father john) (mother john)), ((father john) y))
      Unify((father john), (father john))
        Unify(father ,father)
        return { }
      Unify((john), (john))
        Unify(john, john)
        return { }
      Unify(( ),( ))
      return { }
      Unify((mother john), y))
      return {y/(mother john)}
    return {y/(mother john)}
  return {x/john, y/(mother john)}
return {x/john, y/(mother john)}

```

Původní klauzule:	(Parents x (father x) (mother john)) (Parents john (father john) y)
Vrácená substituce:	{x/john, y/(mother john)}
Výsledné klauzule:	(Parents john (father john) (mother john)) (Parents john (father john) (mother john))

3.3.2 Zobecněná rezoluční metoda

Nechť C_1 a C_2 jsou klauzule, které nemají společné proměnné. Nechť C_1 je klauzule (množina literálů) obsahující unifikovatelnou podmnožinu literálů M_1 , kterou unifikátor λ_1 unifikuje do jediného literálu L_1 , nechť C_2 je klauzule (množina literálů) obsahující unifikovatelnou podmnožinu literálů M_2 , kterou unifikátor λ_2 unifikuje do jediného literálu L_2 a nechť tyto literály tvoří komplementární pár ($L_1 = \neg L_2$). Pak rezolventou klauzulí C_1 a C_2 je klauzule

$$(C_1\lambda_1 - M_1\lambda_1) \vee (C_2\lambda_2 - M_2\lambda_2)$$

Rezoluční metoda zaručuje odvození prázdné klauzule z nesplnitelné množiny výchozích klauzulí ve smyslu, že prázdnou klauzuli je možné odvodit. Závažnou otázkou však zůstává optimální výběr rodičovských klauzulí, neboť jinak může docházet k odvozování značného počtu zbytečných rezolvent. Tím se samozřejmě zvyšují nároky na paměť počítače a prodlužuje se i doba výpočtu. Je známo několik doporučovaných strategií, z nichž nejznámější a nejpoužívanější je lineární strategie.

Lineární strategie používá při odvozování nové rezolventy vždy poslední odvozenou rezolventu. Praktické použití této strategie je ukázáno na následujících příkladech.

Příklad 1: Dokažte nesplnitelnost množiny klauzulí:

- 1) $\neg A(x) \vee B(x) \vee C(x, f(x))$
- 2) $\neg A(x) \vee B(x) \vee D(f(x))$
- 3) $E(a)$
- 4) $A(a)$
- 5) $\neg C(a, y) \vee E(y)$
- 6) $\neg E(x) \vee \neg B(x)$
- 7) $\neg E(x) \vee \neg D(x)$

Postup důkazu:

- | | | | |
|-----|-----------------------------|--------------------|-------------------------|
| 8) | $B(a) \vee C(a, f(a))$ | rezolventa z 1, 4 | substituce $\{x/a\}$ |
| 9) | $C(a, f(a)) \vee \neg E(a)$ | rezolventa z 8, 6 | substituce $\{x/a\}$ |
| 10) | $\neg E(a) \vee E(f(a))$ | rezolventa z 9, 5 | substituce $\{y/f(a)\}$ |
| 11) | $E(f(a))$ | rezolventa z 10, 3 | |
| 12) | $\neg D(f(a))$ | rezolventa z 11, 7 | substituce $\{x/f(a)\}$ |
| 13) | $\neg A(a) \vee B(a)$ | rezolventa z 12, 2 | substituce $\{x/a\}$ |
| 14) | $B(a)$ | rezolventa z 13, 4 | |
| 15) | $\neg E(a)$ | rezolventa z 14, 6 | substituce $\{x/a\}$ |
| 16) | $\square$ | rezolventa z 15, 3 | |

Podařilo se odvodit prázdnou klauzuli a tím je důkaz proveden.

Příklad 2: Z následující množiny vět dokažte tvrzení „Někdo má spokojený život“.

1. Bohatí a zdraví lidé jsou šťastní.
2. Lidé, kteří sportují, jsou zdraví.
3. Emil sportuje a je bohatý.
4. Šťastní lidé mají spokojený život.

Postup při důkazu:

klauzule

1. Bohatí a zdraví lidé jsou šťastní:
 $\forall x((\text{Bohaty}(x) \wedge \text{Zdravy}(x)) \Rightarrow \text{Stastny}(x))$
 $\neg(\text{Bohaty}(x) \wedge \text{Zdravy}(x)) \vee \text{Stastny}(x)$
 $\neg\text{Bohaty}(x) \vee \neg\text{Zdravy}(x) \vee \text{Stastny}(x)$ (1)
2. Lidé, kteří sportují, jsou zdraví:
 $\forall y(\text{Sportuje}(y) \Rightarrow \text{Zdravy}(y))$
 $\neg\text{Sportuje}(y) \vee \text{Zdravy}(y)$ (2)
3. Emil sportuje a je bohatý.
 $\text{Sportuje}(\text{emil}) \wedge \text{Bohaty}(\text{emil})$
 $\text{Sportuje}(\text{emil})$ (3a)
 $\text{Bohaty}(\text{emil})$ (3b)
4. Šťastní lidé mají spokojený život.

$$\begin{aligned} & \forall z(\text{Stastny}(z) \Rightarrow \text{Spokojeny_zivot}(z)) \\ & \neg \text{Stastny}(z) \vee \text{Spokojeny_zivot}(z) \end{aligned} \quad (4)$$

Přidá se klauzule vytvořená negací dokazovaného tvrzení „Někdo má spokojený život“:

$$\begin{aligned} & \neg(\exists v(\text{Spokojeny_zivot}(v)) \\ & \forall v(\neg \text{Spokojeny_zivot}(v)) \\ & \neg \text{Spokojeny_zivot}(v) \end{aligned} \quad (5)$$

a dokáže se nesplnitelnost této (rozšířené) množiny:

$$\neg \text{Stastny}(v) \quad \text{r. 5, 4} \quad \text{s.} = \{z/v\} \quad (6)$$

$$\neg \text{Bohaty}(v) \vee \neg \text{Zdravy}(v) \quad \text{r. 6, 1} \quad \text{s.} = \{x/v\} \quad (7)$$

$$\neg \text{Zdravy}(\text{emil}) \quad \text{r. 7, 3b} \quad \text{s.} = \{v/\text{emil}\} \quad (8)$$

$$\neg \text{Sportuje}(\text{emil}) \quad \text{r. 8, 2} \quad \text{s.} = \{y/\text{emil}\} \quad (9)$$

$$\square \quad \text{r. 9, 3a} \quad (10)$$

Podařilo se odvodit prázdnou klauzuli a tím je důkaz proveden (zkratka „r.“ znamená „rezolventa z“ a zkratka „s.“ „substitute“).

Příklad 3: Z následujících dvou vět dokažte tvrzení „Karel má prarodiče“.

1. Každý má rodiče.
2. Prarodič je rodič rodiče.

Postup při důkazu:

1. Každý má rodiče.

$$\begin{aligned} & \forall x_1 \exists y_1(\text{Rodic}(x_1, y_1)) \\ & \forall x_1(\text{Rodic}(x_1, \text{primy_predek}(x_1))) \\ & \text{Rodic}(x_1, \text{Primy_predek}(x_1)) \end{aligned} \quad (1)$$

2. Prarodič je rodič rodiče.

$$\begin{aligned} & \forall x_2 \forall y_2 \forall z_2(\text{Rodic}(x_2, y_2) \wedge \text{Rodic}(y_2, z_2) \Rightarrow \text{Prarodic}(x_2, z_2)) \\ & \forall x_2 \forall y_2 \forall z_2(\neg \text{Rodic}(x_2, y_2) \vee \neg \text{Rodic}(y_2, z_2) \vee \text{Prarodic}(x_2, z_2)) \\ & \neg \text{Rodic}(x_2, y_2) \vee \neg \text{Rodic}(y_2, z_2) \vee \text{Prarodic}(x_2, z_2) \end{aligned} \quad (2)$$

Přidá se klauzule vytvořená negací dokazovaného tvrzení „Karel má prarodiče“:

$$\neg \text{Prarodic}(\text{karel}, x_3) \quad (3)$$

a dokáže se nesplnitelnost této (rozšířené) množiny:

$$\neg \text{Rodic}(\text{karel}, y_2) \vee \neg \text{Rodic}(y_2, x_3) \quad \text{r. 3, 2} \quad \text{s.} = \{x_2/\text{karel}/, z_2/x_3\} \quad (4)$$

$$\neg \text{Rodic}(\text{primy_predek}(\text{karel}), x_3) \quad \text{r. 4, 1} \quad \text{s.} = \{x_1/\text{karel}, y_2/\text{primy_predek}(\text{karel})\} \quad (5)$$

$$\begin{aligned} & \square \quad \text{r. 5, 1} \\ & \text{s.} = \{x_1/\text{primy_predek}(\text{karel}), x_3/\text{primy_predek}(\text{primy_predek}(\text{karel}))\} \end{aligned} \quad (6)$$

Podařilo se odvodit prázdnou klauzuli a tím je důkaz proveden (opět zkratka „r.“ znamená „rezolventa z“ a zkratka „s.“ „substitute“).

Stojí za povšimnutí, že Skolemova funkce, pomocí které byl odstraněn existenční kvantifikátor v klauzuli 1, má význam bezprostředního předka (při vyhodnocení by vracela bezprostředního předka osoby, která by byla jejím argumentem. Proto jsme tuto funkci nazvali `primy_predek`).

3.4 Hornovy klauzule

Závěrem této kapitoly se stručně zmíníme o Hornových klauzulích, které jsou základem jazyka PROLOG. Hornovy klauzule jsou klauzule, která obsahují nejvíce jeden pozitivní literál, tj. klauzule, které mohou být zapsány buď takto

$$((L_1 \wedge L_2 \wedge, \dots, \wedge L_n) \Rightarrow L) \Leftrightarrow (\neg L_1 \vee \neg L_2 \vee, \dots, \vee \neg L_n \vee L)$$

nebo takto

$$((L_1 \wedge L_2 \wedge, \dots, \wedge L_n) \Rightarrow T) \Leftrightarrow (\neg L_1 \vee \neg L_2 \vee, \dots, \vee \neg L_n)$$

kde $n \geq 0$.

V jazyku PROLOG představuje první z uvedených klauzulí pro $n = 0$ fakt a pro $n > 0$ pravidlo.

Pomocí druhé klauzule se zadává cíl, který se PROLOG snaží splnit.

Dokazování nesplnitelnosti množiny Hornových klauzulí je výrazně jednodušší než dokazování nesplnitelnosti množiny obecných formulí.

4 PROLOG

PROLOG patří mezi tzv. deklarativní programovací jazyky, ve kterých uživatel pouze zadá aktuální znalosti a cíl výpočtu, přičemž postup, jakým se tento výpočet provádí, je ponechán zcela na interpretu jazyka (systému). Základními přístupy, které PROLOG při výpočtu používá, jsou unifikace, rezoluce a backtracking.

PROLOG je považován za jeden ze dvou základních jazyků klasické UI (druhým je jazyk LISP). V této kapitole seznámíme blíže s jazykem SWI PROLOG. Postupně zde bude popsána syntax jazyka, postup při plnění cílů, některé důležité zabudované predikáty, postup při definici nových klauzulí, zvláště klauzulí pro abstraktní datové struktury (zásobník, frontu, množinu), a pro základní prohledávací algoritmy (BFS, DFS, A*).

4.1 Syntax jazyka

Syntax jazyka PROLOG je poměrně jednoduchá:

atom:	posloupnost alfanumerických znaků začínající malým písmenem, která může být uzavřena do apostrofů (například 'mlýnske kolo')
konstanta:	číslo integer číslo real atom
proměnná:	posloupnost alfanumerických znaků začínající velkým písmenem nebo znakem _ (podtržítkem)
term:	konstanta proměnná seznam řetěz predikát
argumenty:	term [, argumenty]
seznam:	[] [argumenty] [argumenty proměnná] [argumenty seznam]
řetězec:	posloupnost znaků uzavřená uvozovkami
predikát:	jméno(argumenty) [term] operátor [term] jméno
jméno:	atom
operátor:	definovaná posloupnost znaků
predikáty:	[not] predikát [, predikáty] [not] predikát [; predikáty]
pravidlo:	hlava :- tělo.
hlava:	predikát
tělo:	predikáty
fakt:	predikát.
klauzule:	fakt pravidlo
databáze:	množina klauzulí
otázka/cíl:	predikáty.
poznámka:	% libovolný řetěz znaků na zbytku řádku

Pomocné symboly (kulaté a hranaté závorky, čárka, středník, tečka) nejsou v předchozím výpisu syntaxe uvedeny samostatně, ale přímo na místech, kde se mohou používat. Mezery jsou nevýznamné.

4.2 Plnění cílů

Interpret PROLOGu vyzve uživatele dvojicí znaků ?- k zadání cíle/otázky. Zadání cíle se ukončí znakem CR (klávesa Enter), který je pokynem pro zahájení výpočtu, tj. činnosti spočívající v plnění cíle. Obvykle je prvním cílem načtení předem připravené databáze. Pokud je již databáze v systému uložena, pak další cíle spočívají v odvozování nových znalostí ze znalostí uložených v této databázi.

Pokud PROLOG zadaný cíl splní, pak odpoví buď slovem true. (pokud v zadaném cíli nebyla volná proměnná), nebo vypíše všechny proměnné s navázanými termy. Pokud tento výpis není ukončen tečkou, pak je možné plnění cíle buď ukončit (zadáním tečky, nebo stiskem klávesy Enter), nebo vyzvat systém k jinému/dalšímu plnění cíle (navázání jiných termů na volné proměnné) stiskem znaku ; (středníkem).

Pokud PROLOG zadaný cíl nesplní, odpoví slovem false. Je nutné poznamenat, že nesplnění cíle neznamená jeho negaci, ale pouze skutečnost, že z dané databáze nelze zadaný cíl splnit.

Plnění zadaného cíle se řídí následujícími pravidly:

1. *Cílem je predikát:*

PROLOG se snaží ztotožnit cíl s faktem nebo s hlavou pravidla (stejně jméno, stejný počet argumentů a stejné argumenty, volná proměnná se může vázat na libovolný term).

Cíl je splněn, jestliže se ztotožní:

- a) s faktem,
- b) s hlavou pravidla a současně lze splnit tělo pravidla (tělo pravidla je chápáno jako nový cíl, databáze se prohledává vždy od počátku).

2. *Cílem je konjunkce predikátů*

Jestliže je cílem konjunkce predikátů, snaží se PROLOG postupně (zleva doprava) splnit všechny tyto predikáty. Každý predikát představuje nový cíl a vázané proměnné jsou globální (stejně proměnné ve všech predikátech musí být vázané na stejný term!). Při prvním pokusu o plnění každého cíle C_i (prochází se zleva doprava) se databáze prochází od počátku:

- Pokud se cíl C_i nesplní, vrátí se PROLOG k předcházejícímu cíli C_{i-1} a pokouší se tento cíl splnit jiným způsobem (pokračuje v prohledávání databáze od aktuálního místa pro tento cíl).
- Pokud se cíl C_{i-1} splní, přechází PROLOG opět k cíli C_i (databázi pro tento cíl začne procházet znovu od začátku).
- Pokud se cíl C_{i-1} nesplní, vrací se PROLOG k předcházejícímu cíli C_{i-2} , neuspěje-li cíl C_{i-2} , vrací se PROLOG k cíli C_{i-3} atd. Jde o typickou aplikaci metody zpětného navracení – backtracking.

Pokud jsou splněny všechny cíle (zadaná konjunkce predikátů), je původní cíl splněn, v opačném případě je nesplněn.

3. *Cílem je disjunkce predikátů*

Jestliže je cílem disjunkce predikátů, snaží se PROLOG postupně (zleva doprava) splnit některý z nich. Databázi prochází pro každý cíl od počátku a proměnné jsou pro každý cíl lokální.

4.2.1 Jednoduchá konverzace

Příklad 1

Databáze:

girl(susan).
girl(anna).
boy(john).
boy(george).
boy(harry).
pair(B, G) :- boy(B), girl(G).

Konverzace:

?- boy(harry).	% V dotazech nejsou žádné volné proměnné
true.	% PROLOG odpovídá true. nebo false.
?- boy(james).	
false.	
?- pair(john, anna).	
true.	
?- pair(anna, john).	
false.	
?- boy(X).	% V dotazech jsou volné proměnné
X = john ;	% PROLOG odpovídá výpisem těchto proměnných
X = george ;	% s navázanými termíny
X = harry.	
?- girl(G).	
G = susan .	
?- pair(X, Y).	
X = john,	
Y = susan ;	
X = john,	
Y = anna ;	
X = george,	
Y = susan ;	
X = george,	
Y = anna ;	
X = harry,	
Y = susan ;	
X = harry,	
Y = anna.	

Příklad 2

Databáze:

likes(kate, roastbeef).
likes(kate, budgie).
likes(jane, juice).
likes(jack, jane).
likes(john, kate).

Konverzace:

```
?- likes(X, Y),likes(Y, budgie).      % Cíl obsahuje konjunkci dílčích cílů
X = john,
Y = kate.

?- likes(X, Y);likes(Y, budgie).      % Cíl obsahuje disjunkci dílčích cílů
X = kate,
Y = roastbeef ;
X = kate,
Y = budgie ;
X = jane,
Y = juice ;
X = jack,
Y = jane ;
X = john,
Y = kate ;
Y = kate.                             % Konec plnění prvního cíle
                                     % Plnění druhého cíle
```

4.3 Práce se seznamy

```
?- L=[a,1,X,b(c,d),[5,6,7],"halo"].    % Prvky seznamu mohou být libovolné termy
L = [a, 1, X, b(c, d), [5, 6, 7], "halo"]. % Mezery mezi termy jsou nevýznamné
?- L=[ ].                               % Prázdný seznam
L = [ ].

?- [a, b, c]=[H|T].                     % Použití operátoru | (svislé čárky)
H = a,                                  % Proměnná H (Head/Hlava) před operátorem |
T = [b, c].                             % se naváže na první prvky seznamu,

?- [a,b,c] = [H1,H2|T].                 % proměnná T (Tail/Tělo) se naváže na
H1 = a,                                 % zbytek seznamu
H2 = b,
T = [c].

?- [a,b,c] = [H1,H2,H3|T].               % proměnná T (Tail/Tělo) se naváže na
H1 = a,                                 % zbytek seznamu
H2 = b,
H3 = c,
T = [ ].

?- [a,b,c] = [H1,H2,H3,H4|T].
false.

?- T = [b,c,d], L = [a|T].               % Seznam lze nejen rozdělit, ale i spojit
T = [b, c, d],
L = [a, b, c, d].

?- [a,b,c,d] = [ _ |T].                  % Samostatné podtržítko je bezejmenná proměnná
T = [b, c, d].

?- [a,b,c,d] = [H| _ ].
H = a.

?- [a,b,c,d] = [H| _A].                  % Jinak jde o normální proměnnou
H = a,
_A = [b, c, d].
```

4.4 Definice nových klauzulí

Programování v jazyku PROLOG spočívá v definicích nových klauzulí/predikátů. Jednoduché klauzule pak mohou být použity ve složitějších programech:

Příklady:

```
prvek seznamu (member(X, S))           % prvek X je prvkem seznamu S
member(X,[X | _]).                       % pokud je jeho prvním prvkem, nebo
member(X,[_ | T]) :- member(X,T).        % pokud je prvkem jeho těla
*****

spojení dvou seznamů (append(S1,S2,S3)) % seznam S3 je spojením seznamů S1 a S2
append([ ],L,L).                         % pokud je první seznam prázdný, nebo
append([H|T1],L2,[H|T3]) :- append(T1,L2,T3). % se první prvek seznamu S1 přidá k výsledku
                                              % spojení seznamů T1 a L2, tj. k T3
*****

výpis termů seznamu (write_list(L))      % Jednotlivé termy seznamu L
write_list([ ]).                          % pro prázdný seznam hotovo, jinak
write_list([H|T]) :- write(H), nl, write_list(T). % se vypíše první prvek a rekurzivně zbytek
                                              % Predikát write vypíše term,
                                              % Predikát nl způsobí přechod na nový řádek
*****

reverzní výpis termů seznamu (reverse_write_list(L))
reverse_write_list([ ]).                  % Oproti předchozímu přímému výpisu se
reverse_write_list([H|T]) :-              % pouze přehodí predikáty ve druhé klauzuli
reverse_write_list(T), write(H), nl.
*****

předek osoby (ancestor(X,Y))             % osoba Y je předkem osoby X, jestliže Y je:
ancestor(X,Y) :- father(X,Y).             % otcem X, nebo
ancestor(X,Y) :- mother(X,Y).             % matkou X
ancestor(X,Y) :- father(X,Z), ancestor(Z,Y). % předkem otce X, nebo
ancestor(X,Y) :- mother(X,Z), ancestor(Z,Y). % předkem matky X
```

Uvažujme databázi, která obsahuje předchozí klauzule pro member, append a ancestor a následující fakty:

```
father(george,charles).
father(charles,robert).
father(robert,john).
father(eve,paul).
mother(george,eve).
mother(charles,jane).
mother(robert,mary).
mother(eve,anna).
```

Konverzace pak může probíhat například takto:

```
?- member(3,[1,2,3,4,5]).
```

```
true .
```

```
?- member(X,[a,b,c]).
```

```
X = a .
```

```
?- member(X,[a,b,c]).
```

```
X = a ;
```

```
X = b ;
```

```
X = c ;
```

```
false.
```

```
?- append([a,b,c],[d,e,f,g],L).
```

```
L = [a, b, c, d, e, f, g].
```

```
?- append(L,[e,f,g],[1,2,e,f,g]).
```

```
L = [1, 2]
```

% Aby byl seznam [1,2,e,f,g] spojením seznamu L
% a seznamu [e,f,g] musí seznam L = [1,2]

```
?- ancestor(george,X).
```

```
X = charles ;
```

```
X = eve ;
```

```
X = robert ;
```

```
X = jane ;
```

```
X = john ;
```

```
X = mary ;
```

```
X = paul ;
```

```
X = anna ;
```

```
false.
```

4.5 Vybrané zabudované predikáty

Konvence zápisů argumentů:

+ARG	Argument musí být navázán na term, který splňuje požadavky na typ tohoto argumentu („vstupní“ argument).
–ARG	Argumentem musí být volná proměnná („výstupní“ argument).
?ARG	Argumentem musí být term příslušného typu (buď „vstupní“, nebo „výstupní“ argument).

4.5.1 Predikáty pro práci s databází

consult(+Filename)	Čte klauzule ze souboru a přidává je na konec databáze. Cesta k souboru musí být v apostrofch
assert(+Clause)	Přidá klauzuli na konec databáze
retract(+Clause)	Odstraní první výskyt klauzule z databáze

Příklady použití:

```
?- consult('g:/prik1ad1.pl').    % V souboru prik1ad1.pl je databáze z předchozí kapitoly  
true.
```

```
?- assert(boy(james)).  
true.
```

```
?- boy(X).
X = james.
?- retract(boy(james)).
true.
?- boy(X).
false.
?- assert(a(X):- ancestor(X,eve)).
true.
?- a(M).
M = george ;
false.
```

4.5.2 I/O predikáty

tell(+Filename)	Otevírá soubor pro zápis
told	Uzavírá soubor otevřený pro zápis
write(+Term)	Zapíše term do souboru otevřeného pro zápis (standardně na obrazovku)
nl	Přejde na nový řádek (v zápisu!)
see(+Filename)	Otevírá soubor pro čtení
seen	Uzavírá soubor otevřený pro čtení
read(-Term)	Čte term ze souboru otevřeného pro čtení (standardně z klávesnice)

Příklady použití

```
?- tell('g:/aa.txt'),write(abc),write("."),write([1,2,3]), write("."),nl,write(a(b,c)),write("."),told.
true.
                                % Obsah souboru aa.txt je
                                %  abc.[1,2,3].
                                %  a(b,c).

?- see('g:/aa.txt'),read(X),read(Y),read(Z),read(V),read(W),seen.
X = abc.[1, 2, 3],
Y = a(b, c),
Z = V, V = W, W = end_of_file.
```

Při čtení termů ze souboru musí být každý term ukončen tečkou a mezi jednotlivými termy musí být oddělovače (mezery, nebo nové řádky). Jinak pokus o čtení způsobí chybu!

4.5.3 Predikáty – operátory

Operátory v jazyku PROLOG lze rozdělit na operátory aritmetické, operátory relační a operátory pracující s termy.

K aritmetickým operátorům patří standardní binární operátory: +, −, *, /, //, mod (sečítání, odečítání, násobení, dělení, celočíselné dělení a zbytek po celočíselném dělení) a operátor is, který aritmetické výrazy s těmito operátory vyhodnocuje.

K relačním operátorům patří standardní relační operátory, které vyhodnocují a porovnávají hodnoty dvou výrazů: <, <=, >, >=, =:=, =\= (menší než, menší nebo rovno, větší, větší nebo rovno, stejné, rozdílné).

Operátory pracující s termy jsou čtyři a jsou specifické pro PROLOG: =, ==, \=, \== (první operátor uspěje, jde-li termy ztotožnit (volné proměnné se přitom naváží!!!), druhý operátor uspěje, jsou-li termy stejné, třetí uspěje, pokud termy nelze ztotožnit a čtvrtý uspěje, pokud termy nejsou stejné. Všechny operátory se standardně zapisují v infixové notaci, mohou se však psát i v prefixové notaci, jako standardní predikáty.

Příklady použití

```
?- X is (15 - 6) * 3.    % Infixový zápis operátoru, vyhodnotí aritmetický výraz a jeho hodnotu
X = 27.                % přiřadí volné proměnné X

?- is(X,(15-6)*3).      % Prefixový zápis operátoru
X = 27.

?- X is 50.0 / 2.5.      % Výsledkem vyhodnocení výrazu je reálné číslo 20.0
X = 20.

?- 25 is 50 // 2.        % Výsledkem vyhodnocení výrazu je celé číslo 25
X = 25.

?- 4+3 > 9-5.           % Výrazy se vyhodnotí, podmínka splněna
true.

?- >(3+8, 4).           % Prefixový zápis operátorů, výrazy se vyhodnotí, podmínka splněna
true.

?- 4+3 < 9-5.           % Výrazy se vyhodnotí, podmínka nesplněna
false.

?- 3+4+2 == 1+1+2.      % Výrazy nejsou stejné
false.

?- 3+4+2 \= 1+1+2.      % Výrazy nejsou stejné
true.

?- S = 3 + 4.           % operátor = navázal volnou proměnnou S na term 3 + 4
S = 3 + 4.

?- L = [a,b], L == X.    % Proměnná L je vázána na seznam, proměnná X je volná,
false.                  % tj. termy (proměnné) nejsou stejné!

?- L = [a,b], L = X, L == X. % proměnná X byla navázána na proměnnou L, tj. termy
L = X, X = [a,b].       % L a X jsou stejné
```

4.5.4 Některé další užitečné predikáty

!	řez (cut); uspěje, ale nepovolí návrat
repeat	vždy uspěje
halt	ukončí konverzaci
not	neguje logickou hodnotu argumentu
bagof(T, +C, -S)	vrací seznam S všech instancí termu T, pro které uspěje cíl C

Příklady použití

Uvažujme stejnou databázi, jako v příkladu 1 (kap. 4.2.1) doplněnou vykřičníkem/řezem v pravidle pair a několika dalšími fakty ma(X,Y) s významem X má/vlastní Y:

```

girl(susan).
girl(anna).
boy(john).
boy(george).
boy(harry).
ma(karel,marii).
ma(honza,evu).
ma(karel,alika).
ma(jiri,zeryka).
ma(karel,nuz).
ma(honza,sirky).
ma(karel,zizen).
pair(BB, G) :- boy(BB),!, girl(G).
co_ma(X,Bag):-bagof(Y,ma(X,Y),Bag).

```

Konverzace pak může probíhat například takto:

```

?- pair(X,Y).
X = john,
Y = susan ;
X = john,
Y = anna.                % řez nedovolí návrat k jinému chlapci (george, harry)
?- co_ma(karel,Karel_ma).
Karel_ma = [marii, alika, nuz, zizen].
?- halt.

```

4.6 Abstraktní datové struktury (zásobník, fronta, množina)

empty([]). Vytvoření prázdného seznamu, nebo test na prázdný seznam
(stejně pro zásobník, frontu i množinu)

V následujících definicích klauzulí pro vložení a výběr prvku je prvním argumentem příslušného predikátu vkládaný, resp. vybíraný prvek (E, Element), druhým argumentem aktuální seznam (zásobník, fronta, množina) a třetím argumentem nový seznam po provedené operaci.

Zásobník (Stack)

push(E, T, [E T]).	Vložení prvku
pop(E, [E T], T).	Výběr prvku

Fronta (Queue)

enqueue(E, [], [E]).	Vložení prvku (na konec fronty)
enqueue(E, [H T2], [H T3]) :- enqueue(E, T2, T3).	Při návratech se přidávají předchozí prvky fronty
dequeue(E, [E T], T).	Výběr prvku

Fronta s prioritou (Priority Queue)

insert(E, [], [E]).	Vložení prvku
insert(E, [H T], [E,H T]) :- precedes(E, H).	
insert(E, [H T2], [H T3]) :- precedes(H, E), insert(E, T2, T3).	

precedes(A, B) :- A < B.

Platí pouze pro čísla!

dequeue(E, [E | T], T).

Výběr prvku

Množina (Set)

add(E, S, S) :- member(E, S), !.

Vložení prvku

add(E, S, [E | S]).

delete(E, [], []).

Výběr prvku

delete(E, [E | T], T) :- !.

delete(E, [H | T2], [H | T3]) :- delete(E, T2, T3).

V následujících klauzulích pro operace se dvěma množinami (průnik, sjednocení, rozdíl) jsou prvními dvěma argumenty predikátů původní množiny a třetím argumentem je výsledná množina.

intersection([], _, []).

Průnik dvou množin

intersection([H | T1], S2, [H | T3]) :- member(H, S2), !, intersection(T1, S2, T3).

intersection([H | T1], S2, S3) :- intersection(T1, S2, S3).

union([], S, S).

Sjednocení dvou množin

union([H | T], S2, S3) :- union(T, S2, S4), add(H, S4, S3).

difference([], _, []).

Rozdíl dvou množin

difference([H | T], S2, S3) :- member(H, S2), difference(T, S2, S3).

difference([H | T1], S, [H | T3]) :- not(member(H, S)), difference(T1, S, T3).

V následujících klauzulích pro porovnání dvou množin (podmnožina, rovnost) jsou argumenty příslušných predikátů obě původní množiny.

subset([], _).

Testuje, zda první množina je

subset([H | T], S) :- member(H, S), subset(T, S).

podmnožinou druhé

equal(S1, S2) :- subset(S1, S2), subset(S2, S1).

Testuje rovnost množin

4.7 Základní prohledávací algoritmy

Se znalostmi získanými z předchozího textu by již mělo být snadné porozumět níže uvedeným programům, které postupně implementují základní prohledávací algoritmy BFS a DFS a A*.

4.7.1 BFS (se seznamem CLOSED) - úloha dvou džbánů

Prvky seznamů Open a Closed jsou dvouprvkové seznamy. Prvním prvkem je daný uzel a druhým prvkem jeho bezprostřední předchůdce, např.: [[3,0],[0,0]], nebo [[0,0],-]]. Symbolem Start je označen počáteční uzel, seznam Goals označuje seznam cílů.

% část programu BFS nezávislá na konkrétní úloze:

StartBFS(Start,Goals):-

% Volání například: startBFS([0,0],[[0,2],[2,0]]).

 bfs([Start,-],[],Goals).

% Argumenty bfs(Open,Closed,Goals)

bfs([],_,_-):

% Fronta Open je prázdná, neúspěšné řešení

 write('No solution was found.').nl,!,

bfs(Open,Closed,Goals):-

% První stav v Open je stavem cílovým:

 dequeue([State,Parent],Open,_),

% vybere se první uzel/stav z Open

 member(State,Goals),

% zjistí, že je cílovým stavem

```

write('Solution has been found! '),
write('The path is:'),nl,
print_solution([State,Parent],Closed).           % vypíše se řešení (cesta)

bfs(Open,Closed,Goals):-                         % První stav v Open není stavem cílovým
    dequeue([State,Parent],Open,Rest_open),      % vybere první uzel/stav z Open
    get_children(State,Rest_open,Closed,Children), % provede se jeho expanze
    append(Rest_open,Children,New_open),          % následníci se připojí na konec Open
    append([[State,Parent]],Closed,New_closed),   % expandovaný uzel se uloží do Closed
    bfs(New_open,New_closed,Goals).              % volá se procedura bfs na nové obsahy
                                                % Open a Closed

dequeue(E,[E|T],T).                             % Výběr prvku z čela fronty

get_children(State,Rest_open,Closed,Children):-   % Expanze uzlu/stavu State
    bagof(Child,moves(State,Rest_open,Closed,Child),Children).

get_children(_,_,[ ]).

moves(State,Rest_open,Closed,[Next,State]):-     % Generování nového následníka, který
    move(State,Next),                             % není v Open (not je predikát negace)
    not(member([Next,_],Rest_open)),              % ani v Closed (not_member je predikát)
    not_member([Next,_],Closed).

not_member(H,L):-                                % Definice predikátu not_member
    member(H,L),!,fail.                          % Je-li prvkem neuspěje,
not_member(_,_).                                % jinak uspěje

member(H,[H|_]).                                % Definice predikátu member
member(H,[_|T]):-                               % (kap. 4.4)
    member(H,T).

append([],S,S).                                  % Definice predikátu append
append([H|T1],T2,[H|T3]):-                      % (kap. 4.4)
    append(T1,T2,T3).

print_solution([State,-],_):-                    % Reverzní výpis řešení
    write(State),nl.

print_solution([State,Parent],Closed):-          % Předchůdci uzlů se hledají v Closed
    member([Parent,Grandparent],Closed),
    print_solution([Parent,Grandparent],Closed),
    write(State),nl.

% část programu BFS závislá na konkrétní úloze: úloha dvou džbánů s obsahy 4 a 3 litry
move([V,M],[4,M]):-V<4.                         % Větší džbán lze naplnit, pokud není plný
move([V,M],[V,3]):-M<3.                         % Menší džbán lze naplnit, pokud není plný
move([V,M],[0,M]):-V>0.                         % atd.
move([V,M],[V,0]):-M>0.
move([V,M],[0,MN]):-V>0,M<3,3-M>=V,MN is M+V.
move([V,M],[VN,3]):-V>0,M<3,3-M<V,VN is V-3+M.
move([V,M],[VN,0]):-M>0,V<4,4-V>=M,VN is M+V.
move([V,M],[4,MN]):-M>0,V<4,4-V<M,MN is M-4+V.

a:-startBFS([0,0],[[0,2],[2,0]]).               % Definice predikátů pro jednodušší spouštění
b:-startBFS([0,0],[[0,1]]).

```

Ukázka řešení:

?- a.

The solution has been found! The path is:

[0,0]

[0,3]

[3,0]

[3,3]

[4,2]

[0,2]

true ;

The solution has been found! The path is:

[0,0]

[0,3]

[3,0]

[3,3]

[4,2]

[0,2]

[2,0]

true ;

No solution was found.

true .

?- b.

The solution has been found! The path is:

[0,0]

[4,0]

[1,3]

[1,0]

[0,1]

true ;

No solution was found.

true .

4.7.2 DFS - úloha dvou džbánů

Prvky seznamu Open jsou seznamy. Prvním prvkem v každém tomto seznamu je daný uzel a dalšími seznamy jsou jeho předchůdci, např.: [[4,3],[0,3],[0,0]]. Symbolem Start je označen počáteční uzel, seznam Goals označuje seznam cílů.

% část programu DFS nezávislá na konkrétní úloze:

startDFS(Start,Goals):-

dfs([Start],Goals).

% Volání například: startDFS([0,0],[[0,2],[2,0]]).

% Argumenty dfs(Open,Goals)

dfs([],_-

write('No solution was found. '),!

% Zásobník Open je prázdný, neúspěšné řešení

dfs(Open,Goals):-

% První stav v Open je stavem cílovým:

pop([State|Ancestors],Open,_),

% vybere první uzel/stav z Open

member(State,Goals),

% zjistí, že je cílovým stavem

write('The solution has been found! '),

write('The path is: '),!,nl,

print_solution([State|Ancestors]).

% vypíše řešení (jednotlivý následníci uzlu)

dfs(Open,Goals):-

% První stav v Open není stavem cílovým

pop([State|Ancestors],Open,Rest_open),

% vybere první uzel/stav z Open

get_children(State,Ancestors,Rest_open,Children),

% provede se jeho expanze

append(Children,Rest_open,New_open),

% následníci se připojí na začátek Open

dfs(New_open,Goals).

% volá se procedura dfs na nový obsah Open

pop(Top,[Top|Stack],Stack).

% Výběr prvku z čela zásobníku Open

get_children(State,Ancestors,Rest_open,Children):-

% Expanze uzlu/stavu State

bagof(Child,moves(State,Ancestors,Rest_open,Child),Children).

get_children(_,_,[]).

```

moves(State,Ancestors,Rest_open,[Next,State|Ancestors]):-
    move(State,Next),                % Generování nového následníka, který
    not_member(Next,Ancestors),      % není mezi předchůdci, ani
    not_member(Next,Rest_open).      % v zásobníku Open

not_member(H,L):-                    % Definice predikátu not_member
    member(H,L),!,fail.

not_member(_,_).

member(E,[E|_]).                    % Definice predikátu member
member(E,[H|_]):-member(E,H).      % pro obecný seznam, tj. seznam
member(E,[_|T]):-member(E,T).      % s vnořenými seznamy

append([],S,S).                     % Definice predikátu append
append([H|T1],T2,[H|T3]):-         % (kap. 4.4)
    append(T1,T2,T3).

print_solution([State]):-           % Reverzní výpis řešení
    write(State),nl.

print_solution([State|T]):-         % Předchůdci uzlů jsou přímo
    print_solution(T),              % v jejich seznamech
    write(State),nl.

% část programu DFS závislá na konkrétní úloze: úloha dvou džbánů s obsahy 4 a 3 litry

move([V,M],[4,M]):-V<4.             % Definice možných změn stavů
move([V,M],[V,3]):-M<3.
move([V,M],[0,M]):-V>0.
move([V,M],[V,0]):-M>0.
move([V,M],[0,MN]):-V>0,M<3,3-M>=V,MN is M+V.
move([V,M],[VN,3]):-V>0,M<3,3-M<V,VN is V-3+M.
move([V,M],[VN,0]):-M>0,V<4,4-V>=M,VN is M+V.
move([V,M],[4,MN]):-M>0,V<4,4-V<M,MN is M-4+V.

a:-startDFS([0,0],[[0,2],[2,0]]).    % Definice predikátů pro jednodušší
b:-startDFS([0,0],[[0,1]]).          % spouštění úloh

```

Ukázka řešení:

```

?- a.
Solution has been found! The path is:
[0,0]
[4,0]
[1,3]
[1,0]
[0,1]
[4,1]
[2,3]
[2,0]
true ;
Solution has been found! The path is:
[0,0]
[0,3]

```

```

?- b.
Solution has been found! The path is:
[0,0]
[4,0]
[1,3]
[1,0]
[0,1]
true .

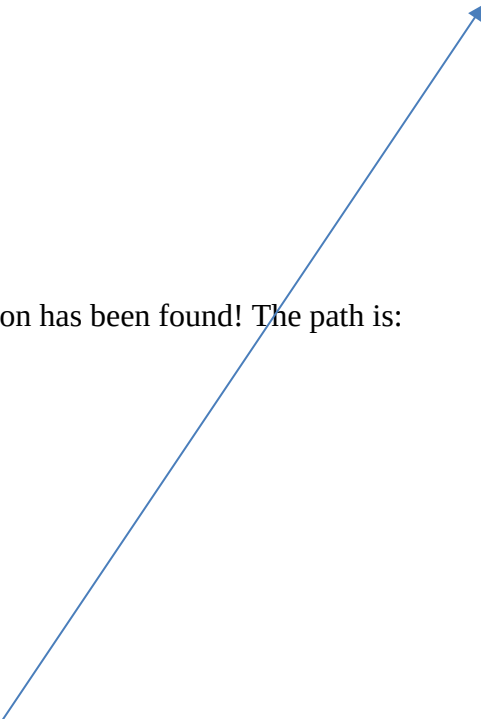
```

Při zadávání středníků najde i pro toto zadání pět dalších a jiných řešení!

```

[4,3]
[4,0]
[1,3]
[1,0]
[0,1]
[4,1]
[2,3]
[2,0]
true ;
Solution has been found! The path is:
[0,0]
[0,3]
[3,0]
[4,0]
[1,3]
[1,0]
[0,1]
[0,3]
[3,0]
[4,0]
[1,3]
[1,0]
[0,1]
[4,1]
[2,3]
[2,0]
true ;
Solution has been found! The path is:
[0,0]
[0,3]
[3,0]
[3,3]
[4,3]
[4,0]
[1,3]
[1,0]
[0,1]
[4,1]
[2,3]
[2,0]
true ;
Solution has been found! The path is:
[0,0]
[0,3]
[3,0]
[3,3]
[4,3]
[4,0]
[1,3]
[1,0]
[0,1]
[4,1]
[2,3]
[2,0]
true ;
No solution was found.
true .

```



4.7.3 Algoritmus A* (se seznamem CLOSED) – Loydův hlavolam

Každý stav je popsán seznamem [State,Parent,G,H,F], kde $F = G + H$. Symbolem Start je označen počáteční uzel, symbolem G je označen cílový uzel. Stav je označen seznamem pozic kamenů a volného políčka (po řádcích), pro Loydovu osmičku například [1,4,2,7,8,3,0,6,5] =

% část programu A* nezávislá na konkrétní úloze:

1	4	2
7	8	3
	6	5

```

startAstar(Start,Goal):-
    estimate(Start,Goal,0,_,H),          % G = 0, vypočítá H, F = G + H = H)
    astar([[Start,nil,0,H,H]],[],Goal).   % Argumenty astar(Open,Closed,Goal)
    astar([ ],_,_):-                     % Dále vše prakticky stejné, jako v metodě
        write('No solution was found.'),!. % BFS s jedinou výjimkou:

astar(Open,Closed,Goal):-
    dequeue([Goal,Parent|_],Open,_),
    write('Solution was found. The path is:'),nl,
    print_solution([Goal,Parent|_],Closed).

astar(Open,Closed,Goal):-
    dequeue([State,Parent,G,H,F],Open,Rest_open),
    get_children(State,G,F,Rest_open,Closed,Goal,[],Children),
    insert_to_open(Rest_open,Children,New_open), % Vložení do fronty s prioritou!!
    append([[State,Parent,G,H,F]],Closed,New_closed), % Na pozici v Closed nezáleží
    astar(New_open,New_closed,Goal).

dequeue(E,[E|T],T).

get_children(State,G,F,Rest_open,Closed,Goal,Temp,Children):- % Expanzi uzlu

```

```

move(State,Next),                                % lze provést i jiným způsobem, bez
not(member([Next|_],Closed)),                    % použití predikátu bagof,
not(member([Next|_],Temp)),
estimate(Next,Goal,G,Gnew,H),                   % navíc s výpočtem hodnot G a H
Fnew is Gnew+H,
get_children(State,G,F,Rest_open,Closed,Goal,    % Každý následník Next se připojí
[[Next,State,Gnew,H,Fnew]|Temp],Children),!.    % k seznamu Temp
get_children(_,_,_,_,_L,L).                     % Pokud již není další následník, tak se seznam Temp
                                                % zkopíruje do proměnné children
max(A,B,A):-A>=B.                                % Platí pouze pro čísla
max(A,B,B):-A<B.

insert_to_open(Open,[],Open).                   % Pro prázdný seznam se Open nemění
insert_to_open(Open,[H|T],Open_new):-           % Jinak se postupně vkládají jednotliví
insert(Open,H,Temp_open),                       % následníci H do Temp_open
insert_to_open(Temp_open,T,Open_new).

insert([],H,[H]).                               % Vložení do fronty podle hodnoty F
insert(Open,[Next,P,G,H,F],New_open):-          % stav již v Open je, musí se porovnat
member([Next,_,_,_],Open),!.                   % jejich hodnoty a v Open ponechat
insert_change(Open,[Next,P,G,H,F],New_open).    % pouze stav s lepším ohodnocení
insert([[[S1,P1,G1,H1,F1]|T1],[S2,P2,G2,H2,F2], % jinak porovnává hodnoty dvou prvních
[[S2,P2,G2,H2,F2],[S1,P1,G1,H1,F1]|T1]]):-      % prvků, pokud má druhý prvek lepší
F1>=F2.                                          % hodnocení, tak se s prvním vymění
insert([[[S1,P1,G1,H1,F1]|T1],[S2,P2,G2,H2,F2], % jinak se predikát insert zanořuje
[[S1,P1,G1,H1,F1]|T3]]):-                      % a hledá místo, kam prvek do fronty
F1<F2,                                          % s prioritou zařadit
insert(T1,[S2,P2,G2,H2,F2],T3).

%insert_change(Open,Child,Open_new), v Open ponechá ze dvou stejných stavů pouze ten
insert_change(Open,[Node,Pnew,Gnew,Hnew,Fnew],Open):- % s lepším ohodnocením
remove(Open,[Node,Pnew,Gnew,Hnew,Fnew],[Node,_,_,_Fold,_],_),
Fold=<Fnew.                                         % Lepší ohodnocení má starý stav
insert_change(Open,[Node,Pnew,Gnew,Hnew,Fnew],Open_new):-
remove(Open,[Node,Pnew,Gnew,Hnew,Fnew],[Node,_,_,_Fold],Open_temp),
Fold>Fnew,                                         % Lepší ohodnocení má nový stav
insert(Open_temp,[Node,Pnew,Gnew,Hnew,Fnew],Open_new).

remove([[[Node,Pold,Gold,Hold,Fold]|T],[Node,_,_,_],
[Node,Pold,Gold,Hold,Fold],T).                   % Odstraňovaný prvek je prvním prvkem
remove([H|T],[Node,_,_,_],[Node,Pold,Gold,Hold,Fold],[H|Ttemp]]:- % Jinak hledá stav
remove(T,[Node,_,_,_],[Node,Pold,Gold,Hold,Fold],Ttemp).         % v těle seznamu
member(H,[H|_]).
member(H,[_|T]):-member(H,T).
append([],S,S).
append([H|T1],T2,[H|T3]):-
append(T1,T2,T3).

```

```

print_solution([State,nil|_|_):-
    write1(State),nl.
print_solution([State,Parent|_|Closed):-
    member([Parent,Grandparent|_|Closed),
    print_solution([Parent,Grandparent|_|Closed),
    write1(State),nl.

% část programu A* závislá na konkrétní úloze (Lloydův hlavolam)
estimate(State,Goal,G,Gnew,H):-
    size(N),
    NN is N*N,
    evaluate(State,Goal,OK),H is NN-1-OK,Gnew is G+1. % Výpočet hodnot G, F a H
                                                    % Loydova osmička, nebo patnáctka
                                                    % G = G + 1, heuristika H1
evaluate([],[],0).
evaluate([o|TState],[o|TGoal],OK):-
    evaluate(TState,TGoal,OK).
evaluate([A|TState],[B|TGoal],OK):-
    A\=B,
    evaluate(TState,TGoal,OK).
evaluate([H|TState],[H|TGoal],OK):-
    H\=o,
    evaluate(TState,TGoal,TOK),
    OK is TOK+1.
                                                    % Zjištění počtu kamenů na správných pozicích
                                                    % Pozice volného políčka se nepočítá
                                                    % Na příslušných pozicích jsou různé kameny
                                                    % Na příslušných pozicích jsou stejné kameny
                                                    % a nejsou to prázdná políčka
                                                    % Zjistí se počet kamenů ve zbytku stavu
                                                    % a inkrementuje se počet OK
write1([]).
write1(L):-size(N),write2(L,N).
                                                    % Výpis řešení
                                                    % ve tvaru 3x3, nebo 4x4
write2([H|T],M):-M>0,write(H), write(' '),MM1 is M-1,write2(T,MM1).
write2(L,_):-nl,write1(L).
move(State,Next):- size(N),
    find_pos(State,SP),
    SP>N,
    NP is SP-N,
    change(State,NP,SP,Next).
                                                    % Přechzení rozměru (3/4 tj. 3x3/4x4)
                                                    % Nalezení pozice volného políčka
                                                    % Pohyb volným políčkem nahoru
                                                    % Nalezení pozice kamene ve stejném sloupci
                                                    % předcházejícího řádku a výměna
move(State,Next):- size(N),
    find_pos(State,SP),
    R is SP mod N,
    R\=0,
    NP is SP+1,
    change(State,NP,SP,Next).
                                                    % Pohyb volným políčkem doprava
move(State,Next):- size(N),
    find_pos(State,SP),
    NN is N*(N-1)+1,
    SP<NN,
    NP is SP+N,
    change(State,NP,SP,Next).
                                                    % Pohyb volným políčkem dolů
move(State,Next):- size(N),
    find_pos(State,SP),
    R is SP mod N,
    R\=1,

```

```

NP is SP-1,                                % Pohyb volným políčkem doleva
change(State,NP,SP,Next).

find_pos([o|_],1).                          % Políčko na první pozice, vrací se jednička,
find_pos([H|T],SP):-                        % pokud není na první pozici, hledá se dále
    H\=o,
    find_pos(T,SPM1),
    SP is SPM1+1.

change(State,NP,SP,Next):-                  % Výměna kamene a volného políčka
    move_sp(State,NP,Piece,Temp_State),     % Přesunutí volného políčka
    move_piece(Temp_State,SP,Piece,Next).    % Přesunutí kamene

move_sp([H|T],1,H,[o|T]).                  % Přesunutí volného políčka na pozici
move_sp([H|T],N,Piece,[H|TN]):-            % kamene a navrácení čísla kamene
    N>1, NM1 is N-1,
    move_sp(T,NM1,Piece,TN).

move_piece([_|T],1,H,[H|T]).                % Přesunutí kamene na původní pozici
move_piece([H|T],N,Piece,[H|TN]):-         % volného políčka
    N>1, NM1 is N-1,
    move_piece(T,NM1,Piece,TN).

a3:-assert(size(0)),retractall(size(_)),assert(size(3)),    % Definice predikátů pro jednodušší
    startAstar([8,1,3,o,2,4,7,6,5],[1,2,3,8,o,4,7,6,5]).    % spouštění

a4:-assert(size(0)),retractall(size(_)),assert(size(4)),
    startAstar([1,2,3,4,5,6,o,8,9,a,7,b,d,e,f,c],[1,2,3,4,5,6,7,8,9,a,b,c,d,e,f,o]).

```

Ukázka řešení:

?- a3.	?- a4.
Solution was found. The path is:	Solution was found. The path is:
8 1 3	1 2 3 4
o 2 4	5 6 o 8
7 6 5	9 a 7 b
	d e f c
o 1 3	
8 2 4	1 2 3 4
7 6 5	5 6 7 8
	9 a o b
1 o 3	d e f c
8 2 4	
7 6 5	1 2 3 4
	5 6 7 8
1 2 3	9 a b o
8 o 4	d e f c
7 6 5	
true.	1 2 3 4
	5 6 7 8
	9 a b c
	d e f o

5 Strojové učení

Strojové učení (ML, *Machine Learning*) je schopnost inteligentního systému měnit své znalosti tak, aby příště vykonával stejnou nebo podobnou činnost účinněji a efektivněji. Strojové učení lze rozdělit do tří základních skupin:

- Učení s učitelem (supervised learning)
- Učení bez učitele (unsupervised learning)
- Posilované učení (reinforcement learning)

Učení s učitelem spočívá v tom, že pro každý krok učení je známá požadovaná odezva a systém je tak okamžitě informován o aktuálním hodnocení jeho poslední akce. Učení se provádí na tzv. trénovací množině příkladů T , kdy každý příklad je představován množinou vstupních hodnot (vstupním vektorem) a množinou správných/požadovaných výstupních hodnot (výstupním vektorem):

$$T = \{(\vec{i}_1, \vec{d}_1), (\vec{i}_2, \vec{d}_2), \dots, (\vec{i}_p, \vec{d}_p)\},$$

$\vec{i}_i$... i -tý vstupní vektor

$\vec{d}_i$... i -tý výstupní vektor

Prvky vstupních i výstupních vektorů mohou nabývat číselných i symbolických hodnot.

Často se množina dostupných dat nepoužívá k trénování celá, ale rozdělí se na dvě nebo tři podmnožiny. První (trénovací, cca 80 % dat) se použije k trénování, druhá (testovací, cca 10 % až 20 % dat) se použije k testování a třetí (cca 10 % dat) se někdy použije k doladění parametrů.

Učení bez učitele spočívá v nalezení podobností ve vstupních vektorech trénovací množiny a žádná podpůrná informace není obvykle dostupná. Při učení se hledají shluky obrazů (*clusters*) představovaných mnohorozměrnými číselnými vstupními vektory trénovací množiny T .

$$T = \{\vec{i}_1, \vec{i}_2, \dots, \vec{i}_p\}$$

Posilované učení se od učení s učitelem odlišuje tím, že systém dostává ohodnocení až po provedení nějaké posloupnosti akcí. Toto ohodnocení může být pozitivní i negativní a také různě veliké. Podstatné je, že každá akce ovlivňuje i následující akce a systém musí sám zjistit, které jeho tahy byly dobré a které naopak špatné.

5.1 Učení s učitelem

V této kapitole budou stručně popsány principy tří metod učení s učitelem:

- Tvorba rozhodovacích stromů (*Decision Tree Building*)
- Prohledávání prostoru verzí (*Version Space Search*)
- Rozpoznávání a klasifikace obrazů (*Pattern Recognition and Classification*)

Všechny tyto metody pracují s vektory, které nabývají symbolických hodnot (případné číselné hodnoty jsou rovněž chápány jako symbolické) a jsou založené na předpokladu, že každá hypotéza, která vyhovuje dostatečně velké množině trénovacích příkladů, bude vyhovovat i dalším, dosud neznámým příkladům.

5.1.1 Rozhodovací stromy

Rozhodovací stromy slouží ke klasifikaci objektů na základě hodnot jejich atributů / vlastností. Jsou vytvářeny ze známé množiny příkladů a jsou známou metodou používanou při dolování dat (získávání znalostí z databází).

Princip metody bude nejlépe patrný z jednoduchého příkladu. Uvažujme tabulku 5.1 se čtrnácti položkami o jednotlivých klientech banky. První čtyři atributy (druhý až pátý sloupec) v každém řádku představují podmínkové atributy (vstupní vektory), atribut v posledním sloupci je rozhodovací atribut a představuje jednorvkový výstupní vektor. Atributy *Příjem*, *Historie úvěrů* a *Risk* jsou tříhodnotové, zbývající dva atributy (*Dluh* a *Ručení*) mají pouze dvě hodnoty.

Předpokládejme, že na základě těchto příkladů má bankovní úředník stanovit hodnotu rozhodovacího atributu (risku úvěru) pro nového klienta, zná-li hodnoty jeho podmínkových atributů.

Tabulka 5.1

Klient	Příjem	Dluh	Historie úvěrů	Ručení	Risk
1	< 15	vysoký	špatná	žádné	vysoký
2	15 – 35	vysoký	neznámá	žádné	vysoký
3	15 – 35	nízký	neznámá	žádné	přiměřený
4	< 15	nízký	neznámá	žádné	vysoký
5	> 35	nízký	neznámá	žádné	nízký
6	> 35	nízký	neznámá	adekvátní	nízký
7	< 15	nízký	špatná	žádné	vysoký
8	> 35	nízký	špatná	adekvátní	přiměřený
9	> 35	nízký	dobrá	žádné	nízký
10	> 35	vysoký	dobrá	adekvátní	nízký
11	< 15	vysoký	dobrá	žádné	vysoký
12	15 – 35	vysoký	dobrá	žádné	přiměřený
13	> 35	vysoký	dobrá	žádné	nízký
14	15 – 35	vysoký	špatná	žádné	vysoký

Úloha se zdá být na první pohled velmi jednoduchá. Je však nutné si uvědomit, že pro jednu kombinaci rozhodovacího atributu by měla úplná tabulka celkem 36 řádků (součin počtu hodnot jednotlivých podmínkových atributů $3 \cdot 2 \cdot 2 \cdot 3 = 36$) a že rozhodovací atribut se třemi různými hodnotami vede k 3^{36} možným různým tabulkám ($3^{36} \cong 1.5 \cdot 10^{17}$)! V reálné praxi pak podobné tabulky mají desítky i stovky vícehodnotových atributů a statisíce až miliony řádků.

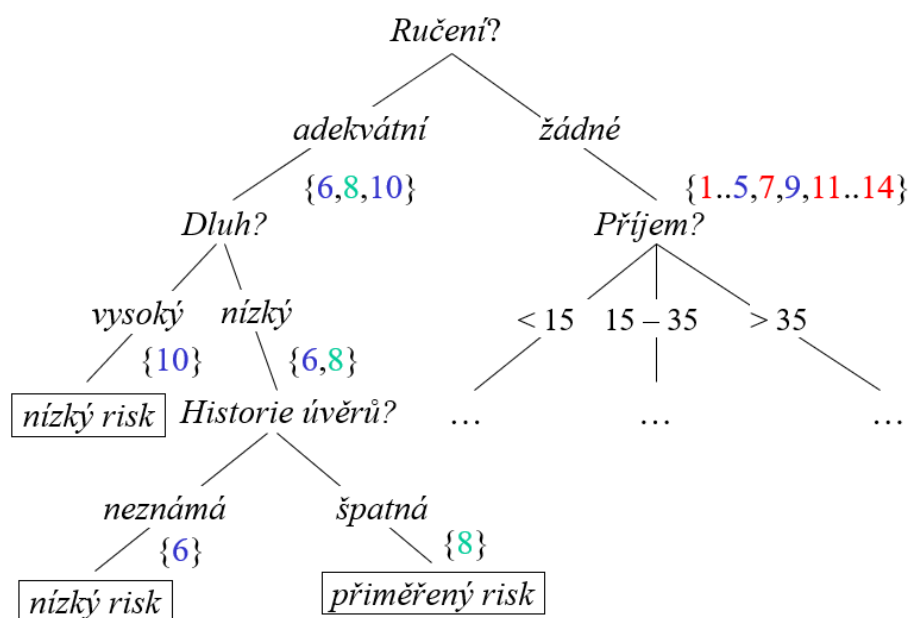
5.1.1.1 Algoritmus Decision Tree

Algoritmus Decision Tree je základním algoritmem pro budování rozhodovacích stromů. Má dva vstupní parametry (množinu příkladů *MP* a množinu podmínkových atributů *MA*) a vrací rozhodovací strom. Příklady se rozumí jednotlivé řádky tabulky.

Algoritmus Decision_tree:

1. Patří-li všechny prvky množiny příkladů MP do stejné třídy, vraťte listový uzel označený touto třídou, jinak pokračujte.
2. Je-li množina atributů MA prázdná, vraťte listový uzel označený disjunkcí všech tříd, do kterých patří prvky v množině příkladů MP , jinak pokračujte.
3. Vyberte atribut A_i , odstraňte jej z množiny atributů MA a učiňte jej kořenem aktuálního stromu. Necht' MA_{-i} je množina atributů MA bez atributu A_i .
4. Pro každou hodnotu h_{ji} vybraného atributu A_i :
 - a. Vytvořte novou větev stromu označenou hodnotou h_{ji} .
 - b. Volejte rekurzivně Decision_tree s parametry MP_{ji} a MA_{-i} , kde množina MP_{ji} je podmnožinou všech prvků množiny příkladů MP , které mají hodnotu h_{ji} atributu A_i .
 - c. Připojte vrácený podstrom/uzel k této větvi.

Algoritmus Decision_tree se zdá být velmi jednoduchý, skrývá však v sobě jeden zásadní problém, kterým je výběr atributu A_i v bodu 3. Nevhodné výběry vedou k hlubokým a neefektivním vyhledávacím stromům (Obr. 5.1), přestože optimální strom může být poměrně jednoduchý (Obr. 5.2).



Obr. 5.1 Příklad budování rozhodovacího stromu

5.1.1.2 Algoritmus ID3

Nejznámějším algoritmem, který řeší problém výběru atributu v kroku 4, je algoritmus známý pod zkratkou ID3 (*Iterative Dichotomiser 3*). Algoritmus ID3 je naprosto stejný jako algoritmus Decision Tree, pouze výběr atributu (bod 3 algoritmu) není náhodný a je založen na maximalizaci informačního zisku.

Z teorie informace je známo, že entropie neboli míra neuspořádanosti zprávy s n možnými odpověďmi $\{o_1, o_2, \dots, o_n\}$, jejichž pravděpodobnosti výskytu jsou $P(o_i)$, je dána výrazem

$$E(MP) = \sum_{i=1}^n -P(o_i) \log_2(P(o_i)) \quad [bit]$$

Zprávu MP představují příklady (hodnoty podmínkových atributů), odpověďmi jsou hodnoty rozhodovacího atributu.

Maximální hodnota entropie pro n možných odpovědí je dána výrazem $E_{\max}(M) = \log_2 n$. Této hodnoty se dosáhne, jsou-li všechny odpovědi stejně pravděpodobné. Pro dvě odpovědi je hodnota entropie 1 bit (tj. 1 bit je potřebný k odstranění neurčitosti zprávy), pro čtyři odpovědi je hodnota entropie 2 bity apod. Pokud je pravděpodobnost některé odpovědi nulová, resp. jedničková, přispívá tato odpověď k entropii nulovou hodnotou ($0 \cdot \log_2 0 = 0$, $1 \cdot \log_2 1 = 0$).

Podmínkový atribut A_i , který má m různých hodnot, rozděluje množinu příkladů MP do m disjunktních podmnožin, z nichž každá obsahuje MP_i příkladů.

Entropie, tj. očekávaná informace potřebná k dokončení stromu bude-li jeho kořenem atribut A_i , je pak dána výrazem

$$E(MP, A_i) = \sum_{i=1}^m \frac{|MP_i|}{|MP|} E(MP_i)$$

a odpovídající informační zisk IG (*Information Gain*) vztahem

$$IG(MP, A_i) = E(MP) - E(MP, A_i)$$

Vybraným atributem A_i podle bodu 3 algoritmu *Decision_tree* je pak atribut, který přináší největší informační zisk.

Podobně se postupuje v každém nelistovém uzlu. Při rekurzivním volání se atribut A_i z množiny atributů odstraní a za množinu příkladů MP se považuje podmnožina MP_i (viz bod 4 algoritmu).

Pro náš příklad platí (čísla na pravých stranách výrazů jsou čísla řádků tabulky 5.1, která jsou barevně označená podle hodnot rozhodovacího atributu):

$$MP = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14\}$$

$$MA = \{\text{Příjem}, \text{Historie úvěrů}, \text{Dluh}, \text{Ručení}\}$$

$$P_1 = P(\text{vysoký risk}) = 6/14, P_2 = P(\text{přiměřený risk}) = 3/14, P_3 = P(\text{nízký risk}) = 5/14$$

Entropie tabulky, tj. entropie množiny MP všech příkladů/řádků tabulky je

$$E(MP) = \sum_{i=1}^3 -P_i \log_2(P_i) = -\frac{6}{14} \log_2\left(\frac{6}{14}\right) - \frac{3}{14} \log_2\left(\frac{3}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right) = 1.531$$

Informační zisk s respektováním hodnot atributu *Příjem* (<15, 15-35, >35) se vypočítá takto:

$$MP_1 = \{1, 4, 7, 11\}, MP_2 = \{2, 3, 12, 14\}, MP_3 = \{5, 6, 8, 9, 10, 13\}$$

Pro všechny čtyři prvky/příklady podmnožiny MP_1 jsou hodnoty rozhodovacího atributu *Risk* stejné (*vysoký risk*), v podmnožině MP_2 se vyskytují dvakrát dva prvky se stejnými hodnotami pro tento atribut (2x *vysoký risk* a 2x *přiměřený risk*) a v podmnožině MP_3 má pět prvků stejnou hodnotu atributu (5x *nízký risk*) a jeden prvek hodnotou jinou (*přiměřený risk*). Pak

$$E(MP_1) = -\frac{4}{4} \log_2\left(\frac{4}{4}\right) = 0$$

$$E(MP_2) = -\frac{2}{4} \log_2\left(\frac{2}{4}\right) - \frac{2}{4} \log_2\left(\frac{2}{4}\right) = 1$$

$$E(MP_3) = -\frac{5}{6}\log_2\left(\frac{5}{6}\right) - \frac{1}{6}\log_2\left(\frac{1}{6}\right) = 0.650$$

$$\begin{aligned} E(MP, Příjem) &= \sum_{i=1}^3 \frac{|MP_i|}{|MP|} E(MP_i) = \frac{4}{14} E(MP_1) + \frac{4}{14} E(MP_2) + \frac{6}{14} E(MP_3) = \\ &= \frac{4}{14} \left(-\frac{4}{4} \log_2\left(\frac{4}{4}\right) \right) + \frac{4}{14} \left(-\frac{2}{4} \log_2\left(\frac{2}{4}\right) - \frac{2}{4} \log_2\left(\frac{2}{4}\right) \right) + \\ &\quad + \frac{6}{14} \left(-\frac{5}{6} \log_2\left(\frac{5}{6}\right) - \frac{1}{6} \log_2\left(\frac{1}{6}\right) \right) = 0.564 \end{aligned}$$

a informační zisk při použití tohoto atributu

$$IG(MP, Příjem) = E(MP) - E(Příjem) = 0.966$$

Informační zisk s respektováním hodnot atributu *Historie úvěrů* (neznámá, špatná, dobrá) se vypočítá stejným způsobem:

$$MP = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14\}$$

$$MP_1 = \{2, 3, 4, 5, 6\}, MP_2 = \{1, 7, 8, 14\}, MP_3 = \{9, 10, 11, 12, 13\}$$

$$E(MP_1) = -\frac{2}{5}\log_2\left(\frac{2}{5}\right) - \frac{1}{5}\log_2\left(\frac{1}{5}\right) - \frac{2}{5}\log_2\left(\frac{2}{5}\right) = 1.522$$

$$E(MP_2) = -\frac{1}{4}\log_2\left(\frac{1}{4}\right) - \frac{3}{4}\log_2\left(\frac{3}{4}\right) = 0.311 + 0.5 = 0.811$$

$$E(MP_3) = -\frac{3}{5}\log_2\left(\frac{3}{5}\right) - \frac{1}{5}\log_2\left(\frac{1}{5}\right) - \frac{1}{5}\log_2\left(\frac{1}{5}\right) = 1.371$$

$$\begin{aligned} IG(MP, Historie úvěrů) &= E(MP) - \sum_{j=1}^3 \frac{|MP_j|}{|MP|} E(MP_j) = \\ &= 1.531 - \frac{5}{14} E(MP_1) - \frac{4}{14} E(MP_2) - \frac{5}{14} E(MP_3) = 0.266 \end{aligned}$$

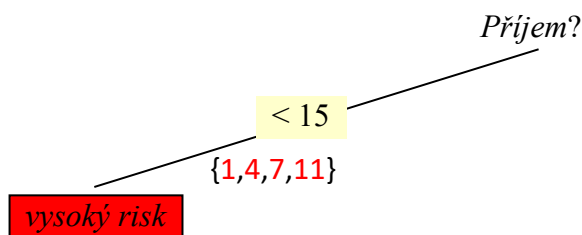
Informační zisk s respektováním odpovědi pro zbývající dva atributy se vypočítají podobně:

$$IG(MP, Dluh) = 0.063 \text{ bitů}$$

$$IG(MP, Ručení) = 0.206 \text{ bitů}$$

Nejvyšší informační zisk přináší výběr atributu *Příjem*. Proto se tento atribut vybere za kořen celého stromu a z množiny atributů *MA* se odstraní $MA = \{Historie\text{ úvěrů}, Dluh, Ručení\}$.

Pro první hodnotu atributu *Příjem* <15 se podle bodu 4 algoritmu ID3 vytvoří nová větev stromu a volá se algoritmus ID3 s množinou prvků $MP = MP_1 = \{1, 4, 7, 11\}$ a novou množinou atributů *MA*. Protože pro všechny čtyři prvky nové množiny *MP* jsou hodnoty rozhodovacího atributu stejné (*vysoký risk*), vrací algoritmus listový uzel označený touto hodnotou.



V množině prvků pro druhou hodnotu atributu $Příjem = 15 - 35$, tj. $MP = MP_2 = \{2,3,12,14\}$ se vyskytují dvakrát dvě různé hodnoty rozhodovacího atributu (*přiměřený risk* a *vysoký risk*) a entropie této množiny je dána vztahem

$$E(MP = MP_2) = -\frac{2}{4} \log_2 \left(\frac{2}{4} \right) - \frac{2}{4} \log_2 \left(\frac{2}{4} \right) = -\log_2 \left(\frac{2}{4} \right) = 1$$

Výpočtem informačního zisku IG pro jednotlivé atributy aktuální množiny MA se zjistí, že tento zisk je největší pro atribut *Historie úvěru*:

Historie úvěru (špatná/neznámá/dobrá): $MP_1 = \{14\}$, $MP_2 = \{2,3\}$, $MP_3 = \{12\}$,

$$E(MP_1) = E(MP_3) = -P_1 \log_2(P_1) = -P_3 \log_2(P_3) = -1 \cdot \log_2 1 = 0$$

$$E(MP_2) = \sum_{i=1}^2 -P_i \log_2(P_i) = -\frac{1}{2} \log_2 \left(\frac{1}{2} \right) - \frac{1}{2} \log_2 \left(\frac{1}{2} \right) = 1$$

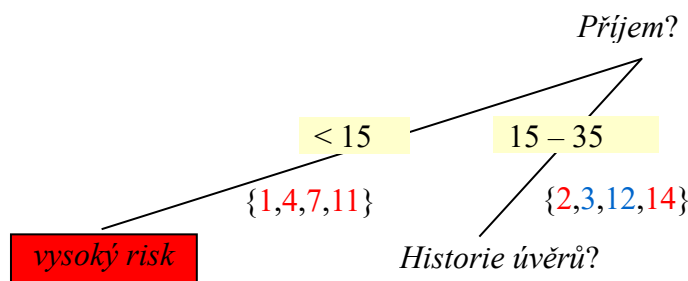
$$\begin{aligned} IG(MP, \text{Historie úvěrů}) &= E(MP) - \sum_{j=1}^3 \frac{|MP_j|}{|MP|} E(MP_j) = \\ &= 1 - \frac{1}{4} \cdot 0 - \frac{2}{4} \cdot 1 - \frac{1}{4} \cdot 0 = 0.5 \end{aligned}$$

Podobně se spočítají informační zisky pro zbývající dva atributy:

$$IG(MP, \text{Dluh}) = 0.311$$

$$IG(MP, \text{Ručení}) = 0$$

Atribut *Historie úvěrů* se proto vybere za počátek nového podstromu a z množiny atributů MA se odstraní ($MA = \{\text{Dluh}, \text{Ručení}\}$)



Pro první hodnotu atributu *Historie úvěrů* (špatná) se podle bodu 4 algoritmu ID3 vytvoří nová větev podstromu a volá se algoritmus ID3 s množinou příkladů $MP = \{14\}$ a aktuální množinou atributů $MA = \{\text{Dluh}, \text{Ručení}\}$.

Podle bodu 1. algoritmu ID3 se vrací listový uzel hodnotu *vysoký risk*.

Pro druhou hodnotu vlastnosti *Historie úvěrů* (neznámá) se vytvoří druhá větev podstromu a volá se algoritmus ID3 s množinou příkladů $MP = \{2,3\}$ a aktuální množinou atributů MA .

Pro atribut *Dluh* (*vysoký, nízký*) $MP_1 = \{2\}$, $MP_2 = \{3\}$ je informační zisk

$$IG(MP, \text{Dluh}) = E(MP) - \sum_{j=1}^2 \frac{|MP_j|}{|MP|} E(MP_j) = 1 - \frac{1}{2} \cdot 0 - \frac{1}{2} \cdot 0 = 1$$

a pro atribut *Ručení* (*žádné, adekvátní*) $MP_1 = \{2,3\}$, $MP_2 = \{\}$ je informační zisk

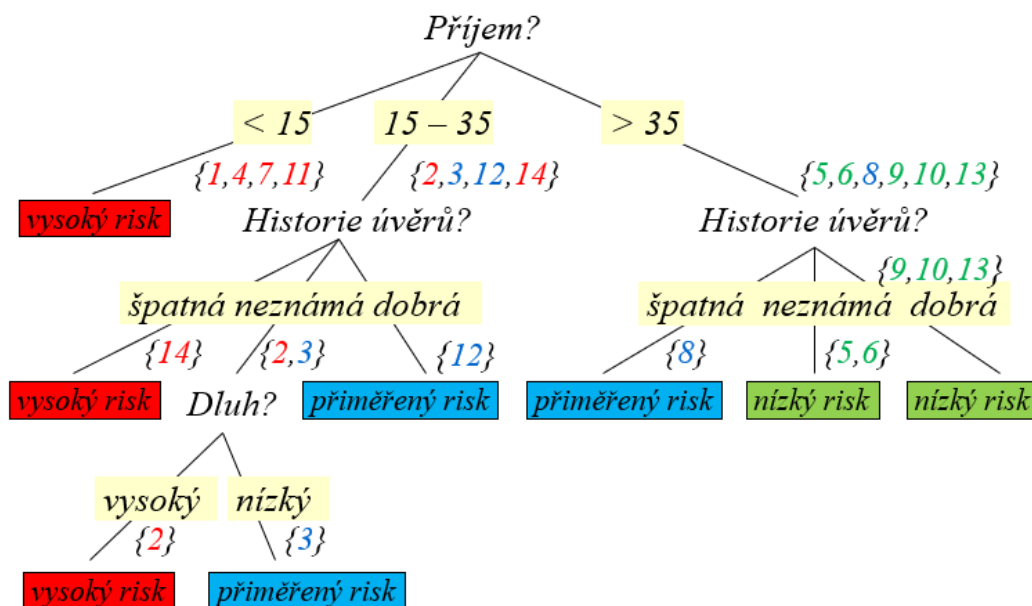
$$IG(MP, Ručení) = E(MP) - \sum_{j=1}^2 \frac{|MP_j|}{|MP|} E(MP_j) = 0 - \frac{1}{2} \cdot 0 - \frac{1}{2} \cdot 0 = 0$$

Informační zisk je nejvyšší pro atribut *Dluh*, a proto se tento atribut zvolí za počátek dalšího podstromu. Z množiny atributů se atribut *Dluh* odstraní ($MA = \{Ručení\}$).

Pro první hodnotu vlastnosti *Dluh* (*vysoký*) se vytvoří nová větev podstromu a volá se algoritmus ID3 s množinou příkladů $MP = \{2\}$ a aktuální množinou atributů $MA = \{Ručení\}$, který vrací listový uzel *vysoký risk*.

Stejně se postupuje pro druhou hodnotu atributu *Dluh* (*nízký*), kdy volaný algoritmus ID3 pro $MP = \{3\}$ a $MA = \{Ručení\}$ vrátí listový uzel *přiměřený risk*.

Podobně se poskytuje dále při dokončení větve Historie úvěrů *dobrá* (pro *Příjem* = 15-35) i při budování poslední větve uzlu *Příjem* > 35. Dokončený strom je ukázán na Obr. 5.2.



Obr. 5.2 Úplný a optimální rozhodovací strom

Vytvořený strom lze pak snadno převést na pravidla:

1. Má-li žadatel příjem nižší než 15 tis. Kč, je risk úvěru vysoký.
2. Má-li žadatel příjem mezi 15 až 35 tis. Kč, pak
 - a) je-li historie splácení jeho úvěrů špatná, je risk úvěru vysoký.
 - b) je-li historie splácení jeho úvěrů dobrá, je risk úvěru přiměřený.
 - c) je-li historie splácení jeho úvěrů neznámá, pak
 - je-li jeho dluh vysoký, je risk úvěru vysoký.
 - je-li jeho dluh nízký, je risk úvěru přiměřený.
3. Má-li žadatel příjem vyšší než 35 tis. Kč, pak
 - a) je-li historie splácení jeho úvěrů špatná, je risk úvěru přiměřený.
 - b) je-li historie splácení jeho úvěrů dobrá, je risk úvěru nízký.
 - c) je-li historie splácení jeho úvěrů neznámá, je risk úvěru nízký.

5.1.2 Prohledávání prostoru verzí

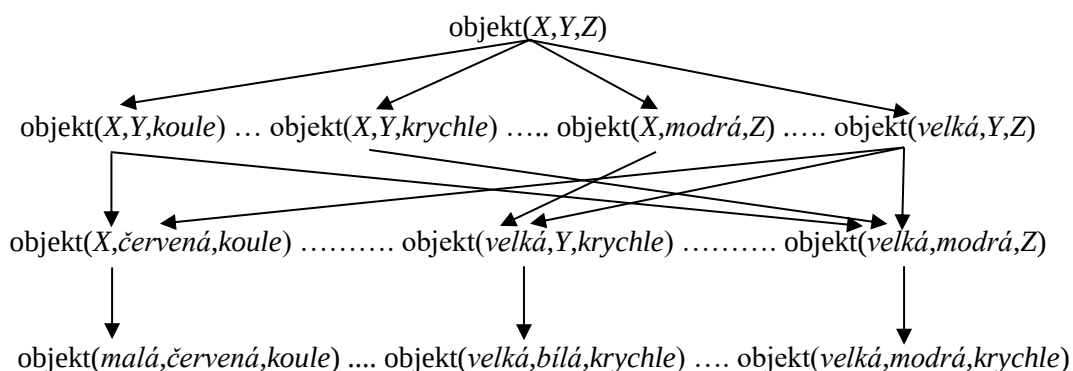
Prohledávání prostoru verzí představuje soubor metod učení významných pojmů/hypotéz na základě pozitivních a negativních příkladů (jde o aplikaci přístupů, které se používají při učení malých dětí). Při učení se hledá takový popis daného pojmu nebo hypotézy, který zahrnuje všechny pozitivní příklady a vylučuje všechny negativní příklady z trénovací množiny příkladů.

Princip metod bude zřejmý z jejich aplikací na jednoduchý příklad:

Předpokládejme obecný pojem *objekt*, u kterého jsme schopni určit velikost, barvu a tvar. Pro jednoduchost dále předpokládejme pouze dvě různé velikosti, tři různé barvy a tři různé tvary:

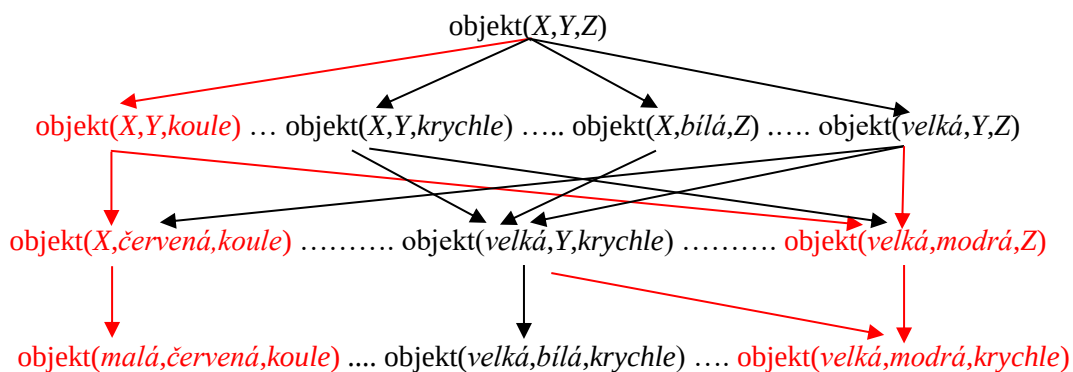
objekt(velikost, barva, tvar),
 velikost = {velká, malá},
 barva = {červená, bílá, modrá},
 tvar = {koule, kvádr, krychle}.

Úplný prostor verzí pro možné objekty výše uvedených vlastností je ukázán na Obr. 5.3.



Obr. 5.3 Prostor verzí pro uvažovaný příklad

Po pozitivním příkladu, například (velká, bílá, krychle) musí být prostor hypotéz/verzí redukován na (Obr. 5.4):



Obr. 5.4 Prostor verzí pro prvním kladným příkladem

Nejobecnějším pojmem je zde pojem objekt(X,Y,Z), méně obecnými (více specifikovanými) pojmy jsou zde pojmy se dvěma proměnnými (například objekt(X,bílá,Z)), ještě méně obecnými (více specifikovanými pojmy) jsou zde pojmy s jednou proměnnou (například objekt(malá,Y,kvádr)), až konečně nejvíce specifickými pojmy jsou zde pojmy bez proměnných

(například objekt(*velká,červená,koule*)). Poznamenejme, že z opačného směru pohledu jsou pojmy s jednou proměnnou méně specifikovanými (obecnějšími) pojmy a pojmy se dvěma proměnnými ještě méně specifikovanými (ještě obecnějšími) pojmy

Při prohledávání prostoru verzí se používají dvě operace: zobecňování (nahrazení konstant proměnnými) a specializace (nahrazení proměnných konstantami).

Předpokládejme nyní, že se umělý systém bude učit, co znamená pojem *míč*. Necht' trénovací příklady (musí být jak kladné, tak i záporné!) jsou tyto:

Příklad

- | | |
|---|---------|
| 1. objekt(<i>malá,červená,koule</i>) | kladný |
| 2. objekt(<i>malá,modrá,kvádr</i>) | záporný |
| 3. objekt(<i>velká,červená,koule</i>) | kladný |
| 4. objekt(<i>velká,červená,krychle</i>) | záporný |
| 5. objekt(<i>malá,modrá,koule</i>) | kladný |
| 6. objekt(<i>malá,bílá,kvádr</i>) | záporný |

Existují tři různé přístupy/metody učení prohledáváním prostoru verzí: od specifického k obecnějšímu pojmu (*specific to general search*), od obecného k více specifickému pojmu (*general to specific search*) a spojení obou předchozích přístupů (*candidate elimination*).

5.1.2.1 Algoritmus Specific_to_general_search

- Vytvořte dvě prázdné množiny S (Specific) a N (Negative).
- Uložte do množiny S první kladný příklad.
- Pro každý další příklad p z trénovací množiny
 - je-li p kladným příkladem, pak pro každý pojem $s \in S$
 - jestliže pojem s nelze unifikovat s příkladem p , pak jej nahraďte jeho nejvíce specifickým zobecněním, které lze unifikovat s příkladem p ,
 - odstraňte z S všechny pojmy s , které jsou více obecné než jiné pojmy v S ,
 - odstraňte z S všechny pojmy s , které lze unifikovat s některým pojmem v N .
 - je-li p záporným příkladem, pak
 - odstraňte z S všechny pojmy s , které lze unifikovat s příkladem p ,
 - přidejte příklad p do N .

Ukázka práce algoritmu Specific_to_general_search na příkladu:

- | | |
|---|--|
| | $S = \{\}, N = \{\}$ |
| 1. $p =$ objekt(<i>malá,červená,koule</i>) | $S = \{\text{objekt}(\textit{malá,červená,koule})\}$ |
| 2. $p =$ objekt(<i>malá,modrá,kvádr</i>) | $N = \{\text{objekt}(\textit{malá,modrá,kvádr})\}$ |
| 3. $p =$ objekt(<i>velká,červená,koule</i>) | $S = \{\text{objekt}(\textit{X,červená,koule})\}$ |
| 4. $p =$ objekt(<i>velká,červená,krychle</i>) | $N = \{\text{objekt}(\textit{malá,modrá,kvádr}),$
$\text{objekt}(\textit{velká,červená,krychle})\}$ |
| 5. $p =$ objekt(<i>malá,modrá,koule</i>) | $S = \{\text{objekt}(\textit{X,Y,koule})\}$ |
| 6. $p =$ objekt(<i>malá,bílá,kvádr</i>) | $N = \{\text{objekt}(\textit{malá,modrá,kvádr}),$
$\text{objekt}(\textit{velká,červená,krychle}),$
$\text{objekt}(\textit{malá,bílá,kvádr})\}$ |

Výsledek: nejvíce specifické zobecnění je **objekt(X,Y,koule)**.

5.1.2.2 Algoritmus General_to_specific_search:

- Vytvořte dvě prázdné množiny G (General) a P (Positive) a uložte do G nejobecnější pojem.
- Pro každý další příklad p z trénovací množiny
 - je-li p záporným příkladem, pak pro každý pojem $g \in G$
 - jestliže pojem g lze unifikovat s příkladem p , pak jej nahraďte jeho nejobecnější specializací, kterou nelze unifikovat s příkladem p ,
 - odstraňte z G všechny pojmy g , které jsou více specializované než jiné pojmy v G ,
 - odstraňte z G všechny pojmy g , které nelze unifikovat s některým pojmem v P .
 - je-li p kladným příkladem, pak
 - odstraňte z G všechny pojmy g , které nelze unifikovat s příkladem p ,
 - přidejte příklad p do P .

Ukázka práce algoritmu General_to_specific_search na příkladu:

	$G = \{\text{objekt}(X,Y,Z)\}, P = \{\}$
1. $p = \text{objekt}(\text{malá}, \text{červená}, \text{koule})$	$P = \{\text{objekt}(\text{malá}, \text{červená}, \text{koule})\}$
2. $p = \text{objekt}(\text{malá}, \text{modrá}, \text{kvádr})$	$G = \{\text{objekt}(\text{velká}, Y, Z),$ $\text{objekt}(X, \text{červená}, Z), \text{objekt}(X, \text{bílá}, Z),$ $\text{objekt}(X, Y, \text{koule}), \text{objekt}(X, Y, \text{krychle})\}$
	$\Rightarrow G = \{\text{objekt}(X, \text{červená}, Z), \text{objekt}(X, Y, \text{koule})\}$
3. $p = \text{objekt}(\text{velká}, \text{červená}, \text{koule})$	$P = \{\text{objekt}(\text{malá}, \text{červená}, \text{koule}),$ $\text{objekt}(\text{velká}, \text{červená}, \text{koule})\}$
4. $p = \text{objekt}(\text{velká}, \text{červená}, \text{krychle})$	$G = \{\text{objekt}(\text{malá}, \text{červená}, Z),$ $\text{objekt}(X, \text{červená}, \text{kvádr}),$ $\text{objekt}(X, \text{červená}, \text{koule}),$ $\text{objekt}(X, Y, \text{koule})\}$
	$\Rightarrow G = \{\text{objekt}(X, Y, \text{koule})\}$
5. $p = \text{objekt}(\text{malá}, \text{modrá}, \text{koule})$	$P = \{\text{objekt}(\text{malá}, \text{červená}, \text{koule}),$ $\text{objekt}(\text{velká}, \text{červená}, \text{koule}),$ $\text{objekt}(\text{malá}, \text{modrá}, \text{koule})\}$
6. $p = \text{objekt}(\text{malá}, \text{bílá}, \text{kvádr})$	$G = \{\text{objekt}(X, Y, \text{koule})\}$

Výsledek: nejvíce obecná specializace je **objekt(X,Y,koule)**

5.1.2.3 Algoritmus Candidate_elimination

- Vytvořte dvě prázdné množiny G (General) a S (Specific) a uložte do G nejobecnější pojem.
- Uložte do množiny S první kladný příklad.
- Pro každý další příklad p z trénovací množiny
 - je-li p kladným příkladem pak
 - odstraňte z G všechny pojmy $g \in G$, které nelze unifikovat s příkladem p ,
 - pro každý pojem $s \in S$
 - jestliže pojem s nelze unifikovat s příkladem p , pak jej nahraďte jeho nejvíce specifickým zobecněním, které lze unifikovat s příkladem p ,
 - odstraňte z S všechny pojmy s , které jsou více obecné než jiné pojmy v S ,
 - odstraňte z S všechny pojmy s , které nejsou více specifické, než některé pojmy v G .

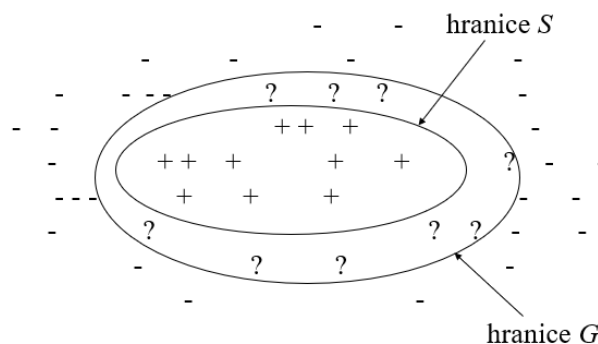
- je-li p záporným příkladem:
 - odstraňte z S všechny pojmy s , které lze unifikovat s příkladem p ,
 - pro každý pojem $g \in G$
 - jestliže pojem g lze unifikovat s příkladem p , pak jej nahraďte jeho nejvíce zobecněnou specializací, kterou nelze unifikovat s příkladem p ,
 - odstraňte z G všechny pojmy g , které jsou více specifické než jiné pojmy v G ,
 - odstraňte z G všechny pojmy g , které nejsou více obecné než některé pojmy v S .
- Jestliže $G = S$ a obě množiny přitom obsahují jediný pojem, pak výsledkem učení je právě tento pojem.

Ukázka práce algoritmu Candidate_elimination na příkladu:

- | | |
|--|---|
| | $G = \{\text{objekt}(X,Y,Z)\}, S = \{\}$ |
| 1. $p = \text{objekt}(\text{malá}, \text{červená}, \text{koule})$ | $S = \{\text{objekt}(\text{malá}, \text{červená}, \text{koule})\}$ |
| 2. $p = \text{objekt}(\text{malá}, \text{modrá}, \text{kvádr})$ | $G = \{\text{objekt}(\text{velká}, Y, Z), \text{objekt}(X, \text{červená}, Z),$
$\text{objekt}(X, \text{bílá}, Z), \text{objekt}(X, Y, \text{koule}),$
$\text{objekt}(X, Y, \text{krychle})\}$ |
| | $\Rightarrow G = \{\text{objekt}(X, \text{červená}, Z), \text{objekt}(X, Y, \text{koule})\}$ |
| 3. $p = \text{objekt}(\text{velká}, \text{červená}, \text{koule})$ | $S = \{\text{objekt}(X, \text{červená}, \text{koule})\}$ |
| 4. $p = \text{objekt}(\text{velká}, \text{červená}, \text{krychle})$ | $G = \{\text{objekt}(\text{malá}, \text{červená}, Z),$
$\text{objekt}(X, \text{červená}, \text{kvádr}),$
$\text{objekt}(X, \text{červená}, \text{koule}),$
$\text{objekt}(X, Y, \text{koule})\}$ |
| | $\Rightarrow G = \{\text{objekt}(X, Y, \text{koule})\}$ |
| 5. $p = \text{objekt}(\text{malá}, \text{modrá}, \text{koule})$ | $S = \{\text{objekt}(X, Y, \text{koule})\}$ |
| 6. $p = \text{objekt}(\text{malá}, \text{bílá}, \text{kvádr})$ | $G = \{\text{objekt}(X, Y, \text{koule})\}$ |

$G = S$ a obě množiny obsahují jediný pojem: **objekt(X,Y,koule)**

Význam a vztah množin S a G je ukázán na Obr. 5.5. Každý pojem, který by byl obecnější než nějaký pojem v G , by zahrnoval některé negativní příklady, každý pojem, který by byl specifitější než nějaký pojem v S , by vylučoval některé pozitivní příklady.

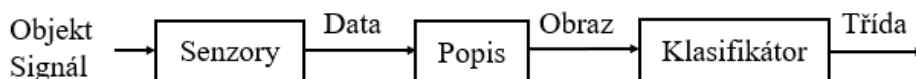


Obr. 5.5 Vztah množin S (Specific) a G (General)

Výsledný pojem $\text{objekt}(X,Y,koule)$ akceptuje všechny kladné příklady a současně vylučuje všechny záporné příklady z trénovací množiny a je u všech algoritmů stejný: míčem je kulatý objekt, na jehož barvě, ani velikosti nezáleží.

5.1.3 Rozpoznávání a klasifikace

Rozpoznávání vzorů (*pattern recognition*) je relativně samostatná část strojového učení, jejímž cílem je naučit se rozpoznávat objekty (jevy, signály apod.) na základě jejich obrazů/vzorů (Obr. 5.6). Rozpoznávání je poměrně široký pojem, který zahrnuje například rozpoznávání obličejů osob, rozpoznávání struktur DNA, rozpoznávání řeči, rozpoznávání objektů na scéně a jejich vzájemných vztahů a jiné. Klasifikací se pak myslí zařazení objektů, resp. jejich obrazů do předem daných tříd, například podle velikosti a barvy objektu, délky trvání děje a podobně.

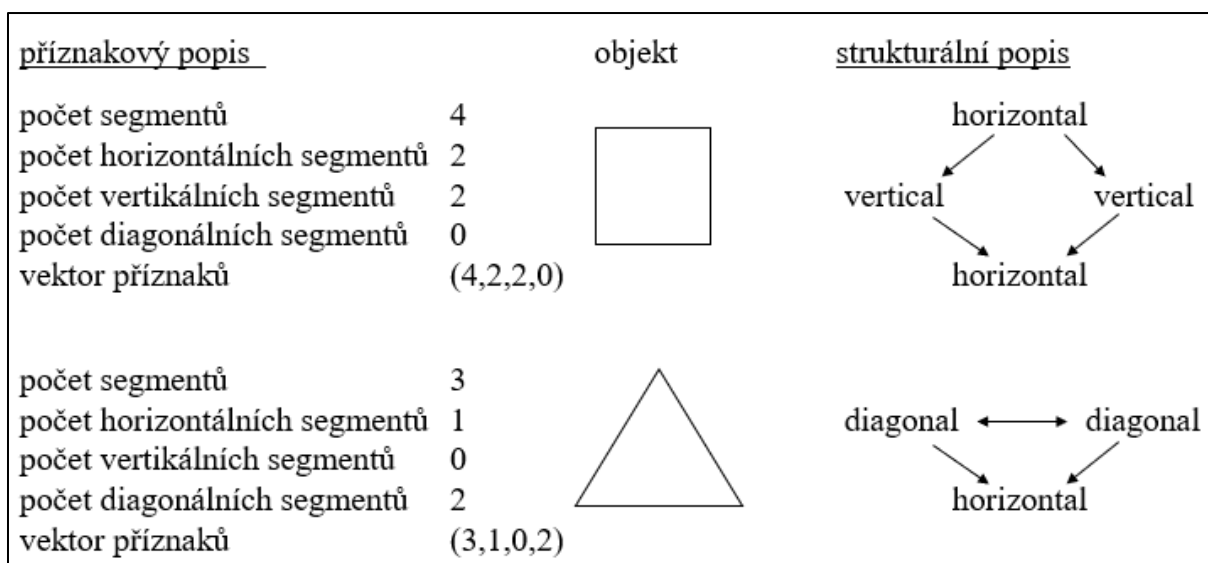


Obr. 5.6. Vztah mezi objektem a jeho obrazem

Existují dva základní typy popisů obrazů (Obr. 5.7):

- Příznakový (popis vektory číselných příznaků)
- Strukturální/syntaktický (popis strukturálními prvky, tzv. primitivy)

V prvním případě se pro rozpoznávání využívá statistických informací o příznacích obrazů obsažených v množině trénovacích dat a jde o tzv. statistické příznakové rozpoznávání, ve druhém případě se pro rozpoznání využívá informací o vztazích mezi příznaky obrazů rozpoznávaných struktur a jde o tzv. strukturální (syntaktické) rozpoznávání.



Obr. 5.7. Příklad příznakového a strukturálního popisu stejných objektů

Zásadním předpokladem úspěšného rozpoznávání je výběr relevantních příznaků, resp. primitiv. Tento výběr je přirozeně problémově závislý a záleží výhradně na zkušenosti uživatele, který příznaky/primitiva popisující objekty nebo jevy vybírá.

5.1.3.1 Příznakové rozpoznávání

Předpokládejme, že obrazy rozpoznávaných objektů nebo jevů jsou n rozměrné číselné vektory

$$\vec{x} = (x_1, x_2, \dots, x_n).$$

Dále předpokládejme, že máme k dispozici trénovací množinu dvojic

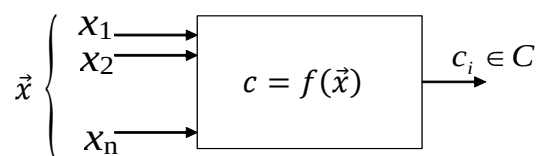
$$T = \{(\vec{x}_1, c_1), (\vec{x}_2, c_2), \dots, (\vec{x}_P, c_k)\}$$

kde první prvek každé dvojice představuje obraz (vektor příznaků) a druhý prvek označuje jeho třídu z předem dané množiny R tříd:

$$C = \{c_1, c_2, \dots, c_R\}$$

Cílem příznakového rozpoznávání/klasifikace je určit třídu c_i , do které vektor příznaků $\vec{x}$ patří $\vec{x} \rightarrow c_i, c_i \in C$.

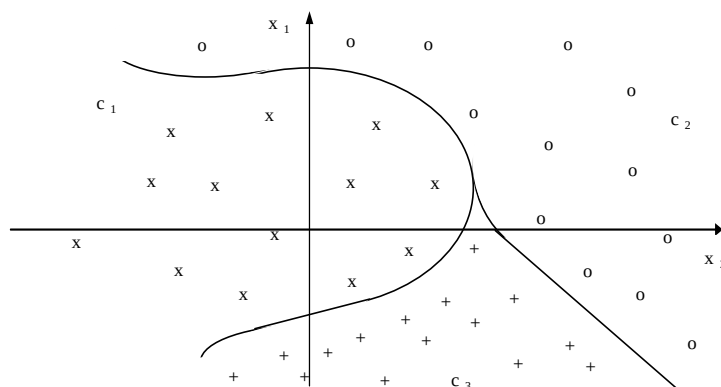
Metody příznakového rozpoznávání/klasifikace vycházejí z předpokladu, že obrazy objektů nebo jevů stejných tříd tvoří v n rozměrném obrazovém prostoru shluky. Tyto shluky mohou být od sebe zřetelně oddělitelné (*separable*), nebo se mohou prolínat a pak jsou neoddělitelné (*inseparable*). Systémy, které se na trénovací množině naučí obrazy rozpoznávat a poté klasifikují nové obrazy, se nazývají klasifikátory (Obr. 5.8).



Obr. 5.8 Příznakový klasifikátor

5.1.3.1.1 Rozpoznávání obrazů reprezentovaných oddělitelnými třídami obrazů

Příklad tří oddělitelných tříd obrazů ve dvourozměrném obrazovém prostoru je na následujícím obrázku (Obr. 5.9).



Obr. 5.9 Oddělitelné třídy obrazů ve dvourozměrném obrazovém prostoru

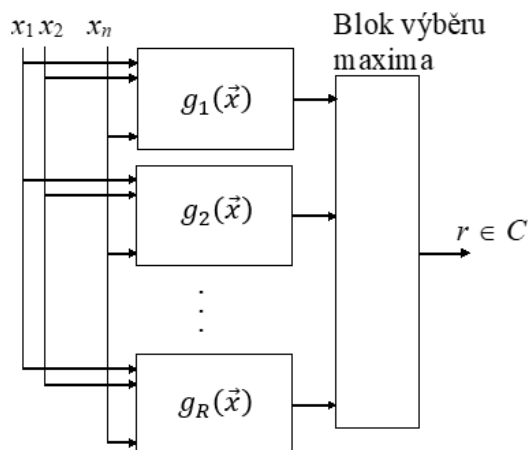
Učení klasifikátoru spočívá v nalezení křivek (obecně n rozměrných ploch), které obrazy jednotlivých tříd od sebe oddělují. Nejčastěji se přitom vychází z diskriminačních/rozlišujících funkcí g , které pro obraz (vektor příznaků) patřící do třídy r splňují podmínku

$$g_r(\vec{x}) > g_s(\vec{x}), \quad r, s \in C, \quad r \neq s.$$

Je zřejmé, že pro plochy rozděluující třídy r a s musí platit vztah

$$g_r(\vec{x}) = g_s(\vec{x}), \quad r, s \in C, r \neq s$$

Činnost klasifikátoru je tedy zcela jednoduchá: Pro daný vstupní obraz/vektor vypočítá hodnoty všech R diskriminačních funkcí a klasifikuje příslušný obraz/vektor do třídy, která přísluší diskriminační funkci s největší výstupní hodnotou (Obr. 5.10).



Obr. 5.10 Příznakový klasifikátor založený na diskriminačních funkcích:

V praxi se často používá klasifikace do dvou tříd (*dichotomie*). Pak je možné použít jedinou diskriminační funkci danou rozdílem dvou diskriminačních funkcí a klasifikovat pouze podle znaménka hodnoty této funkce:

$$\begin{aligned} g(\vec{x}) &= g_1(\vec{x}) - g_2(\vec{x}) \\ g(\vec{x}) &> 0 \quad \text{pro } c_{\vec{x}} = 1 \\ g(\vec{x}) &< 0 \quad \text{pro } c_{\vec{x}} = 2 \end{aligned}$$

Existuje rozsáhlá teorie, která se zabývá problematikou hledání diskriminačních funkcí. Dále si ukážeme pouze algoritmus učení lineárního klasifikátoru pro dichotomii, který hledá koeficienty lineární diskriminační funkce

$$g(\vec{x}) = g_0 + g_1x_1 + g_2x_2 + \dots + g_nx_n = g_0 + \sum_{i=1}^n g_ix_i = g_0 + \vec{g}\vec{x}$$

Učení lineárního klasifikátoru pro dichotomii

Nechť $T = \{(\vec{x}_1, c_1), (\vec{x}_2, c_2), \dots, (\vec{x}_P, c_P)\}$, $\vec{x}_i = (x_{i0}, x_{i1}, x_{i2}, \dots, x_{in})$, $x_{i0} = 1$, $c_j \in \{-1, 1\}$ je trénovací množina a $\mu \in (0, 1)$ je koeficient učení (*Learning rate*):

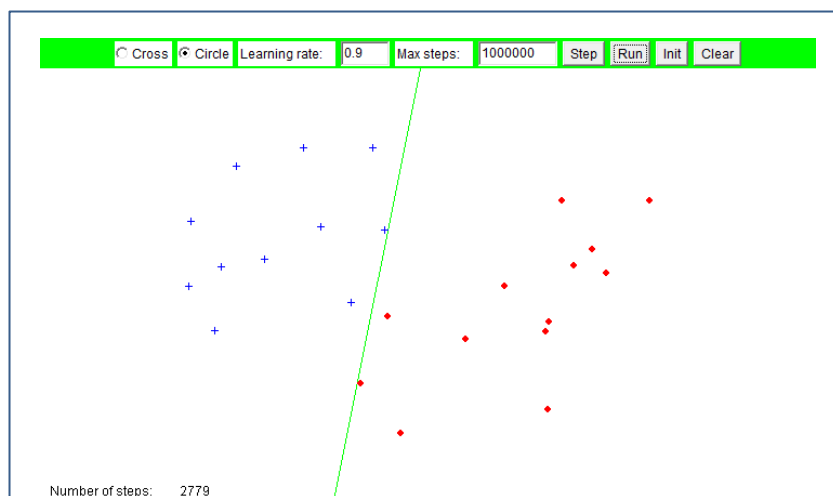
1. Vynulujte vektor vah.
2. Nastavte indikátor změny *modif* = false.
3. Pro každý vektor $\vec{x}_i$ z trénovací množiny ($i = 1 \dots P$), který není správně klasifikován ($\text{sign}(g(\vec{x}_i)) \neq c_i$) upravte vektor vah podle vztahu

$$\vec{g} := \vec{g} + \frac{\mu c_i \vec{x}_i}{\|\vec{x}_i\|} \quad \|\vec{x}\| = \sqrt{x_0^2 + x_1^2 + \dots + x_n^2}$$

a nastavte indikátor změny *modif* = true.

4. Pokud došlo k úpravě vektoru vah (*modif* = true), tak se vraťte na bod 2.

Ukázka rozdělení dvourozměrného obrazového prostoru lineárním klasifikátorem pro dichotonomii je na Obr. 5.11.



Obr. 5.11 Příklad rozdělení dvourozměrného obrazového prostoru klasifikátorem pro dichotonomii

Poznamenejme, že jeden klasifikátor klasifikující do R tříd lze nahradit R klasifikátory pro dichotonomii naučených na klasifikaci „patří/nepatří“ do třídy r , $r \in \{1, \dots, R\}$.

Jiným přístupem ke klasifikaci lineárně oddělitelných obrazů je klasifikace podle minimální vzdálenosti. Pro všechny třídy se nejprve určí etalony (těžiště shluků obrazů jednotlivých tříd) a obrazy se pak klasifikují do třídy, k jejímuž etalonu má klasifikovaný obraz nejbližší. Lze snadno zjistit, že tato klasifikace opět vede na diskriminační funkce:

Pro etalony tříd $\vec{v}_1, \vec{v}_2, \dots, \vec{v}_R$ a pro klasifikaci obrazu/vektoru $\vec{x}$ do třídy r musí platit

$$\|\vec{v}_r - \vec{x}\| = \min_{i=1 \dots R} (\|\vec{v}_i - \vec{x}\|) = \min_{i=1 \dots R} \left(\sqrt{(\vec{v}_i - \vec{x})(\vec{v}_i - \vec{x})} \right)$$

a tedy i

$$\begin{aligned} \|\vec{v}_r - \vec{x}\|^2 &= \min_{i=1 \dots R} (\|\vec{v}_i - \vec{x}\|^2) = \min_{i=1 \dots R} ((\vec{v}_i - \vec{x})(\vec{v}_i - \vec{x})) = \\ &= \min_{i=1 \dots R} (\vec{v}_i \vec{v}_i - 2\vec{v}_i \vec{x} + \vec{x} \vec{x}) = \min_{i=1 \dots R} (\vec{v}_i \vec{v}_i - 2\vec{v}_i \vec{x}) = \\ &= \max_{i=1 \dots R} (-\vec{v}_i \vec{v}_i + 2\vec{v}_i \vec{x}) = \max_{i=1 \dots R} \left(-\frac{1}{2} \vec{v}_i \vec{v}_i + \vec{v}_i \vec{x} \right) = \\ &= g_{r,0} + \vec{g}_r \vec{x} = g_r(\vec{x}) \end{aligned}$$

$$g_{r,0} = -\frac{1}{2} \vec{v}_r \vec{v}_r, \quad \vec{g}_r = \vec{v}_r$$

Rozdělující (hyper)roviny mezi třídami r a s jsou kolmé k vektorům $\vec{v}_r - \vec{v}_s$ ($r, s \in C$, $r \neq s$) a půlí je! Více informací o tomto přístupu ke klasifikaci je uvedeno v kap. 5.2, metoda k -means.

Nelze-li obrazy různých tříd oddělit od sebe lineárními (hyper)plochami, tj. (hyper)rovinami, lze problém řešit některým z následujících postupů:

- Použitím několika lineárních diskriminačních funkcí pro každou třídu (vede k obtížnému učení):

$$g_r(\vec{x}) = \max_{h=1 \dots H} (g_r^h(\vec{x}))$$

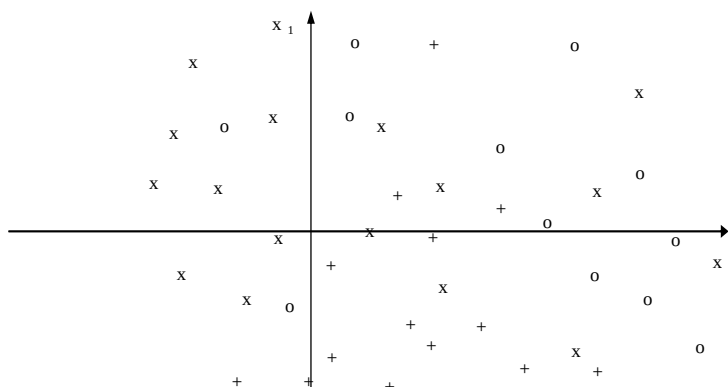
- Použitím obecných diskriminačních funkcí (v praxi obtížně proveditelné):

$$g_r(\vec{x}) = a_r + b_r \ln(x_1) + c_r \operatorname{tg}(d_r x_2) + \dots$$

- Použitím nelineární transformace, která převede původní n rozměrný obrazový prostor na m rozměrný obrazový prostor, ve kterém již lze obrazy separovat lineárními plochami, a pak použít klasifikátor s lineárními diskriminačními funkcemi. Jde o princip používaný u vícevrstvých neuronových sítí (kap 5.1.4.2, Obr. 5.21).
- Použitím několik etalonů pro každou třídu: $\vec{v}_{11}, \vec{v}_{12}, \dots, \vec{v}_{1n}, \vec{v}_{21}, \vec{v}_{22} \dots$ Jde o princip používaný u RCE neuronových sítí (kap. 5.1.4.3)
- Použitím dále popsaného klasifikátoru pro neoddělitelné třídy obrazů.

5.1.3.1.2 Rozpoznávání obrazů reprezentovaných neoddělitelnými třídami obrazů

Příklad tří neoddělitelných tříd obrazů ve dvourozměrném obrazovém prostoru je ukázán na Obr. 5.12.



Obr. 5.12 Neoddělitelné třídy obrazů ve dvourozměrném obrazovém prostoru

V případě neoddělitelných tříd obrazů již nelze rozhodnout, že obraz $\vec{x}$ patří do třídy r , ale lze konstatovat, že obraz $\vec{x}$ patří do třídy r s pravděpodobností $P(r|\vec{x})$.

Při odvozování učícího algoritmu pro tyto případy se vychází z definice ztrátové matice, jejímiž prvky $\lambda(r,s)$ jsou ztráty uživatele, když klasifikuje obraz třídy s nesprávně do třídy r :

$$\lambda = \begin{bmatrix} \lambda(1,1) & \lambda(1,2) & \dots & \lambda(1,R) \\ \lambda(2,1) & \lambda(2,2) & \dots & \lambda(2,R) \\ \vdots & \vdots & \ddots & \vdots \\ \lambda(R,1) & \lambda(R,2) & \dots & \lambda(R,R) \end{bmatrix}$$

Ztráta při správné klasifikaci obrazu je samozřejmě nulová $\lambda(i,i) = 0$. Pro jednoduchost předpokládejme, že ztráta při jakékoliv chybné klasifikaci je $\lambda(i,j) = 1, i \neq j$.

Pak se ztrátová matice se zjednoduší na tvar:

$$\lambda = \begin{bmatrix} 0 & 1 & \dots & 1 \\ 1 & 0 & \dots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \dots & 0 \end{bmatrix}$$

Nechť $P(\vec{x})$ je nepodmíněná pravděpodobnost výskytu obrazu $\vec{x}$. Pak $P(s|\vec{x})$ je podmíněná pravděpodobnost skutečnosti, že obraz $\vec{x}$ patří do třídy s .

Příspěvek ztráty $\lambda(r,s)$ k celkové střední ztrátě způsobené zařazením obrazu třídy s nesprávně do třídy r je dán součinem této ztráty s apriorní pravděpodobností výskytu obrazu $\vec{x}$ a podmíněné pravděpodobnosti, že tento obraz je obrazem objektu třídy s ; na součin obou pravděpodobností se pak aplikuje Bayesovo pravidlo:

$$\lambda(r,s)P(s|\vec{x})P(\vec{x}) = \lambda(r,s)P(\vec{x}|s)P(s)$$

Celková střední ztráta při klasifikaci obrazu $\vec{x}$ do třídy r je dána vztahem

$$L_{\vec{x}}(r) = \sum_{s=1}^R \lambda(r,s)P(\vec{x}|s)P(s)$$

který je možné s respektováním ztrátové matice upravit na tvar (nejprve se uvažuje, že všechny prvky matice jsou jednotkové a pak se odečtou prvky na hlavní diagonále)

$$\begin{aligned} L_{\vec{x}}(r) &= \sum_{s=1}^R \lambda(r,s)P(\vec{x}|s)P(s) = \sum_{s=1}^R P(\vec{x}|s)P(s) - P(\vec{x}|r)p(r) = \\ &= P(\vec{x}) - P(\vec{x}|r)P(r) \end{aligned}$$

Cílem klasifikátoru je přirozeně tuto ztrátu minimalizovat (pravděpodobnost $P(\vec{x})$ nelze minimalizovat, a proto ji lze vypustit):

$$\min(L_{\vec{x}}(r)) = \max(P(\vec{x}|r)P(r))$$

Nejmenší ztrátu tak utrpí uživatel, klasifikuje-li obraz $\vec{x}$ do třídy r , pro níž je hodnota součinu $P(\vec{x}|r)P(r)$ maximální, a proto funkce daná tímto součinem je hledanou diskriminační funkcí

$$g_r(\vec{x}) = P(\vec{x}|r)P(r)$$

Z praktických důvodů je vhodné za diskriminační funkci použít přirozený logaritmus výše uvedeného součinu pravděpodobností (pro $a > b$ je také $\ln(a) > \ln(b)$)

$$g_r(\vec{x}) = \ln(P(\vec{x}|r)P(r)) = \ln(P(\vec{x}|r)) + \ln(P(r))$$

Pro vícerozměrné normální rozložení hustoty pravděpodobnosti

$$P(\vec{x}|r) = \frac{1}{(2\pi)^{\frac{n}{2}}\sqrt{|\bar{\sigma}_r|}} e^{-\frac{1}{2}(\vec{x}-\vec{\mu}_r)^T\bar{\sigma}_r^{-1}(\vec{x}-\vec{\mu}_r)}$$

má diskriminační funkce tvar

$$\begin{aligned} g_r(\vec{x}) &= \ln(P(\vec{x}|r)) + \ln(P(r)) = \\ &= -\frac{1}{2}(\vec{x} - \vec{\mu}_r)^T \bar{\sigma}_r^{-1}(\vec{x} - \vec{\mu}_r) - \frac{1}{2}\ln(|\bar{\sigma}_r|) + \ln(p(r)) \end{aligned}$$

Logaritmus konstanty $1/(2\pi)^{\frac{n}{2}}$ jsme vypustili, protože jeho hodnota je stejná pro všechny obrazy i třídy, a na porovnání proto nemá žádný vliv.

Učení klasifikátoru metodou odhadu (pro n rozměrné normální rozložení):

$T = \{(\vec{x}_1, c_1), (\vec{x}_2, c_2), \dots, (\vec{x}_P, c_P)\}$, $\vec{x}_i = (x_{i1}, x_{i2}, \dots, x_{in})$, $c_i \in \{1, R\}$ je trénovací množina, $\vec{\mu}_i$ jsou vektory středních hodnot, $\vec{v}_i$ a $\vec{w}_i$ jsou pomocné vektory a $\bar{\sigma}_i$ je disperzní matice. Ostatní symboly jsou jednorozměrné pomocné proměnné.

1. Pro všechny třídy r nastavte $\vec{\mu}_r = \vec{0}$; $\vec{\sigma}_r = \vec{0}$; $P_r = 0$;
2. Pro všechny obrazy $\vec{x}_i$ trénovací množiny T počítejte:
 - $\vec{\mu} = \vec{\mu}_{c_i}$
 - $\vec{\mu}_{c_i} = (P_{c_i}\vec{\mu}_{c_i} + \vec{x}_i) / (P_{c_i} + 1)$
 - Jestliže $P_{c_i} > 0$ počítejte:

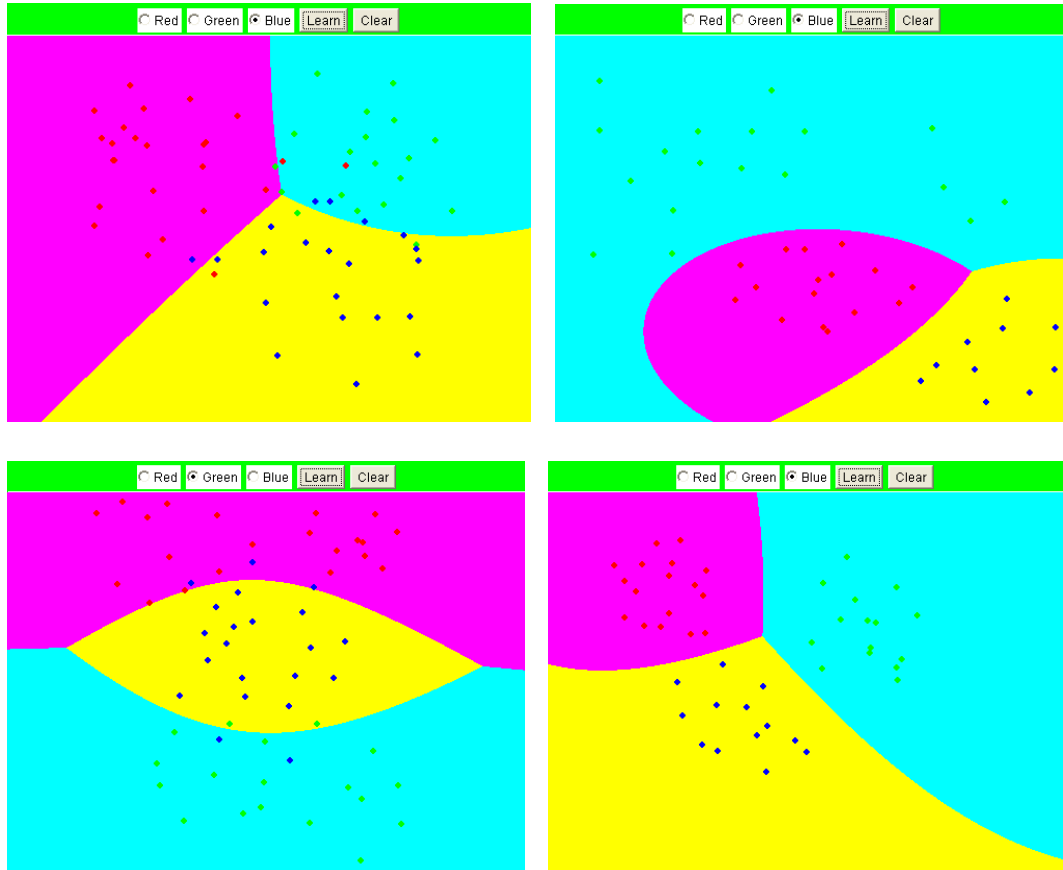
$$\vec{v} = (\vec{x}_i - \vec{\mu}_{c_i})$$

$$\vec{w} = (\vec{\mu} - \vec{\mu}_{c_i})$$

$$\vec{\sigma}_{c_i} = ((P_{c_i} - 1)\vec{\sigma}_{c_i} + \vec{v}^T \vec{v} + P_{c_i} \vec{w}^T \vec{w}) / P_{c_i}$$

$$P_{c_i} = P_{c_i} + 1$$
3. Pro všechny třídy r počítejte $p(r) = P_r / P$
4. $g_r(\vec{x}) = -\frac{1}{2}(\vec{x} - \vec{\mu}_r)^T \vec{\sigma}_r^{-1}(\vec{x} - \vec{\mu}_r) - \frac{1}{2}\ln(|\vec{\sigma}_r|) + \ln(p(r))$

Na Obr. 5.13 je ukázáno rozdělení dvourozměrného obrazového prostoru do tří podprostorů klasifikátorem naučeným výše uvedenou metodou odhadu. Povšimněte si, že klasifikátor pracuje uspokojivě i v případech, kdy normální rozložení obrazů není dodrženo.



Obr. 5.13 Příklady rozdělení dvourozměrného obrazového prostoru klasifikátorem učeným metodou odhadu

5.1.3.2 Strukturální rozpoznávání

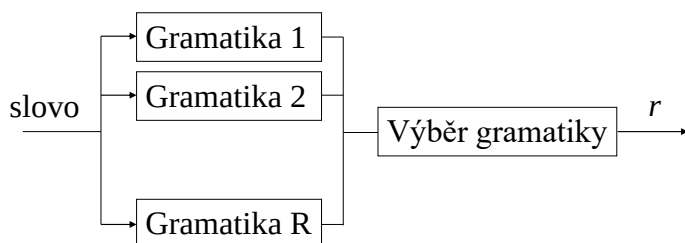
Strukturální rozpoznávání spočívá v rozpoznávání obrazů, které popisují strukturu objektů nebo jevů pomocí primitiv.

Existují dva základní přístupy ke strukturálnímu rozpoznávání obrazů

- rozpoznávání obrazů pomocí gramatik, syntaktické rozpoznávání (*syntactic recognition*)
- rozpoznávání obrazů porovnáním se vzory uloženými v databázi (*template matching*).

Dále se omezíme na popis syntaktického rozpoznávání obrazů představovaných řetězcí znaků. Popisy syntaktického rozpoznávání stromových a grafových obrazů a *template matching* přesahují rámec předmětu.

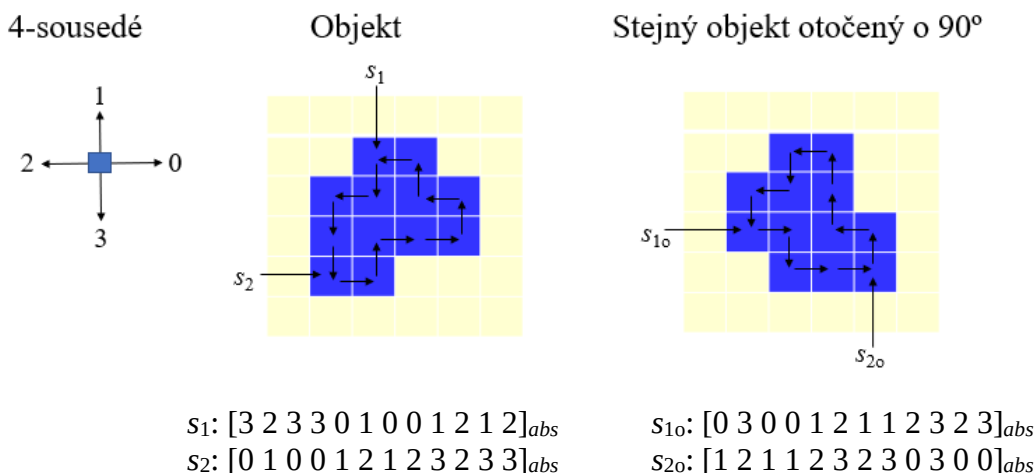
Syntaktické rozpoznávání klasifikuje obrazy do R tříd pomocí syntaktické analýzy. Obrazy, tj. řetězcové popisy objektů nebo jevů, jsou přitom chápány jako slova a množina primitiv jako množina terminálních symbolů. Pokud pak gramatika generuje všechna slova/obrazy, která reprezentují obrazy třídy r , a negeneruje žádné slovo, která reprezentuje obraz jiné třídy, lze klasifikaci převést na problém určení gramatiky, která jako jediná ze všech R gramatik generuje rozpoznávané slovo/obraz (Obr. 5.14).



Obr. 5.14. Princip syntaktického rozpoznávání

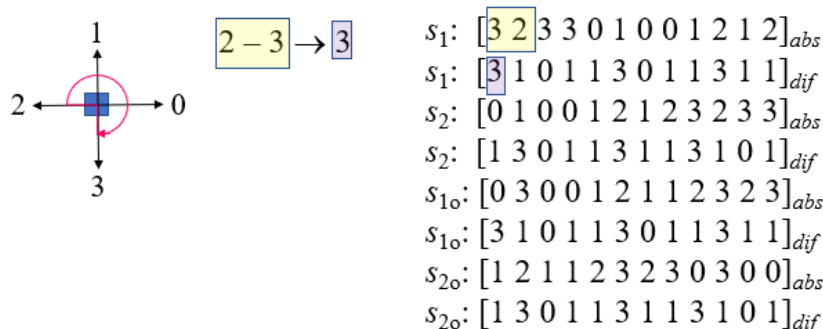
Na úspěšnost strukturálního/syntaktického rozpoznávání má velký vliv výběr primitiv (malá množina primitiv má malou vyjadřovací sílu, velká množina může být nezvládnutelná při učení).

Ke strukturálnímu popisu obrysu objektu se často používá Freemanův řetězcový kód, který za primitiva popisující obrys objektu považuje směry sousedů. Pro čtyři sousedy je možný popis obrysu ukázán na Obr. 5.15, symboly s_1 a s_2 (resp. s_{10} a s_{20} u otočeného obrázku) označují dvě různé startovací pozice popisu. Obrazy vytvořené tímto popisem se nazývají absolutní Freemanovy řetězcové kódy.



Obr. 5.15. Popis obrysu objektu absolutním řetězcovým kódem pro 4 sousedy

Absolutní řetězcový kód je zřejmě invariantní vůči posuvu, je však závislý jak na natočení objektu, tak na startovacím bodu popisu. Pro rozpoznání je proto prakticky nepoužitelný. Problém natočení lze vyřešit pomocí relativního (diferenciálního) kódu, který místo primitiv používá rozdílů/diferencí natočení mezi dvěma sousedními primitivami (následující – aktuální!), přičemž pro určení těchto rozdílů se postupuje v opačném směru (tj. po směru hodinových ručiček). Pro první znak řetězce je předcházejícím znakem znak poslední (Obr. 5.16)



Obr. 5.16. Popis objektu relativním (diferenciálním) řetězcovým kódem

Je vidět, že relativní kódy s_1 a s_{10} jsou stejné, stejně jako jsou stejné kódy s_2 a s_{20} . Relativní kód tak vyřešil problém natočení, problém závislosti na startovacím bodu popisu však zůstal. Tento problém lze pak vyřešit jednoduchou rotací řetězce relativního kódu tak, aby výsledkem bylo největší číslo, které se nazývá číslem tvaru (*shape number*).

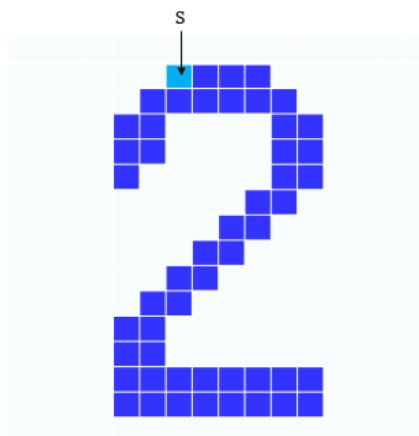
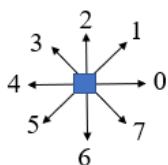
$$s_1 = s_{10} = [3\ 1\ 0\ 1\ 1\ 3\ 0\ 1\ 1\ 3\ 1\ 1]_{dif} \quad s_2 = s_{20} = [1\ 3\ 0\ 1\ 1\ 3\ 1\ 1\ 3\ 1\ 0\ 1]_{dif}$$

shape number: $[3\ 1\ 1\ 3\ 1\ 0\ 1\ 1\ 3\ 0\ 1\ 1]$.

Číslo tvaru je invariantní vůči posunutí, natočení objektu (pro 4 sousedy pouze po 90°) a startovacímu bodu popisu, je však závislé na velikosti objektu.

Popis objektu Freemanovým kódem pro 8 sousedů je ukázán na Obr. 5.17.

8- sousedů



Řetězcový kód pro 8 sousedů:

$$[556612100006665555556660000000244444321111112233444]_{abs} = [010317700060070000010020000002200007770000010101001]_{dif}$$

Číslo tvaru:

$$[777000001010100101031770006007000001002000000220000]$$

Obr. 5.17. Popis objektu Freemanovým řetězovým kódem pro 8 sousedů

Uvažujme gramatiku $G = (V_N, V_T, S, P)$, kde symbol V_N značí množinu neterminálních symbolů, symbol V_T množinu terminálních symbolů a symbol S ($S \in V_N$) startovací symbol. Symbol P označuje množinu přepisovacích pravidel. Abeceda V je dána sjednocením neterminálních a terminálních symbolů $V = V_T \cup V_N$. Symbol V^* označuje množinu všech slov nad abecedou V . Jazyk L je podmnožinou této množiny $L \subseteq V^*$ a pro jazyk $L(G)$ generovaný gramatikou G musí platit $L(G) = \{x \mid x \in V^*, S \rightarrow x\}$. Přepisovací pravidla mají tvar $\alpha \rightarrow \beta$, $\alpha, \beta \in V$, α obsahuje alespoň jeden neterminální symbol.

Číslo tvaru [3 1 1 3 1 0 1 1 3 0 1 1] popisující objekt z Obr. 5.15 by generovala například gramatika s těmito přepisovacími pravidly:

$S \rightarrow 3N$,
 $N \rightarrow 0N$
 $N \rightarrow 1N$
 $N \rightarrow 3N$
 $N \rightarrow 1$

Tato gramatika by však generovala i další řetězce, které by daný objekt nepopisovaly, a proto k rozpoznávání by byla zřejmě nepoužitelná.

Učení syntaktického klasifikátoru zřejmě spočívá v nalezení relevantních přepisovacích pravidel na základě trénovacích dat (tzv. inference gramatik).

Trénovací množina $T = \{S^+, S^-\}$ je dána množinou pozitivních příkladů $S^+ = \{x_i \mid x_i \in L(G)\}$ a množinou negativních příkladů $S^- = \{x_i \mid x_i \notin L(G)\}$.

Inferenční procedura pak může být popsána takto:

0. Necht' $G^{(0)}$ je počáteční odhad gramatiky, $k = 0$.
1. Nastavte $i = 1$, $modif = false$,
2. Vyberte z trénovací množiny slovo x_i .
3. Jestliže $x_i \in S^+$ a $G^{(k)}$ negeneruje x_i tak:
 modifikujte $G^{(k)} \rightarrow G^{(k+1)}$ tak, aby gramatika $G^{(k+1)}$ generovala x_i
 $modif = true$,
 $k = k + 1$.
4. Jestliže $x_i \in S^-$ a $G^{(k)}$ generuje x_i tak:
 modifikujte $G^{(k)} \rightarrow G^{(k+1)}$ tak, aby gramatika $G^{(k+1)}$ negenerovala x_i
 $modif = true$,
 $k = k + 1$.
5. $i = i + 1$.
6. Není-li trénovací množina T prázdná tak se vraťte na bod 2.
7. Byla-li gramatika modifikována ($modif = true$), tak se vraťte na bod 1.

Strukturální/syntaktický popis obsahuje z principu více informací o rozpoznávaném objektu nebo jevu než popis příznakový. V poslední době je však syntaktický přístup k rozpoznávání poněkud v pozadí, a to především z důvodu současných velkých úspěchů konvolučních neuronových sítí. Navíc je strukturální popis značně citlivý na šum, problémem někdy bývá i „falešná“ abeceda nonterminálů a inference gramatik (modifikace gramatik v bodech 3 a 4 předchozího algoritmu) je výrazně složitější než učení příznakových klasifikátorů nebo neuronových sítí.

5.1.4 Neuronové sítě

5.1.4.1 Biologická inspirace

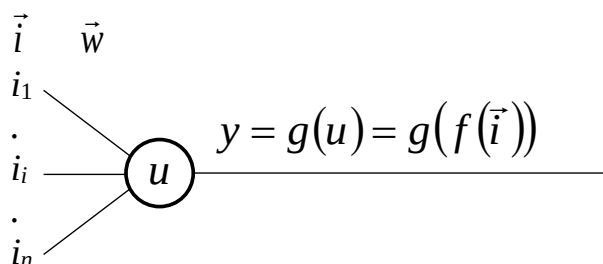
Základními „stavebními“ prvky mozku jsou neurony, které se skládají z těla (sómy), sta až sta tisíc 2–3 mm krátkých výběžků/vstupů (dendritů) a jednoho delšího, až několik desítek cm dlouhého výstupu (axonu), který je zakončen několika terminálními knoflíčky.

Rozhraní mezi dendritem jednoho a axonem druhého neuronu se nazývá synapsí, která zprostředkovává vliv jednoho na druhý neuron. Velikost signálu přenášená dendritem k tělu neuronu tedy závisí jak na aktivitě axonu, kterého se dendrit dotýká, tak na příslušné synaptické váze. Tato váha může být buď posilující/excitační nebo potlačující/inhibiční.

Hodnoty signálů od jednotlivých dendritů se v těle neuronu sčítají. Přesáhne-li jejich celková hodnota jistý práh, neuron vyšle krátký impuls o velikosti asi 100 mV, který postupuje po jeho axonu rychlostí 0.1 až 10 m/s. Pak se neuron na krátkou dobu stane necitlivým. Pokud je po této době celková hodnota vstupního signálu stále větší než práh, impuls se opakuje. Po axonu se tak šíří puls o frekvenci 0.1/s až 100/s.

Lidský mozek má kolem 10^{11} neuronů, tj. přibližně 10^{14} synapsí a učení spočívá výhradně v nastavování synaptických vah.

Pro umělý neuron by byla činnost s podobnou výstupní funkcí obtížně realizovatelná. Proto se používají jednodušší modely, jejichž výstupní signály jsou statické. Klasický model neuronu je ukázán na Obr. 5.18.



Obr. 5.18 Klasický model neuronu

Symbol $\vec{i}$ představuje vstupní vektor, symbol $\vec{w}$ vektor vah/synapsí a symbol y výstup neuronu. Symbol u označuje vnitřní potenciál neuronu, symbol f bázovou funkci a symbol g aktivační funkci. Bázové funkce mohou být lineární nebo radiální, aktivační funkce mohou být skokové nebo spojitě.

5.1.4.2 Neuronové sítě s LBF neurony

Lineární bázová funkce (LBF) je dána vztahem

$$u = \sum_{i=1}^n w_i i_i - \theta = \sum_{i=1}^n w_i i_i + b = \sum_{i=0}^n w_i x_i$$

Symbol θ (*threshold*) značí práh, symbol b předpětí (*bias*), $b = -\theta$ a stanovují hodnotu vnitřního potenciálu u pro nulové hodnoty všech vstupů. Pro praktické výpočty je však výhodnější úprava s novým vstupem s nulovým indexem (váhou w_0 a pevnou hodnotou $x_0 = 1$) tak, jak je uvedeno v pravé části vztahu.

Nejjednodušším umělým neuronem je LBF neuron se skokovou aktivační funkcí

$$y = \begin{cases} 1 & \text{pro } u \geq 0 \\ -1 & \text{pro } u < 0 \end{cases}$$

a nazývá se perceptron.

Perceptron lze naučit klasifikovat lineárně oddělitelné číselné vektory do dvou tříd (podobně jako klasický lineární klasifikátor pro dichotomii).

K jeho učení, tj. k nastavování jeho vah, se používá trénovací množina P dvojic $(\vec{x}_p, d_p)$, kde symboly $\vec{x}_p$ označují vstupní vektory a symboly d_p jejich třídy. Necht'

$$\vec{z}(k) = \begin{cases} +\vec{x}(k) & \text{pro } d(k) = +1 \\ -\vec{x}(k) & \text{pro } d(k) = -1 \end{cases} \quad \text{t.j. } \vec{z}(k) = d(k)\vec{x}(k)$$

kde $d(k) = \pm 1$ je požadovaná klasifikace (třída) a k je krok učení.

Pak základní pravidlo učení perceptronu má tvar:

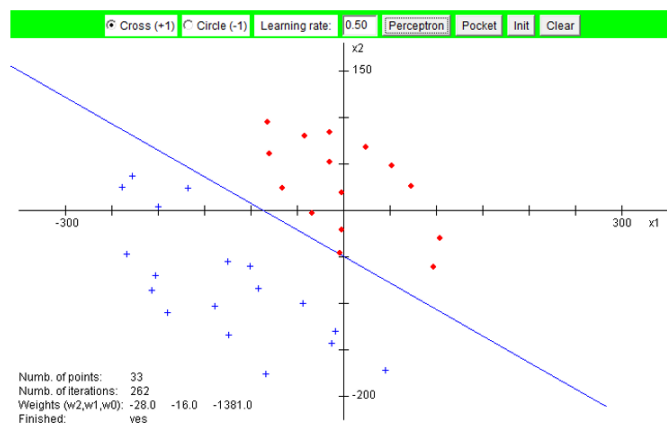
$$\vec{w}(0) = \text{libovolné}$$

$$\vec{w}(k) = \vec{w}(k-1) + 2\mu\vec{z}(k) \quad \text{pro } (\vec{w}(k-1) \cdot \vec{z}(k)) \leq 0$$

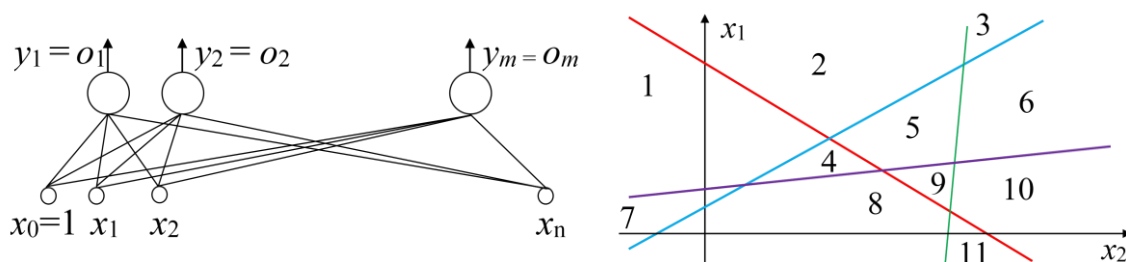
$$\vec{w}(k) = \vec{w}(k-1) \quad \text{jinak}$$

kde μ je tzv. koeficient učení. Na jeho velikosti však prakticky nezáleží, a proto se obvykle volí $\mu = 0.5$.

Ukázka klasifikace obrazů perceptronem je na Obr. 5.19.



Obr. 5.19 Klasifikace obrazů perceptronem



Obr. 5.20 Jednovrstvá perceptronová síť

U jednovrstvých perceptronových sítí (Obr. 5.20) se neurony vzájemně neovlivňují, a proto se každý neuron učí samostatně. Počet tříd R rozpoznatelných touto sítí je dán vztahem

$$R = \begin{cases} 2^m & \text{pro } m \leq n \\ \sum_{i=0}^n \binom{m}{i} & \text{pro } m > n \end{cases}$$

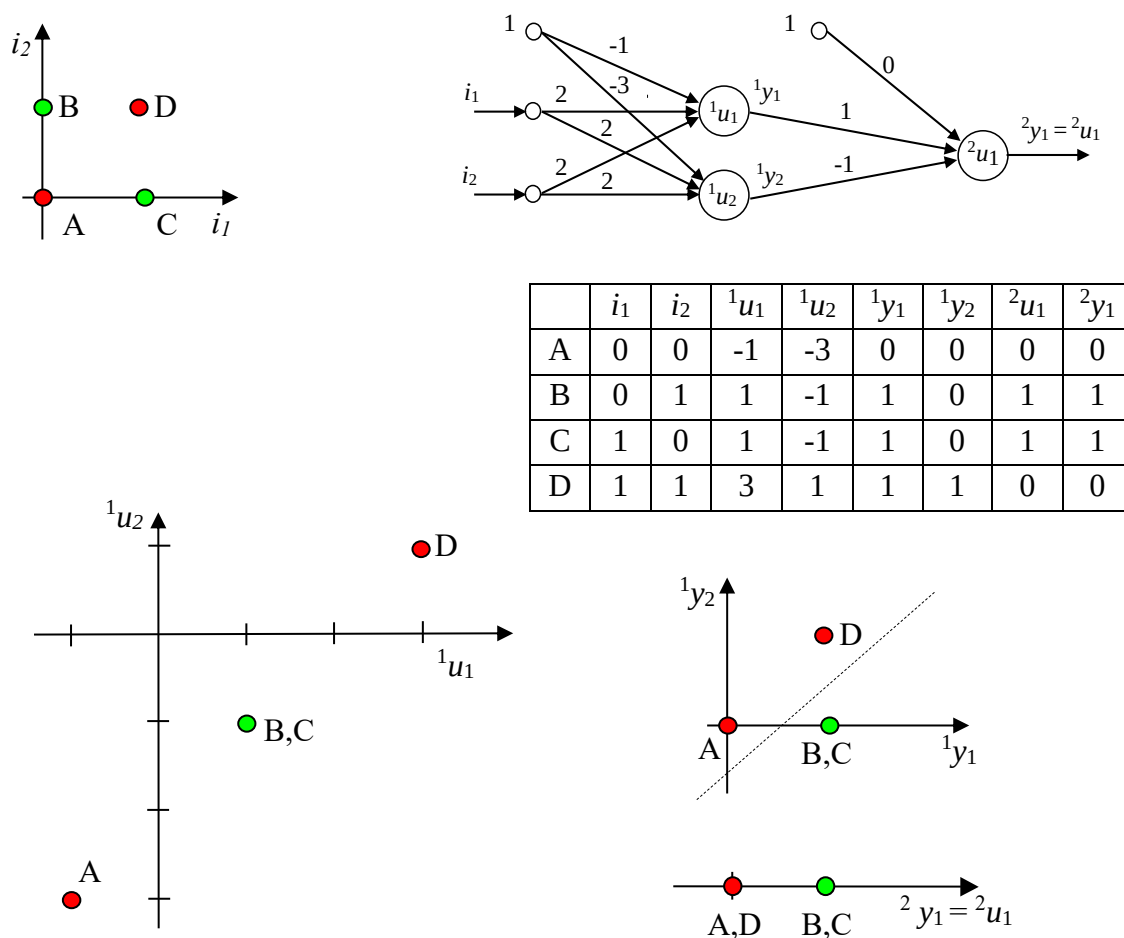
Příklad pro $n = 2, m = 4$ (Obr. 5.20 vpravo):

$$R = \sum_{i=0}^2 \binom{4}{i} = \binom{4}{0} + \binom{4}{1} + \binom{4}{2} = 1 + 4 + 6 = 11$$

kde n je rozměr vstupních vektorů a m je počet neuronů):

Lineárně neoddělitelné číselné vektory lze relativně snadno klasifikovat pomocí vícevrstvých dopředných neuronových sítí (neurony v těchto sítích nemají žádné zpětné vazby k neuronům předcházejících vrstev ani žádné vazby k neuronům ve stejné vrstvě).

Například vektory (resp. jejich koncové body) představující problém XOR ($\vec{i}(A) = (0,0)$, $\vec{i}(B) = (0,1)$, $\vec{i}(C) = (1,0)$, $\vec{i}(D) = (1,1)$) ukázaný na Obr. 5.21 vlevo nahoře, může správně klasifikovat dvouvrstvá síť ukázaná na obrázku vpravo nahoře. Na stejném obrázku je ukázána transformace původního prostoru (i_2, i_1), přes prostor (${}^1u_2, {}^1u_1$) na prostory, ve kterých již lze obrazy lineárně oddělit 2u_1 a 2y_1 . Poznamenejme, že použité neurony mají binární skokové aktivační funkce ($y = 1$ pro $u \geq 0$, $y = 0$, pro $u < 0$).



Obr. 5.21 Řešení problému XOR dvouvrstvou neuronovou sítí

Nejpoužívanějším algoritmem pro učení vícevrstevných dopředných neuronových sítí je algoritmus backpropagation. Tento algoritmus je založen na minimalizaci chyby na výstupu sítě úpravou vah sítě ve směru záporného gradientu (směru klesání) chybové funkce. Aby však bylo možné gradient počítat, musí být aktivační funkce neuronů spojitá, tj. derivovatelná. Velmi často pak neurony dopředných neuronových sítí používají sigmoidální aktivační funkce

$$y = \frac{1}{1+e^{-u}}$$

Předpokládejme trénovací množinu $T = \{(\vec{x}_1, \vec{d}_1), (\vec{x}_2, \vec{d}_2), \dots, (\vec{x}_p, \vec{d}_p)\}$, kde $\vec{x}_p$ jsou vstupní vektory sítě a $\vec{d}_p$ požadované výstupní vektory pro jednotlivé prvky p trénovací množiny. Necht' chyba sítě je definována vztahem (o_{pj} jsou hodnoty jednotlivých neuronů v poslední vrstvě, tj. výstupu sítě).

$$E_p = \sum_{j=1}^m (d_{pj} - o_{pj})^2$$

Pak algoritmus backpropagation vypadá takto:

1. Vynulujte proměnnou E (celkovou chybu sítě).
{Dopředný výpočet}
2. Vyberte z trénovací množiny náhodně p tý prvek.
3. Dejte na vstup sítě vstupní vektor $\vec{x}_p$ a postupně, počínaje první a konče poslední vrstvou sítě ($l = 1 \dots L$), počítejte výstupy jednotlivých neuronů (všechny neurony mají lineární bázové funkce a spojitá sigmoidální aktivační funkce).
4. Vynulujte proměnnou E_p (chybu sítě pro vybraný prvek p).
5. Pro každý výstupní neuron ($j = 1 \dots m$) počítejte
 - a) Jeho příspěvek k chybě $E_p = E_p + 0.5 * (d_j - y_j)^2$
 - b) Derivaci chyby E_p podle jeho vnitřního potenciálu (delta)

$${}^L\delta_j = (d_j - y_j) y_j (1 - y_j)$$

6. Připočtete chybu E_p k celkové chybě $E = E + E_p$.
{Zpětná propagace chyby (vlastní backpropagation)}
7. Postupně, počínaje předposlední a konče první vrstvou sítě ($l = L-1 \dots 1$), počítejte pro každý neuron j příslušné vrstvy derivaci chyby tohoto neuronu podle jeho vnitřního potenciálu (delta)

$${}^l\delta_j = \left(\sum_{k=1}^{n_{l+1}} {}^{l+1}\delta_k {}^{l+1}w_{kj} \right) y_j (1 - y_j)$$

{Opět dopředný výpočet}

8. Postupně, počínaje první a konče poslední vrstvou sítě ($l = 1 \dots L$), počítejte pro každý neuron příslušné vrstvy přírůstky a změny jeho vah a tyto změny proveďte

$$\Delta {}^l w_{ji} = \mu {}^l\delta_j {}^l x_i$$

$${}^l w_{ji} = {}^l w_{ji} + \Delta {}^l w_{ji}$$

9. Pokud není trénovací množina prázdná, tak se vraťte na bod 2, jinak obnovte trénovací množinu.
10. Pokud nejsou splněny ukončovací podmínky (například pokud je celková chyba E větší než předem zvolená konstanta ε , nebo počet průchodů trénovací množinou (epoch) je menší než zvolený maximální počet epoch, tak se vraťte na bod 1.

Poznamenejme, že při klasifikaci do R tříd má výstupní vrstva sítě také R neuronů a požadované výstupní vektory $\vec{d}_p$ mají prvek odpovídající třídě vstupního vektoru $\vec{x}_p$ jedničkový a všechny ostatní prvky nulové. Naučená síť pak klasifikuje neznámé vstupní vektory do třídy, kterou reprezentuje výstupní neuron s nejvyšší hodnotou (tj. hodnotou blíží se hodnotě 1, hodnoty ostatní neuronů by se měly blížit k nule).

5.1.4.3 Neuronové sítě s RBF neurony

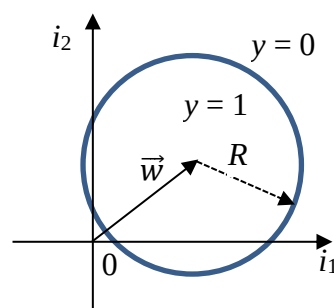
Radiální básová funkce (RBF) je dána vztahem

$$u = \|\vec{i} - \vec{w}\| = \sqrt{\sum_{i=1}^n (i_i - w_i)^2} \quad ,$$

který vyhodnocuje délku vektoru daného rozdílem váhového a vstupního vektoru. Nespojité aktivační funkce pro RBF je dána vztahem

$$y = \begin{cases} 1 & \text{pro } u \leq R \\ 0 & \text{pro } u > R \end{cases}$$

kde parametr R se nazývá poloměrem (obecně n rozměrné hyperkoule). Na Obr. 5.22 je ukázána situace pro neuron se dvěma vstupy: výstupní hodnota nespojité aktivační funkce bude jedničková pro vstupní vektory, jejichž koncové body budou uvnitř kružnice (obecně uvnitř n rozměrné hyperkoule), jinak bude tato výstupní hodnota nulová.



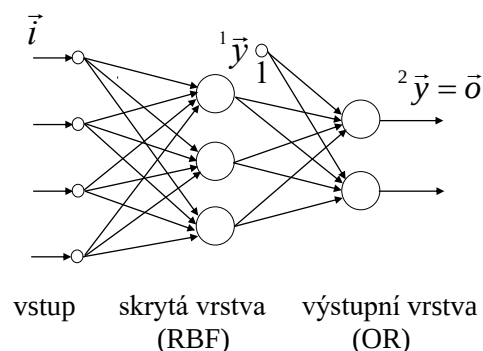
Obr. 5.22 Skoková aktivační funkce pro RBF neurony

Nejpoužívanější neuronovou sítí s RBF neurony pro klasifikaci je síť RCE (*Restricted Coulomb Energy*), která je ukázána na Obr. 5.23. Síť má dvě vrstvy, neurony první vrstvy mají radiální básové funkce, neurony výstupní vrstvy jsou perceptrony realizující logické funkce OR (jedné třídě může odpovídat více neuronů první vrstvy).

K učení RCE sítě se používá trénovací množina

$$T_i = \{(\vec{i}_1, d_1), (\vec{i}_2, d_2), \dots, (\vec{i}_p, d_p)\}$$

kde symboly $\vec{i}_i$ označují vstupní vektory (reálná čísla) a symboly d_i třídy, do kterých mají být tyto vstupní vektory zařazené (přirozená čísla).



Obr. 5.23 Neuronová síť RCE

Algoritmus RCE:

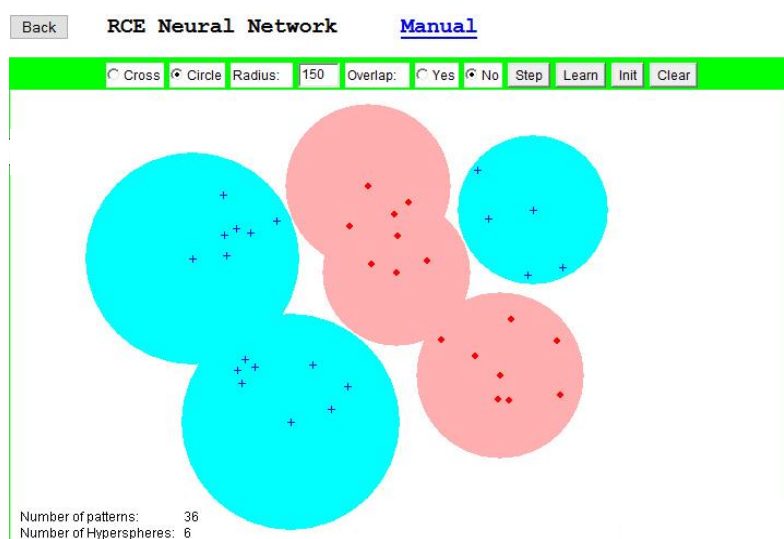
1. Výchozí síť je prázdná, tj. $q = 0$ (počet neuronů skryté vrstvy) a $m = 0$ (počet neuronů výstupní vrstvy). Nastavte výchozí poloměr R_{max} (maximální poloměr hyperkoule).
2. Nastavte indikátor změny sítě $modif = false$.
3. Nastavte index p na první vektor trénovací množiny ($p = 1$).
4. Nastavte indikátor „zásahu“ $hit = false$ a index k neuronu ve skryté vrstvě na první neuron, tj. $k = 1$ (i když na začátku učení je tato vrstva prázdná).
5. Jestliže je $k > q$, tak přejděte na bod 12, jinak pokračujte.

6. Spočítejte vzdálenost u_k mezi vstupním vektorem $\vec{x}_p$ a vektorem vah $\vec{w}_k$ k této neuronu ve skryté vrstvě:

$$u_k = \|\vec{x}_p - \vec{w}_k\| = \sqrt{\sum_{i=1}^n (x_{pi} - w_{ki})^2}$$

7. Je-li vzdálenost u_k větší než aktuální poloměr hyperkoule r_k , tak přejděte na bod 10, jinak pokračujte.
8. Jestliže k -tý neuron reprezentuje stejnou třídu, jako je třída vstupního vektoru, tak nastavte proměnnou $hit = true$ (zásah) a přejděte na bod 10, jinak pokračujte.
9. Zmenšete poloměr r_k tak, aby byl menší než vzdálenost u_k (obvykle se zmenšuje na polovinu této vzdálenosti) a nastavte proměnnou $modif = true$ (v síti byla provedena změna).
10. Inkrementujte index neuronu k , a pokud není větší než aktuální počet neuronů skryté vrstvy q , tak se vraťte na bod 6, jinak pokračujte.
11. Jestliže proměnná $hit = true$ (byl zásah), tak přejděte na bod 14, jinak pokračujte.
12. Inkrementujte počet neuronů skryté vrstvy q , vytvořte nový neuron s vektorem vah $\vec{w}_q = \vec{x}_p$ a poloměrem $r_q = R_{max}$ a nastavte proměnnou $modif = true$ (v síti byla provedena změna).
13. Jestliže třída vstupního vektoru (a tedy i třída nově vzniklého neuronu ve skryté vrstvě) je již reprezentována nějakým výstupním neuronem, pak připojte nový neuron ze skryté vrstvy k tomuto výstupnímu neuronu (jde o neuron s funkcí OR), jinak vytvořte nový výstupní neuron reprezentující třídu aktuálního vstupního vektoru a připojte nový neuron ze skryté vrstvy k tomuto novému výstupnímu neuronu.
14. Inkrementujte index p , a pokud není větší než počet vektorů trénovací množiny P , pak se vraťte na bod 4, jinak pokračujte.
15. Pokud byla síť modifikována ($modif = true$), pak se vraťte na bod 2, jinak učení sítě ukončete.

Přestože se výše uvedený algoritmus může zdát na první pohled složitý, jeho realizace je velmi snadná. Klasifikace sítě RCE je zřejmá z Obr. 5.24, kde klasifikuje body do dvou tříd: křížky a kolečka.



Obr. 5.24 Klasifikace sítě RCE

5.2 Učení bez učitele

Učení bez učitele spočívá v hledání podobností mezi příklady trénovací množiny a v zařazování příkladů s podobnými charakteristikami do skupin. Umělý systém přitom nedostává žádnou informaci o správnosti klasifikace a jedinou informací, kterou má, resp. může mít je počet skupin, do kterých má příklady z trénovací množiny zařazovat/klasifikovat.

Příklady trénovací množiny jsou téměř vždy představované číselnými vektory příznaků klasifikovaných objektů či dějů. Metody učení bez učitele jsou pak založeny na předpokladu, že tyto vektory, resp. jejich koncové body, tvoří v příslušném n rozměrném obrazovém prostoru shluky. Tyto shluky mohou představovat velmi rozmanité n rozměrné útvary, například v několika souřadnicích může jít o zcela kompaktní shluky, zatímco v jiných souřadnicích mohou být značně rozptýlené.

V dalším textu popíšeme princip nejznámější a nejpoužívanější metody, která klasifikuje příklady (číselné vektory) z trénovací množiny do předem daného počtu k shluků a je známá pod názvem metoda/algoritmus *k-means clustering*.

Algoritmus zařazuje vstupní vektor do toho shluku, k jehož středu/těžišti má nejkratší vzdálenost a učí se následujícím postupem: Nejprve náhodně určí k rozdílných vektorů (často tyto vektory vybere z trénovací množiny) a považuje je za středy shluků. Dále zařadí všechny vektory trénovací množiny do příslušných shluků. Pak přepočítá středy shluků, znovu zařadí všechny vektory trénovací množiny do příslušných shluků atd. Učení končí, až všechny vektory zařazuje do stále stejných shluků. Je zřejmé, že na výsledek učení má vliv použitá metrika (Euklidovská, Hammingova, ap.).

Algoritmus *k-means clustering*

Inicializujte k prototypů $\vec{w}_j$ (použijte náhodně vybrané, ale různé vektory $\vec{l}_p$ z trénovací množiny $T_i = \{\vec{l}_1, \vec{l}_2, \dots, \vec{l}_P\}$, tj. $\vec{w}_j = \vec{l}_p$, $j \in \langle 1, k \rangle$, $p \in \langle 1, P \rangle$).

1. Každý vektor $\vec{l}_p$ z trénovací množiny přiřaďte do shluku C_j , $j \in \langle 1, k \rangle$, jehož prototyp $\vec{w}_j$ má od vektoru $\vec{l}_p$ nejmenší vzdálenost, tj.

$$|\vec{l}_p - \vec{w}_j| \leq |\vec{l}_p - \vec{w}_i|, \quad i \in \langle 1, k \rangle$$

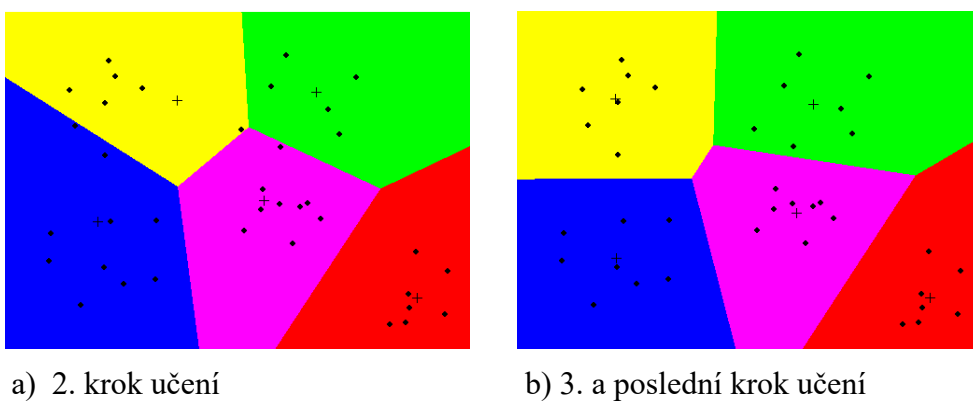
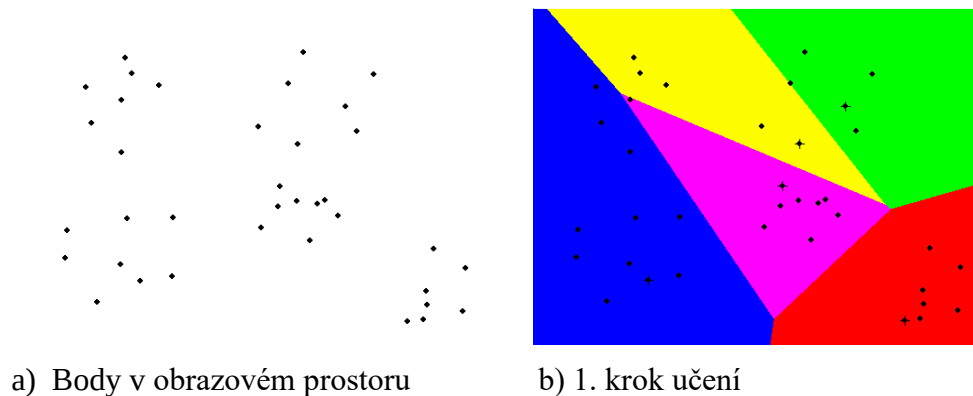
2. Pro každý shluk C_j $j \in \langle 1, k \rangle$ přepočítejte prototyp $\vec{w}_j$ tak, aby byl těžištěm koncových bodů všech vektorů, které jsou k tomuto shluku právě přiřazené (nechť n_j je počet těchto vektorů):

$$\vec{w}_j = \frac{\sum_{\vec{x}_i \in C_j} \vec{x}_i}{n_j}$$

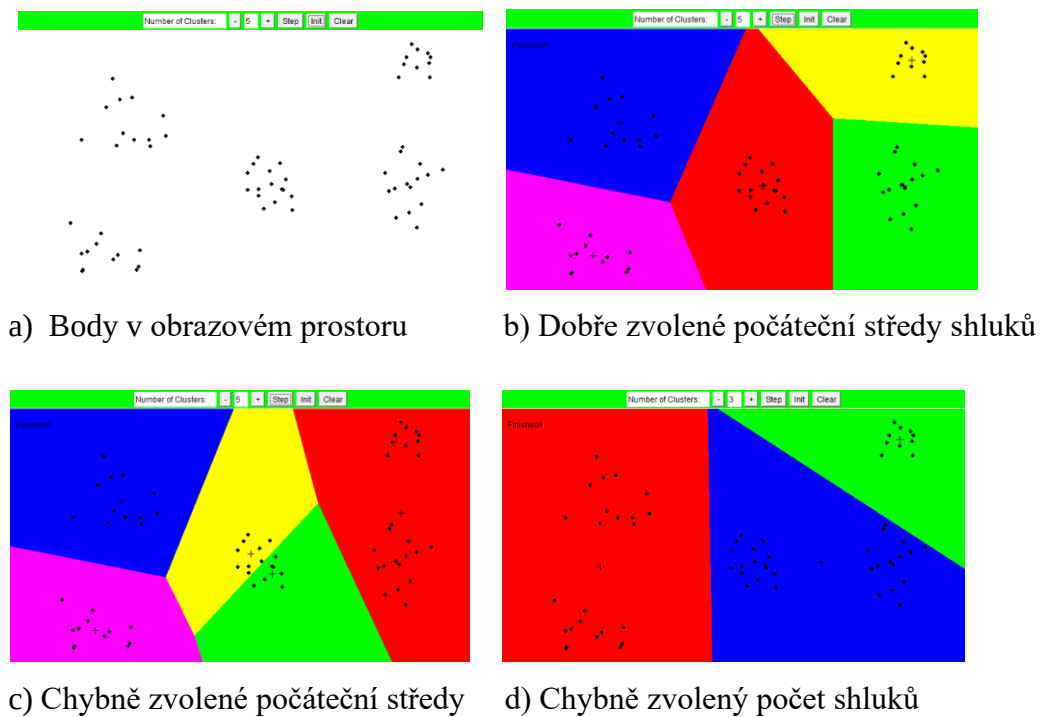
4. Pokud byl některý vektor přeřazen k jinému shluku, vraťte se na bod 2, jinak činnost algoritmu ukončete.

Na obrázku 5.25 je ukázána trénovací množina dvourozměrných číselných vektorů (vlevo nahoře), počáteční náhodná volba pěti středů a následné rozdělení vektorů do shluků (vpravo nahoře), stav během učení (vlevo dole) a výsledek po ukončení učení (vpravo dole).

Algoritmus však může vytvářet i neočekávané/nesprávné shluky, především při nevhodně zvolených počátečních prototypoch nebo při nesprávném nastavení počtu shluků (Obr. 5.26).



Obr. 5.25 Příklad učení algoritmem *k – means clustering*



Obr. 5.26 Příklad učení algoritmem *k – means clustering*

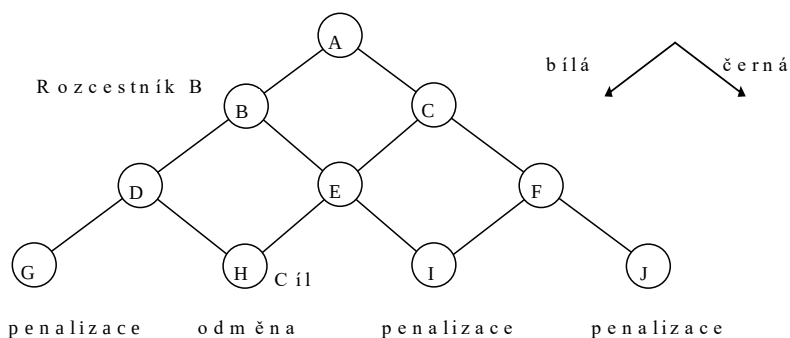
5.3 Posilované učení

Posilované učení se od učení s učitelem odlišuje tím, že systém ohodnocuje své akce na základě penalizací či odměn získaných v koncových stavech a na základě svých hodnocení stavů/akcí získaných vlastními předchozími zkušenostmi. V následujícím textu si ukážeme principy tří metod posilovaného učení používané pod anglickými názvy:

- Policy-only learning
- TD – learning (*Temporal Difference learning*)
- Q – learning
- SARSA (*State–action–reward–state–action*)

5.3.1 Policy-only learning

Police-only learning bylo první prezentovanou metodou posilovaného učení a bylo použito při učení umělého systému hrát Piškvorky. Princip tohoto učení si ukážeme na jednoduchém grafu/stromu (Obr. 5.27), ve kterém z každého uzlu vedou maximálně dvě cesty. Hledá se optimální cesta z počátečního uzlu A do cílového uzlu H.



Obr. 5.27 Graf/strom použitý pro výklad principů Policy-only learning.

Každý uzel (rozcestník) obsahuje schránku s černými a bílými kameny, které se využívají k určení směru cesty. Na začátku učení obsahuje schránka každého rozcestníku stejný a dostatečný počet černých a bílých kamenů ($N_{bílá} = N_{černá}$).

V průběhu učení jsou opakovaně prováděny „procházky“ z počátečního místa A, přičemž pravděpodobnosti pohybu na každém rozcestníku jsou dány výrazy:

$$P_{vlevo} = \frac{N_{bílá}}{N_{bílá} + N_{černá}}, \quad P_{vpravo} = \frac{N_{černá}}{N_{bílá} + N_{černá}}$$

Na konci každé procházky se vyhodnotí cesta takto:

- Cesta vedla do H $\Rightarrow$ odměna (přidání kamenů odpovídajících barev do schránek všech rozcestníků na cestě příslušné procházky)
- Cesta vedla do G, I, J $\Rightarrow$ penalizace (odebrání kamenů odpovídajících barev ze schránek všech rozcestníků na cestě příslušné procházky)

Pravděpodobnost výběru cesty do H se tak postupně zvyšuje, pravděpodobnost všech ostatních cest se naopak snižuje. Po naučení vybírá umělý systém cestu pouze porovnáním počtu kamenů:

je-li ve schránce rozcestníku více bílých kamenů, pokračuje levou cestou jinak pokračuje cestou pravou.

Rozšíření popsaného principu posilovaného učení na problémy s více možnými cestami je snadné – použijí se různobarevné kameny. Barvy kamenů mohou být zcela libovolné, musí však být jednoznačně přiřazeny k některé výchozí hraně každého uzlu.

Policy-only je jednoduchá metoda, ale její použití může vést ke dvěma vážným problémům:

- V případě obecného grafu se může vracet do již dříve vyšetřovaných uzlů (například v bludišti).
- Všechny akce provedené na jedné cestě se hodnotí stejnými vahami, přestože správná ohodnocení mohou být značně rozdílná.

5.3.2 Metoda TD learning (On-Policy)

Metoda TD learning je pasivní metodou, která k přesunu do nového stavu používá předem stanovenou strategii (On-Policy) označovanou symbolem π , kterou si můžeme zjednodušeně představit jako tabulku udávající pravděpodobnosti přechodů mezi jednotlivými stavy. Tyto pravděpodobnosti mohou být stejné (pak se do sousedních stavů přechází zcela náhodně (random policy)), nebo různé (pravděpodobnosti mohou být například dány aktuálními hodnotami sousedních stavů, tj. čím je vyšší ohodnocení sousedního stavu, tím je vyšší pravděpodobnost přechodu do tohoto stavu), v extrémním případě pak přechod do stavu s nejvyšším ohodnocením má pravděpodobnost jedna a ostatní přechody nula (greedy policy). Lze také kombinovat předchozí dva případy, kdy s pravděpodobností danou parametrem ϵ , jehož hodnota musí ležet v rozmezí 0 až 1, se použije random-policy a s pravděpodobností $(1-\epsilon)$ se použije greedy policy (ϵ -greedy policy).

K přepočítávání hodnoty stavu s po každém kroku procházky do stavu s' se používá vztah

$$U^\pi(s) = U^\pi(s) + \alpha \cdot (r(s') + \gamma \cdot U^\pi(s') - U^\pi(s))$$

ve kterém jednotlivé symboly značí

$U^\pi(s)$ ohodnocení (*utility*) stavu s při použití strategie pohybu π ,

$r(s)$ odměnu (*reward*) za dosažení stavu s ,

γ koeficient určující vliv ohodnocení stavu s' na ohodnocení předcházejícího stavu s ,

α koeficient učení, jehož hodnota může klesat s počtem průchodů stavem s .

Algoritmus TD learning

1. Zvolte hodnoty koeficientů α a γ ($0 < \alpha \leq 1$; $0 < \gamma \leq 1$) a vynulujte ohodnocení $U^\pi(s)$ všech stavů. Dále vynulujte počítadlo procházek p a nastavte jejich maximální počet p_{max} . Nastavte $start \rightarrow s$.
2. Generujte nový stav s' s použitím strategie π .
3. Přepočítejte novou hodnotu stavu s pomocí vztahu
$$U^\pi(s) = U^\pi(s) + \alpha \cdot (r(s') + \gamma \cdot U^\pi(s') - U^\pi(s))$$
4. Je-li stav s' cílovým stavem, pak $p + 1 \rightarrow p$, $start \rightarrow s$, jinak $s' \rightarrow s$.
5. Je-li $p < p_{max}$, pak se vraťte na bod 2.

Činnost algoritmu si ukážeme na následujícím jednoduchém příkladu (v obrázcích jsou hodnoty stavů zaokrouhlené na tři desetinná místa).

Uvažujme jednoduchou desku s 3x3 poli, která má dva cílové stavy, a to stav (3,3) s odměnou $r(3,3) = +1$ (žádaný cílový stav) a stav (3,2) s odměnou $r(3,2) = -1$ (nežádoucí cílový stav). Počáteční ohodnocení všech stavů jsou nulová a pro jednoduchost uvažujme, že i odměny $r(s)$ ve všech necílových stavech jsou nulové. Také předpokládejme, že na desce nejsou žádné překážky, a že možné tahy jsou pouze na sousední políčka ve svislém nebo vodorovném směru. Dále nechť $\alpha = 0.1$ a $\gamma = 0.9$. Pro jednoduchost použijeme náhodný výběr kroků procházek (random policy) a všechny procházky budeme startovat ze stavu (1,1), i když obvykle se počáteční body procházek volí náhodně.

1	0	0	0
2	0	0	0.1
3	0	-1	1
	1	2	3

První náhodná procházka: (1,1) → (1,2) → (1,3) → (2,3) → (3,3)

$$U(1,1) = U(1,1) + \alpha \cdot (r(1,2) + \gamma \cdot U(1,2) - U(1,1)) = 0$$

$$U(1,2) = U(1,2) + \alpha \cdot (r(1,3) + \gamma \cdot U(1,3) - U(1,2)) = 0$$

$$U(1,3) = U(1,3) + \alpha \cdot (r(2,3) + \gamma \cdot U(2,3) - U(1,3)) = 0$$

$$U(2,3) = U(2,3) + \alpha \cdot (r(3,3) + \gamma \cdot U(3,3) - U(2,3)) = \\ = 0 + 0.1 \cdot (1 + 0.9 \cdot 0 - 0) = 0.1$$

$$U(3,3) = 0$$

1	0	0	0
2	0	0	0.1
3	0	-1	1
	1	2	3

Druhá náhodná procházka: (1,1) → (1,2) → (2,2) → (1,2) → (1,3) → (2,3) → (3,3)

$$U(1,1) = U(1,1) + \alpha \cdot (r(1,2) + \gamma \cdot U(1,2) - U(1,1)) = 0$$

$$U(1,2) = U(1,2) + \alpha \cdot (r(2,2) + \gamma \cdot U(2,2) - U(1,2)) = 0$$

$$U(2,2) = U(2,2) + \alpha \cdot (r(1,2) + \gamma \cdot U(1,2) - U(2,2)) = 0$$

$$U(1,2) = U(1,2) + \alpha \cdot (r(1,3) + \gamma \cdot U(1,3) - U(1,2)) = 0$$

$$U(1,3) = U(1,3) + \alpha \cdot (r(2,3) + \gamma \cdot U(2,3) - U(1,3)) = \\ = 0 + 0.1 \cdot (0 + 0.9 \cdot 0.1 - 0) = 0.009$$

$$U(2,3) = U(2,3) + \alpha \cdot (r(3,3) + \gamma \cdot U(3,3) - U(2,3)) = \\ = 0.1 + 0.1 \cdot (1 + 0.9 \cdot 0 - 0.1) = 0.19$$

$$U(3,3) = 0$$

1	0	0	0.009
2	0	0	0.190
3	0	-1	1
	1	2	3

Třetí náhodná procházka: (1,1) → (2,1) → (1,1) → (1,2) → (1,3) → (2,3) → (3,3)

$$U(1,1) = U(1,1) + \alpha \cdot (r(2,1) + \gamma \cdot U(2,1) - U(1,1)) = 0$$

$$U(2,1) = U(2,1) + \alpha \cdot (r(1,1) + \gamma \cdot U(1,1) - U(2,1)) = 0$$

$$U(1,1) = U(1,1) + \alpha \cdot (r(1,2) + \gamma \cdot U(1,2) - U(1,1)) = 0$$

$$U(1,2) = U(1,2) + \alpha \cdot (r(1,3) + \gamma \cdot U(1,3) - U(1,2)) = \\ = 0 + 0.1 \cdot (0 + 0.9 \cdot 0.009 - 0) = 0.0008$$

$$U(1,3) = U(1,3) + 0.1 \cdot (r(2,3) + 0.9 \cdot U(2,3) - U(1,3)) = \\ = 0.009 + 0.1 \cdot (0 + 0.9 \cdot 0.19 - 0.009) = 0.0252$$

$$U(2,3) = U(2,3) + 0.1 \cdot (r(3,3) + 0.9 \cdot U(3,3) - U(2,3)) = \\ = 0.19 + 0.1 \cdot (1 + 0.9 \cdot 0 - 0.19) = 0.271$$

$$U(3,3) = 0$$

1	0	0.001	0.025
2	0	0	0.271
3	0	-1	1
	1	2	3

Čtvrtá náhodná procházka: $(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (3,2)$

$$U(1,1) = U(1,1) + \alpha \cdot (r(2,1) + \gamma \cdot U(2,1) - U(1,1)) = 0$$

$$U(2,1) = U(2,1) + \alpha \cdot (r(3,1) + \gamma \cdot U(3,1) - U(2,1)) = 0$$

$$U(3,1) = U(3,1) + \alpha \cdot (r(3,2) + \gamma \cdot U(3,2) - U(3,1)) = \\ = 0 + 0.1 \cdot (-1 + 0.9 \cdot 0 - 0) = -0.1$$

$$U(3,2) = 0$$

1	0	0.001	0.025
2	0	0	0.271
3	-0,100	-1	1
	1	2	3

Pátá náhodná procházka: $(1,1) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (3,2)$

$$U(1,1) = U(1,1) + \alpha \cdot (r(1,2) + \gamma \cdot U(1,2) - U(1,1)) = \\ = 0 + 0.1 \cdot (0 + 0.9 \cdot 0.001 - 0) = 0.00009$$

$$U(1,2) = U(1,2) + \alpha \cdot (r(2,2) + \gamma \cdot U(2,2) - U(1,2)) = \\ = 0.0008 + 0.1 \cdot (0 + 0.9 \cdot 0 - 0.0008) = 0.00072$$

$$U(2,2) = U(2,2) + \alpha \cdot (r(3,2) + \gamma \cdot U(3,2) - U(2,2)) = \\ = 0 + 0.1 \cdot (-1 + 0.9 \cdot 0 - 0) = -0.1$$

$$U(3,2) = 0$$

1	0.000	0.001	0.025
2	0	-0.100	0.271
3	-0,100	-1	1
	1	2	3

Po naučení pak systém přechází z libovolného (necílového) stavu do jeho sousedního stavu, který má nejvyšší hodnotu.

5.3.3 Metoda Q learning (Off-Policy)

Metoda Q learning je podobná metodě TD learning, ale je aktivní metodou, tj. bez předem dané strategie. Agent si akci v každém stavu vybírá sám, obvykle akci s nejvyšším ohodnocením, ale k výběru může použít i svých předchozích zkušeností, případně i zkušeností jiných agentů.

Metoda Q learning místo hodnocení stavů hodnotí akce v těchto stavech a k tomuto hodnocení používá vztah

$$Q(s, a) = Q(s, a) + \alpha \cdot \left(r(s') + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \right),$$

kde značí $Q(s, a)$ označuje ohodnocení akce a provedené ve stavu s a kde výraz $\max_{a'} Q(s', a')$ označuje maximální hodnotu z ohodnocení všech akcí a' , které je možné provést ve stavu s' . Význam ostatních symbolů je stejný jako u metody TD learning.

Algoritmus Q learning

1. Zvolte hodnoty koeficientů α a γ ($0 < \alpha \leq 1$; $0 < \gamma \leq 1$) a vynulujte ohodnocení $Q(s, a)$ všech akcí a ve všech stavech s . Dále vynulujte počítadlo procházek $p = 0$ a nastavte jejich maximální počet p_{max} . Nastavte $start \rightarrow s$.
2. Vyberte akci a , která povede k přechodu ze stavu s do stavu s' .
3. Vypočítejte novou hodnotu vybrané akce a ve stavu s pomocí vztahu:

$$Q(s, a) = Q(s, a) + \alpha \cdot \left(r(s') + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \right)$$

4. Je-li stav s' cílovým stavem, pak $p + 1 \rightarrow p$, $start \rightarrow s$, jinak $s' \rightarrow s$.
5. Je-li $p < p_{max}$, pak se vraťte na bod 2.

Činnost algoritmu si ukážeme na podobném příkladu jako u metody TD learning (deska s 3×3 poli a dvěma cílovými stavy, koeficienty $\alpha = 0.1$, $\gamma = 0.9$). Omezíme se na výběr akce ve stavu (2,2) a přepočítání jejího ohodnocení pro aktuální stav učení ukázaný na obrázku. Pro pohyby nahoru, doprava, dolů a doleva použijeme symboly U , R , D , L (Up, Right, Down, Left).

Pokud by agent použil nejlépe ohodnocenou akci, kterou je ve stavu (2,2) akce $Q((2,2),R)$, tak by přešel do stavu (2,3) a nové ohodnocení akce $Q((2,2),R)$ by pak bylo

$$\begin{aligned} Q((2,2),R) &= Q((2,2),R) + \alpha \cdot (r(2,3) + \gamma \cdot \max(Q((2,3)U), Q((2,3),L), Q((2,3),D)) - Q((2,2),R)) = \\ &= 0.45 + 0.1 \cdot (0 + 0.9 \cdot \max(0.2, -0.22, 0.77) - 0.45) = \\ &= \mathbf{0.4743} \end{aligned}$$

1	0.12	0.22	0.41	0.18
	0.12	0.36	0.59	
2	0.25	0.27	0.20	
	0.31	-0.1	0.45	-0.22
	-0.26	-0.57	0.77	
3	0	-1	1	
		1		
	1	2	3	

1	0.12	0.22	0.41	0.18
	0.12	0.36	0.59	
2	0.25	0.27	0.20	
	0.31	-0.1	0.4743	-0.22
	-0.26	-0.57	0.77	
3	0	-1	1	
		1		
	1	2	3	

Jak však bylo uvedeno dříve, agent může k výběru akce použít i svých předchozích zkušeností, apod., a může tak vybrat i jinou akci, pro kterou by pak nové její ohodnocení bylo

$$\begin{aligned} Q((2,2),U) &= Q((2,2),U) + \alpha \cdot (r(1,2) + \gamma \cdot \max(Q((1,2),L), Q((1,2),D), Q((1,2),R)) - Q((2,2),U)) = \\ &= 0.27 + 0.1 \cdot (0 + 0.9 \cdot \max(0.22, 0.36, 0.41) - 0.45) = 0.2799 \end{aligned}$$

$$\begin{aligned} Q((2,2),L) &= Q((2,2),L) + \alpha \cdot (r(2,1) + \gamma \cdot \max(Q((2,1),R), Q((2,1),U), Q((2,1),D)) - Q((2,2),L)) = \\ &= -0.1 + 0.1 \cdot (0 + 0.9 \cdot \max(0.31, 0.25, -0.26) - 0.1) = -0.0621 \end{aligned}$$

$$\begin{aligned} Q((2,2),D) &= Q((2,2),D) + \alpha \cdot (r(3,2) + \gamma \cdot 0 - Q((2,2),D)) = \\ &= -0.57 + 0.1 \cdot (-1 + 0.9 \cdot \max(0, 0, 0) - 0.57) = -0.613 \end{aligned}$$

Podobným způsobem by se přepočítávaly stavy v jednotlivých krocích všech procházek.

5.3.4 SARSA (On-Policy)

SARSA je přístup ohodnocující akce (podobně jako Q learning), ale k výběru akcí používá strategii π . Vztah pro přepočítávání ohodnocení akcí z předchozí kapitoly se změní na

$$Q(s, a) = Q(s, a) + \alpha \cdot (r(s') + \gamma \cdot Q(s', a') - Q(s, a))$$

tj, místo hledání maxima z možných následujících akcí $\max_{a'} Q(s', a')$ se používá přímo akce

$Q(s', a')$ vybraná také strategií π . Jde tedy o postup s, a, r', s', a' (tj. SARSA).

6 Expertní systémy.

Expertní systémy jsou programové systémy, které simulují činnost expertů při řešení složitých problémově zaměřených úloh. Tyto systémy byly prvními praktickými aplikacemi přístupů umělé inteligence a významně přispěly k obnovení zájmu o tuto problematiku v průběhu sedmdesátých let minulého století.

Použití expertních systémů předpokládá splnění následujících podmínek:

- Problémová oblast musí být dostatečně úzká, aby do expertního systému bylo možné uložit všechny relevantní znalosti.
- Problémová oblast musí být dostatečně složitá, aby expertíza měla smysl.
- Musí existovat experti v dané problematice a musí být ochotni poskytnout své znalosti.
- Přínos expertízy musí převyšovat náklady spojené s tvorbou a provozováním expertního systému.

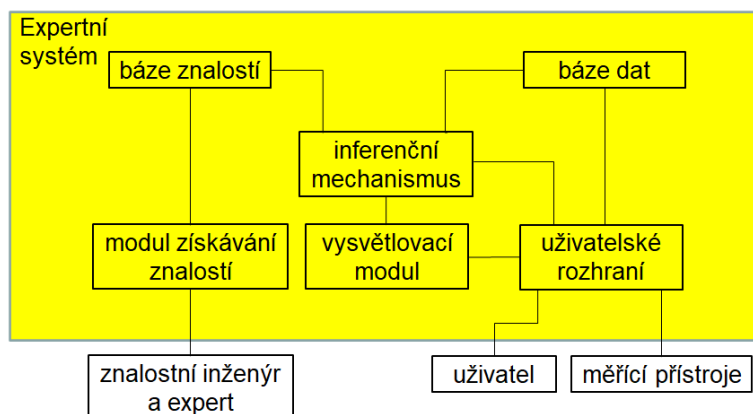
Mezi typické problémové oblasti vhodné pro expertní systémy patří:

- interpretace (rozpoznávání situací, např. havarijních)
- predikce (předpověď důsledků dané situace)
- diagnostika (určování stavu systému)
- konstrukce (skládání/kompozice objektu)
- plánování (stanovení posloupnosti akcí)
- monitorování (sledování údajů)
- řízení (vyhodnocování údajů + akční zásahy)
- učení (výuka) (speciální typ řízení)

Expertní systémy se podle charakteru řešených úloh rozdělují na dvě základní skupiny:

- Diagnostické expertní systémy (jejich úkolem je určit, která z předem daných hypotéz nejlépe koresponduje s daty dané konkrétní úlohy).
- Plánovací/návrhové expertní systémy (jejich úkolem je nalézt posloupnost kroků, kterými lze z daného stavu dosáhnout cílového stavu).

Blokové schéma expertního systému je ukázána na Obr. 6.1.



Obr. 6.1 Blokové schéma expertního systému

Expertní systémy důsledně oddělují báze znalostí a dat od mechanismu jejich využívání (inferenčního mechanismu). Prázdným (*shell*) expertním systémem se pak nazývá systém s prázdnými bázemi znalostí i dat, který je možné po naplnění těchtoází použít k řešení úloh z různých problémových oblastí.

Do báze znalostí se ukládají expertní znalosti, které expertní systém musí být schopen získat od expertů prostřednictvím modulu získávání znalostí. Při získávání znalostí pomáhá expertům znalostní inženýr, který musí být dobře seznámen jak s příslušným expertním systémem, tak i s řešenou problematikou.

Do báze dat (pracovní paměti) expertního systému se ukládají znalosti jedinečné, které se týkají konkrétních řešených úloh. Tato data poskytuje systému přes uživatelské rozhraní buď uživatel nebo/a různé měřicí přístroje a v průběhu výpočtu jsou tyto znalosti doplňovány výroky, resp. hypotézami odvozenými inferenčním mechanismem.

Expertní systém musí být schopen vysvětlit své úvahy a zdůvodnit své rady, analýzy nebo závěry. K tomu slouží vysvětlovací modul.

Nejdůležitější částí každého expertního systému je jeho inferenční (odvozovací) mechanismus, který musí být schopen řešit problémy na základě zadaných faktů pomocí znalostí z báze znalostí.

Typické expertní systémy mají bázi dat tvořenou fakty a bázi znalostí pravidly. Takové systémy se nazývají pravidlové expertní systémy (*Rule-based expert systems*).

6.1 Pravidlové expertní systémy

Pravidla v bázi znalostí se zapisují obvykle ve tvaru $P \rightarrow A$ (if P then A) a interpretují se takto:

- jestliže je splněna podmínka P , pak je možné vykonat akci A
- jestliže je splněna podmínka P , pak je možné odvodit A

Levá část pravidla se nazývá podmínková část (*antecedent*) a mohou se v ní vyskytovat spojky AND a OR. Pravá část pravidla se nazývá konsekvent a může také obsahovat několik akcí nebo závěrů, ale pouze se spojkou AND.

Data v bázi dat jsou reprezentována fakty reprezentujícími aktuální situaci.

Řešení problému spočívá v nalezení řetězce inferencí (*inference chain*), který představuje cestu od definice problému k jeho vyřešení. Existují dvě základní strategie:

- Dopředné řetězení (*forward chaining*) je postup řízený daty
Při této strategii se začíná se všemi známými daty a postupuje se k závěru. Usuzování řízené daty je vhodné pro problémy zahrnující syntézu (plánování, navrhování, rozvrhování, ...).
- Zpětné řetězení (*backward chaining*) je postup řízený cíli; v tomto případě se používá ještě zásobník, do kterého se ukládají dosud nesplněné (pod)cíle.
Tato strategie vybírá možný závěr a pokouší se dokázat jeho platnost hledáním dat, která jej podporují. Je vhodná pro diagnostické problémy, které mají malý počet cílových hypotéz.

Podle povolené maximální délky inferenčního řetězce rozlišujeme mezi mělkou a hlubokou inferencí.

Při inferenci se často v bázi znalostí nachází více aktuálně aplikovatelných pravidel. Tato pravidla představují tzv. konfliktní množinu, ze které se k aplikaci vybere jedno pravidlo, a to:

- Podle priority,
- podle pořadí v bázi znalostí,
- podle časových známek objektů v podmínkové části,
- podle počtu objektů v podmínkové části,
- náhodně.

Obvykle se pro stejnou množinu dat aplikuje pravidlo pouze jednou.

Příklad 1 – odvozování řízené daty (data driven / forward chaining)

Báze znalostí (5 pravidel):

- | | |
|--|-------------------------------|
| 1. $x \wedge y \rightarrow z$ | 4. $e \rightarrow u$ |
| 2. $x \wedge b \wedge c \rightarrow y$ | 5. $f \wedge u \rightarrow v$ |
| 3. $a \rightarrow x$ | |

Nechť báze dat obsahuje fakty a, b, c, d, e a necht' cílem odvozování (inference) je fakt z :

Iterace báze dat (pracovní paměť)	konfliktní množina	vybrané pravidlo
0 $a b c d e$	3, 4	3
1 $a b c d e x$	4, 2	4
2 $a b c d e x u$	2	2
3 $a b c d e x u y$	1	1
4 $a b c d e x u y z$		

Příklad 2 – odvozování řízené cílem (goal driven / backward chaining)

Předpokládejme stejnou bázi znalostí, stejnou počáteční bázi dat i stejný cíl jako v příkladu 1:

Iterace báze dat (pracovní paměť)	konfliktní množina	vybrané pravidlo	zásobník (pod)cílů
0 $a b c d e$	-	-	z
1 $a b c d e$	-	-	$y z$
2 $a b c d e$	3	3	$x y z$
3 $a b c d e x$	2	2	$y z$
4 $a b c d e x y$	1	1	z
5 $a b c d e x y z$			-

Činnost produkčního systému končí buď odvozením cíle (úspěchem), nebo prázdnou konfliktní množinou aplikovatelných pravidel (neúspěchem).

Všimněte si, že v předchozím příkladu byl při dopředném řetězení odvozován fakt u , který na odvození cíle z neměl žádný vliv. Je zřejmé, že v rozsáhlých bázích dat by pak tímto inferenčním postupem bylo zbytečně odvozováno mnoho podobných faktů. Proto se dopředné řetězení prakticky používá pouze v případech, kdy cíle odvozování nejsou známy (například při analýze nebo interpretaci) a v ostatních případech, především pak v diagnostice, se používá výhradně řetězení zpětné.

Příklad 3 – Jednoduchý expertní systém pro rozpoznávání zvířat (v jazyku PROLOG)

```
/* Baze znalosti */
rule(1,zvire,savec,[c1]).           %    if c1 then zvire je savec
rule(2,zvire,savec,[c2]).           %    or if c2 then zvire je savec
rule(3,zvire,ptak,[c3]).            %    if c3 then zvire je ptak
rule(4,zvire,ptak,[c4,c5]).         %    or if c4 and c5 then zvire je ptak
rule(5,savec,selma,[c6]).           %    if c6 then zvire je selma
rule(6,savec,selma,[c7,c8,c9]).     %    or if (c7 and c8 and c9) then zvire je selma
rule(7,savec,kopytnatec,[c10]).      %    atd.
rule(8,savec,sudokopytnik,[c11]).
rule(9,selma,gepard,[c12,c13]).
rule(10,selma,tygr,[c12,c14]).
rule(11,kopytnatec,zirafa,[c15,c16,c12,c13]).
rule(12,kopytnatec,zebra,[c17,c14]).
rule(13,ptak,pstros,[c18,c15,c16,c19]).
rule(14,ptak,tucnak,[c18,c20,c19]).
rule(15,ptak,cap,[c23,c15,c16,c17]).
rule(16,ptak,albatros,[c21]).
rule(17,selma,kocka,[c22,c25]).
rule(18,selma,kocka,[c22,c9]).
rule(19,selma,pes,[c22,c24]).
rule(20,selma,pes,[c22,c26]).

/* Otazky uzivatelum */
ask(c1):-write('Ma srst? ').
ask(c2):-write('Dava mleko? ').
ask(c3):-write('Ma peri? ').
ask(c4):-write('Leta? ').
ask(c5):-write('Klade vejce? ').
ask(c6):-write('Zivi se masem? ').
ask(c7):-write('Ma spicate zuby? ').
ask(c8):-write('Ma drapy? ').
ask(c9):-write('Miri jeho oci dopredu? ').
ask(c10):-write('Ma kopyta? ').
ask(c11):-write('Prezvykuje? ').
ask(c12):-write('Ma zlutohnedou barvu? ').
ask(c13):-write('Ma tmave skvrny? ').
ask(c14):-write('Ma cerne pruhy? ').
ask(c15):-write('Ma dlouhe nohy? ').
ask(c16):-write('Ma dlouhy krk? ').
ask(c17):-write('Ma bilou barvu? ').
ask(c18):-write('Je pravda, ze neumí letat? ').
ask(c19):-write('Je cernobíle? ').
ask(c20):-write('Umi plavat? ').
ask(c21):-write('Leta opravdu dobře? ').
ask(c22):-write('Je to domácí zvíře? ').
ask(c23):-write('Žije u vody? ').
ask(c24):-write('Steka? ').
ask(c25):-write('Mnouka? ').
ask(c26):-write('Chodí na vodítku? ').
```

```

/* Inferencni mechanismus (na uloze nezavisly) */
e:- retrfakt,
    write('Expertni system pro rozpoznavani zvirat. '),nl,
    write('Na otazky odpovidejte ano / cokoliv jineho = ne:'),
    nl,nl,recognition(zvire,0).

retrfakt:- retract(fakt(__,__)),fail.                % Odstraní odpovědi předchozí expertízy
retrfakt.

recognition(X,I):- rule(N,X,Y,Z),conditions(Z),conclusion(X,Y,N), % Navazani promennych
    J is I+1,recognition(Y,J).
recognition(__,__):- I > 0,write('Vic nevim. '),!,nl.
recognition(__,__):- write('Takove zvire neznam. '),nl.

conditions([]).
conditions([X|Y]):- condition(X),conditions(Y).

condition(X):- fakt(X,ano),!.                        % Fakt (kladna odpoved) jiz je v databazi
condition(X):- fakt(X,_),!,fail.                    % Fakt (zaporna odpoved) jiz je v databazi
condition(X):- ask(X),read(Y),assertz(fakt(X,Y)),Y=ano. % Dotaz + odpověď do baze dat

conclusion(X,Y,N):- write('Je to '),write(X),write('-'),write(Y),      % Zaver
    write(' podle pravidla '),write(N),write('. '),nl,nl.

```

Ukázka komunikace:

?- e.

Expertni system pro rozpoznavani zvirat.

Na otazky odpovidejte ano / cokoliv jineho = ne:

Ma srst? ano.

Je to zvire-savec podle pravidla 1.

Zivi se masem? |: ano.

Je to savec-selma podle pravidla 5.

Ma zlutohnedou barvu? |: ne.

Je to domaci zvire? |: ano.

Mnouka? |: ano.

Je to selma-kocka podle pravidla 17.

Vic nevim.

true .

Zásadní problém, se kterým se lze setkat při používání pravidlových expertních systémů, spočívá v nedostatku exaktních znalostí potřebných pro odvozování naprosto spolehlivých závěrů. Zmíněný nedostatek (neurčitost/neúplnost/nejasnost) bývá způsoben nejčastěji těmito příčinami:

- neznámými daty,
- nepřesně definovanými pravidly,
- nepřesně vyjadřovanými závěry expertů („často“, „někdy“ atd.),
- neshodami/odlišnými názory expertů na různá pravidla.

Existuje několik různých přístupů k práci s neurčitostí v expertních systémech. Pro získání základní představy se dále zaměříme na jediný a nejjednodušší z nich, a to na přístup který

spočívá v práci s tzv. faktorem jistoty (jiným častým přístupem k práci s neurčitostí je například použití fuzzy logiky).

6.1.1 Pravidlové systémy pracující s neurčitostí

Podmínkovou část pravidla budeme dále označovat jako předpoklad E (*Evidence*), odvozovaný závěr jako hypotézu H (*Hypothesis*) a důvěru ve správnost pravidla (tj. důvěru v platnost hypotézy H) jako faktor jistoty cf_r (*Certainty Factor in the Rule*):

if E then H $\{cf_r\}$

Faktor jistoty je formálně definován vztahem

$$cf_r = \frac{MB(H,E) - MD(H,E)}{1 - \min(MB(H,E), MD(H,E))}$$

kde $MB(H,E)$ značí tzv. míru důvěry (*Measure of Belief*) a $MD(H,E)$ míru nedůvěry (*Measure of Distrust*) v hypotézu H , pro jistý, tj. pravdivý předpoklad E . Pro takový předpoklad E tedy platí

$$cf(H, E) = cf_r$$

Faktor jistoty ve slovním vyjádření pak může znamenat například:

cf_r	Slovní význam
-1	jistě/určitě ne
-0.8	téměř jistě ne
-0.6	pravděpodobně ne
-0.4	možná ne
-0.2 až 0.2	nevím
0.4	možná
0.6	pravděpodobně
0.8	téměř jistě
1	jistě/určitě

Pro míry důvěry a nedůvěry pak platí vztahy

$$MB(H, E) = \begin{cases} 1 & \text{pro } P(H) = 1 \\ \frac{\max(P(H|E), P(H)) - P(H)}{1 - P(H)} & \text{jinak} \end{cases}$$

$$MD(H, E) = \begin{cases} 1 & \text{pro } P(H) = 0 \\ \frac{\min(P(H|E), P(H)) - P(H)}{-P(H)} & \text{jinak} \end{cases}$$

kde $P(H)$, resp. $P(H|E)$ značí nepodmíněnou, resp. podmíněnou pravděpodobnost hypotézy H .

Pro míru důvěry MB , míru nedůvěry MD a faktor jistoty cf_r platí:

Rozsah	$0 \leq MB \leq 1$
	$0 \leq MD \leq 1$
	$-1 \leq cf_r \leq 1$
Jistě pravdivá hypotéza	$MB = 1$
$P(H E) = 1$	$MD = 0$
$P(\neg H E) = 0$	$cf_r = 1$

Jistě nepravdivá hypotéza	$MB = 0$
$P(H E) = 0$	$MD = 1$
$P(\neg H E) = 1$	$cf_r = -1$
Neznámý předpoklad	$MB = 0$
$P(H E) = P(H)$	$MD = 0$
	$cf_r = 0$

Hodnoty faktorů jistoty se pomocí předchozích vztahů obvykle nepočítají. Buď jsou stanovovány experty ad hoc, nebo jsou v pravidlech $\text{if } E \text{ then } H \{cf_r\}$ s nejistými předpoklady E , tj. s $\{cf(E)\}$, vyhodnocovány pomocí následujících jednoduchých operací:

- Pro negaci nejistého předpokladu

$$cf(\neg E) = -cf(E)$$

- Pro nejistý předpoklad E

$$cf(H, E) = cf_r \cdot cf(E)$$

- Pro konjunkci nejistých předpokladů $E_1, E_2, \dots$ v podmínkové části pravidla:

$$cf(H, E_1 \wedge E_2 \wedge \dots) = cf_r \cdot \min(cf(E_1), cf(E_2), \dots)$$

- Pro disjunkci nejistých předpokladů $E_1, E_2, \dots$ v podmínkové části pravidla:

$$cf(H, E_1 \vee E_2 \vee \dots) = cf_r \cdot \max(cf(E_1), cf(E_2), \dots)$$

- Pro dvě pravidla se stejnou hypotézou (cf_i je zkráceným zápisem $cf_i(H, E_i)$):

$$\text{if } E_1 \text{ then } H \{cf_{r1}\} \quad cf_1 = cf_1(H, E_1) = cf_{r1} \cdot cf(E_1)$$

$$\text{if } E_2 \text{ then } H \{cf_{r2}\} \quad cf_2 = cf_2(H, E_2) = cf_{r2} \cdot cf(E_2)$$

$$cf_{12} = cf_{12}(H, E_1, E_2) = \begin{cases} cf_1 + cf_2 \cdot (1 - cf_1) & \text{if } cf_1 > 0 \text{ and } cf_2 > 0 \\ \frac{cf_1 + cf_2}{1 - \min(|cf_1|, |cf_2|)} & \text{if } (cf_1 \cdot cf_2) < 0 \\ cf_1 + cf_2 \cdot (1 + cf_1) & \text{if } cf_1 < 0 \text{ and } cf_2 < 0 \end{cases}$$

- Pro více pravidel se stejnou hypotézou se postupuje podobně: nejprve se určí cf_{12} pro dvě pravidla, pak se spočítá cf_{123} z cf_{12} a z cf_3 atd. Příklad výpočtu pro tři pravidla se stejnou hypotézou:

$$\text{if } (E_1 \wedge E_2 \wedge E_3) \vee (E_4 \wedge \neg E_5) \text{ then } H \{cf_{r1}\}$$

$$\text{if } E_6 \wedge E_7 \text{ then } H \{cf_{r2}\}$$

$$\text{if } E_8 \text{ then } H \{cf_{r3}\}$$

Nechť $cf(E_1) = 0.9, cf(E_2) = 0.8, cf(E_3) = 0.3, cf(E_4) = -0.5,$
 $cf(E_5) = -0.4 \rightarrow cf(\neg E_5) = 0.6, cf(E_6) = 0.7, cf(E_7) = -0.1, cf(E_8) = 0.6,$
 $cf_{r1} = 0.9, cf_{r2} = 0.7, cf_{r3} = 0.8$

Pak $cf_1 = 0.9 \cdot \max(\min(0.9, 0.8, 0.3), \min(-0.5, 0.4)) = 0.27 > 0$
 $cf_2 = 0.7 \cdot \min(0.7, -0.1) = -0.07 < 0$
 $cf_3 = 0.8 \cdot 0.6 = 0.48 > 0$
 $cf_{12} = (0.27 + (-0.07)) / (1 - \min(0.27, 0.07)) = 0.215 > 0$
 $cf_{123} = 0.215 + 0.48 \cdot (1 - 0.215) = 0.59$

Z praktických důvodů je vhodné upravit pravidla tak, aby jejich podmínkové části byly tvořeny pouze konjunkcemi, nebo disjunkcemi předpokladů (jde o podobnou úpravu, jako byla úprava obecných grafů na AND/OR grafy v kap. 2.3).

Například první pravidlo $\text{if } (E_1 \wedge E_2 \wedge E_3) \vee (E_4 \wedge \neg E_5) \text{ then } H \{cf_{r1}\}$ z předcházejícího příkladu lze upravit pomocí nových pravidel takto:

$\text{if } E_4 \wedge \neg E_5 \text{ then } H_{45} \{cf_{r45} = 1\}$

$\text{if } E_1 \wedge E_2 \wedge E_3 \text{ then } H_{123} \{cf_{r123} = 1\}$

$\text{if } E_{123} \vee E_{45} \text{ then } H \{cf_{r1}\}$

tj.

$$cf(E_{45}) = cf_{r45} \cdot \min(cf(E_4), cf(\neg E_5)) = 1 \cdot \min(-0.5, 0.4) = -0.5$$

$$cf(E_{123}) = cf_{r123} \cdot \min(cf(E_1), cf(E_2), cf(E_3)) = 1 \cdot \min(0.9, 0.8, 0.3) = 0.3$$

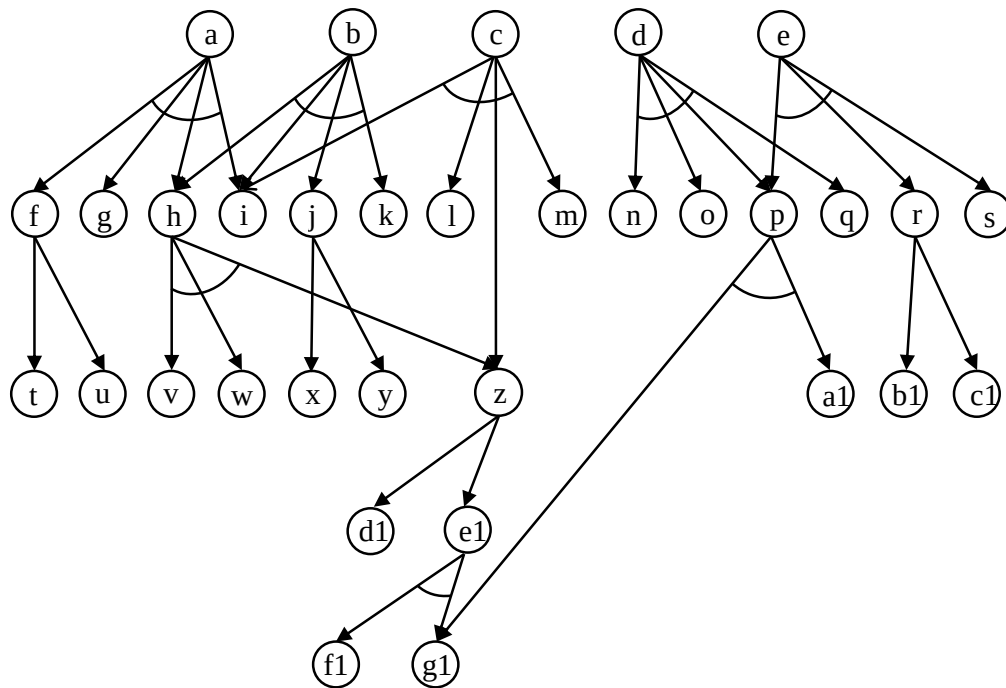
$$cf_1 = cf_{r1} \cdot \max(cf(E_{123}), cf(E_{45})) = 0.9 \cdot \max(-0.5, 0.3) = 0.27$$

Bázi znalostí ES pracujícího s neurčitostí pak lze zobrazit jako AND/OR graf, který má několik kořenových hypotéz a celou řadu vnitřních a listových hypotéz/předpokladů. Předpoklady, resp. hypotézy ohodnocené explicitně uživatelem se stávají fakty.

Práce ES pracujícího s neurčitostí probíhá obvykle takto:

1. Uživatel oznámí systému fakt, který musí odpovídat některému z uzlů AND/OR (tj. některému z předpokladů některého pravidla).
2. Systém hledá kořenové hypotézy, které zadaný fakt ovlivňuje.
3. Systém oznámí uživateli zjištěné kořenové hypotézy a jejich aktuální platnosti (faktory jistoty cf) a nastaví práh ceny verifikace (ověřování) hypotéz na zvolenou minimální hodnotu.
4. Je-li uživatel spokojen, činnost systému končí. Jinak systém zvýší práh pro ceny verifikace a prohledává graf pro každý relevantní kořen (hypotézu) směrem k listům, dokud nenarazí na uzel který:
 - odpovídá faktu zodpovězenému dříve uživatelem,
 - představuje hypotézu s cenou verifikace vyšší, než je aktuální práh. Pak systém použije buď apriorní platnost této hypotézy (pokud je zadána), nebo platnost průměrnou, tj. $cf = 0$.
 - představuje hypotézu s cenou verifikace nižší nebo stejnou, než je aktuální práh. Pak se prohledávání grafu rekurzivně zanořuje a vyčíslí se tak cf této hypotézy.
 - je dosud neznámým listovým předpokladem. Systém se dotáže na jeho platnost a tím se z tohoto předpokladu stane fakt.
5. Systém postupuje zpět ke kořenům a přepočítává platnosti všech vnitřních a kořenových hypotéz.
6. Systém oznámí uživateli nové platnosti všech relevantních kořenových hypotéz a vrací se k bodu 4.

Na Obr. 6.2 je ukázán demonstrační AND/OR graf, který používá expertní systém pracující s neurčitostí naprogramovaný v jazyku PROLOG. Ceny verifikace nastavuje od 0 do 100 po kroku 10.



Obr. 6.2 AND/OR graf pro demonstraci činnosti expertního systému pracujícího s neurčitostí

Příklad 4 – Expertní systém pracující s neurčitostí (v jazyku PROLOG)

/* Baze znalosti */

```
rule(and,[f,g,h,i],a,0.9).           % Definice pravidel
rule(and,[h,i,j,k],b,0.9).
rule(and,[i,l,m,z],c,0.8).
rule(and,[n,o,p,q],d,0.9).
rule(and,[p,r,s],e,0.7).
rule(or,[t,u],f,0.85).
rule(and,[v,w,z],h,0.7).
rule(or,[x,y],j,0.75).
rule(and,[a1,g1],p,0.9).
rule(or,[b1,c1],r,0.8).
rule(or,[d1,e1],z,0.8).
rule(and,[f1,g1],e1,0.9).
```

```
apriori(h,0.4).                      % Stanovení implicitních hodnot pro některé hypotézy
apriori(k,-0.2).
apriori(r,0.5).
apriori(g1,0.1).
price(r,10).
```

```
price(n,20).                         % Stanovení cen pro některé hypotézy
price(g1,20).
price(h,40).
price(t,80).
price(u,80).
```

Ukázka konverzace:

?- expert(a1,0.8). % Spuštění expertízy zadáním faktu a1

The hypothesis n has the default value 0 and price of its verification is 20

Type the validity of o: -0.2.

The hypothesis g1 has the default value 0.1 and price of its verification is 20

The inner hypothesis p has the validity 0.090

Type the validity of q: |: 0.7.

The root hypothesis d has the validity -0.180

The hypothesis g1 has the default value 0.1 and price of its verification is 20

The inner hypothesis p has the validity 0.090

The hypothesis r has the default value 0.5 and price of its verification is 10

Type the validity of s: |: 0.4.

The root hypothesis e has the validity 0.063

Price threshold was 0

Are you satisfied (y/n): |: n.

The hypothesis n has the default value 0 and price of its verification is 20

The hypothesis g1 has the default value 0.1 and price of its verification is 20

The inner hypothesis p has the validity 0.090

The root hypothesis d has the validity -0.180

The hypothesis g1 has the default value 0.1 and price of its verification is 20

The inner hypothesis p has the validity 0.090

Type the validity of b1: |: -0.3.

Type the validity of c1: |: 0.6.

The inner hypothesis r has the validity 0.480

The root hypothesis e has the validity 0.063

Price threshold was 10

Are you satisfied (y/n): |: y.

true .

Pro úplnost a zvědavé čtenáře je níže uveden inferenční systém:

```
/* inference engine */
```

```
:- dynamic found/1. % definice dynamického faktu
```

```
expert(Hyp,CF):- retr_all, asserta(fakt(Hyp,CF)), find_roots(Hyp,Roots), % Start expertízy  
                asserta(threshold(10)), repeat, calculate(Roots,Roots), test_end.
```

```
vypis_roots([H|T]):- write(H), write(' '), vypis_roots(T).
```

```
vypis_roots([]):-nl.
```

```
retr_all:- retrfakt, retrfound, retrnotroot.
```

```
retrfakt:- retract(fakt(_, _)), fail.
```

```
retrfakt.
```

```
retrfound:- retract(found(_)), fail.
```

```
retrfound.
```

```
retrnotroot:- retract(not_root(_)), fail.
```

```
retrnotroot.
```

```
find_roots(Hyp,_):- search_up(Hyp,UpperHyp), assert_1(found(UpperHyp)), fail.
```

```
find_roots(_,L):- collect([],L), !.
```

```
search_up(Hyp,UH):- rule(_,List,H,_), member(Hyp,List), assert_1(not_root(Hyp)),  
                  search_up(H,UH).
```

```
search_up(Hyp,Hyp).
```

```
assert_1(X):- X, !.
```

```

assert_1(X):- asserta(X).
collect(Tmp,L):- retract(found(X)), !, search_root(Tmp,X,L).
collect(L,L).
search_root(Tmp,X,L):- not_root(X), retract(not_root(X)), collect(Tmp,L).
search_root(Tmp,X,L):- collect([X|Tmp],L).
calculate([],_).
calculate([H|T],Roots):- calc(H,Roots,_), calculate(T,Roots).
calc(H,Roots,CF_H):- rule(and,L,H,CF_R), calc_and(L,1,CF_E,Roots),
                        CF_H is CF_E * CF_R, write_calc(H,CF_H,Roots).
calc(H,Roots,CF_H):- rule(or,L,H,CF_R), calc_or(L,-1,CF_E,Roots),
                        CF_H is CF_E * CF_R, write_calc(H,CF_H,Roots).
calc(H,Roots,CF_H):- rule(comb,L,H,CF_R), calc_comb(L,CF_E,Roots),
                        CF_H is CF_E * CF_R, write_calc(H,CF_H,Roots).
write_calc(H,CF_H,Roots):- member(H,Roots),!, write('The root hypothesis '), write(H),
                        write(' has the validity '), format('~3f',CF_H), nl, nl.
write_calc(H,CF_H,_):- write('The inner hypothesis '), write(H),
                        write(' has the validity '), format('~3f',CF_H), nl.
calc_and([F|T],CF_in,CF_out,Roots):- test(F,CF_F,Roots), min(CF_in,CF_F,CF_tmp),
                        calc_and(T,CF_tmp,CF_out,Roots).
calc_and([],CF,CF,_).
calc_or([F|T],CF_in,CF_out,Roots):- test(F,CF_F,Roots), max(CF_in,CF_F,CF_tmp),
                        calc_or(T,CF_tmp,CF_out,Roots).
calc_or([],CF,CF,_).
calc_comb([H1,H2],CF_out,Roots):- test(H1,CF1,Roots), test(H2,CF2,Roots),
                        calc_v(CF1,CF2,CF_out).
calc_comb([H1,H2|T],CF_out,Roots):- test(H1,CF1,Roots), calc_comb([H2|T],CF2,Roots),
                        calc_v(CF1,CF2,CF_out).
calc_v(CF1,CF2,CF):- CF1 >= 0, CF2 >= 0, CF is CF1 + CF2 * (1 - CF1).
calc_v(CF1,CF2,CF):- (CF1 * CF2) < 0, abs(CF1,C1), abs(CF2,C2),
                        min(C1,C2,C), CF is (CF1 + CF2) / (1 - C).
calc_v(CF1,CF2,CF):- CF1 < 0, CF2 < 0, CF is CF1 + CF2 * (1 + CF1).
test(Hyp,CF,_):- fakt(Hyp,CF), !.
test(Hyp,CF,_):- threshold(T), price(Hyp,P), P>T, !, val(Hyp,CF),
                        write('The hypothesis '), write(Hyp),
                        write(' has the default value '), write(CF),
                        write(' and price of its verification is '), write(P), nl.
test(Hyp,CF,Roots):- calc(Hyp,Roots,CF), !.
test(Hyp,CF,_):- write('Type the validity of '),write(Hyp), write(': '),
                        read(CF), asserta(fakt(Hyp,CF)), !.
val(Hyp,CF):- apriori(Hyp,CF), !.
val(_,0).
min(X,Y,X):- X<Y,!.
min(_,Y,Y).
max(X,Y,X):- X>Y,!.
max(_,Y,Y).
abs(X,X):- X >= 0.
abs(X,Y):- X < 0, Y is -X.
test_end:- retract(threshold(T)), write('Price threshold was '), write(T),nl, nl,
                        write('Are you satisfied (y/n): '), read(X),test_e(X,T).
test_e(y,_):- !.
test_e(_,T):- T>=100, !, write('No other solution. '), nl.
test_e(_,T):- Tnew is T+10, asserta(threshold(Tnew)),fail.

```

7 Počítačové vidění

Ještě v šedesátých letech minulého století někteří z prvních průkopníků umělé inteligence věřili, že řešení problému vizuálního vstupu do počítače bude snadným krokem na cestě k řešení obtížnějších problémů, jako je uvažování a plánování na vyšší úrovni. Uvádí se, že v roce 1966 požádal Marvin Minsky z MIT svého doktoranda G. J. Sussmana, aby v průběhu letních prázdnin propojil kameru s počítačem a vytvořil program, který by popsal, co kamera vidí.

Téměř šedesát let později je tento úkol stále úplně nevyřešený, i když počítačové vidění se dnes běžně používá v mnoha různých oblastech: OCR (*Optical Character Recognition*), ANPR (*Automatic number-plate recognition*), Bing maps atd. Problematika počítačového vidění se stala samostatnou disciplínou s vazbami na matematiku, informatiku, psychologii a neurovědy.

V této kapitole se zaměříme pouze na principy zpracování a rozpoznávání černobílých snímků představovaných maticemi $m \times n$ pixelů.

Nechť každý pixel matice může nabývat k možných hodnot jasové funkce (úrovně šedi) a dále necht' označuje:

$I(u,v)$ aktuální hodnotu jasové funkce pixelu na pozici (u,v) ,
 max maximální hodnotu jasové funkce,
 $práh$ zvolenou hodnotu jasové funkce.

7.1 Transformace obrazů

Transformace obrazů je možné rozdělit na

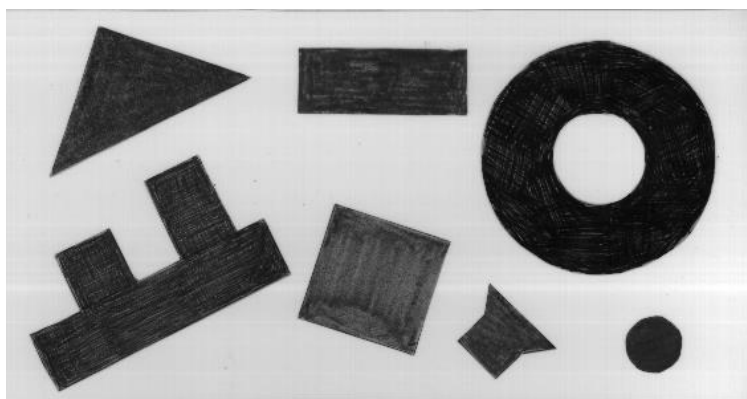
- Monadické (jednobodové) transformace
- Dyadické (dvoubodové) transformace
- Prostorové transformace (filtry, konvoluce)

7.1.1 Monadické transformace

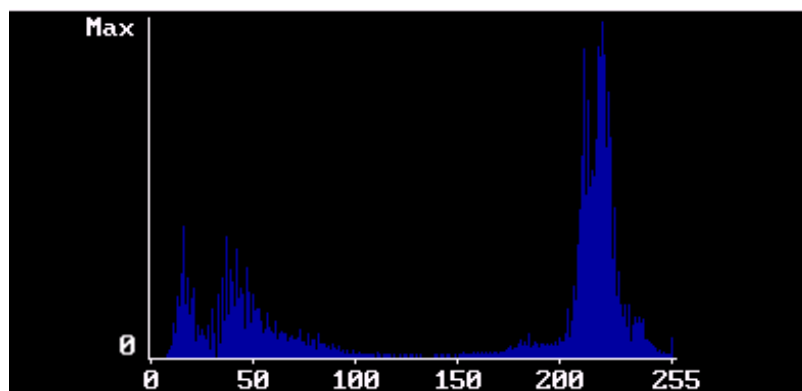
Inverze: $I(u,v) = max - I(u,v)$

Prahování: $I(u,v) = 0$ *pro* $I(u,v) \leq práh$
 $= max$ *pro* $I(u,v) > práh$

Příklady na monadické operace jsou ukázány na Obr. 7.1



Obr. 7.1.a) Šedotónový obrázek



Obr. 7.1.b) Histogram (počet pixelů jako funkce jasu) z obrázku 7.1.a



Obr. 7.1.c) Výsledný černobílý obrázek při nastavení prahu na hodnotu 128



Obr. 7.1.d) Invertovaný černobílý obrázek z Obr. 7.1.c

7.1.2 Dyadické transformace

Dyadické operace mění hodnotu jasu v pixelu podle vztahu $I(u,v) = f(I_1(u,v), I_2(u,v))$:

Součet obrazů (redukce šumu):

$$I(u,v) = f(I_1(u,v) + I_2(u,v))$$

Rozdíl obrazů (detekce změn):

$$I(u,v) = f(I_1(u,v) - I_2(u,v))$$

Násobení obrazů (realizace oken)

$$I(u,v) = I(u,v) \cdot m(u,v)$$

Funkce f současně ořezává výslednou hodnotu operací tak, aby byla v intervalu přípustných hodnot jasové funkce, například v intervalu $< 0, 255 >$.

Maska reálných čísel m má stejnou dimenzi jako obraz, při realizaci oken jsou prvky masky m buď jedničky (v okně) nebo nuly (mimo okno), násobení maskou je ukázáno na Obr. 7.2.



Obr. 7.2 Násobení obrázku maskou

7.1.3 Prostorové transformace

Prostorové operace mění hodnotu jasu v pixelu (u,v) podle vztahu

$$I(u, v) = \sum_{i=-a}^b \sum_{j=-c}^d h(i, j) I(u - i, v - j)$$

$$n = a + b + 1, \quad m = c + d + 1, \quad n = m = 3, 5, \dots$$

Funkce $h(i, j)$ se nazývá konvolučním jádrem, nebo také filtrem (propustí).

V počítačovém vidění se používají dva typy propustí:

Dolní propustí, pro které musí platit podmínka

$$h(i, j) \geq 0, \quad \sum_{i=-a}^b \sum_{j=-c}^d h(i, j) = 1$$

Horní propustí, pro které musí platit podmínka

$$\sum_{i=-a}^b \sum_{j=-c}^d h(i, j) = 0$$

Dále budeme pro jednoduchost uvažovat pouze filtry o rozměrech 3×3 , tj.

$$I(u, v) = \sum_{i=-1}^1 \sum_{j=-1}^1 h(i, j) I(u - i, v - j),$$

7.1.3.1 Dolní propustí

Dolní propustí se používají se především k redukci šumu. Ukážeme si tři filtry, 2 lineární (Box Blur filtr a Gaussův filtr) a jeden nelineární filtr (mediánový filtr):

Box Blur filtr, (lineární „rozmazávací“ filtr, hodnoty $h(i,j)$ jsou stejné):

a) Pixel má vyšší hodnotu jasu než sousední pixely

obraz se šumem	filtr	výsledný obraz																																																																					
<table><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>12</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr></table>	7	7	7	7	7	7	7	7	7	7	7	7	7	7	12	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	<table><tr><td>1/9</td><td>1/9</td><td>1/9</td></tr><tr><td>1/9</td><td>1/9</td><td>1/9</td></tr><tr><td>1/9</td><td>1/9</td><td>1/9</td></tr></table>	1/9	1/9	1/9	1/9	1/9	1/9	1/9	1/9	1/9	<table><tr><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td></tr><tr><td>x</td><td>8</td><td>8</td><td>8</td><td>7</td><td>x</td></tr><tr><td>x</td><td>8</td><td>8</td><td>8</td><td>7</td><td>x</td></tr><tr><td>x</td><td>8</td><td>8</td><td>8</td><td>7</td><td>x</td></tr><tr><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td></tr></table>	x	x	x	x	x	x	x	8	8	8	7	x	x	8	8	8	7	x	x	8	8	8	7	x	x	x	x	x	x	x
7	7	7	7	7	7																																																																		
7	7	7	7	7	7																																																																		
7	7	12	7	7	7																																																																		
7	7	7	7	7	7																																																																		
7	7	7	7	7	7																																																																		
1/9	1/9	1/9																																																																					
1/9	1/9	1/9																																																																					
1/9	1/9	1/9																																																																					
x	x	x	x	x	x																																																																		
x	8	8	8	7	x																																																																		
x	8	8	8	7	x																																																																		
x	8	8	8	7	x																																																																		
x	x	x	x	x	x																																																																		

Přepočtené hodnoty jasu pro označené pixely: $1/9 \cdot (8 \cdot 7 + 1 \cdot 12) = 68/9 = 7,555 \rightarrow 8$

b) Pixel má nižší hodnotu jasu než sousední pixely

obraz se šumem	filtr	výsledný obraz																																																																					
<table><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>5</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr></table>	7	7	7	7	7	7	7	7	7	7	7	7	7	7	5	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	<table><tr><td>1/9</td><td>1/9</td><td>1/9</td></tr><tr><td>1/9</td><td>1/9</td><td>1/9</td></tr><tr><td>1/9</td><td>1/9</td><td>1/9</td></tr></table>	1/9	1/9	1/9	1/9	1/9	1/9	1/9	1/9	1/9	<table><tr><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td></tr><tr><td>x</td><td>7</td><td>7</td><td>7</td><td>7</td><td>x</td></tr><tr><td>x</td><td>7</td><td>7</td><td>7</td><td>7</td><td>x</td></tr><tr><td>x</td><td>7</td><td>7</td><td>7</td><td>7</td><td>x</td></tr><tr><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td></tr></table>	x	x	x	x	x	x	x	7	7	7	7	x	x	7	7	7	7	x	x	7	7	7	7	x	x	x	x	x	x	x
7	7	7	7	7	7																																																																		
7	7	7	7	7	7																																																																		
7	7	5	7	7	7																																																																		
7	7	7	7	7	7																																																																		
7	7	7	7	7	7																																																																		
1/9	1/9	1/9																																																																					
1/9	1/9	1/9																																																																					
1/9	1/9	1/9																																																																					
x	x	x	x	x	x																																																																		
x	7	7	7	7	x																																																																		
x	7	7	7	7	x																																																																		
x	7	7	7	7	x																																																																		
x	x	x	x	x	x																																																																		

Přepočtené hodnoty jasu pro označené pixely: $1/9 \cdot (8 \cdot 7 + 1 \cdot 5) = 61/9 = 6,777 \rightarrow 7$

Gaussův filtr (lineární filtr, lépe koresponduje s dvourozměrnou diskretní Gaussovou funkcí):

obraz se šumem		filtr		výsledný obraz																																																																					
<table><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>12</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr><tr><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td><td>7</td></tr></table>	7	7	7	7	7	7	7	7	7	7	7	7	7	7	12	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	$1/16 \cdot$	<table><tr><td>1</td><td>2</td><td>1</td></tr><tr><td>2</td><td>4</td><td>2</td></tr><tr><td>1</td><td>2</td><td>1</td></tr></table>	1	2	1	2	4	2	1	2	1		<table><tr><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td></tr><tr><td>x</td><td>7</td><td>8</td><td>7</td><td>7</td><td>x</td></tr><tr><td>x</td><td>8</td><td>8</td><td>8</td><td>7</td><td>x</td></tr><tr><td>x</td><td>7</td><td>8</td><td>7</td><td>7</td><td>x</td></tr><tr><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td><td>x</td></tr></table>	x	x	x	x	x	x	x	7	8	7	7	x	x	8	8	8	7	x	x	7	8	7	7	x	x	x	x	x	x	x
7	7	7	7	7	7																																																																				
7	7	7	7	7	7																																																																				
7	7	12	7	7	7																																																																				
7	7	7	7	7	7																																																																				
7	7	7	7	7	7																																																																				
1	2	1																																																																							
2	4	2																																																																							
1	2	1																																																																							
x	x	x	x	x	x																																																																				
x	7	8	7	7	x																																																																				
x	8	8	8	7	x																																																																				
x	7	8	7	7	x																																																																				
x	x	x	x	x	x																																																																				

Přepočtené a zaokrouhlené hodnoty jasu pro označené pixely:

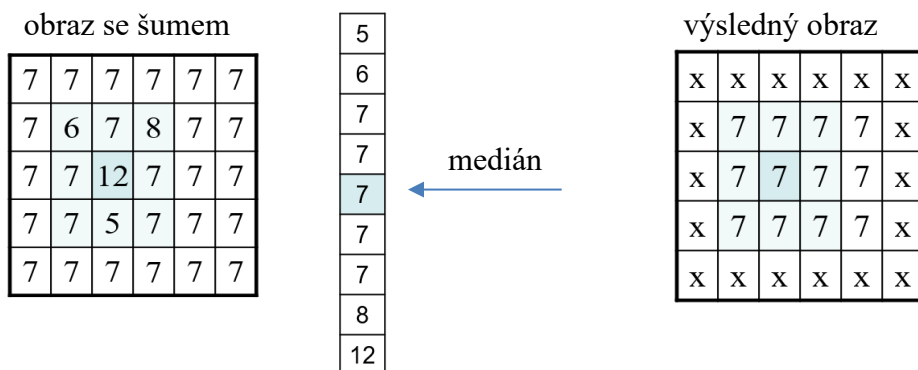
Střední (s původní hodnotou 12): $1/16 \cdot (12 \cdot 7 + 4 \cdot 12) = 132/16 = 8,25 \rightarrow 8$

Sousední (vlevo, vpravo, nahoře a dole): $1/16 \cdot (14 \cdot 7 + 2 \cdot 12) = 122/16 = 7,63 \rightarrow 8$

Sousední na úhlopříčkách: $1/16 \cdot (15 \cdot 7 + 1 \cdot 12) = 112/16 = 7$

Lineární filtry kromě odstraňování šumu vyhlazují obraz jako celek, tj. potlačují hrany a obrysy objektů na snímku. To může být nežádoucí, a proto se k redukci šumu používají častěji nelineární filtry, například mediánový filtr.

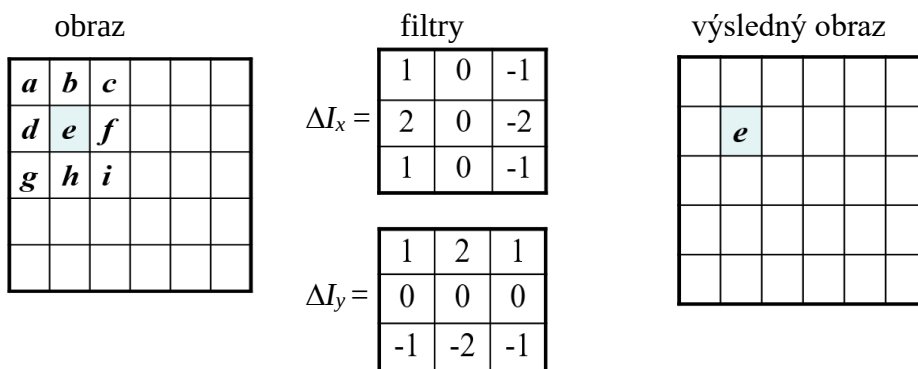
Mediánový filtr (seřadí hodnoty v daném okně a původní hodnotu pixelu nahradí mediánem):



7.1.3.2 Horní propusti

Horní propusti se používají k detekci a zvýrazňování hran. Využívají skutečnosti, že na hranách se výrazně mění hodnoty jasové funkce, a právě tyto změny horní propusti detekují. Horní propusti/filtry se často nazývají operátory.

Sobelův operátor (používá současně dva filtry / horní propusti):



Nová hodnota jasu pixelu *e* se vypočítá podle vztahu

$$\Delta I_x(e) = (I(a) + 2I(d) + I(g)) - (I(c) + 2I(f) + I(i))$$

$$\Delta I_y(e) = (I(a) + 2I(b) + I(c)) - (I(g) + 2I(h) + I(i))$$

$$I(e) = (\Delta I_x(e)^2 + \Delta I_y(e)^2)^{1/2} = ((I(a) + 2I(d) + I(g)) - (I(c) + 2I(f) + I(i)))^2 + ((I(a) + 2I(b) + I(c)) - (I(g) + 2I(h) + I(i)))^2)^{1/2}$$

Pro orientaci hrany procházející pixellem *e* zřejmě platí $\varphi(e) = \arctan(\Delta I_x(e)/\Delta I_y(e))$.

Příklad 1. obraz bez hran

7	7	7	7	7	7
7	7	7	7	7	7
7	7	7	7	7	7
7	7	7	7	7	7
7	7	7	7	7	7

výsledný obraz

x	x	x	x	x	x
x	0	0	0	0	x
x	0	0	0	0	x
x	0	0	0	0	x
x	x	x	x	x	x

Pro všechny pixely v obrázku bez hran platí $\Delta I_x = \Delta I_y$, a proto jejich nové hodnoty jsou nulové.

Příklad 2. obraz s hranou

4	7	7	7	7	7
4	4	7	7	7	7
4	4	4	7	7	7
4	4	4	4	7	7
4	4	4	4	4	7

výsledný obraz

x	x	x	x	x	x
x	13	13	4	0	x
x	4	13	13	4	x
x	0	4	13	13	x
x	x	x	x	x	x

Přepočtené a zaokrouhlené hodnoty jasu pro označené pixely:

Na hraně: $\Delta I_x = -9$ $\Delta I_y = 9$ $(\Delta I_x^2 + \Delta I_y^2)^{1/2} = 12,73 \rightarrow 13$

Blízko hrany: $\Delta I_x = -3$ $\Delta I_y = 3$ $(\Delta I_x^2 + \Delta I_y^2)^{1/2} = 4,24 \rightarrow 4$

Dále od hrany: $\Delta I_x = 0$ $\Delta I_y = 0$ $(\Delta I_x^2 + \Delta I_y^2)^{1/2} = 0$

Úhel hrany ve všech nenulových pixelech $\varphi = \arctan(\Delta I_x / \Delta I_y) = \arctan(-1) = -\pi/4 = -45^\circ$ je stejný a hranou je proto přímka.

Laplaceův operátor (zvýraznění hran):

obraz bez hran

7	7	7	7	7	7
7	7	7	7	7	7
7	7	7	7	7	7
7	7	7	7	7	7
7	7	7	7	7	7

filtr

0	1	0
1	-4	1
0	1	0

výsledný obraz

x	x	x	x	x	x
x	0	0	0	0	x
x	0	0	0	0	x
x	0	0	0	0	x
x	x	x	x	x	x

obraz s hranou

4	7	7	7	7	7
4	4	7	7	7	7
4	4	4	7	7	7
4	4	4	4	7	7
4	4	4	4	4	7

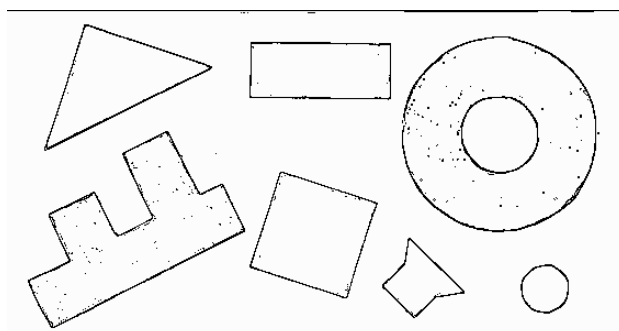
filtr

0	1	0
1	-4	1
0	1	0

výsledný obraz

x	x	x	x	x	x
x	6	-6	0	0	x
x	0	6	-6	0	x
x	0	0	6	-6	x
x	x	x	x	x	x

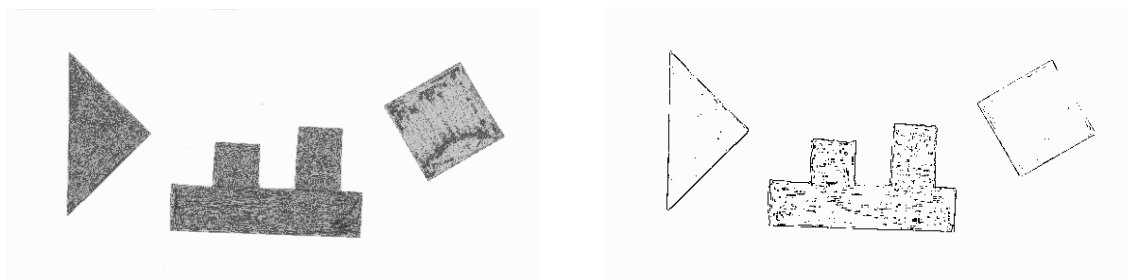
Na Obr. 7.3 je ukázán výsledek detekce hran v Obr. 7.1.a



Obr. 7.3 Detekce hran v Obr. 7.1.a

7.2 Detekce přímek a rohů

Obrázek po detekci hran nevypadá vždy jako na předchozím obrázku. Například na Obr. 7.4. je ukázán výsledek po detekci hran pro jinak nasvícené objekty.

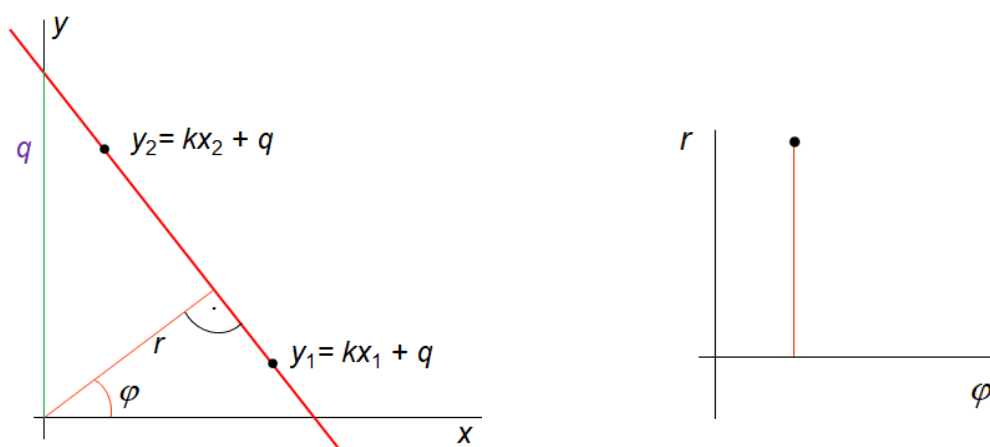


Obr. 7.4. Obrázek a výsledek po detekci hran

Je proto nutné zjistit, které body patří ke stejnému geometrickému objektu, což je velmi obtížná úloha. V dalším textu se omezíme pouze na detekci přímek a rohů (vrcholů).

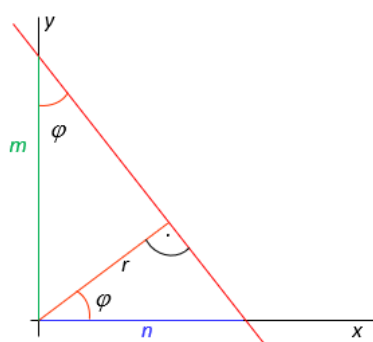
7.2.1 Detekce přímek

Všechny body přímky v prostoru souřadnic (x, y) musí vyhovovat rovnici $y = kx + q$, zatímco všechny body této přímky v prostoru parametrů (r, φ) představují jediný bod (Obr. 7.5).



Obr. 7.5 Přímka v 2D

Transformace z prostoru souřadnic (x, y) do prostoru parametrů (r, φ) je známá pod názvem Houghova transformace. Postup odvození je následující:



$$y = kx + q = -\frac{m}{n}x + m = -\frac{\frac{r}{\sin(\varphi)}}{\frac{r}{\cos(\varphi)}}x + \frac{r}{\sin(\varphi)} =$$

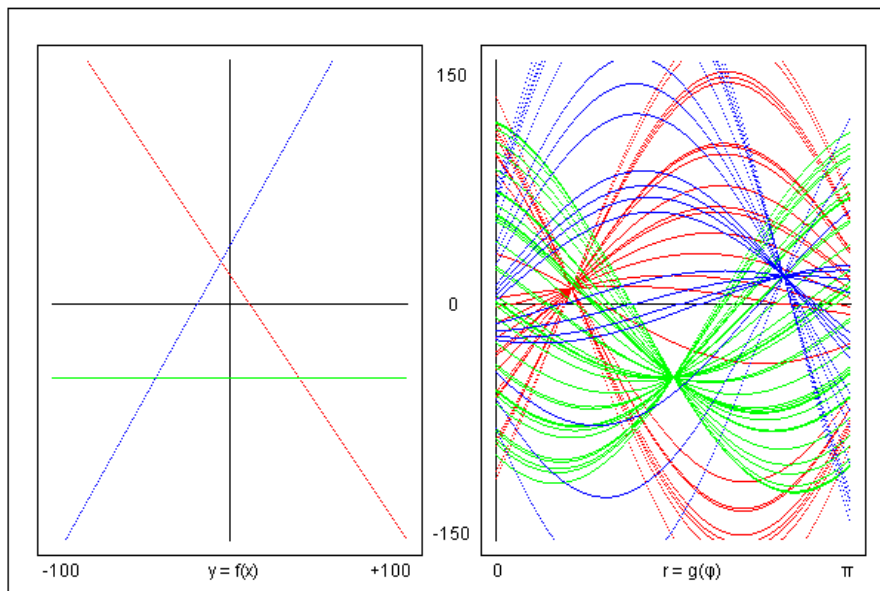
$$= \cot g(\varphi) x + \frac{r}{\sin(\varphi)}$$

odkud $k = \cot g(\varphi)$ a $q = r/\sin(\varphi)$

resp. $\varphi = \operatorname{arccotg}(k)$ a $r = q \sin(\operatorname{arccotg}(k))$

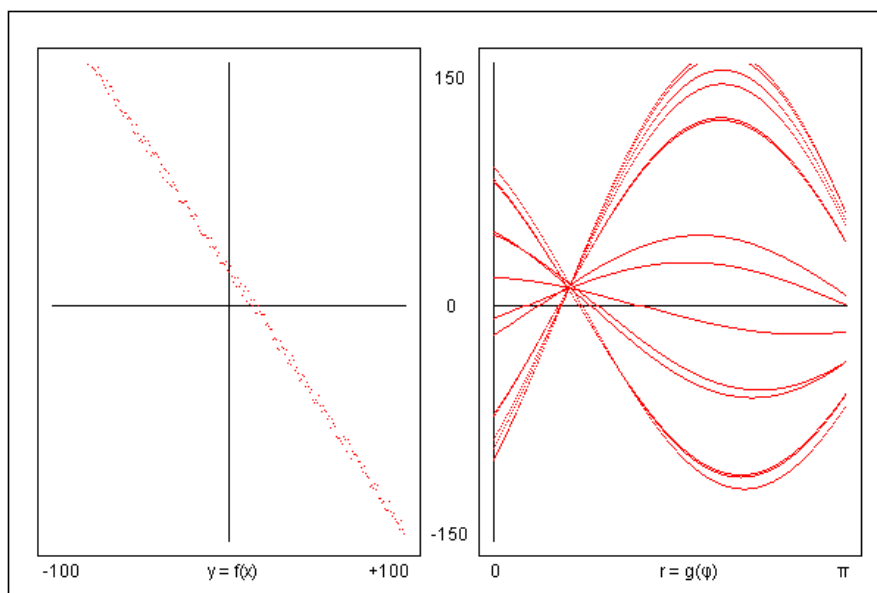
Na Obr. 7.6 je ukázáno zobrazení tří přímek v prostoru souřadnic a zobrazení některých jejich bodů v prostoru parametrů. Všechny křivky reprezentující různé body jedné přímky se nakonec protínají vždy v jednom bodu! Na Obr. 7.7 je pak ukázáno zobrazení „rozmazané“ přímky.

Houghova transformace



Obr. 7.6. Zobrazení tří přímek v prostoru souřadnic a zobrazení některých jejich bodů v prostoru parametrů

Houghova transformace



Obr. 7.7 Zobrazení rozmazané přímky v prostoru souřadnic a zobrazení některých jejích bodů v prostoru parametrů

Algoritmus Hough Transform

- Vytvořte dvourozměrné pole střadačů $\text{Acc}[r, \varphi]$ a všechny jeho prvky vynulujte;
- Pro všechny pixely (u, v) snímku provádějte
 - Je-li (u, v) pixelem hrany pak $(x, y) = (u - u_c, v - v_c)$, kde (u_c, v_c) značí střed snímku
 - Pro $\varphi_i = 0 \dots \pi$ počítejte $r_i = x \cos(\varphi_i) + y \sin(\varphi_i)$ a inkrementujte hodnoty $\text{Acc}[r_i, \varphi_i]$
- Vraťte seznam parametrů (r_i, φ_i) odpovídající nejvýraznějším nalezeným přímkám.

7.2.2 Detekce rohů

Filtry/operátory z kap. 7.1.3.2 byly založeny na skutečnosti, že se na hranách mění intenzita jasové funkce v jednom směru. Pro detekci rohů může být použit podobný přístup, ale nyní se musí detekovat výrazné změny jasové funkce současně v několika směrech.

Nechť I_x značí první parciální derivaci jasové funkce I ve směru osy x

$$I_x(u, v) = \frac{\partial I}{\partial x}(u, v) \cong \frac{\Delta I_x(u, v)}{\Delta x} = \frac{I(u+1, v) - I(u-1, v)}{2}$$

a I_y první parciální derivaci jasové funkce ve směru osy y

$$I_y(u, v) = \frac{\partial I}{\partial y}(u, v) \cong \frac{\Delta I_y(u, v)}{\Delta y} = \frac{I(u, v+1) - I(u, v-1)}{2}$$

Nechť

$$\bar{A} = I_x^2, \quad \bar{B} = I_y^2, \quad \bar{C} = I_x I_y$$

Po vyhlazení těchto funkcí (obvykle Gaussovým filtrem) je označíme symboly A, B, C a definujeme tzv. strukturální matici

$$M = \begin{bmatrix} A & C \\ C & B \end{bmatrix}$$

jejíž vlastní čísla λ_1 a λ_2 se získají z rovnice

$$\begin{bmatrix} A - \lambda & C \\ C & B - \lambda \end{bmatrix} = 0$$

$$\Rightarrow (A - \lambda)(B - \lambda) - C^2 = 0$$

$$\Rightarrow \lambda^2 - (A + B)\lambda + (AB - C^2) = 0$$

$$\Rightarrow \lambda_{1,2} = \frac{1}{2} \left(A + B \pm \sqrt{(A + B)^2 - 4(AB - C^2)} \right)$$

Pokud jsou obě hodnoty vlastních čísel $\lambda_{1,2}$ pro pixel (u, v) malé, je oblast okolo tohoto pixelu plochá, pokud je hodnota λ_1 velká a hodnota λ_2 malá (opačně to být nemůže), může pixel (u, v) náležet hraně a pokud jsou obě hodnoty $\lambda_{1,2}$ velké, stává se pixel (u, v) kandidátem na rohový pixel.

Z tohoto předpokladu vychází Harrisův detektor, který pro hledání rohů používá funkci

$$Q(u, v) = (AB - C^2) - \alpha(A + B)^2$$

Tato funkce vrací největší hodnoty právě pro rohové pixely. Koeficient α se volí z intervalu (0.04, 0.06) s tím, že čím vyšší je jeho hodnota, tím je detektor méně citlivý.

Algoritmus Harris Corner Detector

- Každý pixel (u,v) snímku, pro který je hodnota $Q(u,v)$ větší než zvolený práh, je vybrán za kandidáta na rohový pixel.
- Všichni kandidáti na rohový pixel se umístí do množiny kandidátů, ze které se následně vyřazují všechny pixely, které mají nižší hodnocení než jiné pixely v jejich topologickém sousedství.
- Zbylí kandidáti v množině kandidátů jsou pak považováni za skutečné rohové pixely.

7.3 Metrické a topologické vlastnosti snímku

K metrickým a topologickým vlastnostem snímku patří sousedství pixelů a jejich spojitost, plocha a její hranice, cesta a vzdálenost mezi dvěma pixely:

Sousedé pixelu (u,v)



4-okolí ($N_4(u,v)$): $(u,v+1), (u,v-1), (u+1,v), (u-1,v)$

8-okolí ($N_8(u,v)$): 4-okolí + $(u-1,v+1), (u-1,v-1), (u+1,v+1), (u+1,v-1)$

Spojitosť pixelů ($V = \{g_i, \dots, g_m\}$ je množina úrovní šedi použitá pro definici spojitosti):

4-spojitosť: Dva pixely (s,t) a (u,v) jsou spojitě jestliže
 $((u,v) \in N_4(s,t)) \wedge (I(s,t) \in V) \wedge (I(u,v) \in V)$

8-spojitosť: Dva pixely (s,t) a (u,v) jsou spojitě jestliže
 $((u,v) \in N_8(s,t)) \wedge (I(s,t) \in V) \wedge (I(u,v) \in V)$

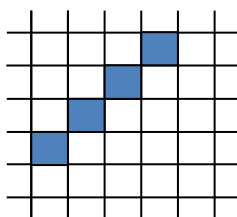
Plocha (oblast)

4-spojitou plochu/oblast tvoří dva 4-spojité pixely a dále pak všechny další pixely, které mají v této ploše alespoň jednoho 4-spojitého souseda.

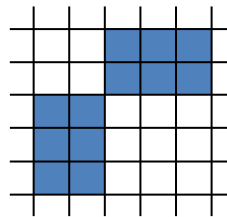
8-spojitou plochu/oblast tvoří dva 8-spojité pixely a dále pak všechny další pixely, které mají v této ploše alespoň jednoho 8-spojitého souseda.

Problémy:

V N_4 nespojitá křivka (přímka)!



V N_8 souvislá plocha!



Hranice plochy (oblasti)

Hraniční pixel 4-spojité plochy má alespoň jednoho 4-sousedu uvnitř a jednoho vně této plochy.
Hraniční pixel 8-spojité plochy má alespoň jednoho 8-sousedu uvnitř a jednoho vně této plochy.

Cesta

4-spojité cesta z pixelu (s,t) do pixelu (u,v) je posloupnost různých pixelů se souřadnicemi $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, pro kterou platí $(x_0, y_0) = (s, t)$, $(x_n, y_n) = (u, v)$ a pro každé $i \in \langle 1, n \rangle$ dále platí, že pixel (x_i, y_i) je 4-spojité s pixelem (x_{i-1}, y_{i-1}) .

8-spojité cesta z pixelu (s,t) do pixelu (u,v) je posloupnost různých pixelů se souřadnicemi $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, pro kterou platí $(x_0, y_0) = (s, t)$, $(x_n, y_n) = (u, v)$ a pro každé $i \in \langle 1, n \rangle$ dále platí, že pixel (x_i, y_i) je 8-spojité s pixelem (x_{i-1}, y_{i-1}) .

Délka cesty je počet pixelů cesty $- 1$ (t.j. délka cesty $= n$).

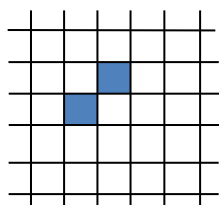
Vzdálenosti pixelů (s,t) , (u,v)

Pro každou vzdálenost mezi dvěma pixely musí platit vztahy:

$D((s,t),(u,v)) > 0$,	je kladná
$D((s,t),(u,v)) = D((u,v),(s,t))$,	je stejná v obou směrech
$D((s,t),(u,v)) \leq D((s,t),(w,z)) + D((w,z),(u,v))$	platí trojúhelníková nerovnost

Nejčastěji se používají tři vzdálenosti:

$D_E((s,t),(u,v)) = \text{Sqrt}((s-u)^2 + (t-v)^2)$	(Euklidovská)
$D_4((s,t),(u,v)) = s-u + t-v $	(4-okolí)
$D_8((s,t),(u,v)) = \max(s-u , t-v)$	(8-okolí)



$$\begin{aligned} D_E &= \sqrt{2} \\ D_4 &= 2 \\ D_8 &= 1 \end{aligned}$$

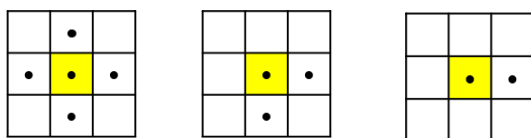
7.4 Morfologické operace

Morfologické operace jsou realizovány jako relace obrazu (bodové množiny) s jinou menší bodovou množinou H , tzv. strukturním elementem. Dále si ukážeme základní morfologické operace na dvojúrovňových/binárních snímcích (binární morfologické operace), ke kterým patří dilatace, erose, otevření a uzavření.

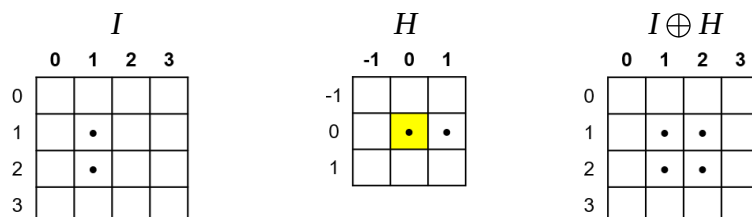
Označme symbolem I množinu všech bodů p snímku, pro které platí $I(p) = 1$ a podobně symbolem H množinu všech bodů q , pro které platí $H(q) = 1$. Pak zmíněné morfologické operace lze popsat takto:

- Dilatace $I \oplus H = \{p + q \mid \text{pro } p \in I \text{ a současně } q \in H\}$
- Eroze $I \ominus H = \{p \mid \text{pro } (p + q) \in I \text{ a každé } q \in H\}$
- Otevření $I \circ H = (I \ominus H) \oplus H$
- Uzavření $I \bullet H = (I \oplus H) \ominus H$

Morfologické masky H mohou být symetrické i nesymetrické, vždy však musí být zřejmý výchozí bod masky (*hot spot*). Příklady morfologických masek (žlutě označený bod se postupně přiloží na každý jedničkový pixel snímku, jedničkové body masky i snímku jsou označeny tečkou):



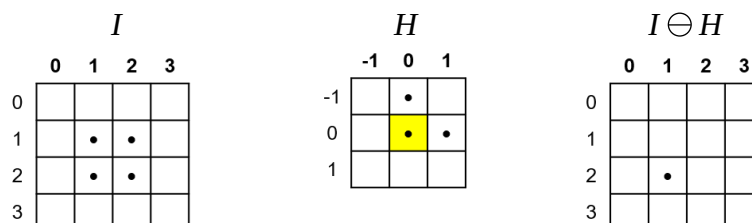
Příklad dilatace ($I \oplus H \equiv \{p + q \mid \text{pro } p \in I \text{ a současně } q \in H\}$):



$$I = \{(1,1), (2,1)\}, \quad H = \{(0,0), (0,1)\}$$

$$I \oplus H = \{(1,1) + (0,0), (1,1) + (0,1), (2,1) + (0,0), (2,1) + (0,1)\} = \{(1,1), (1,2), (2,1), (2,2)\}$$

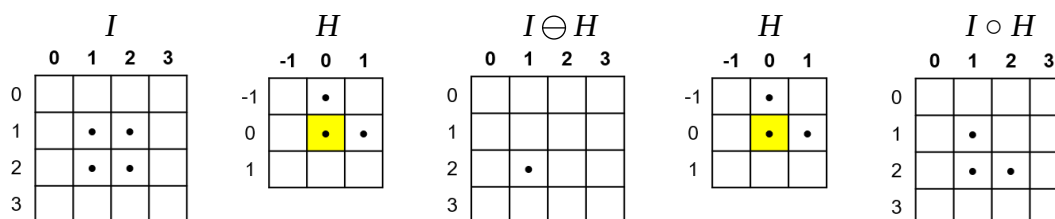
Příklad erose ($I \ominus H \equiv \{p \mid \text{pro } (p + q) \in I \text{ a každé } q \in H\}$):



$$I = \{(1,1), (1,2), (2,1), (2,2)\}, \quad H = \{(-1,0), (0,0), (0,1)\}$$

$$I \ominus H = \{(2,1)\}, \text{ protože pouze } (2,1) + (-1,0) = (1,1) \in I \text{ a současně} \\ (2,1) + (0,0) = (2,1) \in I \text{ a současně} \\ (2,1) + (0,1) = (2,2) \in I.$$

Příklad otevření ($I \circ H = (I \ominus H) \oplus H$):



$$I = \{(1,1), (1,2), (2,1), (2,2)\}, \quad H = \{(-1,0), (0,0), (0,1)\}$$

$$I \ominus H = \{(2,1)\}, \quad H = \{(-1,0), (0,0), (0,1)\}$$

$$I \circ H = (I \ominus H) \oplus H = \{(2,1) + (-1,0), (2,1) + (0,0), (2,1) + (0,1)\} = \{(1,1), (2,1), (2,2)\}$$

Příklad uzavření ($I \bullet H = (I \oplus H) \ominus H$):

I				H			$I \oplus H$				H			$I \bullet H$			
0	1	2	3	-1	0	1	0	1	2	3	-1	0	1	0	1	2	3
0				-1			0				-1			0			
1		•		0		•	1		•	•	0		•	1		•	
2		•		1			2		•	•	1			2		•	
3							3							3			

$$I = \{(1,1), (2,1)\}, \quad H = \{(0,0), (0,1)\}$$

$$I \oplus H = \{(1,1), (1,2), (2,1), (2,2)\}, \quad H = \{(0,0), (0,1)\}$$

$$I \bullet H = (I \oplus H) \ominus H = \{(1,1), (2,1)\}, \text{ protože pouze } (1,1) + (0,0) = (1,1) \in I \oplus H \text{ a současně}$$

$$(1,1) + (0,1) = (1,2) \in I \oplus H \text{ a současně}$$

$$(2,1) + (0,0) = (2,1) \in I \oplus H \text{ a současně}$$

$$(2,1) + (0,1) = (2,2) \in I \oplus H$$

Uvedené binární morfologické operace se používají pro předzpracování obrázku:

Dilatace zaplňuje díry a zálivy menší než strukturní element H a zvětšuje původní velikosti objektů na snímku.

Eroze odstraňuje ze snímku objekty, které jsou menší než strukturní element H a zmenšuje původní velikosti objektů.

Otevření odděluje od sebe objekty spojené úzkou mezerou a zjednodušuje tak rozpoznávání těchto objektů.

Uzavření spojuje objekty, které jsou blízko u sebe, zaplňuje díry menší než strukturní element a vyhlazuje obrysy objektů.

7.5 Analýza scény

Analýza scény spočívá v dekompozici (oddělení) jednotlivých objektů na snímku a následně v jejich rozpoznání. Dále budeme uvažovat pouze černobílé (dvouúrovňové) snímky, na kterých jsou obrazy 2D scény.

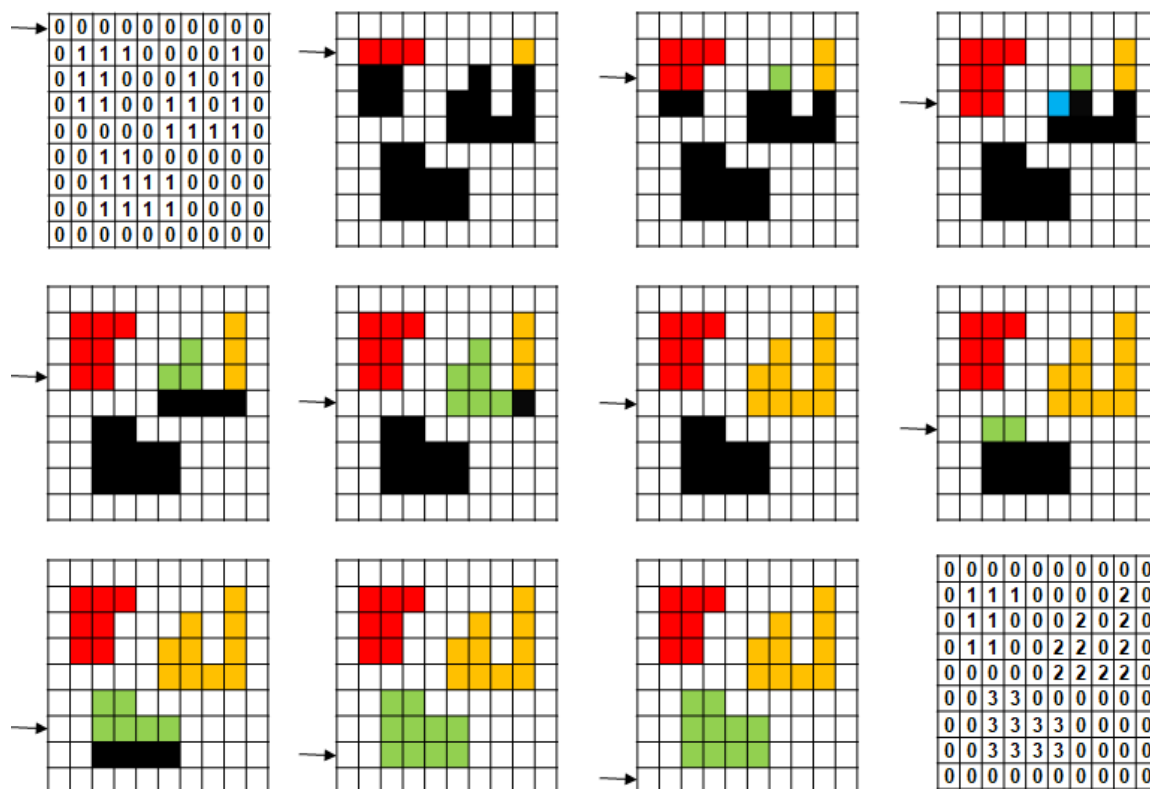
7.5.1 Dekompozice obrazů

Dekompozice obrazů (barvení objektů) se provádí takto:

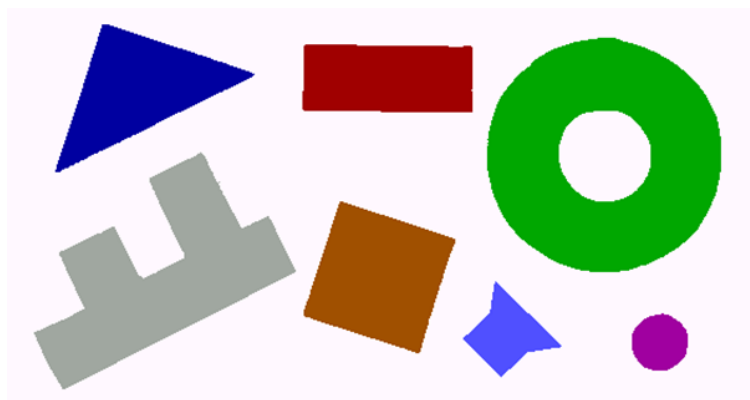
1. Zvolí se výchozí roh snímku, například horní levý, a nastaví se index objektu na hodnotu 1.
2. Postupně pro každý řádek (shora dolů) se prochází a „přebarvují“ se jednotlivé pixely tohoto řádku (zleva doprava):
 - Pokud je hodnota aktuálního pixelu nulová a předchozí pixel měl hodnotu větší než nula, tak se inkrementuje index objektu.
 - Pokud je hodnota pixelu rozdílná od nuly, tak:
 - Pokud je horním sousedem aktuálního pixelu již obarvený pixel, tak se hodnota aktuálního pixelu a dále všech pixelů plochy, do které tento aktuální pixel patří, nastaví na hodnotu horního souseda a pokračuje se v procházení řádku.

- Pokud je hodnota levého souseda aktuálního pixelu rozdílná od nuly, pak se hodnota aktuálního pixelu nastaví na hodnotu tohoto levého souseda a pokračuje se v procházení řádku.
- Pokud je hodnota levého souseda aktuálního pixelu nulová, tak se hodnota aktuálního pixelu nastaví na hodnotu indexu objektu a pokračuje se v procházení řádku.

Příklad (černobílý obraz je vlevo nahoře, výsledek po „obarvení“ objektů pak vpravo dole):



Na Obr. 7.8 je ukázán výsledek dekompozice – „obarvení“ objektů z Obr. 7.1.c.



Obr. 7.8 Výsledek dekompozice objektů z Obr. 7.1.c.

7.5.2 Rozpoznání objektů

Jak bylo uvedeno v kap. 5.1.3, může se k rozpoznávání objektů použít buď příznakový nebo strukturální popis. V této kapitole se zaměříme výhradně na rozpoznávání příznakové.

Používanými příznaky pro rozpoznávání jednotlivých objektů jsou:

- Plocha (počet pixelů)
- Délka hranice (počet pixelů)
- Počet děr
- Plochy děr (počet pixelů)
- Kompaktnost daná podílem $(\text{délka hranice})^2/(\text{plocha})$
- Minimální a maximální vzdálenosti hranice od těžiště
- Průměrná vzdálenost hranice od těžiště
- Poměr ploch děr k celkové ploše
- Momenty

Prvních 8 příznaků nepotřebuje bližší vysvětlení, a proto se zaměříme pouze na momenty.

Obecné momenty jsou dané vztahem

$$m_{pq} = \sum_x \sum_y x^p y^q I(x, y) \quad I(x, y) \in \{0, 1\}$$

Centrální momenty, které jsou invariantní vůči posunutí, jsou dány s použitím předchozího vztahu takto:

$$\mu_{pq} = \sum_x \sum_y (x - \bar{x})^p (y - \bar{y})^q I(x, y)$$

$$\bar{x} = \frac{m_{10}}{m_{00}},$$

$$\bar{y} = \frac{m_{01}}{m_{00}},$$

Normalizované centrální momenty, které jsou invariantní vůči posunutí i vůči změně velikosti, pak lze vypočítat pomocí vztahu

$$\eta_{pq} = \frac{\mu_{pq}}{\mu_{00}^{\frac{p+q}{2}+1}}$$

RST (Rotate/Scale/Translate) invariantní momenty jsou invariantní i vůči natočení. Jsou dány sedmi vztahy, ale pro rozpoznávání se často používají pouze první dva z nich

$$\Phi_1 = \eta_{20} + \eta_{02}$$

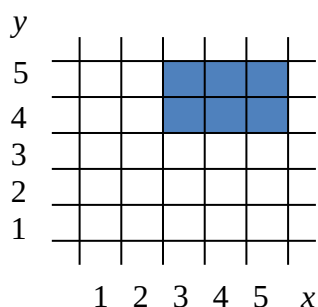
$$\Phi_2 = (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2$$

Dále se používá jejich kombinace, tzv. RST invariantní charakteristická čísla matice druhých momentů:

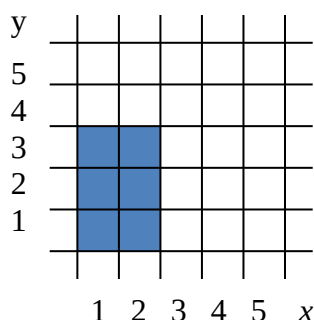
$$v_{min} = 0.5\phi_1 - \sqrt{\phi_2}$$

$$v_{max} = 0.5\phi_1 + \sqrt{\phi_2}$$

Příklad



$$\begin{aligned}
 m_{00} &= \mu_{00} = 6 \\
 \bar{x} &= \frac{\sum x}{6} = \frac{24}{6} = 4 \\
 \bar{y} &= \frac{\sum y}{6} = \frac{27}{6} = 4.5 \\
 \eta_{20} &= \frac{\mu_{20}}{\mu_{00}^2} = \frac{\sum (x - \bar{x})^2}{36} = \frac{2 * (-1)^2 + 2 * (0)^2 + 2 * (1)^2}{36} = \frac{1}{9} \\
 \eta_{02} &= \frac{\mu_{02}}{\mu_{00}^2} = \frac{\sum (y - \bar{y})^2}{36} = \frac{3 * (-0.5)^2 + 3 * (0.5)^2}{36} = \frac{1}{24} \\
 \eta_{11} &= \frac{\mu_{11}}{\mu_{00}^2} = \frac{\sum \sum (x - \bar{x})(y - \bar{y})}{36} = 0 \\
 \phi_1 &= \eta_{20} + \eta_{02} = \frac{1}{9} + \frac{1}{24} = \frac{11}{72} = 0.153 \\
 \phi_2 &= (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2 = \left(\frac{1}{9} - \frac{1}{24}\right)^2 + 0 = 4.82 * 10^{-3}
 \end{aligned}$$



$$\begin{aligned}
 m_{00} &= \mu_{00} = 6 \\
 \bar{x} &= \frac{\sum x}{6} = \frac{9}{6} = 1.5 \\
 \bar{y} &= \frac{\sum y}{6} = \frac{12}{6} = 2 \\
 \eta_{20} &= \frac{\mu_{20}}{\mu_{00}^2} = \frac{\sum (x - \bar{x})^2}{36} = \frac{3 * (-0.5)^2 + 3 * (0.5)^2}{36} = \frac{1}{24} \\
 \eta_{02} &= \frac{\mu_{02}}{\mu_{00}^2} = \frac{\sum (y - \bar{y})^2}{36} = \frac{2 * (-1)^2 + 2 * (0)^2 + 2 * (1)^2}{36} = \frac{1}{9} \\
 \eta_{11} &= \frac{\mu_{11}}{\mu_{00}^2} = \frac{\sum \sum (x - \bar{x})(y - \bar{y})}{36} = 0 \\
 \phi_1 &= 0.153, \quad \phi_2 = 4.82 * 10^{-3}
 \end{aligned}$$

Na Obr. 7.9. je ukázán výsledek rozpoznání objektů z Obr. 7.8., kdy za příznaky byla použita charakteristická čísla matice druhých momentů.

Komponenta	barvy		je: trojuhelnik
Komponenta	barvy		je: mezikruzi
Komponenta	barvy		je: obdelnik
Komponenta	barvy		je: kolecko
Komponenta	barvy		je: ctverec
Komponenta	barvy		je: parnik
Komponenta	barvy		je: trychtyr

Obr. 7.9. Výsledek rozpoznání objektů z Obr. 7.8

Podle ostatních příznaků lze zjistit i natočení jednotlivých objektů a podle polohy jejich těžišť $(\bar{x}, \bar{y})$ i jejich pozice ve snímku/scéně.

8 Zpracování přirozeného jazyka

Podobně jako v případě počítačového vidění nebyla v počátcích UI věnována zpracování přirozeného jazyka téměř žádná pozornost, a stejně jako v počítačovém vidění se později ukázala potřeba práce s přirozeným jazykem i značná obtížnost této problematiky.

Zpracování přirozeného jazyka (NLP, *Natural Language Processing*) zahrnuje dvě relativně samostatné části:

- Zpracování řeči – akustickou analýzu (převod řeči na text) a akustickou syntézu (převod textu na řeč).
- Zpracování textu.

V této kapitole se zaměříme pouze nejjednodušší základy akustické analýzy a přístupů ke zpracování textu.

8.1 Akustická analýza

Akustický signál/zvuk je mechanické vlnění, jehož frekvence, které je chopen vnímat člověk leží přibližně v intervalu 16 Hz až 20 kHz. Pro přenos srozumitelné řeči pak postačuje pásmo výrazně užší. Nejdůležitější složky, které zajišťují srozumitelnost řeči, leží v oblasti 1–3 kHz.

Dynamický rozsah lidského ucha (rozdíl mezi nejhlasitějším a nejtisším vnímatelným zvukem) je uprostřed slyšitelného frekvenčního pásma asi 120 dB, na okrajích pásma je však mnohem menší. Při běžném hovoru je průměrná hlasitost řeči ve vzdálenosti 1 m asi 40 až 60 dB, při velmi hlasitém projevu až 80 dB.

Nyquistův (Shannonův, Kotělnikovův) vzorkovací teorém říká, že dokonalá rekonstrukce signálu je možná pouze tehdy, když je vzorkovací frekvence větší než dvojnásobek maximální frekvence vzorkovaného signálu.

Pro signál s frekvencí 3 kHz je proto nutné vzorkovat s frekvencí 6000 vzorků/s a s uvažováním hlasitosti 60 dB, kdy poměr amplitud je 1:1000, je zapotřebí jednotlivé vzorky ukládat na deseti bitech (0-1023). Z toho vyplývá, že minimální rychlost ukládání vzorkovaných dat je 60 kb/s.

Průměrně rychlá řeč je asi 10 hlásek/s, a s uvažováním 5 bitů/hlásku a lingvistických pravidel je k uložení informace v řeči zapotřebí asi jen 10 až 40 bitů/s. Základní problémem akustické analýzy je tak redukce obrovské redundance řeči.

Příznaky (číselné parametry) akustického signálu se vyhodnocují na mikrosegmentech o délce 10 až 30 ms. Například pro vzorkovací frekvenci 10 kHz je pak v každém mikrosegmentu 200 až 600 vzorků (v dalším textu obecně N vzorků). Pro výpočet parametrů mají vzorky na daném mikrosegmentu buď stejnou váhu (pravoúhlé okénko), nebo vliv vzorků na okrajích segmentu je potlačován (Hammingovo okénko).

Pravoúhlé okénko:

$$w(m) = \begin{cases} 1 & \text{pro } m \in \langle 0, N-1 \rangle \\ 0 & \text{pro ostatní } m \end{cases}$$

Hammingovo okénko:

$$w(m) = \begin{cases} 0.54 - 0.46 * \cos(2\pi \cdot m / (N-1)) & \text{pro } m \in \langle 0, N-1 \rangle \\ 0 & \text{pro ostatní } m \end{cases}$$

Nejjednoduššími příznaky zjistitelnými na mikrosegmentech jsou příznaky v časové oblasti, které se počítají pomocí vztahu

$$F_i = \frac{1}{N} \sum_{k=-\infty}^{\infty} f(k) * w(n - k)$$

kde

i je pořadové číslo segmentu (1, 2, ...)

$n = N * (i - 1)$ je číslo vzorku pro navazující segmenty

$n = N * (i - 1) / 2$ je číslo vzorku pro překrývající se segmenty

Nechť $x(k)$ je hodnota vzorkovaného signálu v čase k . Pak podle funkce $f(k)$ rozeznáváme následující příznaky:

$f(k) = x(k)$ střední hodnotu

$f(k) = |x(k)|$ střední absolutní hodnotu

$f(k) = x^2(k)$ energii (obvykle se měří v několika frekvenčních pásmech)

$f(k) = (\text{sign}(x(k)) - \text{sign}(x(k-1)))$ počet průchodů signálu nulou

Dalšími příznaky mohou například být koeficienty autokorelačních funkcí

$$F_i(r) = \sum_{k=-\infty}^{\infty} x(k) * w(n - k) * x(k - r) * w(n - k - r)$$

a příznaky ve frekvenční oblasti založené na Fourierově transformaci, jejichž podrobnější popis přesahuje rámec předmětu.

Rozpoznávání řeči lze dělit do dvou skupin: na rozpoznávání, které porovnává rozpoznávaná slova se vzory a na rozpoznávání, které je založeno na statistických metodách.

8.1.1 Rozpoznávání porovnáváním se vzory

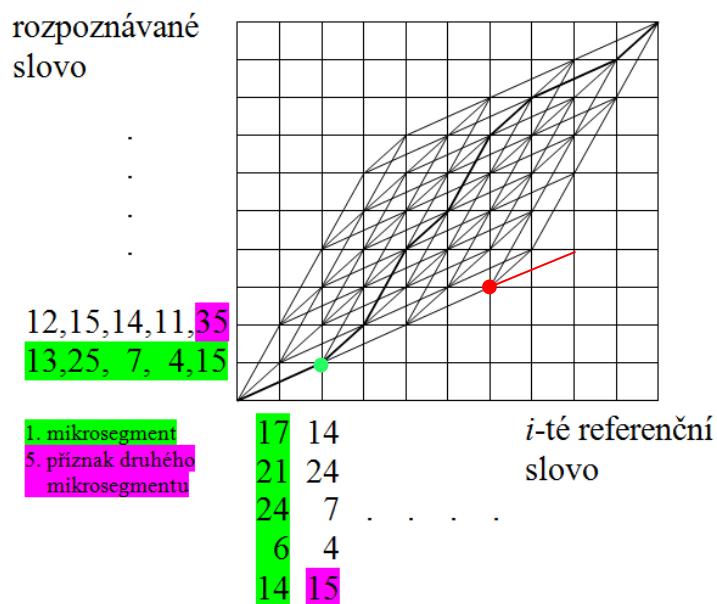
Přibližně do osmdesátých let minulého století se rozpoznávání řeči omezovalo na rozpoznávání izolovaně pronesených slov. Bylo poměrně úspěšné, když počet rozpoznávaných slov byl malý (v extrému 2 slova: *Start* a *Stop*) a když řeč byla pronášena v tichém prostředí osobou, na kterou byl systém natrénován. Vlastní rozpoznávání bylo založeno na porovnávání rozpoznávaných slov se vzory uloženými v databázi (*template matching*), přesněji na porovnávání příznaků získaných z aktuálně rozpoznávaného řečového signálu s příznaky jednotlivých slov, které byly získány trénováním dané množiny slov a jsou uloženy v databázi. Rozpoznávané slovo pak bylo přiřazeno k tomu slovu v databázi, ke kterému bylo nejbližší s respektováním předem daného prahu, jinak toto slovo nebylo rozpoznáno. K vlastnímu rozpoznávání se používala metoda dynamické deformace/borcení časové osy (DTW, *Dynamic Time Warping*).

Vzorová (referenční) i rozpoznávaná slova byla transformována na stejnou časovou délku (stejný počet mikrosegmentů) a na stejnou maximální hlasitost (maximální amplitudu vzorků). Matice příznaků pro i -té slovo (N mikrosegmentů, R příznaků na mikrosegmentu) měly tvar

$$\begin{array}{c} \text{index příznaku} \\ \text{index mikrosegmentu} \end{array} \quad \begin{array}{c} \xrightarrow{\hspace{1cm}} \\ \downarrow \left[\begin{array}{cccc} x_{i11} & x_{i12} & \dots & x_{i1R} \\ x_{i21} & x_{i22} & \dots & x_{i2R} \\ \vdots & & & \vdots \\ x_{iN1} & \dots & \dots & x_{iNR} \end{array} \right] \end{array}$$

kde x_{ijk} je k tý příznak na i -tém mikrosegmentu i -tého slova.

Na Obr. 8.1 je ukázán princip rozpoznávání, tj. porovnávání příznaků, pomocí DTW.



Obr. 8.1 Princip rozpoznávání pomocí DTW.

DTW vychází z předpokladu, že i při stejné délce slova, tj. stejném počtu mikrosegmentů, se mohou vnitřní úseky slova vyskytovat v různých mikrosegmentech (např. slova *mama/máma*). Nejčastěji se předpokládá posun o jeden mikrosegment, jak je naznačeno na obrázku, konec (celková délka) slova se samozřejmě nesmí změnit. Na počátku se proto mohou porovnávat příznaky prvních mikrosegmentů (počítá se vzdálenost D_{11}), pak první mikrosegment rozpoznávaného slova s druhým mikrosegmentem referenčního slova (vzdálenost D_{12}) a druhý mikrosegment rozpoznávaného slova s prvním mikrosegmentem referenčního slova (D_{21}). Vzdálenosti se mohou počítat například součtem absolutních hodnot rozdílů jednotlivých příznaků:

$$D_{11} = |13 - 17| + |25 - 21| + |7 - 24| + |4 - 6| + |15 - 14| = 28$$

$$D_{12} = |13 - 14| + |25 - 24| + |7 - 7| + |4 - 4| + |15 - 15| = 2$$

$$D_{21} = |12 - 17| + |15 - 21| + |14 - 24| + |11 - 6| + |35 - 14| = 47$$

.....

Nejkratší vzdálenost je D_{12} , a proto se výpočet přesune do bodu označeného na obrázku zelenou tečkou. Poznamenejme, že pokud by se výpočet dostal do bodu označeného červenou tečkou (třetí mikrosegment rozpoznávaného slova a šestý mikrosegment referenčního slova, pak by bylo možné pokračovat pouze dvěma směry. Červeně označený směr je zakázaný, protože by neumožnil pokračování cesty do koncového bodu). Za odchylku rozpoznávaného slova od právě porovnávaného i -tého referenčního slova je pak celkový součet jednotlivých vzdáleností.

8.1.2 Rozpoznávání s využitím statistických metod

Rozpoznávání s využitím statistických metod je zaměřeno především na rozpoznávání spojitě řeči, kdy slova, resp. části slov (fonémy), nebo celé promluvy jsou modelovány pomocí skrytých Markovových modelů.

Rozpoznání spojitě řeči, tj. posloupnosti W slov o délce L

$$W = W_{1:L} = (w_1, w_2, \dots, w_L)$$

z posloupnosti vektorů příznaků (*features*) zjištěných na mikrosegmentech akustického signálu

$$F = F_{1:T} = (\vec{f}_1, \vec{f}_2, \dots, \vec{f}_T)$$

je založeno na výběru takové posloupnosti slov $\hat{W}$, pro kterou je pravděpodobnost $P(W|F)$ maximální, tj.

$$\hat{W} = \underset{W}{\operatorname{argmax}} P(W|F)$$

Protože pravděpodobnost $P(W|F)$ není přímo zjistitelná, využije se Bayesova vzorce

$$P(W|F) = P(F|W) P(W) / P(F)$$

Protože pravděpodobnost $P(F)$ je stejná pro všechny posloupnosti slov, nemá vliv na výběr posloupnosti s maximální pravděpodobností, a proto můžeme tuto složku z předchozího vztahu vypustit. Pak

$$\hat{W} = \underset{W}{\operatorname{argmax}} P(W|F) = \underset{W}{\operatorname{argmax}} (P(F|W)P(W))$$

Pravděpodobnost $P(F|W)$ se zjišťuje pomocí akustického modelu a pravděpodobnost $P(W)$ pomocí jazykového modelu.

V plynulém projevu, resp. v jeho akustickém signálu, nelze přímo rozpoznat jednotlivá slova. Rozpoznatelné jsou fonémy/hlásky představující nejmenší jednotku řeči, která může rozlišovat jednotlivá slova (například záměnou fonému p ve slově *pes* za foném v dostaneme slovo *ves*, které má zcela odlišný význam).

Foném může mít několik výslovnostních variant, které se nazývají alofony (například foném d se odlišně vyslovuje ve slovech *dům* a *plod*). Foném je tedy množina všech podobných zvuků (alofonů), které v daném jazyku představují konkrétní podobu jedné hlásky.

8.1.2.1 Skrytý Markovův model

V teorii pravděpodobnosti je Markovův model stochastický model používaný k modelování chování systémů. Předpokládá, že budoucí stavy systému závisí pouze na jeho aktuálním stavu, nikoli na stavech, ve kterých byl systém dříve. Obecně tento předpoklad (Markovova vlastnost) umožňuje výpočet s modelem, který by byl jinak neřešitelný.

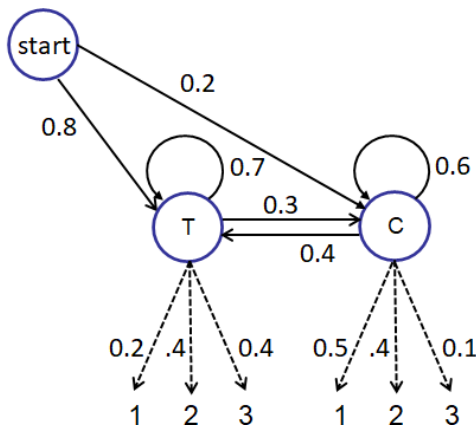
Skrytý Markovův model (HMM, *Hidden Markov Model*) je Markovův model, který modeluje systém se skrytými/nepozorovatelnými vnitřními stavy. Tyto stavy jsou ale zjistitelné nepřímo, prostřednictvím pozorovatelných (*Observable*) proměnných $\vec{o}_t$, které jsou dány podmíněnými pravděpodobnostmi $P(\vec{o}_t | s_t)$, kde proměnná s_t značí stav systému v čase t .

Použití HMM si ukážeme na jednoduchém příkladu: chcete zjistit, jak teplé byly červencové dny v roce 2015 (T - teplé, C - chladné), ale nemáte žádné oficiální záznamy. Máte však k dispozici následující údaje:

- apriorní pravděpodobnosti teplého/chladného dne v červenci $P(T) = 0.8$, $P(C) = 0.2$
- podmíněné pravděpodobnosti, že po teplém dnu T_i bude následovat teplý/chladný den $P(T_{i+1}|T_i) = 0.7$, $P(C_{i+1}|T_i) = 0.3$
- podmíněné pravděpodobnosti, že po chladném dnu bude následovat teplý/chladný den $P(T_{i+1}|C_i) = 0.4$, $P(C_{i+1}|C_i) = 0.6$

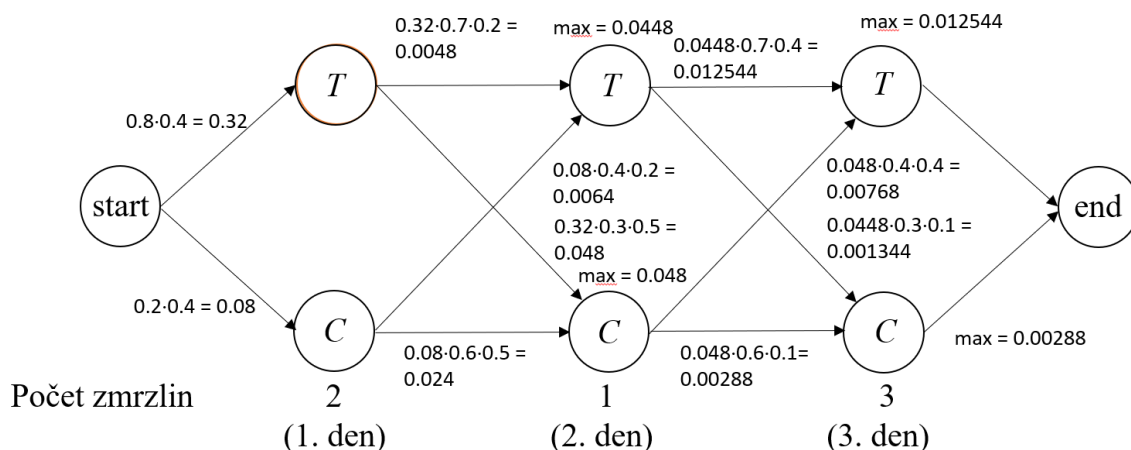
Dále máte deníček s údaji o počtech zmrzlin, které jste si v každém z těchto dnů koupil/a (pozorovatelné proměnné) a víte, že v teplé dny si kupujete zmrzliny s pravděpodobnostmi $P_z(1|T) = 0.2$, $P_z(2|T) = 0.4$ a $P_z(3|T) = 0.4$ a v chladné dny s pravděpodobnostmi $P_z(1|C) = 0.5$, $P_z(2|C) = 0.4$ a $P_z(3|C) = 0.1$.

HMM pak bude vypadat takto (Obr. 8.2):



Obr. 8.2 HMM pro výše uvedený příklad

Uvažujme nyní bez újmy na obecnosti tři po sobě jdoucí dny, ve kterých jste si postupně zakoupil/a 2, 1 a 3 zmrzliny. Z hodnot pozorovaných proměnných (počtů zmrzlin) máme určit, jak teplé tyto tři dny byly. Nejznámějším postupem je použití Viterbiho algoritmu. Ten hledá nejpravděpodobnější posloupnosti skrytých stavů (teplých a chladných dnů), tzv. Viterbiho cestu, která vede k posloupnosti pozorovaných událostí (počtu zmrzlin). Viterbiho algoritmus lze jednoduše znázornit mřížkou, v níž na svislé ose jsou stavy (teplý a chladný den) a na vodorovné ose jednotlivé dny (Obr. 8.3):



Obr. 8.3 Řešení úlohy Viterbiho algoritmem

Pravděpodobnost, že první den bude teplý, je dána součinem apriorní pravděpodobnosti teplého dne a pravděpodobnosti, že jste si v tento den koupil/a dvě zmrzliny. Pravděpodobnost, že první den bude chladný, je dána součinem apriorní pravděpodobnosti chladného dne a pravděpodobnosti, že jste si v tento den koupil/a dvě zmrzliny.

$$\alpha_1 \cdot P(T_1) = P(T) \cdot P_z(2|T) = 0.8 \cdot 0.4 = 0.32$$

$$\alpha_1 \cdot P(C_1) = P(C) \cdot P_z(2|C) = 0.2 \cdot 0.4 = 0.08$$

Koeficient α_1 je dán součtem obou pravděpodobností $\alpha_1 = P(T_1) + P(C_1)$ a normalizuje obě pravděpodobnosti tak, aby jejich součtem byla hodnota jedna:

$$\alpha_1 = (0.32 + 0.08) = 0.4 \Rightarrow P(T_1) = 0.32/0.4 = 0.8, P(C_1) = 0.08/0.4 = 0.2, P(T_1) + P(C_1) = 1.$$

Protože hodnota koeficientů α_i nemá na nalezení Viterbiho cesty žádný vliv, není zapotřebí tuto úpravu provádět.

Pravděpodobnost, že druhý den bude teplý/chladný

$$\begin{aligned} \alpha_2 \cdot P(T_2) &= \max((P(T_1) \cdot P(T_2|T_1) \cdot P_z(1|T_2)), (P(C_1) \cdot P(T_2|C_1) \cdot P_z(1|T_2))) = \\ &= \max((0.32 \cdot 0.7 \cdot 0.2), (0.08 \cdot 0.4 \cdot 0.5)) = \max(0.0448, 0.016) = 0.0448 \end{aligned}$$

$$\begin{aligned} \alpha_2 \cdot P(C_2) &= \max((P(T_1) \cdot P(C_2|T_1) \cdot P_z(1|C_2)), (P(C_1) \cdot P(C_2|C_1) \cdot P_z(1|C_2))) = \\ &= \max((0.32 \cdot 0.3 \cdot 0.5), (0.08 \cdot 0.6 \cdot 0.5)) = \max(0.048, 0.024) = 0.048 \end{aligned}$$

A konečně pravděpodobnost, že třetí den bude opět teplý/chladný

$$\begin{aligned} \alpha_3 \cdot P(T_3) &= \max((P(T_2) \cdot P(T_3|T_2) \cdot P_z(3|T_3)), (P(C_2) \cdot P(T_3|C_2) \cdot P_z(3|T_3))) = \\ &= \max((0.0448 \cdot 0.7 \cdot 0.4), (0.048 \cdot 0.4 \cdot 0.4)) = \max(0.012544, 0.00768) = 0.012544 \end{aligned}$$

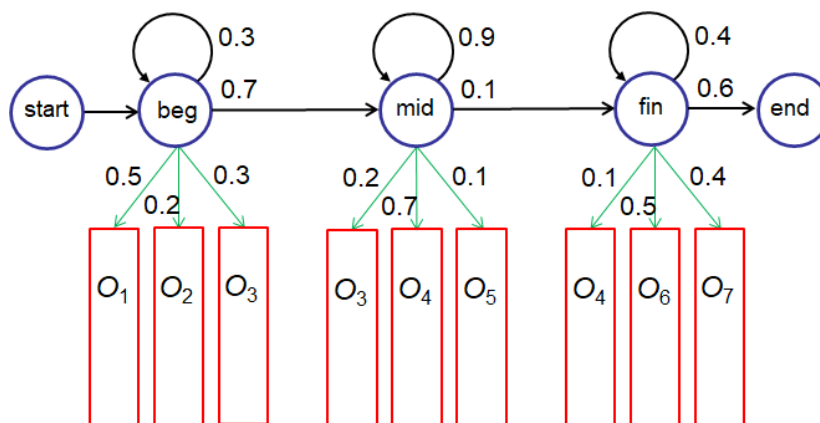
$$\begin{aligned} \alpha_3 \cdot P(C_3) &= \max((P(T_2) \cdot P(C_3|T_2) \cdot P_z(3|C_3)), (P(C_2) \cdot P(C_3|C_2) \cdot P_z(3|C_3))) = \\ &= \max((0.0448 \cdot 0.3 \cdot 0.4), (0.048 \cdot 0.4 \cdot 0.1)) = \max(0.005376, 0.00192) = 0.00192 \end{aligned}$$

Obrácená Viterbiho cesta je tedy T_3, T_2 a T_1 , tzn. že s nejvyšší pravděpodobností budou všechny tři dny teplé.

8.1.2.2 Akustický model

Cílem akustického modelu je určit pravděpodobnost $P(F|W)$, tj. posloupnosti vektorů příznaků F na základě pravděpodobnosti posloupnosti slov W .

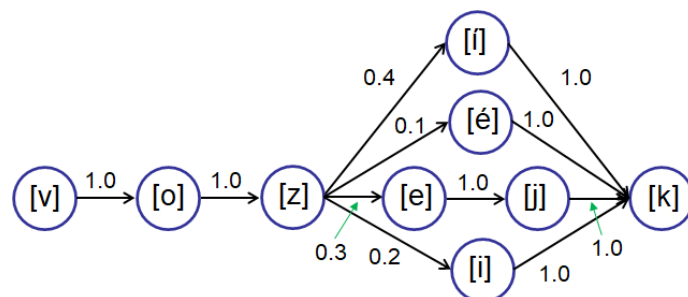
HMM pro foném m pak může vypadat například takto (Obr. 8.4):



Obr. 8.4 Příklad HMM pro rozpoznání hlásky

Označení stavů beg, mid a fin v obrázku znamená začáteční (*begin*), střední (*middle*) a koncový (*final*) stav rozpoznávané hlásky. V červených obdélníčkách jsou pak pozorovatelné proměnné, tj. vektory příznaků na mikrosegmentech.

HMM lze použít i pro rozpoznání slov, například pro slovo vozík (Obr. 8.5). Kroužky zde značí HMM pro jednotlivé fonémy z obrázku na předchozím snímku, paralelní větve pak různé dialekty (vozík, vozék, vozejk, vozik).



Obr. 8.5 HMM pro slovo vozík

Podobně lze HMM rozšířit na celé věty, resp. větné celky, u kterých je možné v plynulé řeči identifikovat počátky a konce (na rozdíl od slov).

8.1.2.3 Jazykový model

Cílem jazykového modelu je určit pravděpodobnost výskytu posloupnosti slov $P(W)$, která se určuje z velkých slovníků (tzv. korpusů) mluvené řeči.

Pravděpodobnost posloupnosti W , která má L slov je obecně dána vztahem

$$P(W) = \prod_{i=1}^L P(w_i | w_{i-1}, w_{i-2}, \dots, w_1)$$

který se pro delší posloupnosti omezuje na $n-1$ předchozích slov

$$P(W) = \prod_{i=1}^n P(w_i | w_{i-1}, w_{i-2}, \dots, w_{i-n+1})$$

Další informace o jazykových modelech jsou v následující kapitole.

8.2 Zpracování textu

Přístupy ke zpracování textu jsou značně různorodé – od relativně jednoduchých korektur textů, přes dolování znalostí z textů a strojové překlady, až po skutečné porozumění obsahu textu.

Většina přístupů ke zpracování textu je založena na jazykových modelech, které předpovídají pravděpodobnostní rozložení jazykových výrazů. Rozeznávají se znakové jazykové modely a slovní jazykové modely.

Nejsložitějším přístupem ke zpracování textu je lingvistický přístup, jehož problematiku stručně zmíníme v kap. 8.2.3

8.2.1 Znakové jazykové modely

Nejjednoduššími jazykovými modely jsou distribuce/rozložení pravděpodobností sekvencí znaků. Pro sekvence n znaků se používá název n -gram, resp. unigram pro 1-gram, bigram pro 2-gram a trigram pro 3-gram. Formálně je n -gram definován jako Markovův řetězec řádu $n-1$.

Podmíněná pravděpodobnost znaku c_i je dána výrazem $P(c_i | c_{i-1}, c_{i-2}, \dots, c_1)$, nebo ve zkráceném zápisu výrazem $P(c_i | c_{1:i-1})$ a pro nejčastěji používané trigramové modely pak výrazy $P(c_i | c_{i-1}, c_{i-2})$, resp. $P(c_i | c_{i-2:i-1})$.

Pro pravděpodobnost sekvence N znaků v trigramovém modelu pak zřejmě platí

$$P(c_{1:N}) = \prod_{i=1}^N P(c_i | c_{i-2:i-1})$$

Pozn.: Jazyk se 100 znaky má milion různých trigramů a pro přijatelné odhady pravděpodobností $P(c_i|c_{i-2:i-1})$ je zapotřebí text o alespoň 10 000 000 znacích.

Identifikace jazyka je jednou z přímých aplikací použití znakových jazykových modelů.

Ze známých textů se nejprve pro každý uvažovaný jazyk L_k zjistí podmíněné pravděpodobnosti $P(c_i|c_{i-2:i-1}, L_k)$. Pro neznámý text o délce N znaků zřejmě platí

$$P(c_{1:N}|L_k) = \prod_{i=1}^N P(c_i|c_{i-2:i-1}, L_k)$$

Jazyk L^* , ve kterém je s nejvyšší pravděpodobností neznámý text napsán, je pak dán výrazem

$$L^* = \underset{L_k}{\operatorname{argmax}} P(L_k|c_{1:N}) = \underset{L_k}{\operatorname{argmax}} P(c_{1:N}|L_k) \frac{P(L_k)}{P(c_{1:N})}$$

a protože pravděpodobnost $P(c_{1:N}) = 1$ (neznámý text je daný), pak

$$L^* = \underset{L_k}{\operatorname{argmax}} P(c_{1:N}|L_k)p(L_k) = \underset{L_k}{\operatorname{argmax}} P(L_k) \prod_{i=1}^N P(c_i|c_{i-2:i-1}, L_k)$$

Apriorní pravděpodobnosti $P(L_k)$ se odhadnou například z náhodně vybraných webových stránek, protože jejich přesná hodnota není kritická vzhledem k řádovým rozdílům pravděpodobností trigramových modelů.

Znakové jazykové modely se podobným způsobem používají například pro *korektury textů*, *rozpoznávání vlastních jmen v textech* atp.

8.2.2 Slovní jazykové modely

Slovní jazykové modely jsou přirozeným rozšířením znakových modelů. Opět rozeznáváme unigramy, bigramy, trigramy a n -gramy, tentokrát samozřejmě nad slovy. Velký problém je však v tom, že zatímco znaků v daném jazyku jsou řádově desítky až stovky, slov v daném jazyku jsou desítky tisíc až několik milionů. Navíc v některých jazycích, například v čínštině, nejsou slova od sebe oddělena mezerami, což použití slovních jazykových modelů v těchto jazycích prakticky vylučuje.

Pozn.: Pro unigramové slovní modely se často používá anglický název „bag of words“. V těchto modelech je text (například věta nebo dokument) reprezentován jako pytel jeho slov, bez ohledu na gramatiku a slovní spojení.

Častým použitím slovních jazykových modelů jsou *klasifikace textů*, například klasifikace e-mailů do dvou tříd *spam/ham*, kde *ham* je synonymem pro *not-spam*. Dobrymi indikátory spamu jsou n -gramy jako *best drug store*, *millions USD*, *pills for potency*, *pounds of gold* atp.

Formálně ($c \in \{spam, ham\}$):

$$\underset{c}{\operatorname{argmax}} P(c|message) = \underset{c}{\operatorname{argmax}} P(message|c) \cdot P(c)$$

Pravděpodobnosti $P(message|spam)$, $P(message|ham)$, $P(spam)$ a $P(ham)$ lze poměrně snadno určit z počtu spamů a ostatních zpráv v poštovních složkách.

Podobným přístupem lze *kategorizovat texty* například podle žánrů, vědeckých oblastí atp.

Další možnou prací s textem je *vyhledávání informací*. Existuje několik různých přístupů, v současné době je zřejmě nejpoužívanějším přístup založený na hodnotící funkci nazvané

BM25 (*Best Matches 25*). Funkce má dva parametry (vyhledávaný výraz a prohledávaný dokument) a vrací číselnou hodnotu, která určuje důležitost dokumentu z pohledu otázky. V této funkci se kombinují tři základní kritéria:

1. Četnost výskytu slov dotazu v prohledávaném dokumentu.
2. Inverzní četnost (eliminace důležitosti častého výskytu spojek, předpon atp).
3. Délku dokumentu (z dokumentů, které obsahují všechna slova dotazu, bývá důležitější dokument kratší).

Funkce BM25 je dána výrazem

$$BM25(d_j, q_{1:M}) = \sum_{i=1}^M IDF(q_i) \frac{TF(q_i, d_j)^{(k+1)}}{TF(q_i, d_j) + k \left(1 - b + b \frac{|d_j|}{L}\right)}$$

kde

$$IDF(q_i) = \log \frac{N - DF(q_i) + 0.5}{DF(q_i) + 0.5}$$

Jednotlivé symboly značí:

d_j	délku j -tého dokumentu (počet slov)
$q_{1:M}$	vyhledávaný výraz skládající se z M slov (q_i je i -té slovo)
N	počet prohledávaných dokumentů
L	průměrnou délku textu v těchto dokumentech
$TF(q_i, d_j)$	četnost výskytu slova q_i (<i>Term Frequency</i>) v dokumentu d_j
$IDF(q_i)$	inverzní četnost výskytu slova q_i
$DF(q_i)$	počet dokumentů (<i>Document Frequency</i>) obsahujících slovo q_i
b, k	parametry (obvykle $b = 0.75$, $k \in \langle 1.2, 2.0 \rangle$)

8.2.3 Lingvistická analýza

Nejsložitějším přístupem ke zpracování textu je lingvistický přístup, který vede k porozumění obsahu textu a může tak být využit například pro *strojové překlady* a *komunikaci v přirozeném jazyku*.

Lingvistickou analýzu můžeme rozdělit na pět relativně samostatných částí, a to na

- lexikální analýzu
- morfologická analýzu
- syntaktická analýzu
- sémantická analýzu
- pragmatická analýzu

Lexikální analýza převádí text na posloupnost slov. Zpracovává například slova rozdělená koncem řádku, zkratky, speciální znaky, číslice apod.

Morfologická analýza přiřazuje jednotlivým slovům slovní druh (podstatné jméno, přídavné jméno, zájmeno atd.), gramatické rysy (jmenné: rod, číslo, pád; slovesné: osoba, číslo, čas atd.), v češtině pak ještě slovní základ (kmen slova), apod.

Syntaktická analýza ověřuje, zda vstupní textový řetězec je větou v daném jazyku. Zpracování češtiny je velmi obtížné vzhledem ke složité morfologii našeho jazyka (k odvozování slov pomocí předpon a přípon), a proto si dále uvedeme základní informace pouze o syntaktické analýze anglického textu.

Slovník/lexicon (nepatrný fragment):

<i>Noun</i> (podstatné jméno)	→ boy [0.05] girl [0.05] plant [0.15] ...
<i>Verb</i> (sloveso)	→ is [0.1] do [0.08] grows [0.1] ...
<i>Adjective</i> (přídavné jm.)	→ right [0.15] blue [0.07] nice [0.05] ...
<i>Adverb</i> (příslovce)	→ here [0.05] ahead [0.06] nearby [0.02] ...
<i>Pronoun</i> (zájmeno)	→ I [0,1] me [0,09] you [0,08] ...
<i>RelPro</i> (relativní zájmeno)	→ that [0.4] which [0.15] who [0.2] ...
<i>Name</i> (jméno)	→ John [0.01] Mary [0.01] Prague [0.002] ...
<i>Article</i> (člen)	→ the [0.4] a [0.3] every [0.05], ...
<i>Prep</i> (předložka)	→ to [0.2] in [0.1] on [0.05] ...
<i>Conj</i> (spojka)	→ and [0.5] or [0.1] but [0.2] ...

Každé slovo uložené ve slovníku může být doplněno pravděpodobností jeho výskytu v dané kategorii. Součet pravděpodobností výskytu slov v dané kategorii (řádku) musí být roven jedné.

Syntaktické kategorie:

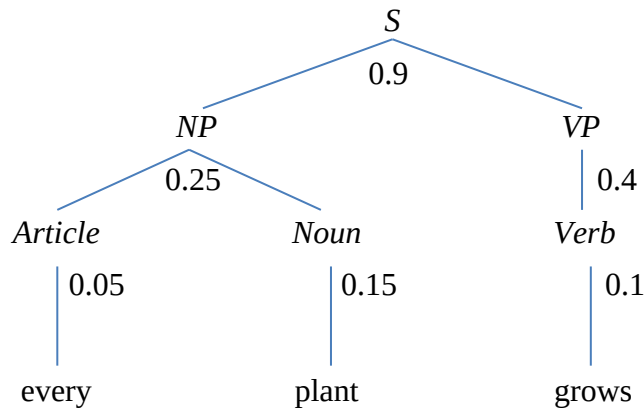
<i>S</i>	Sentence (věta),
<i>NP</i>	Noun Phrase (jmenná fráze),
<i>VP</i>	Verb Phrase (slovesná fráze),
<i>ADJS</i>	Adjectives (přídavná jména),
<i>PP</i>	Prepositional Phrase (předložková fráze),
<i>RelClause</i>	Relative Clause (relativní klauzule)

Gramatika (přepisovací pravidla):

<i>S</i>	→	<i>NP VP</i>	[0.9]	I + feel a breeze
		<i>S Conj S</i>	[0.1]	I feel a breeze + and + it stinks
<i>NP</i>	→	<i>Pronoun</i>	[0.3]	I
		<i>Name</i>	[0.1]	Mary
		<i>Noun</i>	[0.1]	stone
		<i>Article Noun</i>	[0.25]	the box
		<i>Article ADJS Noun</i>	[0.05]	the big red box
		<i>NP PP</i>	[0.1]	the box + on the table
		<i>NP RelClause</i>	[0.1]	the liquid + that is smelly
<i>VP</i>	→	<i>Verb</i>	[0.4]	shows
		<i>Vp Np</i>	[0.35]	feel + a breeze
		<i>VP Adjective</i>	[0.05]	is red
		<i>VP PP</i>	[0.1]	is + on the box
		<i>VP Adverb</i>	[0.1]	go ahead
<i>ADJS</i>	→	<i>Adjective</i>	[0.8]	smelly
		<i>Adjective ADJS</i>	[0.2]	big + red
<i>PP</i>	→	<i>Prep NP</i>	[1]	to + the table
<i>RelClause</i>	→	<i>RelPro VP</i>	[1]	that + is big

Každé přepisovací pravidlo může být také doplněno pravděpodobností jeho používání. Součet pravděpodobností pro pravidla přepisu každého nonterminálu musí být roven jedné.

Například pravděpodobnost věty „*Every plant grows.*“ se pak snadno určí pomocí derivačního stromu



v lineárním zápisu [S [NP [Article every] [Noun plant]] [VP [Verb grows]]]:

$$P(S) = 0.9 \cdot 0.25 \cdot 0.05 \cdot 0.15 \cdot 0.4 \cdot 0.1 = 0.0000675.$$

Tyto pravděpodobnosti se pak mohou využít při hodnocení různých významů jedné věty.

Rozšířené gramatiky (Augmented grammars):

Rozšířené gramatiky řeší poměrně závažný problém bezkontextových gramatik, který spočívá v tom, že vztahy NP a VP nejsou nezávislé. Například věta „John eats banana“ je jistě pravděpodobnější, než věta „John eats bandana“, protože John bude často jíst banány, ale pravděpodobně nikdy nebude jíst šátek.

Rozšíření gramatiky spočívá v tom, že se v přepisovacích pravidlech uvádí pravděpodobnosti pro konkrétní slova, například

$$VP(v) \rightarrow VP(v) NP(n) \quad [P(v,n)]$$

kde pro sloveso v (eats) a podstatné jméno n (banana) bude pravděpodobnost $P(\text{eats, banana})$ výrazně vyšší než pravděpodobnost pro podstatné jméno n (bandanna) $P(\text{eats, bandanna})$.

Problémem předchozího přístupu je velmi velké množství potřebných odhadů $P(v,n)$, například pro slovník s 20 000 podstatnými jmény a 5 000 slovesy je zapotřebí 1000000 těchto odhadů.

Další problém, který je řešitelný pomocí rozšířených gramatik jsou anglická osobní zájmena, kdy záleží na tom, zda jsou na místě podmětu, nebo předmětu ($\text{Case} \in \{\text{objective, subjective}\}$). Pak se přepisovací pravidla mohou upravit takto:

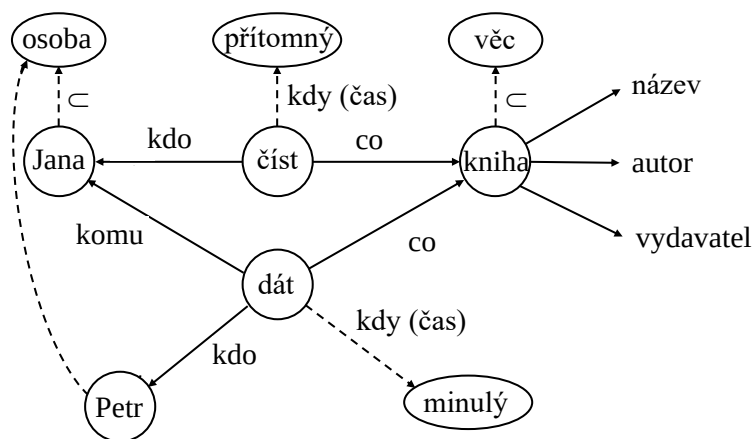
$$\begin{aligned}
 S &\rightarrow NP(\text{subjective}) VP \\
 NP(\text{Case}) &\rightarrow Pronoun(\text{Case}) \mid Name \mid Noun \mid \dots \\
 VP &\rightarrow Verb \mid Verb NP(\text{objective}) \mid \dots \\
 PP &\rightarrow Prep NP(\text{objective}) \\
 &\vdots \\
 Pronoun(\text{subjective}) &\rightarrow I \mid you \mid he \mid she \mid it \mid \dots \\
 Pronoun(\text{objective}) &\rightarrow me \mid you \mid him \mid her \mid it \mid \dots
 \end{aligned}$$

Sémantická analýza přiřazuje větám význam. Jde o velmi obtížnou problematiku, a přestože se řeší již déle než půl století, není dosud uspokojivě vyřešena. Problémem není extrahovat z textu znalosti, ale problémem je, jak tyto znalosti reprezentovat, strukturovat, efektivně v nich hledat a využívat je k odvození dalších znalostí. Pro správné pochopení významů textů je navíc často nutné mít určité znalosti o světě, a to jak znalosti obecné (např. že ryby žijí ve vodě,), tak znalosti konkrétní/odborné (např. že součet délek dvou stran v trojúhelníku nemůže být nikdy menší než délka strany třetí).

Sémantická analýza vyžaduje pochopení lexikální hierarchie, jako například hyponym a hyperonym (slov významem podřazených a nadřazených, například pes a zvíře), synonym (slov různých podob, ale stejných významů, například plakat a brečet), homonym (slov stejných podob ale různých významů, například los je zvíře i poukázka v loterii), antonym (slov opačného významu, například mladý a starý), sémiotiky (znakových systémů, např. dopravních značek, piktogramů apod.) atd.

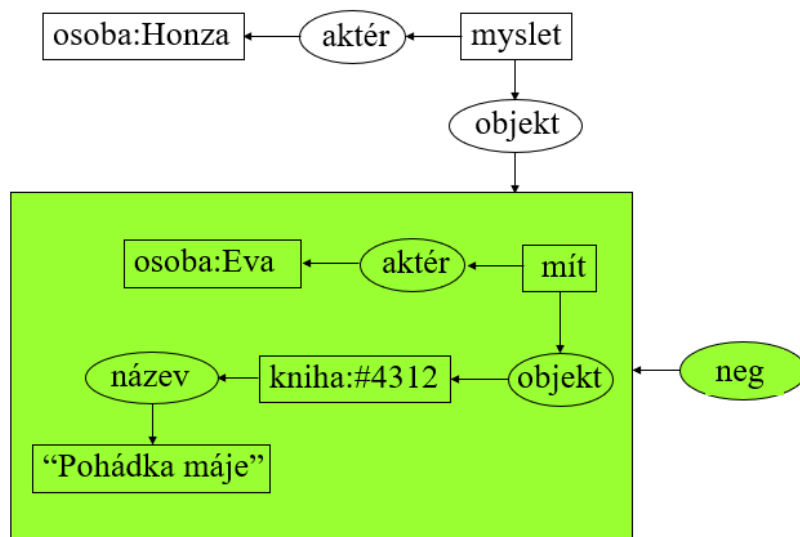
K reprezentaci znalostí získaných sémantickou analýzou se dříve používaly sémantické sítě a pojmové grafy, dnes je věnována pozornost především logice.

Sémantické sítě jsou orientované grafy s ohodnocenými uzly i hranami. Uzly představují objekty nebo pojmy a hrany mezi uzly pak jejich vzájemné vztahy. Základním vztahem je vztah *je*. Používá se obvykle jak pro označení příslušnosti objektu o ke třídě A ($o \in A$), tak i pro označení vztahu pojmu A k obecnějšímu pojmu B , resp. vztahu podtřídy A ke třídě B ($A \subset B$). Základním vztahem je vztah *je* proto, že umožňuje dědění vlastností. Na Obr. 8.6 je ukázán fragment sémantické sítě reprezentující větu *Jana čte knihu, kterou dostala od Petra*.



Obr. 8.6 Příklad sémantické sítě

Pojmové grafy nemají na rozdíl od sémantických sítí ohodnocené hrany. Vzájemné vztahy mezi pojmy zde reprezentují speciální relační uzly, a proto každá hrana musí spojovat uzel reprezentující pojem s uzlem reprezentujícím relaci. Pro snadné rozlišení pojmových uzlů a relačních uzlů jsou první obvykle kresleny jako obdélníky a druhé jako ovály. Na relace není kladeno žádné omezení; mohou být unární, binární atd. Každý pojmový graf reprezentuje libovolně složitý jednoduchý výrok a báze znalostí je pak tvořena množinou těchto grafů. Na Obr. 8.7 je ukázána část pojmového grafu reprezentující větu, ve které je argumentem relace výrok: *Honza si myslí, že Eva nemá knihu „Pohádka máje“*.



Obr. 8.7 Příklad pojmového grafu

Logická schémata používají k reprezentaci znalostí nejčastěji predikátovou logiku prvního řádu, která je prostředkem pro reprezentaci znalostí s velmi dobře definovanou sémantikou i jasnou inferenční strategií. Formule predikátové logiky umožňují poměrně snadnou a efektivní reprezentaci dědičností, výjimek a lze jimi postihnout i hierarchické členění znalostí. Například obecné předpoklady pro třídu ptáků dědí každý příslušník této třídy.

$$\begin{aligned} &\forall x(\text{ptak}(x) \Rightarrow \text{leta}(x)) \\ &\forall x(\text{ptak}(x) \Rightarrow \text{ma_peri}(x)) \\ &\dots \\ &\text{ptak}(\text{skrivan}) \\ &\text{ptak}(\text{kos}) \\ &\text{ptak}(\text{orel}) \end{aligned}$$

Někdy však některý obecný předpoklad nemusí být pravdivý. Nelétá například pštros, pták se zlomeným křídlem, nebo pták, který je mechanickou hračkou. Takové výjimky pak lze ošetřit následovně:

$$\begin{aligned} &\forall x(\text{ptak}(x) \wedge \neg \text{vyjimka_ptak}(x) \Rightarrow \text{leta}(x)) \\ &\text{vyjimka_ptak}(\text{pstros}) \\ &\forall x(\text{ptak}(x) \wedge \text{zlomene_kridlo}(x) \Rightarrow \text{vyjimka_ptak}(x)) \\ &\forall x(\text{ptak}(x) \wedge \text{mechanicka_hracka}(x) \Rightarrow \text{vyjimka_ptak}(x)) \\ &\dots \\ &\text{ptak}(\text{skrivan}) \\ &\text{ptak}(\text{kos}) \\ &\text{ptak}(\text{orel}) \\ &\text{ptak}(\text{pstros}) \end{aligned}$$

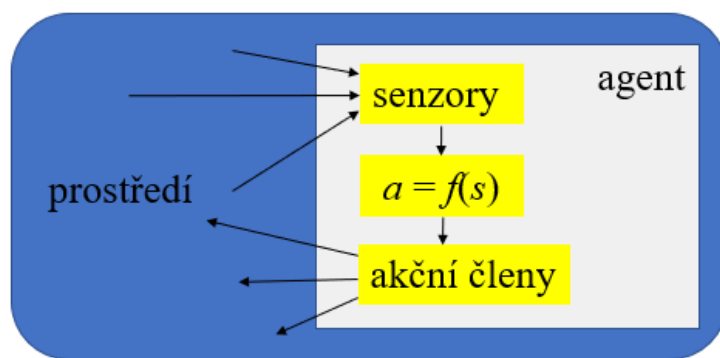
Predikátová logika je logikou monotónní. Proto každá přidaná klauzule musí být pravdivá a odvozené klauzule stále platné. Jinými slovy to znamená, že znalosti lze pouze přidávat, nelze je však jednoduše modifikovat. Tato skutečnost činí problémy při formulaci výroků založených na důvěře, předpokladech a především při formulaci výroků s omezenou časovou platností.

Pragmatická analýza se použije (zatím víceméně teoreticky) v případech, kdy věty obsahují osobní a ukazovací zájmena, příslovečná určení místa a času, nebo jsou samy o sobě nejednoznačné:

- lexikálně: John positioned the dress on the rack.
 Kate wanted the dress on the rack.
- syntakticky: Bob asked John to tell Jane to come on Saturday.
- sémanticky: I ate spaghetti with meatballs.
 I ate spaghetti with salad.
 I ate spaghetti with abandon.
 I ate spaghetti with a fork.
 I ate spaghetti with a friend.
- obsahují metonymie: Fiat announced a new model.
- obsahují metaforu: The silver head.

9 Agentní systémy

Za (umělého) agenta můžeme považovat jakýkoliv systém, který vnímá okolní prostředí, a který může toto prostředí měnit (Obr. 9.1). Právě možnost změny prostředí odlišuje tohoto agenta od jiných inteligentních systémů. Pokud je umělým agentem fyzický systém, pak se tento agent nazývá robotem. Příkladem softwarového agenta je například chatbot, tj. program určený ke komunikaci počítače s lidmi využívaný především v zákaznické podpoře, kde nahrazuje živé operátory.



Obr. 9.1 Schéma agenta

Z matematického hlediska je chování agenta popsáno funkcí $a = f(s)$, která transformuje signály ze senzorů na signály řídící akční členy agenta. Tuto funkci provádí prostřední blok z obrázku a je nejdůležitější částí agenta.

Dále se zaměříme výhradně na racionální inteligentní agenty, jejichž chování spočívá především v netriviálním rozhodování (podmínky z předchozího odstavce splňuje například i obyčejná kalkulačka). Racionální agent by měl na základě svých dosavadních znalostí vybrat pro každou sekvenci vjemů akci, která změní prostředí, pokud možno optimálním způsobem. Hodnocení optimálnosti však nemůže provádět agent sám, protože by pak často mohlo jít o hodnocení, které by bylo neobjektivní a nadhodnocené.

Je zřejmé, že značný vliv na návrh architektury a řídicího programu bude mít prostředí, ve kterém bude agent pracovat. Na toto prostředí může být přitom nahlíženo z několika různých hledisek, jak již bylo naznačeno v úvodu kap. 2.:

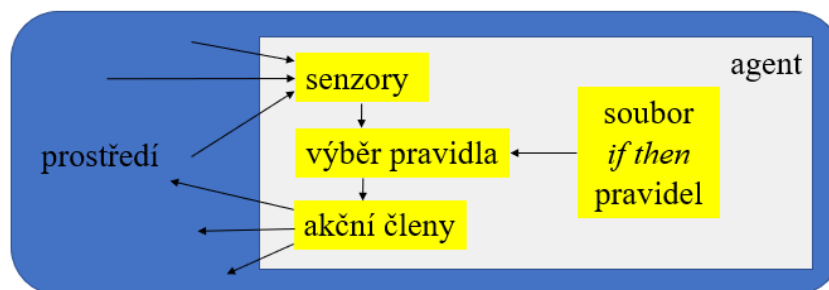
- Pozorovatelnosti
 - plně pozorovatelné (deskové hry, analýza obrázku, ...)
 - částečně pozorovatelné (karetní hry, robotická auta, ...)
 - nepozorovatelné (někdy řešitelné na základě důvěry agenta v jeho aktuální stav a příslušnou akci – například předpis širokospektrálních antibiotik lékařem na základě důvěry, a ne na základě podrobných laboratorních vyšetření pacienta)
- Určitosti
 - deterministické (deskové hry, analýza obrázku, ...)
 - stochastické (karetní hry, medicínská diagnostika, ...)
- Spojitosti
 - diskrétní (deskové hry, karetní hry, ...)
 - spojitě (karetní hry, medicínská diagnostika, ...)

- Časové závislosti
 - statické (deskové hry, analýza obrázku, ...)
 - dynamické (robotická auta, medicínská diagnostika, ...)
- Návaznosti
 - episodické (analýza obrázku, montážní robot, ...)
 - sekvenční (karetní hry, medicínská diagnostika, ...)

Racionální agenty lze rozdělit na dva základní typy:

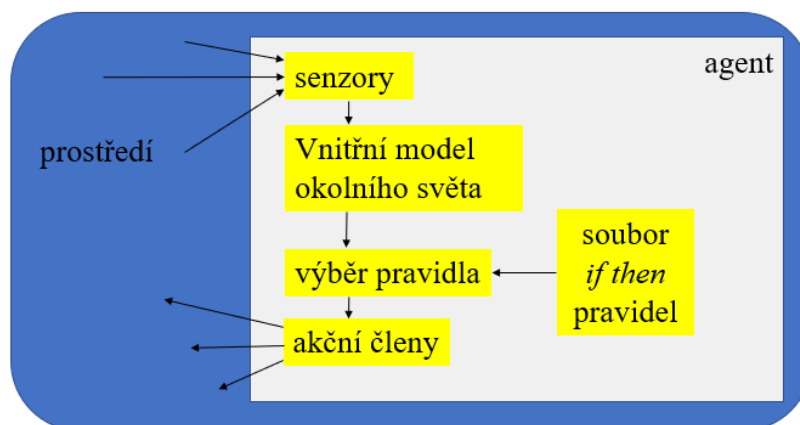
- Jednoduché reflexní agenty (*Simple reflex agents*)
- Agenty založené na modelech (*Model-based agents*)

Jednoduchý reflexní agent (Obr. 9.2) reaguje pouze na bezprostřední podněty, kterými jsou výstupy senzorů. Neukládá si žádné informace o předchozích podnětech, ani o svých předchozích akcích. Přes svou jednoduchost je poměrně často používán, příkladem může být robotický vysavač.



Obr. 9.2 Schéma jednoduchého reflexního agenta

Agent založený na modelu (Obr. 9.3) kombinuje současný vjem se svým vnitřním modelem okolního prostředí (světa), a vytváří si tak aktualizovaný popis tohoto prostředí.



Obr. 9.2 Schéma agenta založeného na modelu

Agent založený na modelu musí řešit tyto čtyři otázky:

- Jaký je okolní svět právě nyní.
- Jak by se změnil, kdybych provedl jednotlivé možné akce.
- Jaký bych měl přínos z každé z těchto změn.
- Jakou akci mám tedy provést.

Agent však velmi často nemůže přesně určit současný stav okolního prostředí (například to vůbec není možné pro částečně pozorovatelné prostředí), a proto vnitřní model obvykle reprezentuje pouze jeho nejlepší odhady. Musí se však rozhodnout a nějakou akci provést.

BDI (Belief-Desire-Intention) agent je zřejmě nejznámější agent založený na modelu. Tento agent se vyznačuje tím, že má tři základní datové struktury:

- Beliefs: Jde o představy agenta o stavu okolního prostředí, které však nemusí odpovídat skutečnému stavu.
- Desires: Jde o přání (touhy) agenta a pokud se naskytne vhodná příležitost, pak se mohou stát cíli jeho chování.
- Intentions: Jde o záměry agenta, které si sestavuje jako strukturu plánů, které mohou vést ke splnění nějakého z jeho cílů.

Důležitou součástí agentů je plánovač, který sestavuje plán, jak dosáhnout vybraného cíle. Někdy mohou být plány pro konkrétní záměry implementovány již v době návrhu agenta a v průběhu jeho činnosti jsou pak pouze vybírány z této zásoby plánů.

9.1 Plánování činnosti agenta

Nejstarším, nejznámějším a dosud používaným systémem pro plánování činnosti samostatných autonomních agentů je STRIPS (*Stanford Research Institute Problem Solver*). STRIPS ke své činnosti, tj. k hledání plánu, používá:

- Databázi (DB), která obsahuje aktuální stav řešeného problému popsany pomocí predikátů.
- Zásobník, který obsahuje cílové predikáty i predikáty dílčích cílů.
- Operátory, které umožňují měnit stav DB. Každý operátor je přitom popsán třemi seznamy: Seznam COND obsahuje predikáty popisující použitelnost operátoru, seznam DEL obsahuje predikáty, které se při použití operátoru z DB odstraní a seznam ADD obsahuje predikáty, které se při použití operátoru do DB přidají.
- Seznam/frontu použitých operátorů (tj. plán řešení úlohy).

Princip činnosti STRIPS je následující:

1. Aktuální stav řešeného problému se uloží do DB, konjunkce predikátů popisujících požadovaný cílový stav se uloží do zásobníku a vytvoří se prázdná fronta pro použité operátory (plán).
2. Pokud je na vrcholu zásobníku konjunkce predikátů, porovnává se postupně každý predikát s predikáty v DB. Pokud se některý predikát v aktuální DB nenachází, uloží se do zásobníku (nový vrchol zásobníku).
3. Pokud se predikát na vrcholu zásobníku v aktuální DB nachází, tak se pouze ze zásobníku odstraní.
4. Pokud predikát na vrcholu zásobníku v aktuální DB není, tak se hledá operátor, který ho tam může uložit. Do zásobníku se pak nejprve uloží nalezený operátor a pak i konjunkce predikátů COND popisujících podmínky jeho použitelnosti.
5. Pokud je na vrcholu zásobníku predikát operátoru, pak se operace provede:
 - a) z DB se odstraní predikáty, které má operátor v seznamu DEL a do DB se naopak přidají predikáty, které má operátor v seznamu ADD.
 - b) Operátor se z vrcholu zásobníku odstraní a uloží se do fronty operátorů.
6. Pokud není zásobník prázdný, pak se postup opakuje od bodu 2, jinak je úloha vyřešena a fronta operátorů pak obsahuje plán, kterým se úloha vyřeší.

Ukázka činnosti STRIPS

Uvažujme čtyři predikáty pro popis stavů:

on(X,Y)	X leží na Y
clear(X)	X je volné, tj. nic na něm neleží
armempty	ruka agenta je prázdná
holding(X)	ruka agenta drží X

a dva operátory, které mohou stavy měnit (původní systém měl čtyři operátory, rozlišoval akce polož na stůl/polož na kostku a zvedni ze stolu/kostky):

pickup(X, Y)	zvedni X z Y
Condition: [on(X, Y), clear(X), armempty]	podmínka
Delete: [on(X, Y), clear(X), armempty]	tyto predikáty se z DB odstraní
Add: [clear(Y), holding(X)]	tyto predikáty se do DB přidají
puton(X, Y)	polož X na Y
Condition: [clear(Y), holding(X)]	podmínka
Delete: [clear(Y), holding(X)]	tyto predikáty se z DB odstraní
Add: [on(X, Y), clear(X), armempty]	tyto predikáty se do DB přidají

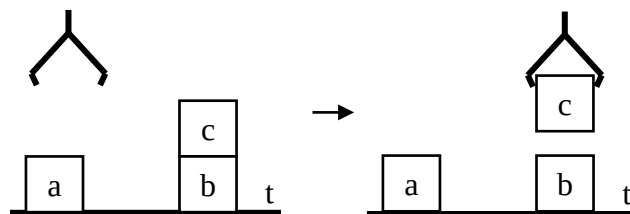
Úloha je zadána počátečním a koncovým stavem, například

Initial state: on(a, t), on(b, t), on(c, b), clear(a),clear(c), armempty, clear(t).

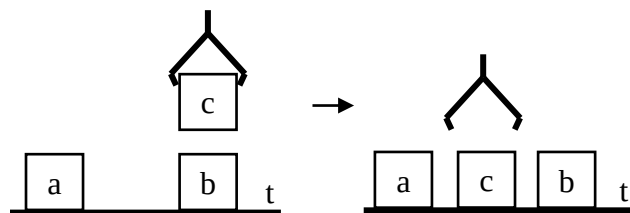
Goal state: on(c, t), on(b, c), on(a, b).

Řešení (předpokládáme, že stůl t (Table) je dostatečně velký, a že proto stále platí clear(t)!):

Zásobník	DB
on(c, t), on(b, c), on(a, b)	on(a, t), on(b, t), on(c, b), clear(a),clear(c), armempty, clear(t)
on(c, t)	
puton(c, t)	
clear(t), holding(c)	
holding(c)	
pickup(c, Y)	
on(c, Y), clear(c), armempty	se substitucí {Y/b} jsou podmínky pro pickup(c, b) splněné a operace se provede



Nový stav zásobníku	Nový stav DB
on(c, t), on(b, c), on(a, b)	on(a, t), on(b, t), clear(a), clear(b), holding(c), clear(t)
on(c, t)	
puton(c, t)	
clear(t), holding(c)	
holding(c)	



podmínky pro puton(c, t) jsou splněné a operace se provede

Nový stav zásobníku	Nový stav DB
$\text{on}(c, t), \text{on}(b, c), \text{on}(a, b)$ $\text{on}(b, c)$ $\text{puton}(b, c),$ $\text{clear}(c), \text{holding}(b)$ $\text{holding}(b)$ $\text{pickup}(b, Y)$ $\text{on}(Y, Z), \text{clear}(Y), \text{armempty} \dots\dots$	$\text{on}(a, t), \text{on}(b, t), \text{clear}(a), \text{clear}(b), \text{on}(c, t), \text{clear}(c), \text{armempty},$ $\text{clear}(t)$

..... se substitucí $\{Y/b, Z/t\}$ jsou podmínky pro $\text{pickup}(b, t)$ splněné a operace se provede

Nový stav zásobníku	Nový stav DB
$\text{on}(c, t), \text{on}(b, c), \text{on}(a, b)$ $\text{on}(b, c)$ $\text{puton}(b, c),$ $\text{clear}(c), \text{holding}(b)$ $\text{holding}(b)$	$\text{on}(a, t), \text{clear}(a), \text{on}(c, t), \text{clear}(c), \text{holding}(b), \text{clear}(t)$

podmínky pro $\text{puton}(b, c)$ jsou splněné a operace se provede

Nový stav zásobníku	Nový stav DB
$\text{on}(c, t), \text{on}(b, c), \text{on}(a, b)$ $\text{on}(a, b)$ $\text{puton}(a, b)$ $\text{clear}(b), \text{holding}(a)$ $\text{pickup}(a, Y)$ $\text{on}(Y, Z), \text{clear}(Y), \text{armempty}$	$\text{on}(a, t), \text{clear}(a), \text{on}(c, t), \text{on}(b, c), \text{clear}(b), \text{armempty}, \text{clear}(t)$

..... se substitucí $\{Y/a, Z/t\}$ jsou podmínky pro $\text{pickup}(a, t)$ splněné a operace se provede

Nový stav zásobníku	Nový stav DB
$\text{on}(c, t), \text{on}(b, c), \text{on}(a, b)$ $\text{on}(a, b)$ $\text{puton}(a, b)$ $\text{clear}(b), \text{holding}(a)$	$\text{on}(c, t), \text{on}(b, c), \text{clear}(b), \text{holding}(a), \text{clear}(t)$

podmínky pro $\text{puton}(a, b)$ jsou splněné a operace se provede.

Nový stav zásobníku

Nový stav DB

on(c, t), on(b, c), on(a, b) on(c, t), on(b, c), clear(b), holding(a), clear(t)
on(a, b)

Všechny predikáty v zásobníku jsou nyní splněné, a ze zásobníku se odstraní. Zásobník je nyní prázdný a úloha je vyřešena. Obsah fronty operátorů (řešení úlohy) je: pickup(c, b), puton(c, t), pickup(b, t), puton(b, c), pickup(a, t), puton(a, b)).

Poznamenejme, že v konjunkci cílových predikátů je výhodné dávat na začátek predikáty pro spodní kostky. Jinak řešení úlohy je výrazně delší a obsahuje řadu zbytečných operací.

Pro zvědavé čtenáře je níže uveden STRIPS v jazyku PROLOG:

```
action(pick(X,Y),
  [on(X,Y),clear(X),armempty],
  [on(X,Y),clear(X),armempty],
  [clear(Y),holding(X)]).
action(put(X,Y),
  [clear(Y),holding(X)],
  [clear(Y),holding(X)],
  [on(X,Y),clear(X),armempty]).
strips(Db,G,Plan):-
  goal_stack(Db,[G],[],Plan).           % G je seznam cilu,[ ] pocatecni plan
goal_stack(_,[ ],P,P).                   % zasobnik cilu prazdny - uloha vyresena
goal_stack(Db,Stack,P,Pnew):-            % provadi postupne TOP polozky zasobniku
  goal_top(Db,Db_crnt,Stack,Stack_crnt,P,P_crnt),!,
  goal_stack(Db_crnt,Stack_crnt,P_crnt,Pnew). % a znovu s novym cilem
goal_top(Db,Db,[H|T],T,P,P):-            % polozka TOP je predikat v databazi -> je OK
  member(H,Db), !.
goal_top(Db,Db,[H|T],[Cond,Action,H|T],P,P):- % polozka TOP je v ADD seznamu akce
  action(Action,Cond,_Add),               % novym cilem na vrcholu zasobniku je akce
  member(H,Add), !.
goal_top(Db,Db_new,[H|T],T,P,P_new):-    % polozka TOP je akce, která se provede
  action(H,_Del,Add),
  del(Del,Db,Db_tmp),
  add(Add,Db_tmp,Db_new),
  appendL(P,[H],P_new), !.
goal_top(Db,Db,[H|T],T,P,P):-            % polozkou TOP je seznam
  testOK(H,Db), !.                       % vsechny polozky jsou splnene
goal_top(Db,Db,[H|T],Snew,P,P):-         % polozkou TOP je seznam
  is_list(H),                             % nesplnene polozky jednotlivě
  push2stack(Db,H,[H|T],Snew), !.        % do zasobniku
testOK([ ],_).
testOK([H|T],Db):-
  member(H,Db),
  testOK(T,Db).
push2stack(_,[ ],S,S).
push2stack(Db,[H|_],[H1|T1],[H,H1|T1]):-
  not(member(H,Db)).
push2stack(Db,[H|T],S,Snew):-
  member(H,Db),
  push2stack(Db,T,S,Snew).
```

```

member(X,[X|_]):-!.
member(X,[_|T]):-
    member(X,T).
add([],Db,Db).
add([H|T],Db,Db_new):-
    add1(H,Db,Db_tmp),
    add(T,Db_tmp,Db_new).
add1(H,Db,Db):-H=clear(t),!.    % predikat clear(t) v DB stále je, a proto se nepridava
add1(H,Db,Db_new):-append(Db,[H],Db_new).
del([],Db,Db).
del([H|T],Db,Db_new):-
    del1(H,Db,Db_tmp),
    del(T,Db_tmp,Db_new).
del1(H,[H|T],[H|T]):-H=clear(t),!.    % predikat clear(t) se nemaze a v DB stále zustava
del1(H,[H|T],T).
del1(H,[X|T],[X|Tnew]):-X\=H,del1(H,T,Tnew).
appendL([],L,L).
appendL([H|T],L,[H|L1]):-
    appendL(T,L,L1).

```

% Uloha z prikladu (reseni se spusti prikazem task1.):

```
db1([on(a,t),on(b,t),on(c,b),clear(a),clear(c),armempty,clear(t)]).
```

```
g1([on(c,t),on(b,c),on(a,b)]).
```

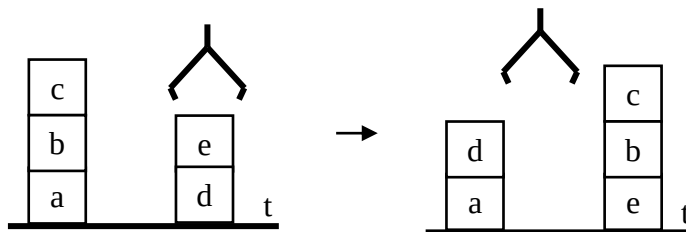
```
task1:-db1(DB),g1(G),tell('task1'),strips(DB,G,P),write(P),told.
```

% Slozitejsi uloha (reseni se spusti prikazem task2.):

```
db2([on(a,t),on(b,a),on(c,b),clear(c),on(d,t),on(e,d),clear(e),armempty,clear(t)]).
```

```
g2([on(e,t),on(b,e),on(c,b),on(a,t),on(d,a)]).
```

```
Task2:-db2(DB),g2(G),tell('task2'),strips(DB,G,P),write(P),told.
```



Obsah souboru task1.vysl:

```
[pickup(c,b),puton(c,t),pickup(b,t),puton(b,c),pickup(a,t),puton(a,b)]
```

Obsah souboru task2.vysl:

```
[pickup(e,d),puton(e,t),pickup(c,b),puton(c,t),pickup(b,a),puton(b,e),pickup(c,t),puton(c,b),
pickup(d,t),puton(d,a)]
```

9.2 Multiagentní systémy

Multiagentní systémy (MAS, *Multi-Agent Systems*) jsou takové systémy, kde se v prostředí pohybuje více inteligentních agentů. Dochází tak nejen k rozšíření interakcí agent ↔ prostředí, na interakce agenti ↔ prostředí, ale i k potřebě vzniku nového typu interakcí agent ↔ agenti.

Podle architektury mohou být MAS rozdělené do čtyř základních skupin:

- centralizované v MAS je jeden řídicí agent, kterému jsou všichni ostatní podřízeni
- hierarchické v MAS se používají různé úrovně řízení
- federované v MAS se používá nepřímá komunikace přes prostředníka
- decentralizované v MAS jsou decentralizovány role i řízení

Každý agent se snaží maximalizovat svůj očekávaný užitek z konaných akcí a ostatní agenti chtějí přirozeně totéž. Jednotliví agenti, resp. skupiny agentů v MAS proto mohou buď spolupracovat (například autonomní samořizené automobily, jejichž společným zájmem je vyhnout se kolizím), nebo mezi sebou soutěžit (například robotický fotbal, kdy cílem skupiny je porazit soupeře), nebo mohou být cíle (skupin) agentů současně kooperativní i soutěživé (například samořizená taxi se budou vyhýbat kolizím, ale budou chtít zaujmout nejvýhodnější místa na stanovištích).

Interakce mezi agenty v MAS může být přímá, nebo nepřímá, a to buď přes prostředí, nebo přes zprostředkovatele (*facilitator*). Zprostředkovatelem může být buď

- Dohazovač (*matchmaker*), který hledá vhodné agenty pro požadovanou službu. Agent, který poskytovatele nějaké služby hledá pak na základě informace od dohazovače kontaktuje přímo agenta, který požadovanou službu poskytuje.
- Makléř (*broker*), který také hledá agenty pro požadovanou službu, ale tuto službu i objedná a poptávajícímu agentovi pak předá výsledek poskytnuté služby.
- Prostředník (*mediator*), který pomáhá agentům řešit spory.

Pro vzájemnou komunikaci agentů bylo navrženo několik komunikačních jazyků (ACLs, *Agent Communication Languages*), nejpoužívanějším standardem je norma FIPA (*Foundation for Physical Intelligent Agents*).

Základní typy komunikace v MAS se liší podle cíle zpráv:

- adresné posílání zpráv zpráva se posílá konkrétním agentům
- všesměrové posílání zpráv zpráva se posílá všem agentům
- selektivní posílání zpráv zpráva se posílá určité skupině agentů

Vlastní komunikaci (CA, *communicative act*) lze považovat za speciální typ akce realizované zasláním zprávy. Jednotlivé zprávy je možné klasifikovat do následujících sedmi základních kategorií:

- oznamovací Zpráva byla odeslána
- příkazovací Odešli zprávu
- zavazující Pošlu zprávu
- pověřující Může poslat zprávu
- zakazující Nesmí poslat zprávu
- deklarativní Potvrzuji, že zpráva byla odeslána
- expresivní Přeji si odeslat zprávu

Spolupráce mezi agenty má tyto etapy:

1. Rozpoznání situace vhodné pro spolupráci, například, když se v prostředí objeví situace, kdy agent nemůže dosáhnout cíle bez vědomé pomoci ostatních agentů.
2. Formování týmu. Tým se buduje z agentů, kteří jsou za dohodnutých podmínek ochotni spolupracovat. Nový tým má pak společný záměr dosáhnout cíle, pro který byl sestaven.
3. Skupinové plánování. Podle typů řešeného problému sestavují agenti plány pro dosažení záměru, a to buď samostatně, nebo koordinovaně.
4. Skupinové konání. Jedná se o řízení (koordinované) vykonávání sestaveného plánu.

Vytváření koalic:

- Pokud přetrvává společný zájem více agentů o vzájemnou spolupráci, je žádoucí, aby se agenti dohodli dodržovat předem ustanovená pravidla, a to
 - Pravidla pro sdílení prostředků, která by měla zamezit ztrátám kvůli opakovanému vzniku konfliktů.
 - Pravidla pro spolupráci při společném plnění úkolů, které nemohou agenti plnit samostatně.
 - Pravidla pro sdílení informací, která zavazují agenty šířit aktuální informace v rámci celé koalice.

Penalizace v rámci koalice (při porušení pravidel):

- Odebrání prostředků. Agentovi mohou být odebrány přidělené prostředky, například strojový čas, paměťové prostředky, komunikační a transportní kanály apod.
- Zrušení závazků. Agenti mohou zrušit své závazky vůči agentovi, který porušil pravidla.
- Ztráta reputace. Reputace je informace o důvěryhodnosti agenta, která je šířena v multiagentním prostředí. Špatná reputace snižuje šance agenta na dohody v budoucnu.
- Izolace agenta. Zamítnutí přístupu agenta do koalic.

Konflikty v MAS mohou být fyzické (spory o prostředky, data, služby, přístup do kritické sekce apod), nebo mentální (konflikty v představách, přáních, závazcích, záměrech apod), například:

Řešení konfliktů

- Útěk. Všichni v konfliktu se vzdají řešení a ustoupí. Nikdo sice neprohrál, ale záměr nebyl splněn.
- Zničení oponenta. Poražen je slabší agent, jeho ztráta v MAS je nevratná.
- Podmanění oponenta. Slabší agent je manipulací, výhrůžkami, podplácením apod. donucen ke spolupráci s vítězem.
- Delegace konfliktu. O řešení konfliktu rozhodne třetí strana a všichni zúčastnění jsou zavázáni rozhodnutí přijmout.
- Kompromis: Jednotliví účastníci konfliktu sleví ze svých cílů tak, aby se situace změnila na nekonfliktní.
- Konsensus. Konsensu je dosaženo tím, že je zvoleno jiné řešení, které vyhovuje oběma stranám (nejlepší, ale nejobtížnější řešení).

Etapy konfliktů

- Detekce konfliktu. Jde o schopnost agenta nebo agentů rozpoznat, že konflikt existuje.
- Rozpoznání konfliktu. Jde o rozpoznání toho, o jaký konflikt se konkrétně jedná.
- Vyhnutí se konfliktu. Agenti si sami zvolí, který zabrání vzniku konfliktu.
- Vyřešení konfliktu. Komunikací je společně dohodnut postup, který eliminuje konflikt.
- Řízení konfliktu. Konfliktní situace přetrvává, ale jsou určena pravidla, jak v těchto případech postupovat.
- Poučení se z konfliktu. Změnaází znalostí, nebo představ agentů o vzniku a průběhu konfliktu.
- Analýza průběhu konfliktu.
- Analýza způsobu řešení konfliktu a vhodnosti zvoleného řešení.

Protokoly pro řešení konfliktů

- Dražby. Více agentů má zájem na využití sdíleného prostředku nebo služby, a soutěží v dražbě o jeho/její přidělení.

- Volby. Jedná se o situaci, kdy existuje několik řešení reprezentovaných agenty, kteří různá řešení navrhuji. Je voleno řešení na základě preferencí jednotlivých agentů.
- Vyjednávání. Agenti se snaží najít kompromis pro řešení konfliktů, většinou způsobem, že slevují ze svých cílů. Výsledkem komunikace je konflikt nebo dohoda, ve které se jeden agent zavazuje druhému vykonat nějakou činnost. Vyjednává se buď mezi rovnoprávnými agenty, častěji však v hierarchické struktuře MAS.
- Argumentace. Jde o řešení mentálních konfliktů. Dvě teorie jsou v rozporu a předmětem sporu je znalost. Agenti se snaží podpořit své znalost na základě argumentů.

Vyjmenované problémy dostatečně ukazují na složitost MAS i na obtížnost jejich návrhu a programování. Řešení těchto problémů přesahuje rámec předmětu IZU, a proto se touto problematikou nebudeme podrobněji zabývat.

10 Závěr

Věřím, že tento učební text bude užitečnou pomůckou při studiu předmětu Základy umělé inteligence a že také přispěje ke zvýšení zájmu o studium specializace Inteligentní systémy v magisterském studiu na FIT. Profilující a zajímavé předměty této specializace (Agentní a multiagentní systémy, Inteligentní systémy, Soft Computing a Teorie her) si mohou zapsat a absolvovat i studenti jiných specializací.

Čtenářům předem děkuji za upozornění na případné chyby, nepřesnosti a překlepy, které mohly uniknout mé kontrole tohoto textu.

V Brně 1. 6. 2021

autor

Literatura

- [1] Ertel, W.: Introduction to Artificial Intelligence, Springer, second edition, Springer 2017, ISSN 1863-7310
- [2] Russell, S., Norvig, P.: Artificial Intelligence, A Modern Approach, Prentice-Hall, Inc., 1995, ISBN 0-13-360124-2, second edition 2003, ISBN 0-13-080302-2, third edition 2010, ISBN 0-13-604259-7